

Basel Halak *Editor*

Convolutional Neural Network Accelerators

From Basic Design Principles
to Advanced Security Applications

 Springer

Editor
Basel Halak

Convolutional Neural Network Accelerators

From Basic Design Principles to Advanced Security Applications



OceanofPDF.com

Editor

Basel Halak

University of Southampton, Southampton, UK

ISBN 978-3-032-08513-9

e-ISBN 978-3-032-08514-6

<https://doi.org/10.1007/978-3-032-08514-6>

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

This work is subject to copyright. All rights are solely and exclusively licensed by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.

The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.

The publisher, the authors and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, expressed or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

This Springer imprint is published by the registered company Springer Nature Switzerland AG

The registered company address is: Gewerbestrasse 11, 6330 Cham,
Switzerland

[OceanofPDF.com](https://oceanofpdf.com)

*To
Suzanne, Hanin, and Sophia
with Love*

OceanofPDF.com

Preface

The exponential growth of artificial intelligence applications, particularly in computer vision, has positioned Convolutional Neural Networks (CNNs) at the forefront of technological advancement. CNNs have revolutionized image processing tasks, such as classification, object detection, and segmentation, enabling remarkable progress across industries including healthcare, automotive, and telecommunications. However, the widespread deployment of CNNs comes with significant computational challenges. These networks require intensive processing power, memory, and energy, making traditional CPU- and GPU-based systems increasingly inadequate for real-time or resource-constrained environments.

The escalating demand for efficient CNN execution has led to a critical need for specialized hardware solutions. Dedicated CNN hardware accelerators, such as Field-Programmable Gate Arrays (FPGAs), Application-Specific Integrated Circuits (ASICs), and custom neural processing units, have emerged as essential tools. These solutions offer tailored architectures designed specifically to enhance the performance and energy efficiency of CNN implementations. The transition toward specialized hardware is not merely an optimization but a necessity to sustain the growth and effectiveness of AI-driven technologies.

In addressing these challenges, this book aims to provide a comprehensive exploration of CNN fundamentals, advanced hardware implementations, and innovative optimization strategies. The book is structured into three parts, addressing design, application, and security of CNN, respectively. Each part of which provides foundational concepts, practical methodologies, and illustrative case studies, supported by extensive references.

Contents at Glance

Part I: CNN Design and Implementation Principles

Chapter [1](#) introduces the fundamentals of CNN architectures, detailing their operational principles, layer functions, and typical structures, as well as motivating the need for hardware acceleration.

Chapter [2](#) provides an extensive tutorial covering CNN design and implementation, both in software frameworks such as TensorFlow and in hardware using SystemVerilog.

Chapter [3](#) discusses performance optimization methods, particularly through approximate multipliers and error injection techniques, enhancing computational efficiency without significant accuracy loss.

Part II: Applications of CNN in Hardware Security

Chapter [4](#) presents a detailed tutorial on simulating Denial-of-Service (DoS) attacks in Network-on-Chip architectures using Gem5, providing insights into security vulnerabilities and defenses of this communication architecture, which will be subsequently used to integrated CNN accelerators.

Chapter [5](#) explores practical CNN applications and security monitors for detecting and localizing DoS attacks in Network-on-Chip environments, highlighting the power of CNNs in hardware security contexts.

Chapter [6](#) compares various machine learning approaches for DoS detection, showcasing cascading models and graph-based methods, along with their respective performance evaluations.

Part III: Threat Analysis of AI Production Chain

Chapter [7](#) provides a comprehensive review of the security threats inherent to each stage of the AI models production process, from initial data acquisition to post-deployment archiving. Risks associated with software and hardware implementation are also discussed.

Chapter [8](#) presents a case study on the mitigation of hardware Trojans in CNN accelerators with exemplary hardware implementation, and detailed performance evaluations.

Book Audience

The intended audience includes graduate students, academic researchers, hardware design engineers, and practitioners in the field of artificial intelligence and embedded development. By bridging theoretical knowledge and practical application, this book aspires to serve as both a comprehensive textbook and a reference for ongoing research and development efforts in CNN hardware acceleration.

I sincerely hope this work contributes to the understanding and advancement of CNN hardware acceleration, encouraging further innovation and development in this rapidly evolving field.

Basel Halak
Southampton, UK
July 2025

OceanofPDF.com

Acknowledgements

I would like to thank all of those who contributed to the emergence, creation, and correction of this book. Firstly, I gratefully acknowledge the valuable contributions from all the authors, for taking the time to share their knowledge and for being very accommodating throughout the publication process. Special thanks go to the graduate students at the University of Southampton for the many hours they have spent working in their labs to generate the experimental results. Of course, the book would not be successful without the contributions of many researchers and experts in hardware security and machine learning. Finally, I would like to thank the great team at Springer for their help and support throughout the publication process.

OceanofPDF.com

Acronyms

AC Approximate Computing

AI Artificial Intelligence

ASIC Application-Specific Integrated Circuit

AxM Approximate Multipliers

BO Buffer Occupancy

BOC Buffer Operation Counts

CNN Convolutional Neural Network

CPU Central Processing Unit

EMEI Error Matrix-Based Error Injection

FC Fully Connected

FNR False Negative Rate

FPGA Field-Programmable Gate Array

FPR False Positive Rate

GANs Generative Adversarial Networks

GCN Graph Convolutional Network

GNN Graph Neural Networks

GPU Graphics Processing Unit

HT Hardware Trojan

LLM Large Language Models

MAC Multiply-and-Accumulate

ML Machine Learning

MNoC Malicious NoC

MPSoC Multi-Processor Systems-on-Chip

MSB Most significant bit

PAC Packet Arrival Curves

PARSEC Princeton Application Repository for Shared-Memory Computers

PE Processing Element

PERCEP Perceptron

ReLU Rectified Linear Unit

RFC Random Forest Classifier

RISC Reduced Instruction Set Computer

ROI Region-of-Interest

ROM Read-Only Memory

RPV Routing-Path Victims

RTL Register-Transfer Level

SCA Side-Channel Attack

SSDM Shuffle and Substitution-Based Defense Mechanism

OceanofPDF.com

Contents

Part I CNN Design and Implementation Principles

[1 Convolutional Neural Networks: Operation Principles and Applications](#)

Haosen Yu, Peiyao Sun, Bingkai Ding and Basel Halak

[2 A Tutorial on CNN Design from Model Training to Hardware Implementation](#)

Peiyao Sun, Haosen Yu, Tom J. Kazmierski and Basel Halak

[3 Performance Optimisation of CNN Hardware](#)

Peiyao Sun, Haosen Yu, Tom J. Kazmierski and Basel Halak

Part II Applications of CNN in Hardware Security

[4 A Gem5 Tutorial on Simulating DoS Attacks in Network-on-Chip](#)

Haoyu Wang and Basel Halak

[5 A Case Study of CNN Applications in Network-on-Chip Security](#)

Haoyu Wang, Ahmad Atamli and Basel Halak

[6 Comparative Analysis on Machine Learning for DoS Detection in Network-on-Chip](#)

Haoyu Wang, Jianjie Ren, Shengkai Hu, Ahmad Atamli and Basel Halak

Part III Threat Analysis of AI Production Chain

[7 AI Production Process Security Attacks and Mitigation Techniques](#)

Haosen Yu and Basel Halak

[8 A Case Study on Shuffle and Substitution-Based Defense Mechanism](#)

Haosen Yu, Peiyao Sun and Basel Halak

[Index](#)

Editors and Contributors

Dr. Basel Halak

is an Associate Professor of Secure Electronics and the Director of the Cyber Security Academy with the University of Southampton. He is also a leading European Masters in Embedded Computing Systems (EMECS). He is a visiting scholar at the Technical University of Kaiserslautern, the Norwegian University of Science and Technology, and the Polytechnic di Torino. He previously served as a visiting professor at the Kazakh-British Technical University 2017. His expertise spans Digital Systems Design, Hardware Security, and Applied Cryptography, and he has authored over 120 refereed conference and journal papers and 7 books, including the first textbook on Physically Unclonable Functions and the first book on Hardware Supply Chain Security. Beyond academia, he has collaborated extensively with industry such as ARM, Arqit, Schneider Electric, and Ericsson. He is the recipient of the Industrial Fellowship from the Royal Academy of Engineering and the National Teaching Fellowship awarded by the Advance Higher Education(HE) Academy. He actively contributes to the global research community as a member of technical program committees for leading conferences such as HOST, IEEE DATE, IEEE DAC, IVSW, ICCCA, ICCCS, MTV, and EWME. He is an Associate Editor of *IEEE Access* and a Guest Editor of the *IET Circuits, Devices & Systems* journal. As of July 2025, he has supervised 18 PhD students and 7 postdoctoral scholars.

Contributors

Ahmad Atamli

School of Electronics and Computer Science, University of Southampton,
Southampton, UK

Bingkai Ding

School of Electronics and Computer Science, University of Southampton,
Southampton, UK

Basel Halak

School of Electronics and Computer Science, University of Southampton,
Southampton, UK

Shengkai Hu

School of Electronics and Computer Science, University of Southampton,
Southampton, UK

Tom J. Kazmierski

School of Electronics and Computer Science, University of Southampton,
Southampton, UK

Jianjie Ren

School of Electronics and Computer Science, University of Southampton,
Southampton, UK

Peiyao Sun

School of Electronics and Computer Science, University of Southampton,
Southampton, UK

Haoyu Wang

School of Electronics and Computer Science, University of Southampton,
Southampton, UK

Haosen Yu

School of Electronics and Computer Science, University of Southampton,
Southampton, UK

OceanofPDF.com

Part I

CNN Design and Implementation

Principles

OceanofPDF.com

1. Convolutional Neural Networks: Operation Principles and Applications

Haosen Yu¹✉, Peiyao Sun¹✉, Bingkai Ding¹✉ and Basel Halak¹✉

(1) School of Electronics and Computer Science, University of
Southampton, Southampton, UK

✉ Haosen Yu

Email: Haosen.Yu@soton.ac.uk

✉ Peiyao Sun

Email: ps2m24@soton.ac.uk

✉ Bingkai Ding

Email: Bingkai.Ding@soton.ac.uk

✉ Basel Halak (Corresponding author)

Email: Basel.Halak@soton.ac.uk

Keywords CNN – Neural networks – Convolutional layers – Pooling layers – Activation functions – Fully connected layers – Normalization – CNN architectures – Feature extraction – Image classification – Hardware accelerators – FPGA – ASIC – GPU limitations – Energy efficiency

1.1 Introduction

Convolutional neural networks (CNNs) [1] have emerged as a cornerstone of modern deep learning, demonstrating unparalleled success across a diverse array of applications, including computer vision, natural language processing, and autonomous systems [2]. Their hierarchical structure,

inspired by the human visual system, allows CNNs to learn spatial hierarchies of features, making them exceptionally adept at handling complex data [3]. However, the computational intensity of CNN workloads, characterized by operations such as convolutions, pooling, and matrix multiplications, presents significant challenges for traditional computing platforms [4].

While general-purpose processors such as central processing units (CPUs) and graphics processing units (GPUs) have driven early progress in deep learning, their limitations have become increasingly evident as CNN models grow in size and complexity [5]. CPUs, optimized for sequential processing, struggle to parallelize the extensive computations required by CNNs. GPUs, despite their parallel processing capabilities, face challenges related to high power consumption, latency, and inefficiencies in handling large-scale datasets. These constraints necessitate the development of hardware accelerators tailored specifically to the unique demands of CNN workloads [6].

1.2 Chapter Overview

This chapter introduces into the principles, design, and implementation of CNN hardware accelerators. It begins with an overview of CNN architectures and their key components in Sect. [1.3](#), establishing the foundation for understanding the computational requirements of these models. Next, it examines the limitations of conventional platforms, emphasizing the need for dedicated accelerators in Sects. [1.4.1](#) and [1.4.2](#). Section [1.4.3](#) then explores existing hardware solutions, including GPUs, application-specific integrated circuits (ASICs), field-programmable gate arrays (FPGAs), and hybrid architectures, comparing their performance, scalability, and energy efficiency. Section [1.4.4](#) highlights emerging trends and challenges, such as energy-efficient designs, edge AI solutions, and the Network-on-Chip (NoC) architectures in hardware design, offering insights into the future of CNN hardware acceleration. Finally, Sect. [1.5](#) provides a summary of this chapter.

1.3 Convolutional Neural Network

A convolutional neural network (CNN) is a class of deep, feed-forward artificial neural networks predominantly employed in the analysis of visual imagery [\[7\]](#). They are specifically designed to process data with a grid-like topology, such as images, represented as grids of pixels. CNNs serve as a cornerstone technology in the field of computer vision and possess robust capabilities for processing multimedia data. Their applications span diverse domains, ranging from facial recognition to medical image analysis [\[8\]](#).

CNNs offer several distinct advantages that contribute to their widespread applicability. First, parameter sharing enables feature detectors, such as edge detectors, to function effectively across an entire image rather than being restricted to specific sections, thereby reducing the model's parameters, improving computational efficiency, and minimizing overfitting. Second, CNNs exhibit translation invariance, whereby identified features can be recognized regardless of their spatial location within the image. Finally, their hierarchical learning ability allows them to tackle complex tasks, including image recognition, by progressively learning patterns at different levels of abstraction [\[3\]](#).

A typical convolutional neural network (CNN) architecture consists of several key layers that work together to extract and process features hierarchically. Convolutional layers serve as the backbone, detecting spatial features like edges and patterns. Pooling layers reduce spatial dimensions to preserve essential information while minimizing computation and overfitting. Normalization layers, such as batch normalization, improve training stability and generalization. Activation layers (e.g., ReLU, Sigmoid, Tanh) introduce nonlinearity, enabling the network to learn complex patterns [9]. Fully connected layers consolidate features for tasks like classification, and the classification layer outputs the final predictions, such as class probabilities. These components collectively allow CNNs to excel in tasks like image recognition and object detection [10].

In summary, CNNs have revolutionized the field of computer vision by providing an efficient and highly accurate framework for analyzing visual data. Their architecture, inspired by the human visual system, enables the automatic and adaptive learning of spatial hierarchies of features from input images, making them indispensable tools for numerous visual data applications.

1.3.1 Operational Principles of CNNs

Within the architecture of convolutional neural networks, image data is initially input into CNN through the input layer. Following this, the network employs a sequence of layers, including the convolutional layer, batch normalization layer, ReLU layer, and pooling layer, to systematically extract and learn features from the input images [9].

After features are extracted, the final layers (often fully connected layers) classify the image based on the detected features. Once the network produces an output, a loss function measures the difference between the predicted output and the actual labels. Common loss functions include cross-entropy for classification tasks and mean squared error (MSE) or mean absolute error (MAE) for regression tasks [11].

Then, perform backpropagation and optimization. During this process, training begins with the computation of gradients, utilizing the chain rule to ascertain the gradient of the loss with respect to the network's weights. This is followed by a weight adjustment phase, where optimization algorithms, such as stochastic gradient descent (SGD) or its advanced counterparts like Adam and RMSprop, refine the weights to minimize the loss. This intricate

dance of forward (from input to loss determination) and backward (involving backpropagation and optimization) passes is iteratively executed over several epochs, continuing until the network reaches an optimal or satisfactory performance threshold [12].

CNNs are trained using backpropagation with gradient descent optimization. The goal is to adjust the weights to minimize the difference between the predicted output and the actual labels, typically represented by a loss function. Regularization techniques, such as dropout and weight decay, are often employed to prevent overfitting [13].

1.3.2 Input Layer

The input layer in a convolutional neural network (CNN) serves as the initial stage where raw data is introduced into the model for subsequent processing. Typically, the input layer accepts multidimensional data, such as images represented by height, width, and channel dimensions. This layer plays a critical role in defining the structure of the data pipeline, as it must accommodate the specific dimensions and format of the dataset under consideration. For instance, an input layer for grayscale images would require a single channel, whereas for RGB images, it would require three channels corresponding to red, green, and blue. The input layer neither performs computations nor alters the data but serves as a conduit to prepare and structure the input for downstream convolutional layers, ensuring compatibility and facilitating efficient feature extraction in subsequent stages [9].

1.3.3 Convolutional Layer

Convolutional layer is the core building block of a CNN. The layer's primary function is to detect local patterns using learned kernels (or filters). When processing an image, these kernels slide (or convolve) around the input data (such as an image) to produce a feature map, effectively transforming the input data into a form that makes it easier for the network to understand. The result of this convolution operation is a set of feature maps that give the spatial arrangement for the detected feature at different locations in the input [14].

1.3.3.1 Core Concepts of Convolutional Layer

Kernels/Filters Kernels (also called filters) are small, trainable matrices that slide across the input data to extract features such as edges, textures, or more abstract patterns. Each kernel is applied to the input data via convolution, producing a feature map as output. For example, a 3×3 kernel slides over an image, performing element-wise multiplication and summing the results.

Stride Stride determines how much the kernel shifts at each step during the convolution operation. Larger strides reduce the size of the output feature map but may lose fine-grained details. Typical strides: $S = 1$ (no skip) or $S = 2$ (downsampling).

Padding Padding is a technique used in convolutional operations to add extra rows and columns around the edges of the input tensor, enabling the preservation or modification of spatial dimensions after the operation. Common types of padding include the following:

- **Valid Padding (no padding):** In this approach, no additional rows or columns are added, resulting in an output tensor smaller than the input tensor due to the reduction caused by the kernel size.
- **Same Padding:** Here, padding is added such that the spatial dimensions of the output tensor match those of the input tensor. This is particularly useful for maintaining consistency in the feature map size across layers.
- **Full Padding:** This type extends the input tensor by padding to ensure that every element of the input is fully convolved, leading to an output tensor larger than the input tensor.

Formula for output size with stride S and padding P :

$$\text{Output size} = \frac{(N+2P-F)}{S} + 1 \quad (1.1)$$

Where N = input size and F = filter size.

Padding choices significantly impact the receptive field, computational efficiency, and the overall architecture of convolutional neural networks.

1.3.3.2 Mathematical Perspective

The convolution operation, a fundamental process in deep learning, involves applying a kernel K to an input tensor X to produce an output tensor Y . Mathematically, this is expressed as

$$Y[i, j] = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} X[i + m, j + n] \cdot K[m, n] \quad (1.2)$$

where the kernel slides over the input tensor, performing element-wise multiplications and summing the results.

Additionally, convolutional layers benefit from weight sharing, wherein a single set of kernel parameters is applied across all spatial locations of the input tensor. This parameter-sharing mechanism significantly reduces the number of trainable parameters, thereby improving the model's computational efficiency and mitigating the risk of overfitting. A sample diagram about convolutional layer is shown in Fig. [1.1](#).

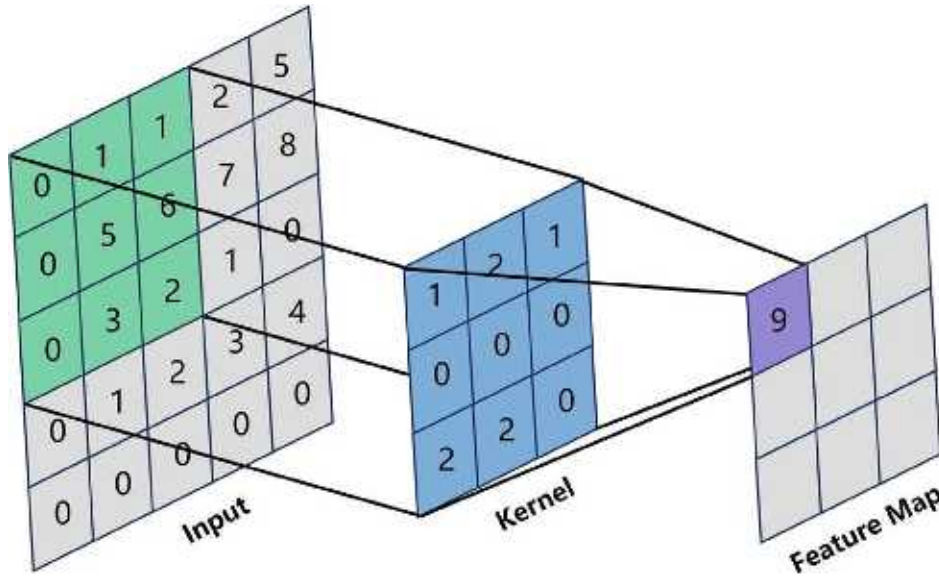


Fig. 1.1 Convolutional layer

1.3.3.3 Feature Maps and Receptive Fields

Feature maps are the outputs generated by applying a kernel to an input tensor, effectively highlighting regions where the kernel's learned pattern is most prominent. These maps serve as representations of specific features within the input data, facilitating the detection of patterns such as edges, textures, or more complex structures [\[9\]](#).

Each value in a feature map corresponds to a receptive field, which represents the specific region of the input image that influences that value. As the network depth increases, the receptive fields expand, allowing deeper layers to capture high-level, abstract features by aggregating information from larger regions of the input. This hierarchical

representation is essential for learning complex patterns and achieving strong performance in tasks such as object recognition and semantic segmentation [15].

1.3.3.4 Types of Convolutions

Convolutions are versatile operations in deep learning, tailored to different types of data and tasks.

1D convolutions are used for sequential data such as time series, signals, or text, with applications in tasks like speech recognition [16].

2D convolutions, the most common type, are applied to image data, processing 2D grayscale or RGB images to extract spatial features. For processing 3D data, such as volumetric scans (e.g., CT or MRI) or video frames [17].

3D convolutions introduce depth as a third dimension, capturing spatial-temporal information [18].

Dilated (atrous) convolutions expand the receptive field by introducing gaps between kernel elements, enabling models to capture broader contextual information without increasing computational parameters, which is particularly useful in semantic segmentation [19].

Lastly, depthwise separable convolutions decompose standard convolution into two steps: depthwise convolution, which applies a separate kernel to each input channel, and pointwise convolution, which uses a 1×1 kernel to combine channel outputs. This approach significantly reduces computational cost while maintaining accuracy, making it a key component in efficient architectures like MobileNet [20].

1.3.4 Pooling Layer

Pooling layers are an essential component of convolutional neural networks (CNNs) used to reduce the spatial dimensions of feature maps. This chapter provides a detailed understanding of their purpose, types, and mathematical concepts.

1.3.4.1 Purpose of Pooling Layers

Pooling layers serve several critical purposes in convolutional neural networks (CNNs). First, they facilitate dimensionality reduction by reducing the size of feature maps, thereby decreasing computational costs and memory requirements in subsequent layers. This ensures the network

remains efficient, even with large input images. Second, pooling enhances translation invariance by summarizing features within a region, making the model robust to small shifts or distortions in the input. Lastly, pooling contributes to preventing overfitting by reducing spatial dimensions, acting as a form of implicit regularization that improves generalization [9].

1.3.4.2 Mathematical Perspective

Let $x^l \in \mathbb{R}^{H^l \times W^l \times D^l}$ be the input to the l -th layer, which is now a pooling layer. The pooling operation requires no parameter (i.e., w^i is null; hence, parameter learning is not needed for this layer). The spatial extent of the pooling ($H \times W$) is specified in the design of the CNN structure. Assume that H divides H^l and W divides W^l and the stride equals the pooling spatial extent, the output of pooling will be an order three tensor of size $H^{l+1} \times W^{l+1} \times D^{l+1}$, with

$$H^{l+1} = \frac{H^l}{H}, \quad W^{l+1} = \frac{W^l}{W}, \quad D^{l+1} = D^l \quad (1.3)$$

This formula ensures precise control over the spatial dimensions of the feature maps after pooling, which is essential for designing and understanding CNN architectures. A sample diagram about pooling layer is shown in Fig. 1.2.

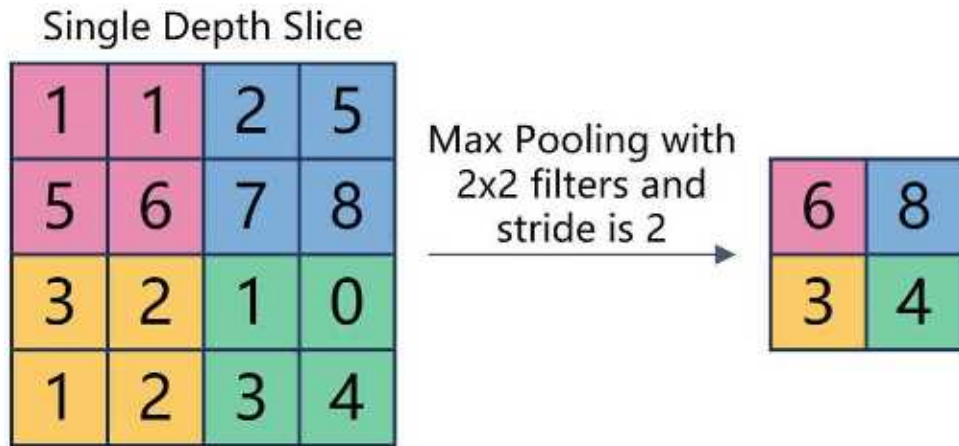


Fig. 1.2 Max pooling layer

1.3.4.3 Padding and Stride in Pooling

Pooling layers often use stride and padding to control the operation's behavior and output dimensions.

The stride specifies the step size of the pooling window as it moves across the feature map. A stride of 2, for example, reduces the spatial dimensions by approximately half, enabling efficient dimensionality reduction while summarizing features.

Padding adds extra rows and columns around the edges of the feature map, similar to its role in convolution. This ensures that the pooling operation includes the boundary regions of the feature map, preserving spatial alignment and avoiding loss of information at the edges, especially in cases where maintaining the feature map's structure is crucial [9].

1.3.4.4 Types of Pooling

Pooling operations in convolutional neural networks (CNNs) come in various forms, each designed to summarize feature maps in different ways while reducing dimensions.

1. **Max Pooling:** This method retains the maximum value from a pooling region, effectively highlighting the most prominent feature. For a pooling window of size 2×2 , the operation $\begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix} \rightarrow 4$ demonstrates the selection of the maximum value. It is defined as Eq. 1.4:

$$Y[i, j] = \max (X[i : i + F, j : j + F]), \quad (1.4)$$

where F is the window size.

Max pooling takes the maximum value from each $F \times F$ window of the input and assigns it to the output position (i, j) .

2. **Average Pooling:** In contrast, this method computes the average of values within the pooling region, capturing the general trend of features. Using the same 2×2 window, the operation $\begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix} \rightarrow 2.5$ is defined as Eq. 1.5:

$$Y[i, j] = \frac{1}{F^2} \sum_{k=0}^{F-1} \sum_{l=0}^{F-1} X[i + k, j + l] \quad (1.5)$$

This formula defines average pooling in a neural network. Here, X is the input feature map, and $Y[i, j]$ is the output at position (i, j) . A square window of size $F \times F$ slides over the input, and the values within the window are averaged. The result is assigned to the corresponding position in the output Y .

3. **Global Pooling:** Global pooling reduces each feature map to a single value, either by taking the maximum (global max pooling) or the average (global average pooling) over all spatial dimensions. This approach is commonly used in architectures like ResNet to replace fully connected layers, simplifying the model while maintaining performance [21].
4. **Adaptive Pooling:** Adaptive pooling dynamically adjusts the output size to a fixed dimension, regardless of input size, making it particularly useful for handling variable-sized inputs.
For example, in PyTorch, `nn.AdaptiveAvgPool2d((5, 5))` produces a 5×5 output from any input dimension [21].

These pooling techniques collectively contribute to efficient dimensionality reduction, feature extraction, and model adaptability.

1.3.4.5 Advanced Pooling Techniques

Advanced pooling methods extend the capabilities of standard pooling to address specific challenges in computer vision tasks [21]:

- **Region of Interest (ROI) Pooling:** This technique is commonly used in object detection models like Faster R-CNN to dynamically pool features from regions corresponding to detected objects, enabling precise localization and classification.
- **Feature Pyramid Pooling:** Employed in networks such as PSPNet, this method extracts multi-scale features by pooling at different scales, enhancing the model's ability to recognize objects of varying sizes and improving performance in tasks like semantic segmentation.
- **Fractional Pooling:** Designed to handle non-integer downsampling ratios, fractional pooling interpolates between neighboring values, allowing finer control over output dimensions and adapting to diverse input sizes more flexibly.

These techniques enhance pooling's adaptability, improving the performance of CNNs in complex tasks like object detection, segmentation, and multi-scale feature extraction.

1.3.5 Normalization Layer

Normalization layers are essential for stabilizing and accelerating the training of deep neural networks. This chapter provides a detailed explanation of normalization, the types of normalization layers, and their mathematical foundation.

1.3.5.1 Purpose of Normalization Layers

Normalization in neural networks involves adjusting the distribution of data, such as activations or inputs, to improve training and performance. It ensures stabilized learning by maintaining consistent distributions across layers, allowing gradients to flow predictably and reducing the risk of issues like exploding or vanishing gradients. Additionally, normalization facilitates faster convergence by minimizing internal covariate shift, which leads to smoother and more efficient optimization. Finally, it enhances generalization, enabling the model to perform better on unseen data by encouraging robust and consistent learning of features [15].

1.3.5.2 Types of Normalization

1. **Batch Normalization (BN):** Batch normalization normalizes activations within a mini-batch by adjusting their distribution to have a mean of zero and a variance of one. Specifically, it computes the batch mean

$$\mu = \frac{1}{m} \sum_{i=1}^m x_i \quad (1.6)$$

and variance

$$\sigma^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu)^2, \quad (1.7)$$

where m is the mini-batch size. Each activation is then normalized as

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}, \quad (1.8)$$

with a small constant ϵ added for numerical stability. Finally, learnable parameters γ (scale) and β (shift) are applied to allow the network to recover useful representations:

$$y_i = \gamma \hat{x}_i + \beta. \quad (1.9)$$

Batch normalization reduces internal covariate shift, accelerates convergence, and provides implicit regularization. However, it may

perform poorly with very small batch sizes and introduces computational overhead during training.

A sample diagram about batch normalization layer is shown in Fig. 1.3.

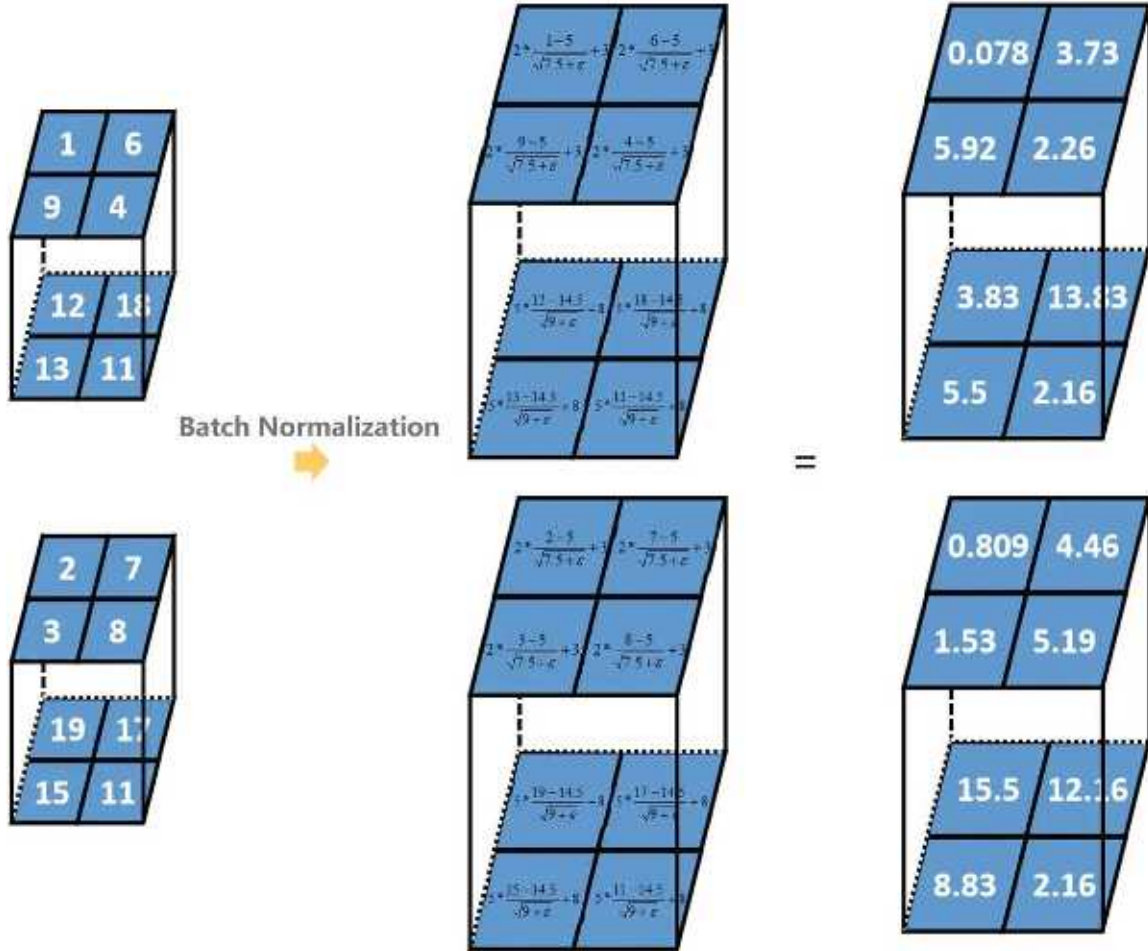


Fig. 1.3 Batch normalization layer

- Layer Normalization (LN):** Layer normalization normalizes activations across all features of a single input instance, making it independent of batch size. For an input with H features, the mean and variance are computed as

$$\mu = \frac{1}{H} \sum_{j=1}^H x_j \quad (1.10)$$

and

$$\sigma^2 = \frac{1}{H} \sum_{j=1}^H (x_j - \mu)^2. \quad (1.11)$$

Each feature is normalized using

$$\hat{x}_j = \frac{x_j - \mu}{\sqrt{\sigma^2 + \epsilon}}, \quad (1.12)$$

followed by scaling and shifting with learnable parameters $y_j = \gamma \hat{x}_j + \beta$. This method works well for tasks like NLP and reinforcement learning, where batch dependencies are less relevant. Layer normalization is particularly useful in transformer architectures and recurrent neural networks, offering stable learning regardless of batch size.

3. **Instance Normalization (IN):** Instance normalization focuses on normalizing the spatial dimensions of each channel independently, making it effective for style transfer and image generation tasks [22]. For a feature map of height H and width W , the mean and variance for each channel c are calculated as

$$\mu_c = \frac{1}{HW} \sum_{i=1}^H \sum_{j=1}^W x_{i,j,c} \quad (1.13)$$

and

$$\sigma_c^2 = \frac{1}{HW} \sum_{i=1}^H \sum_{j=1}^W (x_{i,j,c} - \mu_c)^2 \quad (1.14)$$

The spatial dimensions are then normalized using

$$\hat{x}_{i,j,c} = \frac{x_{i,j,c} - \mu_c}{\sqrt{\sigma_c^2 + \epsilon}} \quad (1.15)$$

followed by scaling and shifting:

$$y_{i,j,c} = \gamma \hat{x}_{i,j,c} + \beta \quad (1.16)$$

Instance normalization normalizes each feature map independently, removing instance-specific variations while preserving structural features. This makes it especially effective for tasks involving stylized outputs [22].

1.3.5.3 Advanced Normalization Techniques

In addressing the inherent limitations of conventional normalization strategies, advanced normalization techniques and their alternatives have emerged, presenting novel approaches to significantly enhance neural network performance. Batch renormalization stands as a notable extension of batch normalization (BN), specifically engineered to mitigate the instability encountered with smaller batch sizes. This method introduces

adaptive scaling and shifting parameters, which serve to stabilize training by ensuring consistent statistical properties across a wider range of batch sizes, thereby improving the reliability and generalization of models trained under such conditions [23]. Furthermore, switchable normalization represents a dynamic approach wherein the network learns to optimally select from a repertoire of normalization techniques, including BN, layer normalization (LN), and instance normalization (IN), on a per-layer basis during training. This adaptive mechanism allows for a tailored normalization strategy, optimizing performance across diverse tasks and architectural configurations [24]. Concurrently, a burgeoning area of research explores the feasibility of normalization-free networks, which entirely circumvent the need for explicit normalization layers. A prominent example is FixUp initialization, which employs specialized parameter initialization schemes and concomitant training adjustments. This methodology enables the effective training of deep neural networks without recourse to normalization, consequently reducing computational overhead and simplifying the overall model architecture [25].

1.3.6 Activation Layer

Activation layers are a critical component of neural networks, introducing nonlinearity to the model, enabling it to learn complex patterns, and transforming raw inputs into useful features. This chapter explores the types of activation functions, their mathematical formulations, characteristics, and practical use cases.

1.3.6.1 Purpose of Activation Layer

The activation layer serves critical purposes in a neural network. It introduces nonlinearity, allowing the network to model complex, nonlinear relationships in data; without it, even a deep network would behave like a linear model. Activation functions also perform feature transformation, converting raw weighted sums into meaningful outputs for subsequent layers to process. Additionally, they influence gradient flow during backpropagation, playing a vital role in convergence and stability by ensuring gradients are neither too large nor too small. These functions make activation layers essential for effective learning and performance [9].

1.3.6.2 Types of Activation Functions

1. **Sigmoid Function:** The sigmoid function, defined as

$$f(x) = \frac{1}{1+e^{-x}} \quad (1.17)$$

which maps inputs to the range $(0, 1)$. It is often used in binary classification tasks as it provides a probability-like output. Sigmoid offers smooth gradients and bounded outputs, which are useful for probabilistic interpretations. However, it suffers from the vanishing gradient problem for large $|x|$, causing slow learning. It is typically used in output layers of binary classifiers [26].

2. **Hyperbolic Tangent (Tanh):** The Tanh function, defined as

$$f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (1.18)$$

which maps inputs to the range $(-1, 1)$. Unlike sigmoid, it is zero centered, which aids in faster optimization. Tanh has strong gradients for values near zero but also suffers from the vanishing gradient problem for large $|x|$. It is commonly used in hidden layers of traditional RNNs [27].

3. **Rectified Linear Unit (ReLU):** ReLU, defined as

$$f(x) = \max(0, x) \quad (1.19)$$

It is the most widely used activation function due to its simplicity and effectiveness. It preserves positive values while setting negative values to zero, helping mitigate the vanishing gradient problem. However, it may encounter the “dying ReLU” problem, where neurons become inactive with zero gradients. ReLU is extensively used in CNNs and DNNs [28].

4. **Leaky ReLU:** Leaky ReLU addresses the dying ReLU problem by allowing a small gradient for negative inputs, defined as

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases} \quad (1.20)$$

Where α is a small positive constant (e.g., 0.01). This retains computational simplicity while ensuring neurons remain active. It is a useful alternative to ReLU when dying neurons are a concern [29].

5. **Parametric ReLU (PReLU):** PReLU extends Leaky ReLU by making the slope α a learnable parameter during training. This flexibility allows the network to adaptively adjust the negative slope for improved

performance. It is often used in deeper architectures where a more adaptable activation is beneficial.

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases} \quad (1.21)$$

Where α is learned during training.

6. **Exponential Linear Unit (ELU)**: ELU, defined as

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha(e^x - 1) & \text{if } x \leq 0 \end{cases} \quad (1.22)$$

Where $\alpha > 0$.

It introduces smooth transitions for negative values. It mitigates the vanishing gradient problem and improves gradient flow, particularly in deep networks. However, it has slightly higher computational costs and is used where faster convergence is desired [30].

7. **Swish**: Swish, defined as

$$f(x) = x \cdot \sigma(x) = x \cdot \frac{1}{1 + e^{-x}} \quad (1.23)$$

It is a smooth, non-monotonic activation function that often outperforms ReLU in deep architectures like EfficientNet. It enhances gradient flow but comes with a slightly higher computational cost. Swish is commonly used in modern CNNs and transformer-based models [31].

1.3.6.3 Comparison of Activation Functions

Choosing an activation function depends on the specific requirements of the task and architecture. ReLU (rectified linear unit) is the most commonly used activation function due to its simplicity, efficiency, and ability to mitigate the vanishing gradient problem, making it a default choice for many networks like CNNs and DNNs. Variants of ReLU, such as Leaky ReLU and Parametric ReLU, address issues like the “dying ReLU” problem. For outputs requiring bounded ranges, functions like Tanh or sigmoid are more appropriate. In modern architectures, Swish has shown improved performance in certain scenarios, especially when its higher computational cost is acceptable [32]. Ultimately, selecting the best activation function often relies on experimentation, as empirical results can vary based on the dataset and application (Table 1.1).

Table 1.1 Comparison of activation functions

Function	Range	Gradient	Advantages	Disadvantages	Use cases
Sigmoid	$(0, 1)$	Small for extremes	Smooth, probabilistic output	Vanishing gradients	Output layer for binary tasks
Tanh	$(-1, 1)$	Small for extremes	Zero centered	Vanishing gradients	RNNs
ReLU	$[0, \infty)$	Constant (1 for $x > 0$)	Sparse, efficient	Dying ReLU problem	CNNs, general DNNs
Leaky ReLU	$(-\infty, \infty)$	Constant	Mitigates dying ReLU problem	Requires tuning of α	Variants of CNNs, DNNs
ELU	$(-\alpha, \infty)$	Smooth for $x < 0$	Faster convergence	Slightly computationally expensive	Deep networks
Swish	$(-\infty, \infty)$	Smooth, adaptive	State-of-the-art performance	Higher computational cost	Modern architectures (e.g., EfficientNet)

1.3.7 Fully Connected Layer

Fully connected (FC) layers are fundamental components of neural networks, especially in traditional architectures. They connect every input neuron to every output neuron, enabling the network to learn global patterns and make predictions.

1.3.7.1 Purpose of Fully Connected Layer

A fully connected (FC) layer is a dense neural layer where every neuron is connected to all neurons in the previous layer, playing a critical role in aggregating and utilizing learned features. FC layers combine spatially distributed features extracted by convolutional layers into a compact representation, enabling high-level decision-making tasks such as classification or regression. They act as a transition point by flattening feature maps into a 1D vector before further processing. FC layers provide a global representation by integrating information from all preceding neurons, making them essential for tasks requiring comprehensive feature aggregation. However, they are computationally expensive due to the large number of parameters, proportional to the product of the input size and the number of neurons, which increases the risk of overfitting. Additionally, FC

layers are vital for dimensionality transformation, reshaping data to suit downstream tasks effectively [33].

1.3.7.2 *Mathematical Perspective*

The output of a fully connected (FC) layer is computed as

$$y = f(Wx + b) \quad (1.24)$$

where W is the weight matrix of size $[\text{output_neurons} \times \text{input_neurons}]$, x is the input vector, b is the bias vector, and f is the activation function.

For an input vector x of size N and a fully connected (FC) layer with M neurons, the layer's functionality is defined by its weights and biases, output computation, and parameter count. The weight matrix W has dimensions $M \times N$, representing the connections between each input and neuron, while the bias vector b has size M , adding a trainable offset to each neuron. The output is computed in two steps: first, a linear transformation is applied, $z = Wx + b$, which aggregates the input features weighted by the learned parameters. Then, the activation function f is applied to the result, yielding the final output $y = f(z)$.

The total number of parameters in the FC layer, which includes $N \times M$ weights and M biases, is calculated as $\text{parameters} = (N \times M) + M$. For example, with an input vector of size 128 and an FC layer comprising 64 neurons, the parameter count would be $(128 \times 64) + 64 = 8256$. This high parameter count highlights the computational expense and potential overfitting risk associated with fully connected layers, making them a critical yet resource-intensive component of neural networks.

A sample diagram about fully connected layer is shown in Fig. 1.4.

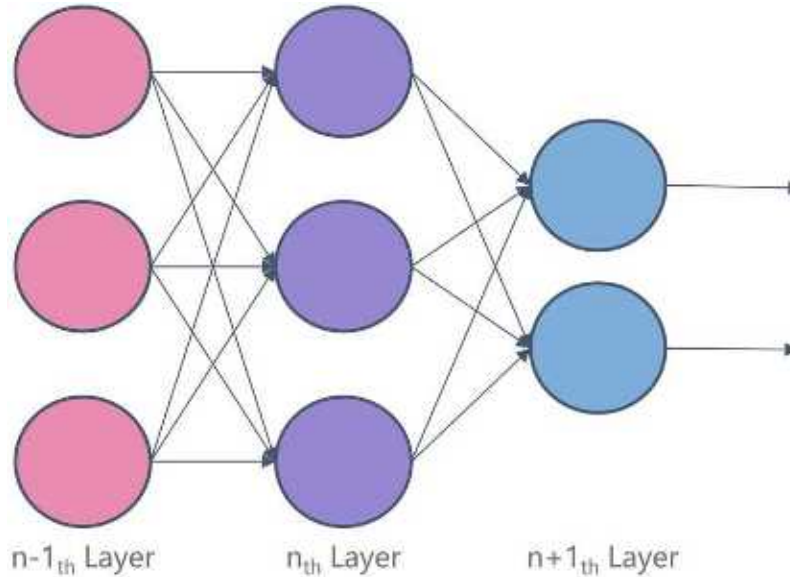


Fig. 1.4 Fully connected layer

Fully connected layers play a crucial role in aggregating features and making predictions. Despite their simplicity, they are computationally intensive and prone to overfitting. Modern architectures often replace or minimize their use with techniques like global pooling, attention mechanisms, or sparse connections.

1.3.8 Classification Layer

In the end, a CNN often has a classification layer to make a prediction. If, for instance, the task is to classify an image among ten categories, this layer would output ten values, each one representing the probability of the image belonging to that class. In convolutional neural networks (CNNs), fully connected (FC) layers serve as a crucial bridge between the feature extraction process and high-level decision-making tasks such as classification or regression. The transition to FC layers involves flattening the multidimensional feature maps produced by convolutional layers into a one-dimensional vector. For example, a $7 \times 7 \times 64$ feature map is reshaped into a 3136-dimensional vector, enabling the network to process spatially distributed features in a compact form. This step consolidates all the learned spatial features, preparing them for classification or other downstream tasks [9].

FC layers are frequently employed as the final layers in CNNs, particularly in image classification tasks, where they act as output layers. In these scenarios, the final FC layer typically consists of C neurons, where C

represents the number of target classes. A softmax activation function is then applied to this layer to produce class probabilities, ensuring the outputs are interpretable and suitable for classification. By integrating spatially learned features from convolutional layers into a global representation, FC layers enable CNNs to make accurate predictions, highlighting their essential role in combining feature extraction and task-specific decision-making [15].

1.3.9 Typical Model Structure of CNNs

The fundamental principle underpinning convolutional neural networks (CNNs) lies in their utilization of shared local receptive fields and weights. This core concept enables efficient feature extraction from spatially correlated data. The seminal work in this field, LeNet, introduced by LeCun et al. in 1998, is widely recognized as the archetype of CNN architectures [34].

Over the past decade, convolutional neural networks (CNNs) have undergone significant evolution, driven by the need for improved performance, scalability, and efficiency in computer vision tasks. Each advancement in CNN architecture has introduced novel design principles, addressing critical challenges such as vanishing gradients, computational cost, and overfitting. Starting with AlexNet, which demonstrated the power of deep learning on large-scale image datasets, modern architectures like ResNet, DenseNet, and EfficientNet have pushed the boundaries of depth, parameter efficiency, and computational optimization. This section explores the innovations and design philosophies of key CNN architectures, highlighting their contributions to the field and their practical applications in solving complex visual tasks [35].

1. LeNet (1998):

The original LeNet model featured a straightforward architecture comprising five convolutional layers directly linked to corresponding pooling layers. Its initial application focused on the recognition of handwritten and printed characters, demonstrating remarkable performance for its time. It is worth noting that contemporary implementations of LeNet often incorporate modifications from the original design to suit a broader range of applications [34].

2. AlexNet (2012):

AlexNet marked a milestone in deep learning by winning the 2012 ImageNet competition, demonstrating the potential of deep CNNs for image classification. It introduced ReLU activation to accelerate training, dropout to reduce overfitting, and leveraged GPUs for efficient computation. The architecture consists of five convolutional layers and three fully connected layers, using large filters such as 11×11 and 5×5 . Despite its breakthrough, AlexNet's large parameter count (~ 60 M) and computational requirements were significant drawbacks [36].

3. VGG (2014):

The VGG network demonstrated that deeper networks with smaller filters (3×3) could significantly improve performance. VGG-16 and VGG-19 are the most common configurations, comprising 13–16 convolutional layers followed by three fully connected layers. Max pooling is used for spatial downsampling. While VGG is simple and effective, its parameter count is exceedingly high (e.g., 138 M for VGG-16), making it resource-intensive in terms of computation and memory [37].

4. Inception (GoogLeNet, 2014):

Inception networks introduced inception modules, which apply convolutions of varying kernel sizes in parallel to capture multi-scale features. The architecture reduces computation through 1×1 convolutions for dimensionality reduction. Auxiliary classifiers improve gradient flow in deeper networks. Despite its efficiency and strong performance, the complexity of its design poses implementation challenges [38].

5. ResNet (2015):

ResNet introduced residual connections (skip connections) to address the vanishing gradient problem, enabling the training of extremely deep networks, such as ResNet-50 and ResNet-152. Residual connections allow gradients to flow directly through layers by learning residual mappings ($F(x) + x$), significantly improving training dynamics and performance. However, the increased depth of ResNet comes at the cost of high computational complexity [39].

6. MobileNet (2017):

MobileNet was designed for mobile and embedded systems, introducing depthwise separable convolutions to reduce computation. Depthwise convolution applies a single filter per input channel, followed by pointwise convolution to combine channels. This lightweight design significantly reduces model size and complexity, albeit with a trade-off in accuracy for highly constrained models. Variants like MobileNetV2 and V3 further enhance performance and efficiency [40].

The structure of MobileNet V2 is shown in Fig. 1.5.

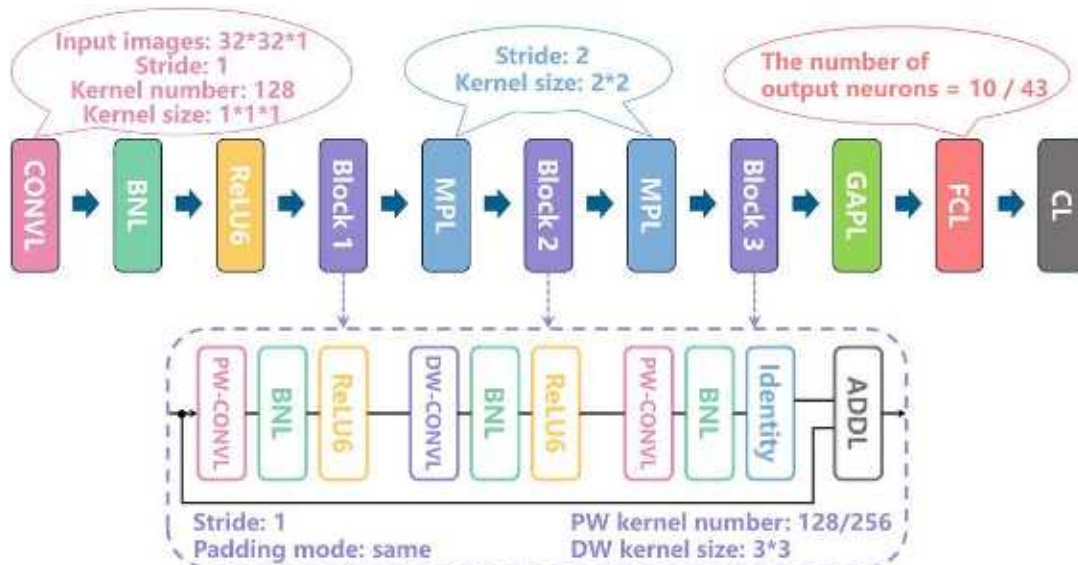


Fig. 1.5 Structure of MobileNetV2

7. ShuffleNet (2017):

ShuffleNet targeted efficient computation on mobile devices by introducing grouped convolutions and channel shuffle mechanisms. Grouped convolutions divide channels into subsets for parallel computation, while channel shuffle ensures information exchange across groups. This design achieves a balance between accuracy and computational efficiency, though its implementation is more complex compared to simpler architectures [41].

The structure of ShuffleNet V2 is shown in Fig. 1.6.

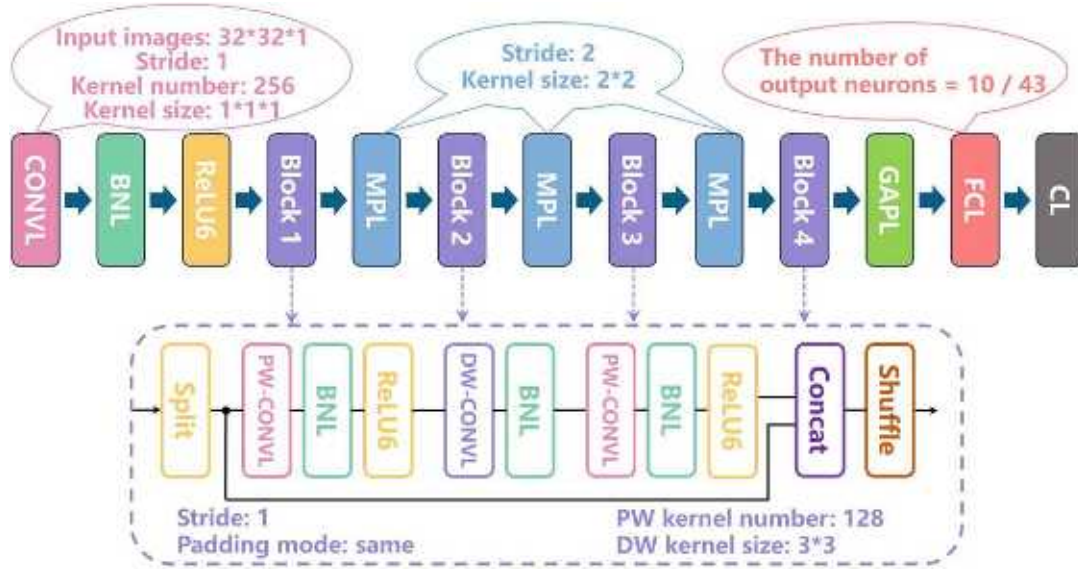


Fig. 1.6 Structure of ShuffleNetV2

8. GhostNet (2020):

GhostNet optimized lightweight CNNs by introducing ghost modules, which generate additional feature maps through inexpensive linear transformations instead of full convolutions. This reduces computational cost and parameter count while maintaining competitive accuracy. GhostNet is highly effective for resource-constrained devices, though extreme compression may lead to slight performance degradation [42].

The structure of GhostNet is shown in Fig. 1.7.

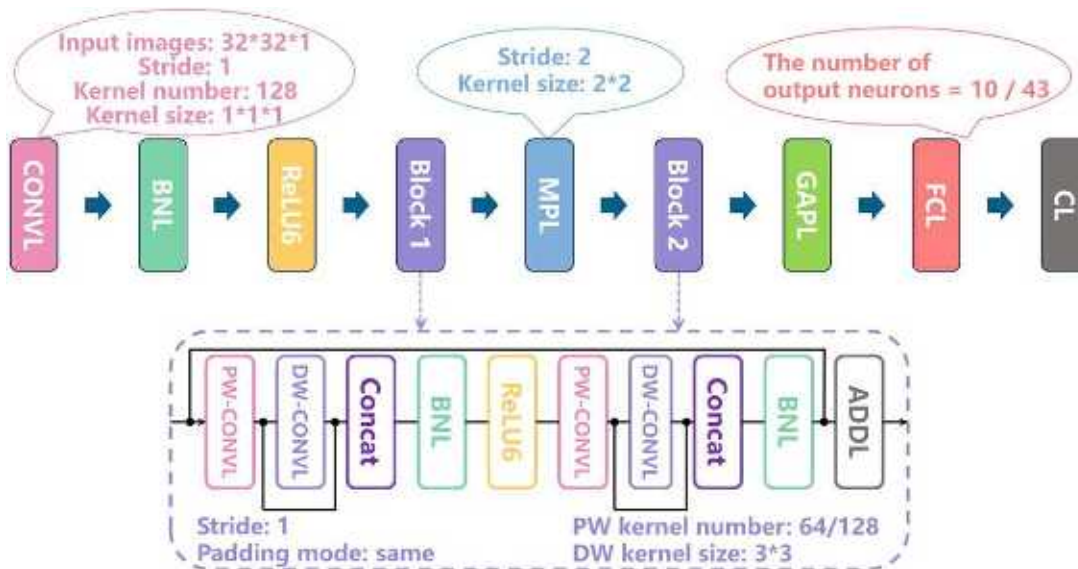


Fig. 1.7 Structure of GhostNet

1.3.10 Innovative Techniques in CNN Architectures

Advancements in convolutional neural network (CNN) architectures have been propelled by innovative techniques that address challenges like vanishing gradients, parameter efficiency, computational cost, and feature representation. This chapter explores these key innovations, detailing their mechanics, mathematical foundations, and contributions to modern deep learning.

1.3.10.1 Residual Connections

Residual connections, first introduced by ResNet, are a key innovation in deep learning that simplify optimization and enable the training of very deep networks [43]. Instead of learning direct mappings, residual connections allow networks to learn residual mappings, where the output is the sum of the input and a learned function. Mathematically, in a residual block, the output is expressed as

$$y = F(x) + x \quad (1.25)$$

Here, $F(x)$ is the residual mapping, and x represents the input to the block. This formulation ensures that the identity mapping is preserved, improving gradient flow during backpropagation and addressing the vanishing gradient problem common in deep networks.

The advantages of residual connections include the ability to train very deep architectures, such as ResNet-152, without degradation in accuracy. They also facilitate stable gradient propagation, allowing the model to achieve better optimization and performance with increasing depth. Residual connections are widely applied in state-of-the-art models, including ResNet, Inception-ResNet, and transformer-based architectures, demonstrating their versatility and importance in modern deep learning [43].

1.3.10.2 Dense Connections

Dense connections, introduced by DenseNet, connect each layer in a dense block to all its subsequent layers, ensuring that features from earlier layers are reused throughout the network. This design promotes feature reuse,

encourages efficient gradient flow, and reduces redundant computation by directly leveraging previously learned features [44].

Mathematically, the output of the l -th layer in a dense block is given by

$$x_l = H_l([x_0, x_1, \dots, x_{l-1}]) \quad (1.26)$$

Here, $[x_0, x_1, \dots, x_{l-1}]$ denotes the concatenation of the feature maps from all preceding layers, and H_l represents the transformation function (e.g., convolution, batch normalization, activation). This formulation ensures that each layer has direct access to all preceding feature maps, leading to improved feature propagation and gradient flow.

Dense connections offer several advantages. They reduce parameter redundancy by reusing features, making DenseNet more parameter efficient compared to traditional architectures with feed-forward connections. Additionally, they enhance gradient propagation, which facilitates better learning in deeper networks.

Dense connections are widely applied in DenseNet for image classification and are particularly effective in tasks that benefit from feature reuse, such as segmentation, where multi-scale and detailed features are crucial. This approach exemplifies how densely connected architectures can achieve high performance with fewer parameters compared to conventional deep networks [44].

1.3.10.3 Attention Mechanisms

Attention mechanisms dynamically focus on the most relevant parts of an input, enhancing feature extraction and representation [45]. They play a vital role in both convolutional neural networks (CNNs) and transformer architectures by selectively emphasizing important spatial regions, feature channels, or long-range dependencies. Key variants include spatial attention, which highlights significant regions in the input for tasks like object detection, channel attention, which amplifies critical feature channels, and self-attention, which computes relationships between all input elements, capturing global dependencies commonly used in transformers [46].

Notable implementations include Squeeze-and-Excitation (SE) blocks, which recalibrate channel-wise feature representations. SE blocks use a global pooled feature map to generate attention weights through the formula $s = \sigma(W_2 \delta(W_1 z))$, where W_1 and W_2 are learnable weights, δ is a ReLU activation, and σ is sigmoid. These blocks are integral to models like

EfficientNet. Nonlocal networks extend this idea by computing relationships across all positions in the input, capturing long-range dependencies. Attention mechanisms are widely applied in models like EfficientNet (SE blocks), Vision Transformers (ViT) (self-attention), and YOLOv5 (spatial and channel attention), showcasing their adaptability and effectiveness in modern deep learning [45].

1.3.10.4 Depthwise Separable Convolutions

Depthwise separable convolutions are an efficient variant of standard convolutions, designed to reduce computational complexity while maintaining accuracy. They decompose the standard convolution operation into two distinct steps: depthwise convolution, which applies a separate spatial filter to each input channel, and pointwise convolution, which combines the outputs of depthwise convolution using 1×1 convolutions. For an input with C_{in} channels, this approach performs spatial filtering and channel combination separately, significantly reducing the number of computations compared to standard convolutions [47].

The primary advantages of depthwise separable convolutions include reduced computational cost and memory usage, making them particularly effective for resource-constrained environments. Additionally, they preserve spatial and channel information effectively, ensuring minimal trade-offs in performance. These convolutions are widely adopted in efficient network architectures like MobileNet, Xception, and EfficientNet, where optimizing for speed and resource efficiency is critical. Their ability to balance accuracy and efficiency has made them a cornerstone in modern lightweight and scalable deep learning models [47].

Innovative techniques in CNNs have transformed the way neural networks process and represent data. From residual and dense connections that improve gradient flow to attention mechanisms and efficient convolutions that optimize computational costs, these advancements have enabled deeper, faster, and more accurate models. Each technique has found its niche in modern architectures, forming the backbone of state-of-the-art models used across industries [3].

1.3.11 Applications of CNNs

Convolutional neural networks (CNNs) have revolutionized numerous fields by offering unparalleled performance in processing visual and spatial

data. Their ability to learn hierarchical features and extract intricate patterns has enabled groundbreaking applications across industries. This chapter explores key real-world applications and case studies, demonstrating the transformative impact of CNNs.

1.3.11.1 Real-World Applications and Case Studies

Image classification is a fundamental task in computer vision where convolutional neural networks (CNNs) assign labels to images based on learned features. By extracting hierarchical patterns from raw image data, CNNs enable accurate categorization and identification across a wide range of applications. In medical imaging, CNNs classify X-rays, MRIs, and CT scans to assist in diagnosing diseases such as pneumonia or detecting tumors. For autonomous vehicles, they identify road signs, pedestrians, and objects to support safe navigation. In retail and e-commerce, CNNs categorize products based on visual attributes, enhancing search and recommendation systems [3]. A notable advancement in image classification is the introduction of ResNet in 2015, which set new benchmarks on the ImageNet dataset [48]. ResNet's use of residual connections allowed deeper architectures to be trained effectively by learning residual mappings instead of direct mappings. This innovation significantly improved classification accuracy while reducing overfitting, highlighting the potential of deep architectures to advance the field of image classification.

Object detection is a key task in computer vision that combines classification and localization to identify objects in an image and draw bounding boxes around them. This task extends beyond merely recognizing what is in an image by also determining where the objects are located. It has diverse applications across various domains. In surveillance, object detection helps identify suspicious activities or unauthorized access in security footage. For autonomous vehicles, it is essential for real-time identification of vehicles, pedestrians, and obstacles to ensure safe navigation. In agriculture, object detection aids in identifying pests and diseases or estimating crop yield, improving efficiency and productivity [49]. A significant milestone in object detection is the development of YOLO (You Only Look Once), a single-stage detection pipeline that revolutionized real-time object detection [50]. By combining detection and classification into a single step, YOLO dramatically reduced inference time

while maintaining high accuracy, making it suitable for applications requiring low-latency detection. Its efficiency and reliability have made YOLO a widely adopted framework in real-world scenarios demanding real-time performance.

Face recognition systems are designed to identify or verify individuals by analyzing facial features from images or video streams. These systems have become integral in various domains. In security, face recognition is used for access control and surveillance, ensuring only authorized individuals can enter restricted areas. In consumer electronics, facial unlocking features enhance convenience and security in smartphones and laptops. In retail, face recognition enables personalized customer experiences by analyzing facial data to understand preferences and behaviors [51]. A notable advancement in this field is FaceNet, which employs deep embeddings to map faces into a high-dimensional feature space [52]. This approach allows for highly accurate face verification and clustering of identities, enabling robust authentication systems and efficient video analytics. FaceNet's deployment in applications like secure authentication and surveillance highlights its effectiveness in real-world face recognition tasks.

Autonomous vehicles rely extensively on convolutional neural networks (CNNs) for real-time scene understanding, integrating multiple vision tasks to ensure safe and efficient operation. Core tasks include object detection using models like YOLO for identifying vehicles, pedestrians, and obstacles in real-time, semantic segmentation with architectures like DeepLab to classify road scenes into meaningful regions and monocular depth estimation using CNNs to predict depth information from single-camera inputs. These capabilities enable autonomous systems to interpret complex environments dynamically [53]. A prominent example is Tesla's Autopilot, which uses deep CNNs for multi-camera vision processing. This system powers advanced driver assistance features such as lane keeping, obstacle detection, and adaptive cruise control, forming the foundation for achieving full vehicle autonomy. By combining these techniques, autonomous vehicles demonstrate the potential of CNNs in transforming transportation [54].

1.3.11.2 Challenges in Real-World Applications

Despite their widespread success, convolutional neural networks (CNNs) face significant challenges when applied to real-world scenarios. Scalability is a major issue, as efficiently processing and managing large-scale data remains a bottleneck in deploying CNNs in practical systems. Another critical concern is robustness, where ensuring reliable performance under diverse conditions, such as changes in lighting, noise, or adversarial attacks, is essential for real-world reliability. Data privacy is also a growing challenge, especially in sensitive domains like healthcare, where balancing model performance with ethical considerations and privacy preservation is paramount [2]. Lastly, hardware constraints pose difficulties in deploying CNNs on edge devices or systems with limited computational resources, necessitating model optimization and lightweight architectures. Addressing these challenges is crucial for the seamless integration of CNNs into practical applications [6].

1.3.11.3 Future Directions

Convolutional neural networks (CNNs) continue to evolve, with several promising directions shaping their future applications and developments. One key trend is the integration with transformers, where CNNs are combined with transformer architectures to handle multimodal tasks such as image captioning and vision-language modeling, leveraging the strengths of both for enhanced performance [55]. Another area of growth is federated learning, which enables collaborative model training across decentralized datasets while preserving privacy, particularly in sensitive domains like medical imaging and financial analysis [56]. Additionally, real-time processing is gaining attention, with efforts focused on optimizing CNNs for low-latency applications such as augmented reality (AR), virtual reality (VR), and robotics. These advancements highlight the growing adaptability of CNNs to meet the demands of diverse, cutting-edge applications [57].

CNNs have transformed industries with applications spanning healthcare, autonomous systems, and creative domains. Their ability to model complex patterns and integrate seamlessly with real-world systems has unlocked unprecedented opportunities [3]. By exploring successful case studies and addressing deployment challenges, this chapter highlights the versatility and potential of CNNs in solving real-world problems.

1.4 CNN Hardware Accelerator

1.4.1 Motivation for CNN Hardware Accelerators

The rapid proliferation of convolutional neural networks (CNNs) in applications such as computer vision, natural language processing, and autonomous systems has highlighted the limitations of traditional computing platforms like central processing units (CPUs) and graphics processing units (GPUs). While these platforms offer significant flexibility and computational power, they are not inherently optimized for the unique demands of CNN workloads [58]. This subsection introduces into the primary limitations of these platforms and explores how dedicated hardware accelerators address these challenges.

1.4.1.1 Limitations of CPUs and GPUs for CNN Workloads

CPUs are designed for general-purpose computing and excel at sequential processing tasks. However, their inability to efficiently parallelize operations limits their suitability for the highly parallel and data-intensive computations required by CNNs [59]. GPUs, on the other hand, are well suited for parallel workloads due to their architecture, which includes thousands of cores optimized for simultaneous data processing [6].

In spite of the aforementioned advantages, GPUs are not without their limitations. One significant concern is their high power consumption, a characteristic shared with CPUs, which renders both unsuitable for deployment in energy-constrained environments such as edge devices or battery-powered systems. Furthermore, latency arising from the data transfer between memory and processing units frequently presents a bottleneck, particularly critical in real-time applications such as autonomous driving and medical imaging. Lastly, processing large-scale datasets or high-resolution inputs can severely strain the memory bandwidth and computational resources of conventional platforms, inevitably leading to a degradation in performance [60].

1.4.1.2 The Role of Hardware Accelerators

Hardware accelerators represent a significant advancement in computational design, specifically engineered to overcome the inherent limitations of general-purpose processing platforms. These specialized units

bridge the critical gap between the demanding computational requirements of convolutional neural network (CNN) workloads and the capabilities of existing hardware by meticulously tailoring their architecture to the unique characteristics of these operations [61]. Key attributes distinguishing hardware accelerators include optimized computational units, which are dedicated processing elements meticulously designed for the efficient execution of ubiquitous CNN operations such as convolution and activation functions. Furthermore, these accelerators prioritize reduced power consumption through energy-efficient designs that effectively lower power requirements without compromising computational performance. Crucially, they incorporate custom dataflow architectures, which are meticulously optimized to enhance data locality and significantly reduce memory access overhead, thereby boosting overall efficiency and throughput [62].

1.4.1.3 Performance Metrics for Hardware Accelerators

The effectiveness of hardware accelerators is rigorously assessed through the evaluation of several critical performance metrics. Throughput quantifies the overall processing capability by measuring the number of operations completed per unit of time. Latency, conversely, represents the time required to process a single input, a metric of paramount importance for applications demanding real-time responsiveness. Power efficiency evaluates the computational performance achieved per unit of power consumed, typically expressed as operations per watt, highlighting the energy proportionality of the accelerator. Finally, scalability assesses the accelerator's capacity to sustain performance levels as the size of the workload or the complexity of the hardware configuration increases [63].

1.4.2 Challenges in Hardware Design

Designing efficient hardware accelerators for CNN workloads involves overcoming several technical challenges. This subsection highlights the primary issues faced in hardware design and their implications for performance and cost.

Designing hardware accelerators for CNN workloads involves several significant technical challenges, primarily categorized into three areas. First, handling large memory requirements is crucial. CNN models often comprise millions of parameters and demand substantial memory for storing intermediate feature maps during both inference and training.

Efficiently managing these memory demands is critical for minimizing memory bandwidth bottlenecks and reducing energy consumption. High bandwidth is essential for data transfer between memory and processing units; without it, contention and reduced throughput can occur. Furthermore, accessing off-chip memory consumes significantly more power than on-chip operations, necessitating innovative memory hierarchies and caching strategies [64].

Second, efficiently managing data movement is a major concern. Data movement, both within the accelerator and between the accelerator and external memory, accounts for a substantial portion of power consumption. Effective dataflow strategies are vital for minimizing latency and enhancing locality. Optimizing interconnects and memory layouts can significantly reduce the time it takes for data to reach processing units, while designing architectures that maximize data reuse and minimize redundant transfers can greatly improve data locality [65].

Finally, balancing performance with cost and area constraints is paramount. Hardware accelerators must strike a delicate balance between achieving high performance and adhering to practical design considerations. Key trade-offs include cost efficiency, physical area, and the balance between flexibility and specialization [66]. Achieving performance goals without escalating manufacturing costs is particularly important for consumer-grade devices. Simultaneously, designing compact accelerators to meet integration requirements within System-on-Chip (SoC) platforms is essential. Moreover, ensuring the accelerator can support a range of CNN architectures without incurring significant performance overheads is a critical aspect of balancing flexibility with specialization [67].

1.4.3 Existing CNN Accelerator Hardware Solutions

Convolutional neural networks (CNNs) have established themselves as a foundational technology in modern deep learning applications, enabling advancements in areas ranging from image and speech recognition to autonomous systems and natural language understanding [2]. However, the computational intensity of CNNs, which arises primarily from operations like convolution and matrix multiplication, poses significant challenges for conventional computing architectures. To address these challenges, a diverse range of CNN hardware accelerators has been developed, encompassing both general-purpose and domain-specific solutions [58].

This section provides a comprehensive overview of existing CNN accelerator hardware solutions, focusing on architectural innovations, performance trade-offs, and practical implementations.

1.4.3.1 Details of the Proposed Hardware Implementation

CNN accelerators are broadly classified into three major categories: general-purpose processors (e.g., GPUs), domain-specific accelerators (e.g., ASICs and FPGAs), and hybrid systems. Each category employs unique architectural approaches to optimize performance for CNN workloads.

1. Graphics Processing Units (GPUs):

Graphics processing units (GPUs) have played a pivotal role in the growth of deep learning by enabling highly parallel computations critical for training and deploying convolutional neural networks (CNNs). NVIDIA's CUDA architecture has been particularly transformative, providing a platform for the efficient execution of matrix operations that are central to CNNs [68]. Modern GPUs, such as NVIDIA's Ampere architecture, further enhance deep learning performance with specialized Tensor Cores, which are designed to accelerate computations for tasks like training and inference. These Tensor Cores support mixed-precision arithmetic (e.g., FP16 and INT8), striking a balance between accuracy and computational speed, and leverage sparsity, exploiting the redundancy in CNN models to improve efficiency [69].

GPUs remain highly versatile, capable of handling both training and inference across a wide range of neural network architectures. Their flexibility and performance make them indispensable in deep learning research and development, powering advancements in applications from image classification to real-time object detection and autonomous systems [6].

2. Application-Specific Integrated Circuits (ASICs):

Application-specific integrated circuits (ASICs) are custom-designed chips optimized for specific computational tasks, offering unmatched performance and efficiency for convolutional neural network (CNN) workloads [70]. A notable example is the Google TPU (tensor processing unit), which employs a systolic array architecture to accelerate matrix multiplication, a core operation in CNNs [71]. TPUs

feature dedicated hardware for both dense and sparse tensor operations, on-chip memory buffers to reduce latency and power consumption, and deliver high throughput for training and inference tasks, making them a key component in large-scale deep learning applications [60].

For resource-constrained environments, edge TPUs are tailored to provide a balance between performance and energy efficiency. These chips are optimized for deployment in low-power devices, such as IoT systems and mobile platforms, enabling CNNs to operate effectively in real-time and off-grid scenarios. Together, TPUs and edge TPUs illustrate the potential of ASICs in advancing deep learning by addressing diverse computational requirements across cloud and edge settings [72].

3. **Field-Programmable Gate Arrays (FPGAs):**

Field-programmable gate arrays (FPGAs) provide a flexible alternative to application-specific integrated circuits (ASICs), offering developers the ability to customize hardware configurations to meet the specific demands of convolutional neural network (CNN) architectures. One of their key advantages is reconfigurability, which allows FPGAs to be reprogrammed for new algorithms or workloads, making them highly adaptable to evolving deep learning techniques. Additionally, their fine-grained parallelism enables efficient utilization of hardware resources, optimizing performance for tasks like matrix multiplication and convolutional operations in CNNs [73].

Prominent FPGA solutions, such as Xilinx's Versal AI Core [74] and Intel's Stratix series [75], are increasingly being deployed in both edge and cloud environments. These platforms leverage the flexibility and efficiency of FPGAs to accelerate deep learning tasks in diverse applications, from low-power IoT devices to high-performance data centers, bridging the gap between adaptability and computational power.

4. **Hybrid Architectures:**

Hybrid architectures leverage the strengths of multiple hardware types, such as CPUs, GPUs, and ASICs, to build heterogeneous systems optimized for end-to-end deep learning workflows. These architectures are designed to balance computational efficiency,

scalability, and flexibility across various stages of deep learning, from data preprocessing to model training and inference [76].

One notable example is NVIDIA's DGX systems, which integrate high-performance GPUs with high-speed interconnects, enabling efficient distributed training for large-scale models. These systems are optimized for parallel processing, allowing researchers to accelerate training times significantly [77]. Another example is AMD's ROCm ecosystem, which facilitates seamless CPU-GPU coprocessing. ROCm provides an open platform for deep learning workflows, enabling efficient data sharing and parallel computation between processors [78].

Hybrid architectures have become essential for handling the increasing complexity of deep learning tasks, offering tailored solutions for diverse applications in both research and industry.

1.4.3.2 Benchmarks, Results, and Comparisons

Evaluating convolutional neural network (CNN) accelerators necessitates a comprehensive assessment of key performance metrics, including throughput (measured in operations per second, such as FLOPS for floating-point operations), latency (the time required for a single inference or training iteration), energy efficiency (performance per watt, which is critical for edge and mobile deployments), and scalability (the ability to maintain performance across varying model sizes and workloads) [63].

Standardized benchmarks, such as MLPerf [79], are instrumental in facilitating objective comparisons across diverse hardware platforms. Key insights from these evaluations reveal distinct strengths among leading accelerators. For instance, Google TPUs demonstrate superior performance in large-scale model training, particularly for workloads like BERT and ResNet-50, with their energy efficiency and seamless integration with TensorFlow making them ideal for enterprise applications [71]. Conversely, NVIDIA GPUs excel in versatility, supporting a broad spectrum of workloads beyond CNNs. Ampere GPUs, in particular, showcase superior performance for mixed-precision training, positioning them as a preferred choice for research endeavors [69]. When comparing ASICs and FPGAs, ASICs typically achieve higher peak performance for specific tasks, while FPGAs offer greater adaptability and lower development costs, rendering them suitable for prototyping and edge applications. These comparative

results underscore the inherent trade-offs between different hardware platforms, emphasizing that the optimal accelerator choice is frequently determined by the specific requirements of the application.

1.4.4 Trends and Advancements in CNN Hardware Accelerators

The field of CNN hardware accelerators is characterized by rapid innovation, driven by the increasing complexity of deep learning models and the demand for real-time, energy-efficient solutions [80]. Several key trends and advancements are shaping this landscape.

As deep learning models continue to evolve, there's a strong focus on specialized architectures for emerging models. New hardware architectures are being designed to meet the unique demands of transformer-based models and sparse neural networks. These emerging architectures aim to optimize performance and efficiency for workloads characterized by large-scale matrix operations and irregular data access patterns [81]. One notable example is NVIDIA's Hopper architecture, which introduces advanced features tailored for transformer-based models. Hopper includes Transformer Engine, which leverages mixed-precision arithmetic and optimized hardware pipelines to accelerate transformer workloads, such as GPT and BERT, while maintaining high accuracy. It also supports dynamic sparsity to improve efficiency when processing sparse neural networks [82]. These specialized architectures reflect a growing focus on optimizing hardware for the next generation of machine learning models, addressing their unique computational challenges and enabling more efficient training and deployment at scale.

Another significant trend is energy-efficient computing. As sustainability becomes a key priority in computing, hardware accelerators are increasingly integrating energy-efficient technologies to reduce power consumption while maintaining performance. These innovations aim to address the high energy demands of modern deep learning workloads, making them more environmentally and economically sustainable. One promising approach is the use of nonvolatile memory (NVM) technologies, such as ReRAM (Resistive RAM), which consume significantly less power compared to traditional memory systems. By retaining data without continuous power, NVM reduces energy consumption for data storage and retrieval, especially in applications like edge computing and mobile AI [83].

These energy-efficient technologies represent significant strides toward greener AI solutions, aligning computational advancements with the growing need for sustainable practices in technology.

The rapid expansion of IoT and edge devices has fueled the development of edge AI solutions. These specialized accelerators are designed to deliver low-power, high-performance inference directly on edge hardware, enabling real-time AI processing without relying on cloud connectivity. This addresses critical concerns around latency, privacy, and energy efficiency. Notable examples include Apple's Neural Engine, integrated into the company's custom processors, which provides efficient on-device computation for tasks like image recognition, natural language processing, and augmented reality. It is optimized for low power consumption, ensuring seamless performance in devices like iPhones and iPads [84]. Similarly, Qualcomm's AI Engine powers advanced AI capabilities in mobile and IoT platforms. By combining heterogeneous processing units, including GPUs, CPUs, and DSPs, it supports versatile workloads such as voice recognition, camera enhancements, and predictive analytics [85]. Edge AI solutions like these enable powerful AI capabilities in resource-constrained environments, making them essential for applications in smart homes, autonomous devices, and industrial IoT systems. They represent a shift toward more distributed AI architectures, empowering devices to process and act on data locally with efficiency and reliability.

Network-on-Chip (NoC) technologies are emerging as a critical component in hardware accelerators, addressing the growing demands of on-chip communication in deep learning and other high-performance computing applications. As the complexity of hardware accelerators increases, efficient NoC designs are essential to ensure smooth data flow, low latency, and high throughput across processing cores and memory [86]. Key advancements in NoC architectures include the development of high-bandwidth, low-latency interconnects that optimize data transfer between processing elements, minimizing bottlenecks in communication-intensive workloads [87]. Additionally, adaptive routing algorithms have been introduced to dynamically manage workloads and reduce congestion, enabling more efficient utilization of on-chip resources. Scalable NoC designs further ensure that performance remains efficient across a range of hardware sizes, from compact edge devices to large-scale data center

accelerators [88]. These advancements are driving the next generation of hardware accelerators, enabling them to handle the ever-increasing demands of modern machine learning applications with improved efficiency and scalability.

In summary, CNN hardware accelerators are evolving to meet the demands of next-generation AI applications, balancing performance, scalability, and sustainability. The ongoing integration of emerging technologies and design paradigms promises to further enhance their capabilities, paving the way for more efficient and accessible deep learning solutions.

1.5 Conclusions

The rapid advancement of convolutional neural networks has reshaped the landscape of artificial intelligence, driving transformative changes across industries. However, the computational demands of these models have outpaced the capabilities of traditional hardware, underscoring the critical need for specialized hardware accelerators. This chapter has explored the principles and implementation of CNN hardware accelerators, shedding light on their pivotal role in overcoming the limitations of general-purpose platforms.

By examining key components of CNN architectures and their computational requirements, the chapter established a foundation for understanding the unique challenges faced by hardware designers. Through an in-depth analysis of existing solutions, including GPUs, ASICs, and FPGAs, the chapter demonstrated how tailored architectures enhance throughput, reduce latency, and improve energy efficiency. Furthermore, emerging trends such as energy-efficient computing, edge AI, and network-on-chip architectures were discussed, highlighting the innovative approaches shaping the future of CNN hardware acceleration.

Looking ahead, the integration of advanced materials, energy-efficient computing, and edge AI promises to further enhance the capabilities of CNN accelerators. As artificial intelligence continues to evolve, the synergy between algorithmic and hardware innovation will remain essential in meeting the growing demands of next-generation applications. By addressing current challenges and embracing emerging opportunities, CNN

hardware accelerators will continue to drive progress in deep learning, enabling breakthroughs across scientific, industrial, and consumer domains.

References

1. O'shea, K., Nash, R.: An introduction to convolutional neural networks (2015). arXiv preprint arXiv:1511.08458
2. Sahu, M., Dash, R.: A survey on deep learning: convolution neural network (CNN). In: Intelligent and Cloud Computing: Proceedings of ICICC 2019, vol. 2, pp. 317–325. Springer (2020)
3. Alzubaidi, L., Zhang, J., Humaidi, A.J., Al-Dujaili, A., Duan, Y., Al-Shamma, O., Santamaría, J., Fadhel, M.A., Al-Amidie, M., Farhan, L.: Review of deep learning: concepts, cnn architectures, challenges, applications, future directions. *J. Big Data* **8**(1), 53 (2021) [[Crossref](#)]
4. Yu, H., Sun, P., Halak, B., Shanthakumar, K., Kazmierski, T.: Tamper resistant design of convolutional neural network hardware accelerator. In: 2023 Asian Hardware Oriented Security and Trust Symposium (AsianHOST), pp. 1–5 (2023). <https://doi.org/10.1109/AsianHOST59942.2023.10409319>
5. Zhang, Q., Zhang, M., Chen, T., Sun, Z., Ma, Y., Yu, B.: Recent advances in convolutional neural network acceleration. *Neurocomputing* **323**, 37–51 (2019) [[Crossref](#)]
6. Hu, Y., Liu, Y., Liu, Z.: A survey on convolutional neural network accelerators: GPU, FPGA and ASIC. In: 2022 14th International Conference on Computer Research and Development (ICCRD), pp. 100–107. IEEE (2022)
7. Elngar, A.A., Arafa, M., Fathy, A., Moustafa, B., Mahmoud, O., Shaban, M., Fawzy, N.: Image classification based on CNN: a survey. *J. Cybersecur. Inf. Manag.* **6**(1), 18–50 (2021)
8. Purwono, P., Ma'arif, A., Rahmaniar, W., Fathurrahman, H.I.K., Frisky, A.Z.K., ul Haq, Q.M.: Understanding of convolutional neural network (CNN): a review. *Int. J. Robot. Control Syst.* **2**(4), 739–748 (2022)
9. Wu, J.: Introduction to convolutional neural networks. *Natl. Key Lab Novel Softw. Technol. Nanjing Univ. China* **5**(23), 495 (2017)
10. Hossain, M.A., Sajib, M.S.A.: Classification of image using convolutional neural network (CNN). *Glob. J. Comput. Sci. Technol.* **19**(2), 13–14 (2019)
11. Wang, Q., Ma, Y., Zhao, K., Tian, Y.: A comprehensive survey of loss functions in machine learning. *Ann. Data Sci.* **9**(2), 187–212 (2022) [[Crossref](#)]
12. Zhou, X.: Understanding the convolutional neural networks with gradient descent and backpropagation. In: *Journal of Physics: Conference Series*, vol. 1004, p. 012028. IOP Publishing, Bristol (2018)

13. Santos, C.F.G.D., Papa, J.P.: Avoiding overfitting: a survey on regularization methods for convolutional neural networks. *ACM Comput. Surv. (CSUR)* **54**(10s), 1–25 (2022)
14. Khan, S., Rahmani, H., Shah, S.A.A., Bennamoun, M., Medioni, G., Dickinson, S.: *A Guide to Convolutional Neural Networks for Computer Vision*. Springer, Cham (2018)
[Crossref]
15. Tombe, R., Viriri, S.: Effective processing of convolutional neural networks for computer vision: a tutorial and survey. *IETE Tech. Rev.* **39**(1), 49–62 (2022)
[Crossref]
16. Kiranyaz, S., Ince, T., Abdeljaber, O., Avci, O., Gabbouj, M.: 1-d convolutional neural networks for signal processing applications. In: *ICASSP 2019-2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 8360–8364. IEEE (2019)
17. Cheng, C., Parhi, K.K.: Fast 2D convolution algorithms for convolutional neural networks. *IEEE Trans. Circ. Syst. I Regul. Pap.* **67**(5), 1678–1691 (2020)
[MathSciNet][Crossref]
18. Yang, H., Yuan, C., Li, B., Du, Y., Xing, J., Hu, W., Maybank, S.J.: Asymmetric 3D convolutional neural networks for action recognition. *Pattern Recognit.* **85**, 1–12 (2019)
[Crossref]
19. Lei, X., Pan, H., Huang, X.: A dilated CNN model for image classification. *IEEE Access* **7**, 124087–124095 (2019)
[Crossref]
20. Srivastava, H., Sarawadekar, K.: A depthwise separable convolution architecture for CNN accelerator. In: *2020 IEEE Applied Signal Processing Conference (ASPCON)*, pp. 1–5. IEEE (2020)
21. Zafar, A., Aamir, M., Mohd Nawi, N., Arshad, A., Riaz, S., Alruban, A., Dutta, A.K., Almotairi, S.: A comparison of pooling methods for convolutional neural networks. *Appl. Sci.* **12**(17), 8643 (2022)
[Crossref]
22. Nam, H., Kim, H.E.: Batch-instance normalization for adaptively style-invariant neural networks. In: *Advances in Neural Information Processing Systems*, 31 (2018)
23. Tian, C., Xu, Y., Zuo, W.: Image denoising using deep cnn with batch renormalization. *Neural Netw.* **121**, 461–473 (2020)
[Crossref]
24. Luo, P., Zhang, R., Ren, J., Peng, Z., Li, J.: Switchable normalization for learning-to-normalize deep representation. *IEEE Trans. Pattern Anal. Mach. Intell.* **43**(2), 712–728 (2019)
[Crossref]
25. Zhang, H., Dauphin, Y.N., Ma, T.: Fixup initialization: residual learning without normalization (2019). arXiv preprint arXiv:1901.09321

26. Pratiwi, H., Windarto, A.P., Susliansyah, S., Aria, R.R., Susilowati, S., Rahayu, L.K., Fitriani, Y., Merdekawati, A., Rahadjeng, I.R.: Sigmoid activation function in selecting the best model of artificial neural networks. In: *Journal of Physics: Conference Series*, vol. 1471, p. 012010. IOP Publishing, Bristol (2020)
27. Karateke, S.: Some mathematical properties of flexible hyperbolic tangent activation function with application to deep neural networks. *Revista de la Real Academia de Ciencias Exactas, Físicas y Naturales. Serie A. Matemáticas* **119**(2), 42 (2025)
28. Agarap, A.F.: Deep learning using rectified linear units (ReLU) (2018). arXiv preprint arXiv:1803.08375
29. Dubey, A.K., Jain, V.: Comparative study of convolution neural network's relu and Leaky-ReLU activation functions. In: *Applications of Computing, Automation and Wireless Systems in Electrical Engineering: Proceedings of MARC 2018*, pp. 873–880. Springer (2019)
30. He, K., Zhang, X., Ren, S., Sun, J.: Delving deep into rectifiers: surpassing human-level performance on imagenet classification. In: *Proceedings of the IEEE International Conference on Computer Vision*, pp. 1026–1034 (2015)
31. Mercioni, M.A., Holban, S.: P-swish: activation function with learnable parameters based on swish activation function in deep learning. In: *2020 International Symposium on Electronics and Telecommunications (ISETC)*, pp. 1–4. IEEE (2020)
32. Dasgupta, R., Chowdhury, Y.S., Nanda, S.: Performance comparison of benchmark activation function ReLU, Swish and Mish for facial mask detection using convolutional neural network. In: *Intelligent Systems: Proceedings of SCIS 2021*, pp. 355–367. Springer (2021)
33. Basha, S.S., Dubey, S.R., Pulabaigari, V., Mukherjee, S.: Impact of fully connected layers on performance of convolutional neural networks for image classification. *Neurocomputing* **378**, 112–119 (2020)
[Crossref]
34. Lecun, Y., Bottou, L., Bengio, Y., Haffner, P.: Gradient-based learning applied to document recognition. *Proc. IEEE* **86**(11), 2278–2324 (1998). <https://doi.org/10.1109/5.726791>
[Crossref]
35. Cong, S., Zhou, Y.: A review of convolutional neural network architectures and their optimizations. *Artif. Intell. Rev.* **56**(3), 1905–1969 (2023)
[Crossref]
36. Krizhevsky, A., Sutskever, I., Hinton, G.E.: Imagenet classification with deep convolutional neural networks (AlexNet). *Commun. ACM* **60**, 84–90 (2017)
[Crossref]
37. Simonyan, K., Zisserman, A.: Very deep convolutional networks for large-scale image recognition (2014). arXiv preprint arXiv:1409.1556
38. Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., Erhan, D., Vanhoucke, V., Rabinovich, A.: Going deeper with convolutions. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 1–9 (2015)

39. He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 770–778 (2016)
40. Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., Chen, L.C.: Mobilenetv2: inverted residuals and linear bottlenecks. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 4510–4520 (2018)
41. Zhang, X., Zhou, X., Lin, M., Sun, J.: Shufflenet: an extremely efficient convolutional neural network for mobile devices. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 6848–6856 (2018)
42. Han, K., Wang, Y., Tian, Q., Guo, J., Xu, C., Xu, C.: Ghostnet: more features from cheap operations. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 1580–1589 (2020)
43. He, K., Zhang, X., Ren, S., Sun, J.: Identity mappings in deep residual networks. In: European Conference on Computer Vision, pp. 630–645. Springer (2016)
44. Huang, G., Liu, Z., Van Der Maaten, L., Weinberger, K.Q.: Densely connected convolutional networks. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 4700–4708 (2017)
45. Guo, M.H., Xu, T.X., Liu, J.J., Liu, Z.N., Jiang, P.T., Mu, T.J., Zhang, S.H., Martin, R.R., Cheng, M.M., Hu, S.M.: Attention mechanisms in computer vision: a survey. *Comput. Vis. Media* **8**(3), 331–368 (2022) [\[Crossref\]](#)
46. Niu, Z., Zhong, G., Yu, H.: A review on the attention mechanism of deep learning. *Neurocomputing* **452**, 48–62 (2021) [\[Crossref\]](#)
47. Chollet, F.: Xception: deep learning with depthwise separable convolutions. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 1251–1258 (2017)
48. Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M., Berg, A.C., Fei-Fei, L.: ImageNet large scale visual recognition challenge. *Int. J. Comput. Vis. (IJCV)* **115**(3), 211–252 (2015). <https://doi.org/10.1007/s11263-015-0816-y>
49. Zhiqiang, W., Jun, L.: A review of object detection based on convolutional neural network. In: 2017 36th Chinese Control Conference (CCC), pp. 11104–11109. IEEE (2017)
50. Jiang, P., Ergu, D., Liu, F., Cai, Y., Ma, B.: A review of yolo algorithm developments. *Proc. Comput. Sci.* **199**, 1066–1073 (2022) [\[Crossref\]](#)
51. Lawrence, S., Giles, C.L., Tsoi, A.C., Back, A.D.: Face recognition: a convolutional neural-network approach. *IEEE Trans. Neural Netw.* **8**(1), 98–113 (1997) [\[Crossref\]](#)

52. Schroff, F., Kalenichenko, D., Philbin, J.: Facenet: a unified embedding for face recognition and clustering. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 815–823 (2015)
53. Chen, L., Lin, S., Lu, X., Cao, D., Wu, H., Guo, C., Liu, C., Wang, F.Y.: Deep neural network based vehicle and pedestrian detection for autonomous driving: a survey. *IEEE Trans. Intell. Transp. Syst.* **22**(6), 3234–3246 (2021)
[\[Crossref\]](#)
54. Ingle, S., Phute, M.: Tesla autopilot: semi autonomous driving, an uptick for future autonomy. *Int. Res. J. Eng. Technol.* **3**(9), 369–372 (2016)
55. Guo, J., Han, K., Wu, H., Tang, Y., Chen, X., Wang, Y., Xu, C.: Cmt: Convolutional neural networks meet vision transformers. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 12175–12185 (2022)
56. He, C., Annavaram, M., Avestimehr, S.: Group knowledge transfer: federated learning of large cnns at the edge. *Adv. Neural Inf. Proces. Syst.* **33**, 14068–14080 (2020)
57. Cortes, D., Bermejo, B., Juiz, C.: The use of cnns in VR/AR/MR/XR: a systematic literature review. *Virtual Real.* **28**(3), 154 (2024)
[\[Crossref\]](#)
58. Chen, Y., Xie, Y., Song, L., Chen, F., Tang, T.: A survey of accelerator architectures for deep neural networks. *Engineering* **6**(3), 264–274 (2020)
[\[Crossref\]](#)
59. Datta, D., Mittal, D., Mathew, N.P., Sairabanu, J.: Comparison of performance of parallel computation of CPU cores on CNN model. In: *2020 International Conference on Emerging Trends in Information Technology and Engineering (ic-ETITE)*, pp. 1–8. IEEE (2020)
60. Wang, Y.E., Wei, G.Y., Brooks, D.: Benchmarking TPU, GPU, and CPU platforms for deep learning (2019). arXiv preprint arXiv:1907.10701
61. Wang, C., Lou, W., Gong, L., Jin, L., Tan, L., Hu, Y., Li, X., Zhou, X.: Reconfigurable hardware accelerators: opportunities, trends, and challenges (2017). arXiv preprint arXiv:1712.04771
62. Dhilleswararao, P., Boppu, S., Manikandan, M.S., Cenkeramaddi, L.R.: Efficient hardware architectures for accelerating deep neural networks: survey. *IEEE Access* **10**, 131788–131828 (2022)
[\[Crossref\]](#)
63. Velasco-Montero, D., Fernández-Berni, J., Rodríguez-Vázquez, A.: Relevant hardware metrics for performance evaluation. In: *Visual Inference for IoT Systems: A Practical Approach*, pp. 61–88. Springer (2022)
64. Siu, K., Stuart, D.M., Mahmoud, M., Moshovos, A.: Memory requirements for convolutional neural network hardware accelerators. In: *2018 IEEE International Symposium on Workload Characterization (IISWC)*, pp. 111–121. IEEE (2018)

65. Ma, Y., Cao, Y., Vrudhula, S., Seo, J.S.: Optimizing the convolution operation to accelerate deep neural networks on FPGA. *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* **26**(7), 1354–1367 (2018)
66. Krichen, M.: Convolutional neural networks: a survey. *Computers* **12**(8), 151 (2023) [[Crossref](#)]
67. Lin, Y.J., Chang, T.S.: Data and hardware efficient design for convolutional neural network. *IEEE Trans. Circ. Syst. I: Regul. Pap.* **65**(5), 1642–1651 (2017)
68. Kirk, D., et al.: NVIDIA cuda software and GPU parallel computing architecture. In: *ISMM*, vol. 7, pp. 103–104 (2007)
69. Pei, Z., Li, C., Qin, X., Chen, X., Wei, G.: Iteration time prediction for CNN in multi-GPU platform: modeling and analysis. *IEEE Access* **7**, 64788–64797 (2019) [[Crossref](#)]
70. Machupalli, R., Hossain, M., Mandal, M.: Review of asic accelerators for deep neural network. *Microprocess. Microsyst.* **89**, 104441 (2022) [[Crossref](#)]
71. Armoni, M.: Tensor processing units (TPU): a technical analysis and their impact on artificial intelligence. *Tech4Future Information Technology Report* (2023)
72. Seshadri, K., Akin, B., Laudon, J., Narayanaswami, R., Yazdanbakhsh, A.: An evaluation of edge TPU accelerators for convolutional neural networks. In: *2022 IEEE International Symposium on Workload Characterization (IISWC)*, pp. 79–91. *IEEE* (2022)
73. Zhao, R., Song, W., Zhang, W., Xing, T., Lin, J.H., Srivastava, M., Gupta, R., Zhang, Z.: Accelerating binarized convolutional neural networks with software-programmable fpgas. In: *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, pp. 15–24 (2017)
74. Gaide, B., Gaitonde, D., Ravishankar, C., Bauer, T.: Xilinx adaptive compute acceleration platform: versaltm architecture. In: *Proceedings of the 2019 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, pp. 84–93 (2019)
75. He, K., Liu, B., Zhang, Y., Ling, A., Gu, D.: Fecaffe: FPGA-enabled caffe with opencl for deep learning training and inference on intel stratix 10 (2019). *arXiv preprint arXiv:1911.08905*
76. Yan, M., Deng, L., Hu, X., Liang, L., Feng, Y., Ye, X., Zhang, Z., Fan, D., Xie, Y.: HyGCN: a GCN accelerator with hybrid architecture. In: *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pp. 15–29. *IEEE* (2020)
77. Gawande, N.A., Daily, J.A., Siegel, C., Tallent, N.R., Vishnu, A.: Scaling deep learning workloads: NVIDIA DGX-1/pascal and intel knights landing. *Futur. Gener. Comput. Syst.* **108**, 1162–1172 (2020) [[Crossref](#)]
78. Otterness, N., Anderson, J.: AMD GPUs as an alternative to NVIDIA for supporting real-time workloads. In: *Proceedings of the 32nd Euromicro Conference on Real-Time Systems* (2020)

79. Reddi, V.J., Cheng, C., Kanter, D., Mattson, P., Schmuelling, G., Wu, C.J., Anderson, B., Breughe, M., Charlebois, M., Chou, W., et al.: MLPerf inference benchmark. In: 2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA), pp. 446–459. IEEE (2020)
80. Reuther, A., Michaleas, P., Jones, M., Gadepally, V., Samsi, S., Kepner, J.: AI accelerator survey and trends. In: 2021 IEEE High Performance Extreme Computing Conference (HPEC), pp. 1–9. IEEE (2021)
81. Huang, S., Tang, E., Li, S., Ping, X., Chen, R.: Hardware-friendly compression and hardware acceleration for transformer: a survey. *Electron. Res. Arch.* **30**(10), 3755–3785 (2022) [[MathSciNet](#)][[Crossref](#)]
82. Elster, A.C., Haugdahl, T.A.: NVIDIA hopper GPU and grace CPU highlights. *Comput Sci Eng* **24**(2), 95–100 (2022) [[Crossref](#)]
83. He, Y., Wang, Y., Wang, Y., Li, H., Li, X.: An agile precision-tunable CNN accelerator based on ReRAM. In: 2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD), pp. 1–7. IEEE (2019)
84. Hübner, P., Hu, A., Peng, I., Markidis, S.: Apple vs. oranges: evaluating the apple silicon m-series SoCs for HPC performance and efficiency (2025). arXiv preprint arXiv:2502.05317
85. Chatha, K.: Qualcomm® cloud al 100: 12tops/w scalable, high performance and low latency deep learning inference accelerator. In: 2021 IEEE Hot Chips 33 Symposium (HCS), pp. 1–19. IEEE (2021)
86. Gao, J., Shao, Q., Deng, F., Wang, Q., Jing, N., Jiang, J.: An NoC-based CNN accelerator for edge computing. In: 2023 IEEE 15th International Conference on ASIC (ASICON), pp. 1–4. IEEE (2023)
87. Tiwari, B., Yang, M., Wang, X., Jiang, Y., Muthukumar, V.: Improving the performance of a NoC-based CNN accelerator with gather support. In: 2020 IEEE 33rd International System-on-Chip Conference (SOCC), pp. 72–77. IEEE (2020)
88. Biglari, S., Hosseini, F., Upadhyay, A., Zhao, H.: Survey of network-on-chip (NoC) for heterogeneous multicore systems. In: 2024 IEEE 17th International Symposium on Embedded Multicore/Many-core Systems-on-Chip (MCSoc), pp. 155–162. IEEE (2024)

2. A Tutorial on CNN Design from Model Training to Hardware Implementation

Peiyao Sun¹✉, Haosen Yu¹✉, Tom J. Kazmierski¹✉ and Basel Halak¹✉
(1) School of Electronics and Computer Science, University of Southampton,
Southampton, UK

✉ **Peiyao Sun**
Email: ps2m24@soton.ac.uk

✉ **Haosen Yu**
Email: Haosen.Yu@soton.ac.uk

✉ **Tom J. Kazmierski**
Email: T.J.Kazmierski@soton.ac.uk

✉ **Basel Halak (Corresponding author)**
Email: Basel.Halak@soton.ac.uk

Keywords CNN design – TensorFlow – SystemVerilog – CNN training – Hardware modeling – Software implementation – Hardware simulation – Testbench design – Layer functions – Network evaluation – Dataflow architecture – RTL design – FPGA implementation – CNN deployment – Deep learning frameworks

2.1 Introduction

Convolutional neural networks (CNNs) are widely used in the field of image classification. While many recent works and books focus on theoretical aspects, there is a lack of detailed tutorials to guide beginners in building their first models. However, for beginners in this field, whether focusing on hardware or software, one of the primary challenges is the lack of a comprehensive and integrated tutorial that guides them through the process of implementing their designs. This

chapter aims to provide a clear, practical guide to help beginners quickly understand mainstream training frameworks, understand key design considerations, and implement their first CNN model effectively.

This chapter covers both software-level and hardware-level aspects of CNN design. At the software level, it introduces Python-based frameworks such as TensorFlow, while at the hardware level, how to utilize Vitis AI to deploy CNN models on Xilinx platforms quickly and how to design custom CNN hardware accelerators using SystemVerilog. By the end of this chapter, readers will have a solid foundation for training CNN models and implementing them on hardware platforms, which helps them to develop their first CNN-related implementation quickly.

TensorFlow, developed by Google [1], and PyTorch, developed by Meta AI [2], are the most popular learning frameworks in both research and industrial fields. TensorFlow supports various hardware platforms such as CPUs, GPUs, and TPUs. It combines static computation graphs for optimized performance with Eager Execution for dynamic debugging and prototyping. It also offers comprehensive Toolkits such as TensorBoard for visualization, TensorFlow Serving for inference, and the TensorFlow Model Optimization Toolkit for quantization and pruning [1, 3]. PyTorch's advantage is its flexibility, offering a dynamic computation graph that simplifies debugging and supports innovative neural network designs. In addition, PyTorch is also highly integrated with Python, performs well in terms of GPU-accelerated computations, provides tools like TorchScript for model deployment, and supports ONNX for high transferability [4].

Deploying neural networks directly on FPGAs using HDLs, such as SystemVerilog, provides unique advantages in flexibility and customization. While VITIS AI relies on predefined hardware architectures for CNNs, direct implementation using HDLs allows developers to design customized CNN hardware accelerators. This approach is particularly advantageous for creating specialized hardware structures or implementing uncommon components, such as utilizing TOSAM approximate multipliers [5] instead of normal accuracy multipliers. For developers aiming to design highly customized neural network accelerators, the use of HDLs remains an essential and irreplaceable option.

2.2 Chapter Overview

This chapter provides a comprehensive tutorial on utilizing various tools and frameworks for training and deploying CNNs. In Sect. [2.3](#), we detail the process of using TensorFlow to train a CNN. Subsequently, Sect. [2.4](#) explores the implementation of custom CNN hardware accelerators using SystemVerilog. The example code is available in the GitHub repository.¹

2.3 Software-Level CNN Model Training Based on TensorFlow Framework

This section provides an overview of TensorFlow tutorials, focusing on key aspects of training a network using TensorFlow. We will explore common layer functions, the framework model structure, guidelines for writing code, and the parameter definitions required for function implementation. All codes in this section are Python Code, and TensorFlow will be imported by the code shown in Fig. [2.1](#)

```
import tensorflow as tf
```

Fig. 2.1 Importing TensorFlow

2.3.1 Layer Functions

2.3.1.1 *Convolutional Layers*

In widely used neural networks such as MobileNet, ResNet, and ShuffleNet, three types of convolutional layers are employed: standard convolutional layers, pointwise convolutional layers, and depthwise convolutional layers. This section describes the structure and input parameters of the layer functions in TensorFlow that correspond to these three types of convolutional layers. For a standard convolutional layer and a pointwise convolutional layer, the layer function is `tf.keras.layers.Conv2D` whose default and full structure is shown in Fig. [2.2](#).

```
tf.keras.layers.Conv2D(
    filters,
    kernel_size,
    strides=(1, 1),
    padding='valid',
    data_format=None,
    dilation_rate=(1, 1),
    groups=1,
    activation=None,
    use_bias=True,
    kernel_initializer='glorot_uniform',
    bias_initializer='zeros',
    kernel_regularizer=None,
    bias_regularizer=None,
    activity_regularizer=None,
    kernel_constraint=None,
    bias_constraint=None,
    **kwargs
)
```

Fig. 2.2 TensorFlow Conv2D layer definition

Let us start with the basic parameter Settings, as shown in Fig. 2.2, including filters, kernel_size, strides, padding, data_format, group, and activation.

- **Filter:** Filter is the number of kernels in this layer, and the data format is *int*. A common number of filter values is 16, 32, 64, 128, and 256.
- **kernel_size:** The kernel size represents the size of the convolution kernel of this layer, which can be in int format or tuple format with two elements. For kernel size Kernel_size = 3 is equivalent to Kernel_size = (3,3), but if the Kernel of the current convolution layer is not square, use the tuple format as needed, such as Kernel_size = (3,5). The common value of the kernel size is 3,5,7,9, and 11.
- **padding:** In the `tf.keras.layers.Conv2D` convolutional layer function, padding takes two inputs: “valid” or “same.” “valid” means not padding the input of the layer, while same means padding the layer so that the size of the input layer’s two-dimensional space is equal to the size of the output layer’s two-dimensional space. For the situation of the special padding, such as adding a specific size of the padding, or the padding size is not the same in all directions, you can use `keras.layers.ZeroPadding2D` function.
- **data_format:** The `data_format` parameter in `tf.keras.layers.Conv2D` specifies the arrangement of input dimensions. Two modes are available: `channels_last` and `channels_first`. If not explicitly defined, the default is `channels_last`. In TensorFlow, the input to `tf.keras.layers.Conv2D` is a four-dimensional tensor representing

batch size, height, width, and channels. For `channels_last`, the data format is (batch size, height, width, channels), whereas for `channels_first`, the format is (batch size, channels, height, width).

- **group:** In `tf.keras.layers.Conv2D`, the `group` parameter specifies whether to perform grouped convolution and determines the number of groups. A `group` value of 1 indicates no grouped convolution, while higher values divide the convolution operation into the specified number of groups. For example, setting `group = 2` splits the convolution into two groups. By default, `group` is set to 1 if not explicitly specified.
- **activation:** The `activation` parameter specifies whether an activation function is applied to process the output of a convolution operation. Setting it to "None" indicates that no activation function will be used. TensorFlow provides several predefined activation functions, including "deserialize", "elu", "exponential", "gelu", "get", "hard_sigmoid", "hard_silu", "hard_swish", "leaky_relu", "linear", "log_softmax", "mish", "relu", "relu6", "selu", "serialize", "sigmoid", "silu", "softmax", "soft- plus", "softsign", "swish", and "tanh". The desired activation function can be specified in the format `activation="relu"`.
- **use_bias:** The `use_bias` parameter is of type `bool`. When set to `True`, it adds a bias term to the data generated by the convolution. If `use_bias=True` and the `activation` parameter is not `None`, the bias is added first, followed by processing through the specified activation function.

The parameters `kernel_initializer` and `bias_initializer` specify how the initial kernel weights and biases are generated. Supported initialization methods are introduced in the Table [2.1](#)

Table 2.1 Descriptions of common Keras weight initializers

Initializer	Description
TruncatedNormal	Samples from a normal distribution, but values more than two standard deviations from the mean are redrawn. Avoids extreme weights
Zeros	All weights initialized to zero. Not suitable for weights due to symmetry
Ones	All weights initialized to one. Sometimes used for bias initialization
GlorotNormal	Xavier normal initializer: normal distribution scaled by the number of input and output units (fan-in, fan-out)
GlorotUniform	Xavier uniform initializer: same scaling principle as <code>GlorotNormal</code> , but uses a uniform distribution

Initializer	Description
HeNormal	Kaiming normal initializer: scales variance by fan-in.
	Suitable for ReLU and variants
HeUniform	Uniform version of He initializer
Orthogonal	Initializes weights with orthogonal matrices. Preserves variance
Constant	Sets all weights to a user-specified constant value
VarianceScaling	General-purpose initializer. Scaling can adapt to fan-in, fan-out, and mode. Glorot, He, and Lecun are special cases
LecunNormal	Scaled normal distribution based on fan-in. For <code>selu</code> activation
LecunUniform	Uniform variant of <code>LecunNormal</code>
Identity	Initializes square weight matrices as identity. For recurrent layers

Additionally, user-defined initialization methods are supported.

The parameters `kernel_regularizer`, `bias_regularizer`, and `activity_regularizer` allow for the addition of regularization terms to the kernel weights, biases, and outputs. Predefined regularization methods include:

User-defined regularization methods can also be applied. By the way, the default mode of the regularization is none, which means the regularization is not applied.

The parameters `bias_constraint` and `kernel_constraint` specify the conditions to be applied to the trained bias and kernel weights during optimization. The available constraint functions are introduced in Table [2.2](#).

Table 2.2 Descriptions of common Keras regularizers

Regularizer	Description
L1	Adds a penalty equal to the absolute value of the weights ($\lambda \sum w $)
	Encourages sparsity and feature selection
L2	Adds a penalty proportional to the square of the weights ($\lambda \sum w^2$)
	Encourages smaller weights and reduces overfitting
L1L2	Combines both L1 and L2 penalties
	Offers a balance between sparsity and weight decay
OrthogonalRegularizer	Penalizes deviation from orthogonality between weight vectors
	Useful for constraining weight matrices in some specialized models

The default mode is none, which means the constraint is not applied.

We have already provided an example of setting `tf.keras.layers.Conv2D` with none of the arguments set to default mode,

in the GitHub repository.² `tf.keras.layers.DepthwiseConv2D` shares most of its parameters with `tf.keras.layers.Conv2D`. The only differing parameter is `depth_multiplier`, which is an integer that specifies the number of output channels generated for each input channel in depthwise convolution.

2.3.1.2 Batch Normalization Layers

Now that we have covered some basic functions and settings of convolutional layers, we will now look at functions related to normalization layers. In TensorFlow, the related layer is called `tf.keras.layers.BatchNormalization`. The full and default version is shown in Fig. 2.3.

```
tf.keras.layers.BatchNormalization(  
    axis=-1,  
    momentum=0.99,  
    epsilon=0.001,  
    center=True,  
    scale=True,  
    beta_initializer='zeros',  
    gamma_initializer='ones',  
    moving_mean_initializer='zeros',  
    moving_variance_initializer='ones',  
    beta_regularizer=None,  
    gamma_regularizer=None,  
    beta_constraint=None,  
    gamma_constraint=None,  
    synchronized=False,  
    **kwargs  
)
```

Fig. 2.3 BatchNormalization layer configuration

Many parameters in this layer are consistent with the settings of convolutional layers, including regularization and initialization configurations. Therefore, this section will focus on the parameters that are unique to this layer and are not included in the convolutional layer settings. These parameters are `axis`, `momentum`, `epsilon`, `center`, `scale`, and `synchronized`.

- **Synchronized:** This parameter is only applicable when using the TensorFlow backend. If set to `True`, it synchronizes the global batch statistics (mean and variance) across all devices during each training step in a distributed training strategy. If set to `False` (default), each replica uses its local batch statistics.

- **Axis:** An integer specifying the axis to be normalized, typically the feature axis. For example, after a Conv2D layer with `data_format="channels_first,"` the value of `axis` should be set to 1. The default value is `axis=-1`.
- **Momentum:** Defines the momentum for the moving average used to compute the mean and variance. The default value is `momentum=0.99`.
- **Epsilon:** A small float value added to the variance to prevent division by zero during normalization. The default value is `epsilon=0.001`.
- **Center:** If set to `True` (default), the layer adds an offset (β) to the normalized tensor. If set to `False`, β is ignored.
- **Scale:** If set to `True` (default), the normalized tensor is multiplied by a scale factor (γ). If set to `False`, γ is not used. This can be disabled when the subsequent layer is linear, as scaling will be handled by that layer.

We have already provided an example of setting `tf.keras.layers.`

`BatchNormalization` with none of the arguments set to default mode, in the GitHub repository.³

2.3.1.3 Activation Layers

Next, we will introduce some commonly used activation layers, including `ReLU` and `Softmax`. These two kinds of activation layers are called `tf.keras.layers.ReLU` and `tf.keras.layers.Softmax`. The full and default versions of `ReLU` and `Softmax` are shown in Figs. [2.4](#) and [2.5](#).

```
tf.keras.layers.ReLU(
    max_value=None,
    negative_slope=0.0,
    threshold=0.0,
    **kwargs
)
```

Fig. 2.4 ReLU activation layer configuration

```
tf.keras.layers.Softmax(
    axis=-1,
    **kwargs
)
```

Fig. 2.5 Softmax activation layer configuration

The parameters in `tf.keras.layers.ReLU` include `max_value`, `threshold`, and `negative_slope`, which are explained as follows:

- **max_value:** A float value (≥ 0) that specifies the maximum activation value. If set to `None`, the activation value is unlimited. Setting this parameter can achieve certain variants of ReLU, such as ReLU6, which helps to limit excessively large values in neural networks. The default value is `None`.
- **negative_slope:** A float value (≥ 0) that defines the negative slope coefficient. The default value is `0.0`. Configuring this parameter enables variants of ReLU, such as Leaky ReLU, which helps address the “dying neuron” problem by allowing small gradients for negative inputs.
- **threshold:** A float value (≥ 0) representing the threshold for threshold activation. Inputs below this threshold are either set to zero or multiplied by the `negative_slope` coefficient. The default value is `0.0`.

For the `tf.keras.layers.Softmax` function, it has only one input parameter, `axis`. This parameter can be an integer or a list of integers, specifying the axis (or axes) along which the softmax normalization is applied.

2.3.1.4 Pooling Layers

Next, we will introduce functions related to pooling layers. Specifically, we will discuss the functions for four commonly used pooling layers: average pooling layer, max pooling layer, global average pooling layer, and global max pooling layer. These functions are named `tf.keras.layers.AveragePooling2D`, `tf.keras.layers.MaxPool2D`, `tf.keras.layers.GlobalAveragePooling2D`, and `tf.keras.layers.GlobalMaxPool2D`, respectively. Their default configurations and full parameter presentation are shown in Figs. [2.6](#), [2.7](#), [2.8](#), and [2.9](#).

```
tf.keras.layers.AveragePooling2D(  
    pool_size,  
    strides=None,  
    padding='valid',  
    data_format=None,  
    name=None,  
    **kwargs  
)
```

Fig. 2.6 AveragePooling2D layer configuration

```
tf.keras.layers.MaxPool2D(
    pool_size=(2, 2),
    strides=None,
    padding='valid',
    data_format=None,
    name=None,
    **kwargs
)
```

Fig. 2.7 MaxPool2D layer configuration

```
tf.keras.layers.GlobalAveragePooling2D(
    data_format=None,
    keepdims=False,
    **kwargs
)
```

Fig. 2.8 GlobalAveragePooling2D layer configuration

```
tf.keras.layers.GlobalMaxPool2D(
    data_format=None,
    keepdims=False,
    **kwargs
)
```

Fig. 2.9 GlobalMaxPool2D layer configuration

For `tf.keras.layers.AveragePooling2D` and `tf.keras.layers.MaxPool2D`, the parameters `padding`, `data_format`, and `strides` have the same configuration methods and data types as the parameters with the same names in `tf.keras.layers.Conv2D`. Similarly, the parameter `pool_size` is identical to `kernel_size` in its functionality. The `pool_size` specifies the size of the filter or kernel used in the pooling operation.

Additionally, a new parameter, `name`, is introduced. This parameter allows the user to assign a specific name to the pooling layer by setting its value. The un-default mode of these two-layer functions is represented on GitHub.

For `tf.keras.layers.GlobalAveragePooling2D` and `tf.keras.layers.GlobalMaxPool2D`, a new parameter, `keepdims`, is

introduced. This parameter is a boolean value that controls whether the height and width dimensions of the output are retained:

- If `keepdims=True`, the output shape retains the height and width dimensions with a size of 1. The output shape will be `(batch_size, 1, 1, channels)`.
- If `keepdims=False` (default), the height and width dimensions are compressed, and the output shape will be `(batch_size, channels)`.

2.3.1.5 Fully Connected Layer

In TensorFlow and Keras, fully connected layers are commonly referred to as dense layers, represented by the class `tf.keras.layers.Dense`. The default configuration is shown in Fig. [2.10](#).

```
tf.keras.layers.Dense(  
    units,  
    activation=None,  
    use_bias=True,  
    kernel_initializer='glorot_uniform',  
    bias_initializer='zeros',  
    kernel_regularizer=None,  
    bias_regularizer=None,  
    activity_regularizer=None,  
    kernel_constraint=None,  
    bias_constraint=None,  
    lora_rank=None,  
    **kwargs  
)
```

Fig. 2.10 Dense layer configuration

For `tf.keras.layers.Dense`, the following parameters share the same data type and configuration methods as in `tf.keras.layers.Conv2D`:

- `activation`
- `use_bias`
- `kernel_initializer`
- `bias_initializer`
- `kernel_regularizer`
- `bias_regularizer`
- `activity_regularizer`
- `kernel_constraint`
- `bias_constraint`

In addition, two new parameters are introduced:

- **units**: Specifies the dimensionality of the output space (i.e., the number of neurons). This parameter determines the size of the output tensor. For example, setting `units=128` indicates that the layer will output a tensor of size 128.
- **lora_rank**: A parameter used for low-rank decomposition (e.g., in LoRA techniques) for efficient training of large models. **Type:** `int` or `None`. **Default:** `None`. When set to a value other than `None`, low-rank decomposition is used to optimize weight updates, reducing the number of parameters and computational overhead.

An example of a non-default configuration for this layer is shown on GitHub.

2.3.2 Network Training and Evaluation

In the previous sections, we introduced how common layers in CNN networks are defined in TensorFlow. Next, we will demonstrate how to construct, train, and evaluate a custom network using the layer functions discussed above. Specifically, we will use MobileNet as a foundation to illustrate the process of training a CNN network, including both transfer learning and training from scratch.

2.3.2.1 Transfer Learning

Transfer learning refers to fine-tuning a pretrained model to quickly obtain a model that meets specific objectives. TensorFlow provides a wide range of pretrained models, offering an excellent foundation for implementing transfer learning. Next, we will use a pretrained model provided by TensorFlow to train our model on the CIFAR-10 dataset [6].

A. Load CIFAR-10 Dataset

The CIFAR-10 dataset consists of 60,000 32×32 RGB images divided into ten classes. We use `tf.keras.datasets.cifar10.load_data()` to load the dataset, splitting it into 50,000 training samples and 10,000 test samples. This step is shown in Fig. 2.11.

```
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()
```

Fig. 2.11 Load CIFAR-10 dataset

B. Data Preprocessing

Before retraining the network, the loaded data should be preprocessed. To prepare the data for MobileNet:

- The images are normalized using `preprocess_input` from `tensorflow.keras.applications.mobilenet`. The input pixels will be scaled between -1 and 1 .
- The labels are converted into a binary class matrix format using `tf.keras.utils.to_categorical`.

The detailed code is shown in Fig. [2.12](#).

```
x_train = tf.keras.applications.mobilenet.preprocess_input(x_train)
x_test = tf.keras.applications.mobilenet.preprocess_input(x_test)
y_train = tf.keras.utils.to_categorical(y_train, 10)
y_test = tf.keras.utils.to_categorical(y_test, 10)
```

Fig. 2.12 Data preprocessing

C. Load Pretrained MobileNet

Next, we need to load the pretrained MobileNet model. For loading the MobileNet model pretrained on ImageNet, we need to use:

- `weights='imagenet'`: Load weights trained on the ImageNet dataset.
- `include_top=False`: Exclude the top classification layers.
- `input_shape=(32, 32, 3)`: Match the input shape of CIFAR-10 images.

The base model's weights are frozen to retain the learned features during training, which means the weights of the convolutional layer will not be updated. The code of this part is shown in Fig. [2.13](#).

```
mobilenet_base = tf.keras.applications.MobileNet(
    weights='imagenet',
    include_top=False,
    input_shape=(32, 32, 3)
)
mobilenet_base.trainable = False
```

Fig. 2.13 Load pretrained MobileNet

D. Build the Custom Model

Based on the MobileNet base, we need to add a global average pooling layer and fully connected layers to fit the CIFAR-10 dataset:

- A `GlobalAveragePooling2D` layer to reduce spatial dimensions.

- A dense layer with 128 units and ReLU activation for high-level feature learning.
- An output layer with 10 units and Softmax activation for classification.

The detailed code is shown in Fig. [2.14](#).

```
model = tf.keras.models.Sequential([
    mobilenet_base,
    tf.keras.layers.GlobalAveragePooling2D(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
])
```

Fig. 2.14 Build the custom model

E. Compile the Model

After completing the above operations, the network's basic structure is established. Next, the model is compiled using the Adam optimizer, categorical cross-entropy loss, and accuracy as the evaluation metric. The code is shown in Fig. [2.15](#).

```
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

Fig. 2.15 Compile the model

F. Train the Model

After compilation, the model can be trained. The configuration of the training is: ten epochs with a batch size of 64. Reserve 20% of the training data for validation. The code is shown in Fig. [2.16](#).

```

history = model.fit(
    x_train, y_train,
    batch_size=64,
    epochs=10,
    validation_split=0.2,
    verbose=2
)

```

Fig. 2.16 Train the model

G. Evaluate the Model

After training the model, we need to utilize the test image sets to evaluate the model in terms of accuracy and loss. The code is shown in Fig. [2.17](#).

```

test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
print(f"Test accuracy: {test_acc:.4f}")
print(f"Test accuracy: {test_acc:.4f}")

```

Fig. 2.17 Evaluate the model

H. Make Predictions

Then, we can utilize `model.predict()` function to check the classification result. The code is shown in Fig. [2.18](#).

```

predictions = model.predict(x_test[:5])
print("Predicted classes:",
      tf.argmax(predictions, axis=1).numpy())

```

Fig. 2.18 Make predictions

2.3.2.2 Training from Scratch

In this section, we will introduce how to build our complex and deep CNN model from scratch without relying on transfer learning . We will still use the MobileNet architecture as the foundation and the CIFAR-10 dataset for demonstration. In addition, the code, which demonstrates how to build a deep CNN model from scratch using TensorFlow, has already been uploaded to GitHub.

A. Load CIFAR-10 Dataset

This step is the same as in the Transfer learning part, loading the CIFAR-10 images from TensorFlow.

B. Data Preprocessing

The pixel values of the images need to be converted into float32 format and then normalized to the range [0, 1], and the class labels need to be converted to a binary matrix format. The code of this step is shown in Fig. [2.19](#).

```
# load CIFAR-10 data
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()

# normalization+one-hot coding
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0
y_train = tf.keras.utils.to_categorical(y_train, 10)
y_test = tf.keras.utils.to_categorical(y_test, 10)
```

Fig. 2.19 Data preprocessing

C. Define a Depthwise Separable Convolution Block

Then, we will build a depthwise separable convolution block, which consists of:

- **DepthwiseConv2D**: Applies depthwise convolutions for spatial filtering.
- **BatchNormalization**: Normalizes activations to improve convergence.
- **ReLU**: Applies nonlinear activation.
- **Conv2D**: Implements pointwise convolutions to combine features.

The code is shown in Fig. [2.20](#).

```
def depthwise_separable_conv_block(x, filters, kernel_size=(3, 3), strides=(1, 1),
alpha=1.0):
    filters = int(filters * alpha)

    x = tf.keras.layers.DepthwiseConv2D(
        kernel_size=kernel_size,
        strides=strides,
        padding='same'
    )(x)
    x = tf.keras.layers.BatchNormalization()(x)
    x = tf.keras.layers.ReLU()(x)

    x = tf.keras.layers.Conv2D(
        filters,
        kernel_size=(1, 1),
        strides=(1, 1),
        padding='same'
    )(x)
    x = tf.keras.layers.BatchNormalization()(x)
    x = tf.keras.layers.ReLU()(x)

    return x
```

Fig. 2.20 Define depthwise separable convolution block

D. Build the MobileNet Model

Then, the MobileNet model is constructed using:

- An initial convolution layer with 32 filters.
- Multiple depthwise separable convolution blocks with varying filter sizes and strides.
- A global average pooling layer for dimensionality reduction.
- A fully connected layer with a softmax activation for classification.

The code is shown in Fig. [2.21](#).

```

def build_mobilenet(input_shape, num_classes, alpha=1.0):
    inputs = tf.keras.Input(shape=input_shape)

    x = tf.keras.layers.Conv2D(32, (3, 3), strides=(2, 2), padding='same')(inputs)
    x = tf.keras.layers.BatchNormalization()(x)
    x = tf.keras.layers.ReLU()(x)

    x = depthwise_separable_conv_block(x, 64, alpha=alpha)
    x = depthwise_separable_conv_block(x, 128, strides=(2, 2), alpha=alpha)
    x = depthwise_separable_conv_block(x, 128, alpha=alpha)
    x = depthwise_separable_conv_block(x, 256, strides=(2, 2), alpha=alpha)
    x = depthwise_separable_conv_block(x, 256, alpha=alpha)
    x = depthwise_separable_conv_block(x, 512, strides=(2, 2), alpha=alpha)

    for _ in range(5):
        x = depthwise_separable_conv_block(x, 512, alpha=alpha)

    x = depthwise_separable_conv_block(x, 1024, strides=(2, 2), alpha=alpha)
    x = depthwise_separable_conv_block(x, 1024, alpha=alpha)

    x = tf.keras.layers.GlobalAveragePooling2D()(x)
    outputs = tf.keras.layers.Dense(num_classes, activation='softmax')(x)

    return tf.keras.Model(inputs, outputs)

```

Fig. 2.21 Build MobileNet model

E. Compile, Train, Evaluate, and Predict the Model

They are the same as in the transfer learning; the model needs to be compiled by the `model.compile()` at first and then be trained by the `model.fit()`. After completing the training step, we can use `model.evaluate()` to evaluate the performance of the trained model and use `model.predict()` to check the inference result of the trained model. The detail of these steps is shown in [Fig. 2.22](#).

```

def main():
    # load CIFAR-10 data
    (x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()

    # normalization+one-hot coding
    x_train = x_train.astype('float32') / 255.0
    x_test = x_test.astype('float32') / 255.0
    y_train = tf.keras.utils.to_categorical(y_train, 10)
    y_test = tf.keras.utils.to_categorical(y_test, 10)
    # build model
    model = build_mobilenet(input_shape=(32, 32, 3), num_classes=10, alpha=1.0)
    model.summary()

    # compile model
    model.compile(optimizer='adam',
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])

    # train model
    model.fit(x_train, y_train,
              validation_data=(x_test, y_test),
              epochs=10,
              batch_size=64)

    # test model
    test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
    print(f'\nTest accuracy: {test_acc:.4f}')

if __name__ == '__main__':
    main()

```

Fig. 2.22 Compile, train, evaluate, and predict the model

2.4 Hardware-Level CNN Model Implementation on System Verilog

In this section, we will provide a detailed tutorial to guide new CNN hardware developers in quickly developing their first CNN hardware accelerator. This article will show how to use System Verilog to build a customized small CNN hardware accelerator digital logic circuit.

This section will introduce the preparation step and divide a CNN hardware accelerator into smaller modules. These modules include the data reading module, the convolution layer module, the pooling layer module, the fully connected layer

module, the activation layer module, and the classification module. The overview structure of the following CNN hardware accelerator is shown in Fig. 2.23.

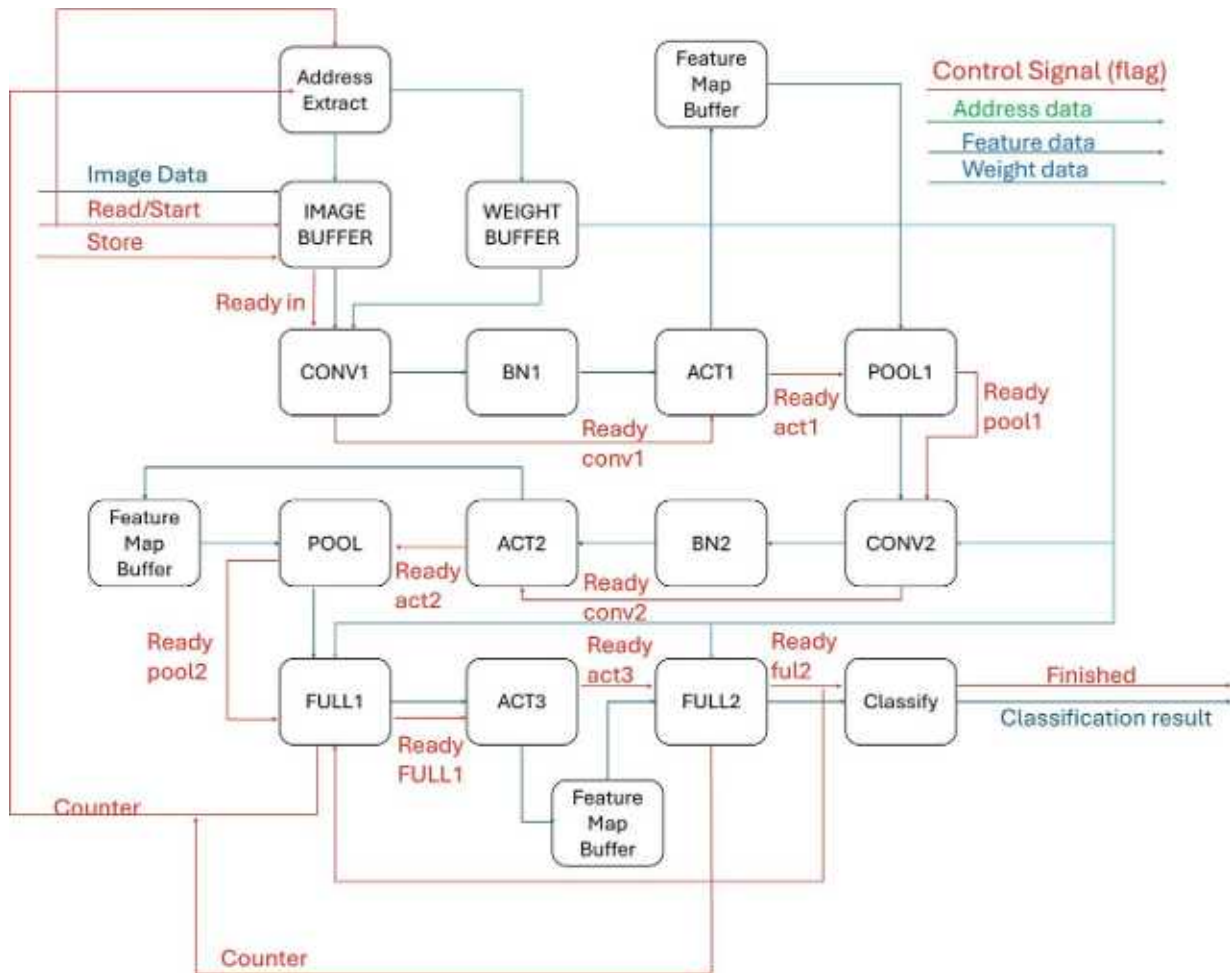


Fig. 2.23 The structure of a sample CNN hardware accelerator

2.4.1 Preparation

The preparation step can be regarded as a fixed-point quantization process, as illustrated in Fig. 2.24. This process is applied to all data utilized within the CNN hardware accelerator. The overall workflow of dynamic fixed-point quantization during CNN inference is as follows: a set of test images is fed into the CNN model to perform inference, during which the output feature maps of each layer are captured. For every layer, the maximum absolute value is calculated across all feature maps, weights, and biases. The integer bit width is then determined based on the number of bits required to represent the integer part of this maximum value (including the sign bit), while the remaining bits, determined by the total bit width supported by the hardware, are allocated to the fractional part. Using this fixed-point format, all weights and biases are quantized and exported in binary form for

deployment on hardware. Additionally, the pixel data to be fed into the CNN hardware accelerator must also be converted into this fixed-point binary format. It is important to note that the pixel data should be stored in memory in a column-by-column sequence.

```
Pseudocode: Dynamic Fixed-Point Quantization for CNN Inference

Inputs:
- test_images[]           // Test images dataset
- model_layers[]          // CNN layers, each with feature map, weights, and
bias
- BIT_WIDTH = 16          // Total bit width for fixed-point

For each layer in model_layers:
    max_abs = 0

    // 1. Run inference for all test images and update max absolute value for this
    layer
    For each test_image in test_images:
        feature = inference(test_image, model_layers, layer) // Get current
        layer's feature map
        max_abs = max(max_abs, maximum absolute value of feature)

    max_abs = max(max_abs, maximum absolute value of layer.weights, maximum
    absolute value of layer.bias)

    // 2. Determine integer and fractional bit widths
    int_width = number of bits required for floor(max_abs) + 1 // Include sign
    bit
    frac_width = BIT_WIDTH - int_width

    // 3. Quantize and export weights and bias in fixed-point binary
    quantized_weights = quantize layer.weights to (int_width, frac_width)
    quantized_biases = quantize layer.bias to (int_width, frac_width)
    save quantized_weights to "layerN_weights.bin"
    save quantized_biases to "layerN_bias.bin"
```

Fig. 2.24 Preparation step: Fixed-point quantization

2.4.2 Data Reading Module

The data reading module can be divided into three parts: the address extraction module, the image read module(image buffer), and the weight extraction module.

2.4.2.1 Address Extraction Module

Figure [2.25](#) presents the pseudocode for the address extraction module, which determines the required data addresses for input images in subsequent

computations. To facilitate efficient fusion of adjacent layers, the address extraction sequence is carefully designed such that each time a block of $conv1_kernel_size \times conv1_kernel_size$ pixels is extracted, the first convolutional layer produces one output feature. Likewise, whenever the first convolutional layer generates a block of $pool1_kernel_size \times pool1_kernel_size$ features, the first pooling layer yields one pooled feature. This pattern continues: each time the first pooling layer outputs a $conv2_kernel_size \times conv2_kernel_size$ block, the second convolutional layer computes one feature; and for every $pool2_kernel_size \times pool2_kernel_size$ features generated by the second convolutional layer, the second pooling layer produces a single output.

```

Module address_extract

Input:
- start          // Control signal to begin address generation

Output:
- address        // Current pixel address for processing

Internal Parameters:
- image_size     // Width of the input image
- kernel_size1   // Size of the first convolution kernel
- pool_size1     // Size of the first pooling window
- kernel_size2   // Size of the second convolution kernel
- pool_size2     // Size of the second pooling window

Internal Variables:
- row_out2       // Row index in the output feature map after the second
pooling layer
- col_out2       // Column index in the output feature map after the second
pooling layer
- i, j           // Row/column indices within the first convolution window
- i_p, j_p       // Row/column indices within the first pooling window
- i_c, j_c       // Row/column indices within the second convolution window
- i_p2, j_p2     // Row/column indices within the second pooling window

Main Logic:
  While start == 1:
    For row_out2 in 1 .. kernel_size2:
      For col_out2 in 1 .. kernel_size2:
        For i_p2 in 1 .. pool_size2:
          For j_p2 in 1 .. pool_size2:
            For i_c in 1 .. kernel_size2:
              For j_c in 1 .. kernel_size2:
                For i_p in 1 .. pool_size1:
                  For j_p in 1 .. pool_size1:
                    For i in 1 .. kernel_size1:
                      For j in 1 .. kernel_size1:
                        // Calculate the memory address of
the current pixel

                        address = j
                        + (j_p - 1)
                        + pool_size1 * (
                          (j_c - 1)
                          + (j_p2 - 1)
                          + pool_size2 * (col_out2 -
1)
                        )
                        + image_size * (
                          i
                          + (i_p - 1)
                          + pool_size1 * (
                            (i_c - 1)
                            + (i_p2 - 1)
                            + pool_size2 *
(row_out2 - 1)
                          )
                        - 1
                      )
                    // Output or use the computed
address as needed

```

Fig. 2.25 Module: Address extract

To achieve this, we use the indices of the output feature map produced by the second pooling layer as the basis to back-calculate the pixel addresses needed for extraction. The pseudocode in Fig. 2.25 details this operation. The main parameters include the input image width (`image_size`), the kernel sizes for both convolution layers (`kernel_size1`, `kernel_size2`), and the pooling window sizes (`pool_size1`, `pool_size2`). The variables `row_out2` and `col_out2` represent the indices of the output feature map after the second pooling layer, while other indices (`i`, `j`, `i_p`, `j_p`, `i_c`, `j_c`, `i_p2`, `j_p2`) enumerate positions within their respective kernel or pooling windows.

The address for each pixel is then calculated using a linear indexing formula, as shown in Eq. 2.1, which uniquely maps the multidimensional loop variables to a one-dimensional address within the flattened image or feature map:

$$\begin{aligned}
 J &= j + (j_p - 1) + \text{pool_size1} \times [(j_c - 1) + (j_{p2} - 1) + \\
 &\quad \text{pool_size2} \times (\text{col_out2} - 1)] \\
 I &= i + (i_p - 1) + \text{pool_size1} \times [(i_c - 1) + (i_{p2} - 1) + \\
 &\quad \text{pool_size2} \times (\text{row_out2} - 1)] \\
 \text{address} &= J + \text{image_size} \cdot (I - 1)
 \end{aligned} \tag{2.1}$$

This hierarchical and parameterized address generation scheme enables support for varied kernel and pooling sizes without modification. Such an approach is particularly advantageous for pipelined processing in CNN hardware accelerators.

2.4.2.2 Image Read Module

The calculated address from Fig. 2.25 is provided to the image read Module, which retrieves the corresponding pixel data from memory. In addition to supplying data for subsequent computations, the image read module also serves as a buffer for storing input pixel data from external sources. The module stores incoming pixel data into the buffer when the input flag `store` is set. Conversely, when `store` is 0, `read` is set, and the internal counter reaches `image_size` $\times$ `image_size`, the module fetches pixel data based on the input address and supplies it to downstream processing. The detailed working principle of this module is shown in Fig. 2.26

```

Module image_read

Inputs:
- store      // Write enable flag for incoming data
- read       // Read enable flag for supplying data
- address    // Input address for pixel retrieval
- data_in    // Pixel data from external source

Output:
- data_out   // Pixel data supplied to downstream processing

Parameters:
- image_size // Width (and height) of the image

Internal Variables:
- buffer[image_size * image_size] // Buffer for image data
- counter                        // Tracks number of pixels written
- store_last                     // Tracks previous value of store

Initialization:
counter = 0
store_last = 0

Main Logic (every clock cycle):
If store == 1 and store_last == 0:           // Detect rising edge of store
    counter = 0                             // Reset counter on first store
If store == 1:
    buffer[counter] = data_in
    counter = counter + 1
Else if store == 0 and read == 1 and counter == image_size * image_size:
    data_out = buffer[address]
Else:
    data_out = 0 // Or hold previous value

store_last = store // Update store_last for edge detection

```

Fig. 2.26 Module: Image read

2.4.2.3 Extracting Weight

1. Module Definition of Kernel_Conv

The `kernel_conv1` module, as shown in Fig. 2.27, is responsible for managing and outputting the weights and biases required by the first convolutional layer of the accelerator. Upon initialization or reset, the module loads all kernel weights and bias values for each output channel from external files into internal buffers.

```

Module kernel_conv1

Inputs:
- start          // Start signal for convolution weight loading
- n_reset        // Active-low reset
- clk            // Clock signal

Outputs:
- weight_conv1[kernel_num1] // Weights for each output channel
- bias_conv1[kernel_num1]   // Bias for each output channel

Parameters:
- kernel_size1 // Kernel width/height (assumed square)
- kernel_num1  // Number of output feature maps (output channels)

Internal Variables:
- count        // Index for traversing kernel values
- kernel[kernel_num1][kernel_size1 * kernel_size1] // 2D array for all kernels
- bias[kernel_num1] // Bias values
- weight[kernel_num1] // Current weight values

Initialization:
- Load kernel weights for each output channel from external files
- Load bias values for each output channel from external file

Main Logic (on each rising edge of clk or falling edge of n_reset):
If n_reset == 0:
    count = 0
    weight = {0, 0, ..., 0}
Else if start == 1:
    If count == kernel_size1 * kernel_size1:
        count = 1
    Else:
        count = count + 1
    For channel in 1 .. kernel_num1:
        weight[channel] = kernel[channel][kernel_size1 * kernel_size1 - count]
Else:
    count = 0
    weight = {0, 0, ..., 0}

Combinational Output:
If start == 1:
    For channel in 1 .. kernel_num1:
        weight_conv1[channel] = weight[channel]
        bias_conv1[channel] = bias[channel]
Else:
    weight_conv1 = {0, 0, ..., 0}
    bias_conv1 = {0, 0, ..., 0}

```

Fig. 2.27 Module: Buffer of weight for the first convolutional layer

During operation, when the `start` signal is set, the module sequentially outputs the kernel weights for each output channel. It uses an internal counter to iterate through all weight values associated with each kernel. For each clock cycle with `start` active, the counter increments, and the corresponding weight value for each output channel is provided at the output. If the counter reaches the maximum kernel size ($\text{kernel_size1} \times \text{kernel_size1}$), it wraps around and restarts from the first weight value. The associated bias for each output channel is also provided at the output in parallel with the weights.

If either the module is not in the active state (`start` unset) or a reset is triggered (`n_reset` is low), the output weights and biases are cleared to zero, and the internal counter is reset. This ensures that weight streaming only occurs during the appropriate active periods and is robust to resets. In addition, the `kernel_conv2` module, as shown in Fig. [2.28](#), is responsible for managing and streaming the weights and biases required by the second convolutional layer in the accelerator. At initialization or resetting, the module loads all kernel weights and bias values for each output channel from external files into internal buffers. The kernel weights are organized such that each output channel contains a three-dimensional weight tensor covering all input channels and all spatial positions within the kernel.

```

Module kernel_conv2

Inputs:
- start                // Start signal for weight loading
- n_reset              // Active-low reset
- clk                  // Clock signal
- ready                // Ready flag for synchronizing input

Outputs:
- weight_conv2[out_ch][in_ch]  // 2D array: weights for output channel
out_ch, input channel in_ch
- bias_conv2[out_ch]           // Bias for each output channel

Parameters:
- kernel_size2            // Size of the convolution kernel (assume square)
- kernel_num2             // Number of output feature maps (output channels)
- kernel_channel2         // Number of input feature maps (input channels)

Internal Variables:
- count                   // Tracks position within the kernel
- kernel[kernel_num2][kernel_size2 * kernel_size2 * kernel_channel2] // All
weights
- bias[kernel_num2]       // All
biases

Initialization:
- Load kernel weights for each output channel from external files
- Load bias values for each output channel from external file

Main Logic (on each rising edge of clk or falling edge of n_reset):
If n_reset == 0:
    count = 1
    For out_ch in 1 .. kernel_num2:
        For in_ch in 1 .. kernel_channel2:
            weight_conv2[out_ch][in_ch] = 0
Else if start == 1 and ready == 1:
    If count == kernel_size2 * kernel_size2:
        count = 1
    Else:
        count = count + 1
Else:
    count = count

Combinational Output:
For out_ch in 1 .. kernel_num2:
    For in_ch in 1 .. kernel_channel2:
        If start == 1 and ready == 1:
            // Each output channel has a kernel, each kernel slices for each
input channel
            // For example: kernel[out_ch][total_offset - (in_ch-
1)*kernel_size2*kernel_size2 - count]
            weight_conv2[out_ch][in_ch] =
                kernel[out_ch][kernel_size2 * kernel_size2 * kernel_channel2
- (in_ch-1)*kernel_size2*kernel_size2
- count]
        Else:
            weight_conv2[out_ch][in_ch] = 0
    For out_ch in 1 .. kernel_num2:
        bias_conv2[out_ch] = bias[out_ch]

```

Fig. 2.28 Module: Buffer of weight for the second convolutional layer

During normal operation, when both the `start` and `ready` signals are set, the module sequentially outputs the kernel weights for every combination of output and input channels. An internal counter tracks the current position within the kernel window for each channel pair. On each clock cycle with `start` and `ready` active, the counter increments and extracts the weights from the internal buffers based on the counter value. If the counter reaches the kernel window size ($\text{kernel_size2} \times \text{kernel_size2}$), it wraps around and begins a new cycle. The corresponding bias value for each output channel is provided in parallel with the weights.

If a reset (`n_reset`) occurs, the internal counter is reset, and all output weights are cleared to zero, ensuring a well-defined initial state. If `start` or `ready` is not set, the module maintains its current state, and the output weights remain at zero.

2. Module Definition of WEIGHT_FULL

The `weight_full` module, as shown in Fig. [2.29](#), is responsible for providing the weights to the current computation in the first FC layer. All weights are organized as a one-dimensional array, which is initialized by loading data from an external file (such as `weight_full.txt`). The module receives as inputs the index of the current computation-related output neuron (`count_full`) and the index of the current computation-related input neuron (`count_finish`). The main configuration parameters include the number of input channels (`input_channels`), the number of input neurons per channel (`input_neurons`), the total number of output neurons (`output_neurons`), and the total number of weights (`total_weights` = `input_channels` $\times$ `input_neurons` $\times$ `output_neurons`).

```

Module weight_full1

Inputs:
- count_full      // Index of the current output neuron (1..output_neurons)
- count_finish    // Index of the current input neuron (1..input_neurons)

Output:
- weight_full1[input_channels] // Weights for all input channels for the
current FC connection

Parameters:
- input_channels    // Number of input channels
- input_neurons     // Number of input neurons per channel
- output_neurons    // Number of output neurons
- total_weights = input_channels * input_neurons * output_neurons

Internal Variables:
- weight[total_weights:0] // Flat array holding all FC weights

Initialization:
- Load weight data from "weight_full.txt" into weight array.
- weight[total_weights] = 0

Combinational Logic:
If count_full == 0:
    For ch in 1 .. input_channels:
        weight_full1[ch] = 0
Else:
    For ch in 1 .. input_channels:
        // Each input channel offset by (ch-1) * input_neurons *
output_neurons
        base = total_weights - (ch-1)*input_neurons*output_neurons
        address = base - (count_finish-1)*output_neurons - count_full
        weight_full1[ch] = weight[address]

```

Fig. 2.29 Module: Buffer of weight for the first fully connected layer

During operation, if `count_full` = 0, the output weights for all input channels (`weight_full1`) are set to zero. Otherwise, for each input channel *ch*, the module calculates a base address using,

$$\text{base} = \text{total_weights} - (ch-1) \times \text{input_neurons} \times \text{output_neurons}$$
and then computes the full address as

$$\text{address} = \text{base} - (\text{count_finish}-1) \times \text{output_neurons} - \text{count_full}$$
The output weight for channel *ch* is then given by

`weight_full1[ch] = weight[address]`

The `weight_full2` module, as shown in Fig. [2.30](#), is responsible for providing the appropriate weights for the calculation in the second fully connected layer. All weights are stored in a one-dimensional array, which is initialized by loading data from an external file (such as `weight_full2.txt`). During operation, the module receives the indices of the current input neuron (`count_2`) and output neuron (`count_1`), as well as an enable flag (`flag_full2`). When the enable flag is not set, the output weight is set to zero. When the enable flag is set, the module computes the corresponding address in the flattened weight array according to the formula

`address = total_weights - count_1 - output_neurons × (count_2 - 1)`
, where `total_weights = input_neurons × output_neurons`. The corresponding value from the weight array is then provided at the output.

```

Module weight_ful2

Inputs:
- count_2      // Index of current input neuron for the second FC layer
(1..input_neurons)
- count_1      // Index of current output neuron for the second FC layer
(0..output_neurons-1)
- flag_ful2    // Enable flag for weight output

Output:
- weight_ful2  // Weight for the specified FC2 connection

Parameters:
- input_neurons  // Number of input neurons
- output_neurons // Number of output neurons
- total_weights = input_neurons * output_neurons

Internal Variables:
- weight[total_weights:0] // Flat array holding all FC2 weights

Initialization:
- Load weight data from "weight_ful2.txt" into weight array.
- weight[total_weights] = 0

Combinational Logic:
If flag_ful2 == 0:
    weight_ful2 = 0
Else:
    // Compute address in flattened array
    // Address = total_weights - count_1 - output_neurons * (count_2 - 1)
    address = total_weights - count_1 - output_neurons * (count_2 - 1)
    weight_ful2 = weight[address]

```

Fig. 2.30 Module: Buffer of weight for the second fully connected layer

2.4.3 Convolution Layer Module

This part will introduce the basic element of the convolution layer, the convolutional processing element, and how to combine multiple convolution processing elements to build a convolutional layer module.

2.4.3.1 Convolution Processing Element

The conv_pe module, as shown in Fig. 2.31, implements a multichannel convolution PE that computes the output feature value for one output channel. The main parameters are the kernel size (kernel_size) and the number of input channels (input_channel).

```

Module conv_pe

Parameters:
- kernel_size      // Total number of multiply-accumulate operations per PE
- input_channel    // Number of input channels

Inputs:
- input_featuremap[input_channel] // Current input feature values for all
channels
- weight[input_channel]           // Corresponding weights for all input
channels
- bias                           // Bias for this output channel
- n_reset, clk                   // Active-low reset and clock signals
- start                         // Start signal for computation
- ready_pool                 // Handshake signal from pool layer (input
data available)

Outputs:
- output_featuremap           // Computed output feature for this PE
- flag                        // Flag indicating computation complete

Internal Variables:
- sum                        // Accumulator for convolution result
- counter                    // Counts number of computation steps

Main Logic (on each clock cycle or off n_reset):
If n_reset == 0:
    counter = 0
    sum = 0
Else if start == 1:
    If counter == kernel_size:
        counter = 0
        sum = 0
    Else:
        If ready_pool == 1:
            counter = counter + 1
            sum = sum + (for i in 1 .. input_channel: input_featuremap[i] *
weight[i])
        Else:
            counter = counter // Hold
            sum = sum // Hold
    Else:
        counter = 0
        sum = 0

Combinational Output:
If counter == kernel_size:
    flag = 1 // Computation done for one output value
    output_featuremap = sum + bias
Else:
    flag = 0
    output_featuremap = 0

```

Fig. 2.31 Module: Convolutional processing element

Inputs to the module include the current input feature values for all channels (`input_featuremap [input_channel]`), the corresponding weights (`weight [input_channel]`), a bias term (`bias`), the active-low reset and clock signals (`n_reset`, `clk`), a start control signal (`start`), and a handshake signal from the pooling layer (`ready_pool`) indicating that valid input data is available.

Internally, the module uses an accumulator (`sum`) to store the partial convolution result and a counter (`counter`) to track the number of multiply-accumulate operations performed. On each clock cycle, if reset is triggered, the accumulator and counter are cleared. When computation starts (`start` is high), if the counter reaches the kernel size, the module resets the counter and accumulator to begin a new computation window. Otherwise, when `ready` is set, the counter is incremented and the accumulator is updated as follows:

$$\text{sum} = \text{sum} + \sum_{i=1}^{\text{input_channel}} \text{input_featuremap}[i] \times \text{weight}[i]$$

If `ready` is not set, both the counter and accumulator hold their current values.

The combinational output logic assigns the completion flag (`flag`) to 1 and outputs the computed feature (`output_featuremap`) when the counter reaches the kernel size, where the output is given by the accumulated sum plus bias:

$$\text{output_featuremap} = \text{sum} + \text{bias}$$

Otherwise, the flag is not set, and the output is set to zero.

2.4.3.2 Convolution Layer

The `conv_layer` module, as shown in Fig. [2.32](#), implements a parallel convolution computation for a multichannel input using multiple PEs. It receives as inputs the current input feature value (`input_feature`), a set of weights and biases for each output channel (`weight[num_channels]` and `bias[num_channels]`), clock and reset signals (`clk`, `n_reset`), a start control signal (`start`), and a handshake input (`ready_in`) from the preceding image read module to indicate data availability.

```

Module conv_layer

Inputs:
  - input_feature           // Current input feature value (shared for all
  PEs)
  - weight[num_channels]    // Weights for each convolution kernel (per
  output channel)
  - bias[num_channels]      // Bias values for each output channel
  - clk, n_reset            // Clock and active-low reset signals
  - start                   // Start signal to begin convolution
  computation
  - ready_in                // Handshake signal from Image Read module.
                           // Indicates input feature data is available
  for processing.

Outputs:
  - output_feature[num_channels] // Output feature values from each PE (one per
  output channel)
  - ready                     // Output flag. Asserted when all PEs have
  produced new feature data.

Parameters:
  - num_channels    // Number of convolution kernels/output channels
  - kernel_size     // Size of each convolution kernel (assumed square)

Internal Variables:
  - feature[num_channels] // Results from each PE
  - flag[num_channels]    // Done flag from each PE

// Instantiate num_channels convolution processing elements (PEs)
// Each PE computes the convolution for one output channel.
For ch in 1 .. num_channels:
  conv_pe #(kernel_size * kernel_size) pe (
    .input_featuremap(input_feature),
    .weight(weight[ch]),
    .bias(bias[ch]),
    .clk(clk),
    .n_reset(n_reset),
    .start(start),
    .ready_in(ready_in),
    .output_featuremap(feature[ch]),
    .flag(flag[ch])
  )

// Output logic: Synchronize data output with PE completion
Combinational Output Logic:
  If all flag[ch] == 1 (for ch = 1..num_channels):
    For ch in 1 .. num_channels:
      output_feature[ch] = feature[ch]
    ready = 1 // All PEs have produced new feature data, output is valid
  Else if any flag[ch] == 1:
    // Error: Partial computation detected, which should not occur in correct
    operation
    Display "Running Error"
    ready = 0
  Else:
    For ch in 1 .. num_channels:
      output_feature[ch] = 0
    ready = 0 // Computation not yet started or all PEs idle

```

Fig. 2.32 Module: First convolutional layer

The number of output channels is given by the parameter `num_channels`, and each kernel is assumed to be square with size `kernel_size`.

Internally, the module instantiates `num_channels` convolution PEs (`conv_pe`), each responsible for computing the convolution operation for one output channel. For each channel ch ($1 \leq ch \leq \text{num_channels}$), the PE is provided with the current input feature, the corresponding weight and bias, and control signals, and produces an output feature value (`feature[ch]`) along with a done flag (`flag[ch]`).

After all PEs have finished their computations (i.e., all `flag[ch]` are set and every PEs generates a new feature data), the module outputs the results from each PE on `output_feature[ch]` and sets the ready signal (`ready`) to high, indicating that valid output data is available. If only a subset of `flag[ch]` signals are set, this indicates an error condition, and the module outputs an error message and sets `ready` to low. If none of the PEs have finished, the outputs are held at zero, and `ready` remains low.

The `conv_layer2` module, as shown in Fig. [2.33](#), implements a parallel multichannel convolution operation for a set of input feature maps. The inputs include the current input feature map values (`input_feature[in_channels]`), a flattened array of weights for all convolution kernels (`weight[kernel_num*in_channels]`), and bias values for each output channel (`bias[kernel_num]`). The module is synchronized by clock and reset signals (`clk`, `n_reset`), a start signal (`start`), and a handshake signal from the preceding pooling layer (`ready_pool`) to indicate when the input data is ready for processing.

```

Module conv_layer2

Inputs:
- input_feature[in_channels]    // Input feature map, with in_channels per
data point
- weight[kernel_num * in_channels] // Flattened weights for all kernels
- bias[kernel_num]              // Bias values, one per kernel/output channel
- clk, n_reset                  // Clock and active-low reset signals
- start                          // Start signal for convolution computation
- ready_pool                    // Handshake signal from previous (pool)
layer.
                                // Indicates input feature maps are available
                                and processing can start.

Outputs:
- output_feature[kernel_num]    // Output feature map for each kernel/output
channel
- ready                          // Flag asserted when all kernels have
finished computation and output data is valid

Parameters:
- in_channels                    // Number of input feature map channels
- kernel_num                    // Number of kernels/output channels
- kernel_size                    // Size of each convolution kernel (assumed square)

Internal Variables:
- feature[kernel_num]           // Feature result from each PE
- flag[kernel_num]              // Done flag from each PE

// Instantiate kernel_num convolution processing elements (PEs), each with
in_channels inputs
For k in 1 .. kernel_num:
    // Each PE gets its own slice of weights and corresponding bias
    conv_pe2 #(kernel_size * kernel_size) pe (
        .input_featuremap(input_feature[1:in_channels]),
        .weight(weight[(k*in_channels):(k-1)*in_channels+1]),
        .bias(bias[k]),
        .clk(clk),
        .n_reset(n_reset),
        .start(start),
        .ready_pool(ready_pool),
        .output_featuremap(feature[k]),
        .flag(flag[k])
    )

Combinational Output Logic:
If all flag[k] == 1 (for k = 1..kernel_num):
    For k in 1 .. kernel_num:
        output_feature[k] = feature[k]
    ready = 1 // All PEs have finished, output is valid
Else if any flag[k] == 1:
    // Error: Inconsistent state, only part of the PEs are done
    Display "Running Error"
    ready = 0
Else:
    For k in 1 .. kernel_num:
        output_feature[k] = 0
    ready = 0 // Idle or reset state

```

Fig. 2.33 Module: Second convolutional layer

The main parameters are the number of input channels (`in_channels`), the number of output channels or kernels (`kernel_num`), and the size of each convolution kernel (`kernel_size`, assumed square).

Internally, the module instantiates `kernel_num` convolution processing elements (`conv_pe`), each receiving its slice of the weights and the corresponding bias. For kernel k ($1 \leq k \leq \text{kernel_num}$), the PE is connected to the complete set of input channels, receives the relevant weights (`weight` indexed for kernel k), and produces an output feature value (`feature[k]`) along with a completion flag (`flag[k]`).

When all completion flags (`flag[k]`) are set, the module outputs the results from all PEs on `output_feature[k]` and sets the ready flag (`ready`) to indicate valid output data. If only a subset of completion flags are set, this indicates an error, and the module sets `ready` low and displays a runtime error message. If none of the PEs have completed, all output features are held at zero, and `ready` remains low.

2.4.4 Pooling Layer Module

This part will introduce the basic element of the MaxPooling layer, the MaxPooling processing element, and how to combine multiple MaxPooling processing elements to build a MaxPooling layer module.

2.4.4.1 MaxPooling Processing Element

The `pool_pe` module, as shown in Fig. [2.34](#), implements a processing element for max pooling over a fixed-size window. The main parameter is the pooling window size (`pool_size`).

```

Module pool_pe

Parameters:
- pool_kernel_size    // Size of pooling window
Inputs:
- input_featuremap     // Input value for current window position
- clk, n_reset         // Clock and active-low reset
- start                // Start signal for pooling computation
- enable               // Enable signal, advances pooling operation

Outputs:
- output_featuremap    // Pooled result (maximum value in window)
- flag                 // Flag indicating pooling is complete

Internal Variables:
- max_value            // Stores current maximum value in pooling window
- counter              // Counts number of values processed (pooling window
size)

Main Logic (on each clock edge or off n_reset):
If n_reset == 0:
    max_value = 0
    counter = 0
Else:
    If start == 1:
        If counter == pool_kernel_size:
            counter = 0
            max_value = input_featuremap // Start next window with current
input
        Else:
            If enable == 1:
                counter = counter + 1
                If input_featuremap >= max_value:
                    max_value = input_featuremap
                Else:
                    max_value = max_value // Hold
            Else:
                counter = counter // Hold
                max_value = max_value // Hold
        Else:
            max_value = 0
            counter = 0

Combinational Output:
If counter == pool_kernel_size:
    flag = 1 // Pooling complete for current window
    output_featuremap = max_value
Else:
    flag = 0
    output_featuremap = 0

```

Fig. 2.34 Module: Max pooling processing element

Inputs to the module include the input value for the current window position (`input_featuremap`), clock and active-low reset signals (`clk`, `n_reset`), a start signal (`start`) to initiate pooling, and an enable signal (`enable`) to advance the pooling operation.

Internally, the module maintains a register (`max_value`) that tracks the maximum value in the current pooling window and a counter (`counter`) to track the number of values processed. If reset is triggered, both registers are cleared. During operation, if `start` is high and the counter equals the pooling window size, the module resets the counter and initializes `max_value` to the current input value (starting a new window). Otherwise, when `enable` is high, the counter is incremented. The module compares the input value to `max_value`, updating it if the new value is larger.

The combinational output logic assigns the completion flag (`flag`) to high and outputs the maximum value (`output_featuremap`) when the pooling window is complete (i.e., `counter` equals `pool_size`). Otherwise, the flag is low, and the output is set to zero.

2.4.4.2 MaxPooling Layer

The `pool_layer` module, as shown in Fig. [2.35](#), implements a parallel pooling operation over a multichannel input feature map. It receives as inputs the current feature values from the previous layer (`input_feature[num_channels]`), clock and active-low reset signals (`clk`, `n_reset`), a start signal (`start`) to initiate pooling, and an enable signal (`ready`) indicating that input data is available.

```

Module pool_layer

Inputs:
- input_feature[num_channels] // Input feature map from previous layer
- clk, n_reset                // Clock and active-low reset
- start                       // Start signal for pooling operation
- ready                       // Enable signal indicating input data is
available

Outputs:
- output_feature[num_channels] // Pooled feature map for each channel
- ready_pool                   // Output flag asserted when all pooling
elements are done

Parameters:
- num_channels // Number of input/output channels

Internal Variables:
- feature[num_channels] // Output from each pool_pe
- flag[num_channels]    // Completion flag from each pool_pe

// Instantiate num_channels pooling processing elements (PEs)
For ch in 1 .. num_channels:
    pool_pe pe (
        .input_featuremap(input_feature[ch]),
        .clk(clk),
        .n_reset(n_reset),
        .enable(ready),
        .start(start),
        .output_featuremap(feature[ch]),
        .flag(flag[ch])
    )

Combinational Output Logic:
If all flag[ch] == 1 (for ch = 1..num_channels):
    For ch in 1 .. num_channels:
        output_feature[ch] = feature[ch]
    ready_pool = 1 // All pool PEs have completed
Else if any flag[ch] == 1:
    // Error: Inconsistent state, only part of the PEs are done
    Display "Running Error"
    output_feature = undefined // (or X state)
    ready_pool = 0
Else:
    For ch in 1 .. num_channels:
        output_feature[ch] = 0
    ready_pool = 0 // Idle or reset state

```

Fig. 2.35 Module: Max pooling layer

The module parameter `num_channels` defines the number of channels processed in parallel. Internally, the module instantiates `num_channels` pooling

processing elements (`pool_pe`), each operating independently on its corresponding input channel. For each channel ch ($1 \leq ch \leq \text{num_channels}$), the PE receives the relevant feature map value, clock and reset, and control signals and produces an output feature (`feature[ch]`) and a completion flag (`flag[ch]`).

The combinational output logic monitors the completion flags. When all `flag[ch]` signals are set, indicating that all pooling elements have finished processing, the module outputs the results from all PEs on `output_feature[ch]` and sets `ready_pool` to high to indicate valid output data. If only a subset of the completion flags are high, this represents an error state; in this case, the module outputs an error message, the output feature is undefined, and `ready_pool` is set low. If none of the PEs have completed, all output features are held at zero, and `ready_pool` remains low.

2.4.5 Activation Layer Module

This part will introduce the basic element of the ReLU6 layer, how the ReLU6 processing element works, and how to combine multiple ReLU6 processing elements to build a ReLU6 layer module.

2.4.5.1 ReLU6 Processing Element

The `Relu6_pe` module, as shown in Fig. [2.36](#), implements the ReLU6 activation function for a given input. The main parameter is the upper bound for the ReLU6 activation (`relu6_upper_bound`), which represents the value 6.0 in the system's fixed-point data format.

```

Module Relu6_pe

Parameters:
- relu6_upper_bound // Upper bound for RelU6 activation (e.g., 6144,
representing 6.0 in fixed-point)

Inputs:
- input_featuremap // Input value to be activated
- clk, n_reset     // Clock and active-low reset
- start           // Start signal for activation
- enable          // Enable signal

Outputs:
- output_featuremap // Activated value after applying RelU6
- flag              // Output valid flag

Main Logic (on each clock edge or off n_reset):
If n_reset == 0:
    output_featuremap = 0
    flag = 0
Else:
    If start == 0:
        output_featuremap = 0
        flag = 0
    Else:
        flag = enable
        If input_featuremap > 0:
            If input_featuremap < relu6_upper_bound:
                output_featuremap = input_featuremap
            Else:
                output_featuremap = relu6_upper_bound
        Else:
            output_featuremap = 0

```

Fig. 2.36 Module: ReLU6 processing element

Inputs include the value to be activated (`input_featuremap`), clock and active-low reset signals (`clk`, `n_reset`), a start control signal (`start`), and an enable signal (`enable`).

On each clock edge or falling edge of `n_reset`, the following logic is executed. If reset is triggered (`n_reset` is low), the output (`output_featuremap`) and the output flag (`flag`) are cleared. If the start signal is not high, both outputs remain cleared. Otherwise, the flag is set according to the value of `enable`, and the module computes the ReLU6 activation as follows: if `input_featuremap` is positive and less than `relu6_upper_bound`, the output is set to the input value. If the input exceeds the upper bound, the output is saturated to `relu6_upper_bound`. If the input is negative or zero, the output is set to zero.

2.4.5.2 ReLU6 Layer

The `Relu6_layer` module, as shown in Fig. [2.37](#), applies the ReLU6 activation function in parallel to each channel of an input feature map. The inputs include the feature values for all channels (`input_feature[num_channels]`), clock and active-low reset signals (`clk`, `n_reset`), a start signal (`start`) to initiate activation computation, and an enable signal (`ready`) indicating data availability.

```

Module Relu6_layer

Inputs:
- input_feature[num_channels]    // Input feature map for all channels (e.g.,
3 channels)
- clk, n_reset                   // Clock and active-low reset
- start                          // Start signal for activation computation
- ready                          // Enable signal for activation processing

Outputs:
- output_feature[num_channels]    // Activated feature values for all channels
- ready_activation                // Output flag: asserted when all activations
complete

Parameters:
- num_channels                   // Number of input channels (equals number of activation
PEs)

Internal Variables:
- feature[num_channels]          // Output from each Relu6_pe
- flag[num_channels]             // Completion flag from each Relu6_pe

// Instantiate num_channels activation PEs (one per channel)
For ch in 1 .. num_channels:
    Relu6_pe ap (
        .input_featuremap(input_feature[ch]),
        .clk(clk),
        .n_reset(n_reset),
        .enable(ready),
        .start(start),
        .output_featuremap(feature[ch]),
        .flag(flag[ch])
    )

Combinational Output Logic:
If all flag[ch] == 1 (for ch = 1..num_channels):
    For ch in 1 .. num_channels:
        output_feature[ch] = feature[ch]
    ready_activation = 1    // All activation PEs have completed, output is
valid
Else if any flag[ch] == 1:
    // Error: Inconsistent completion, only part of the activation PEs are
done
    Display "Running Error"
    output_feature = undefined // or X state
    ready_activation = 0
Else:
    For ch in 1 .. num_channels:
        output_feature[ch] = 0
    ready_activation = 0    // Idle or reset state

```

Fig. 2.37 Module: ReLU6 layer

The number of channels is specified by the parameter `num_channels`, which determines how many activation processing elements (PEs) are instantiated. Each channel has a corresponding instance of the `Relu6_pe` module. For each channel ch ($1 \leq ch \leq \text{num_channels}$), the PE processes its respective input, providing an activated output (`feature[ch]`) and a completion flag (`flag[ch]`).

The combinational output logic checks all completion flags. If all `flag[ch]` signals are high, indicating that all activation PEs have finished computation, the module outputs the activated values on `output_feature[ch]` and asserts the `ready_activation` signal. If only some flags are high, this indicates an error condition; the module outputs an error message, sets the output to an undefined state, and deasserts `ready_activation`. If none of the PEs have finished, the outputs are held at zero, and `ready_activation` remains low.

2.4.6 Fully Connected Layer Module

This part will introduce the basic elements of the fully connected layer, the fully connected processing element, and how to combine multiple fully connected processing elements to build a fully connected layer module.

2.4.6.1 Fully Connected Processing Element

The `full_pe` module, as shown in Fig. 2.38, implements a single multiply-and-accumulate (MAC) unit for a FC layer, typically used in neural network accelerators. It receives as inputs the value of the current input neuron (`input_featuremap`) and the corresponding weight (`weight`), both represented in fixed-point format and clock (`clk`), active-low reset (`n_reset`), and a start signal (`start`) to control operation.

```

Module: full_pe

Inputs:
- input_featuremap      // The value of the current input neuron (fixed-point)
- weight                // The corresponding weight for this connection
(fixed-point)
- clk, n_reset          // Clock and active-low reset signals
- start                 // Start signal to trigger the computation

Output:
- output_featuremap      // Output after multiplication and truncation (fixed-
point)

Internal Variables:
- mul                   // Stores the intermediate multiplication result
(wider bit width than inputs)

Main Logic (on each rising edge of clk or off n_reset):
If n_reset == 0:
    mul = 0
Else:
    If start == 0:
        mul = 0
    Else:
        mul = input_featuremap * weight

Combinational Output:
// Apply truncation or rounding according to hardware/fixed-point design
output_featuremap = mul

```

Fig. 2.38 Module: Fully connected processing element

On each rising edge of the clock or falling edge of *n_reset*, the main logic operates as follows: if reset is active (*n_reset* is low), the internal multiplication register (*mul*) is cleared. If the start signal is not high, *mul* remains zero. Otherwise, the module computes the product of the input feature and weight and stores it in *mul*.

The output (*output_featuremap*) is generated by truncating or rounding the multiplication result to match the fixed-point hardware design requirements.

2.4.6.2 Fully Connected Layer

The *full_layer* module, as shown in Figs. [2.39](#) and [2.40](#), implements a parallel FC layer. In each clock cycle, it receives a vector of input neurons from the previous layer (*input_feature[in_channels]*) and their corresponding weights (*weight_full[in_channels]*). The parameter *in_channels* determines the number of input neurons processed in parallel.

The module computes and accumulates contributions to every output neuron (output_feature[out_neurons]) across all input batches.

```
Module full_layer

Inputs:
- input_feature[in_channels] // Parallel input neurons from previous
layer
- weight_full[in_channels] // Corresponding weights for each input
- clk, n_reset // Clock and active-low reset
- start // Start signal for FC computation
- ready // Indicates valid input data is available
for accumulation
- finish // Reset/finish flag from final FC layer
(clears accumulators)

Outputs:
- output_feature[out_neurons] // Output of all FC neurons
- count_1 // Index of the output neuron currently
being updated
- count_2 // Index of the input neuron batch
currently being processed
- ready_full // Output valid flag: asserted when all
accumulations are complete

Parameters:
- in_channels // Number of input neurons processed in parallel
- out_neurons // Number of output neurons
- input_batches // Number of input batches

Internal Variables:
- in[in_channels] // Latched input neurons for current
operation
- neuron_hidden[out_neurons] // Accumulators for each output neuron
- bias_full[out_neurons] // Bias for each output neuron
- feature[in_channels] // Output from each full_pe (product input
× weight)
- count, count1, count2 // Counters for output neuron, input
neuron batch, and other purposes
- flag, flag1 // Control flags

Initialization (off n_reset):
- Load bias_full from file
- Set neuron_hidden, counters, and flags to zero

Instantiate in_channels processing elements (PEs) for parallel MAC:
For ch in 1 .. in_channels:
    full_pe fp (
        .input_featuremap(in[ch]),
        .weight(weight_full[ch]),
        .clk(clk),
        .n_reset(n_reset),
        .start(start),
        .output_featuremap(feature[ch])
    )
```

Fig. 2.39 Module: First fully connected layer part 1

```
Main Sequential Logic (per clock cycle or reset):
  If n_reset == 0:
    count = 0
    count2 = 0
    in = {0, ..., 0}
    neuron_hidden = {0, ..., 0}
    flag = 0
    flag1 = 0
  Else:
    If finish == 1:
      count2 = 0
      neuron_hidden = {0, ..., 0}
    If ready == 1:
      count2 = count2 + 1
      flag = 1
    If flag == 1:
      // Only update the current output neuron (count)
      neuron_hidden[count] = neuron_hidden[count] +
SUM_over_ch(feature[1..in_channels])
      // Output neuron index loop
      If count == 0:
        count = 0
      Else:
        count = count + 1
      If count == out_neurons:
        flag = 0
        count = 0
    Else:
      in = input_feature    // Latch new batch of input neurons when idle
      count = 0

      // Completion detection: all batches and all neurons processed
      If count2 == input_batches and count == out_neurons:
        flag1 = 1
      Else:
        flag1 = 0

Combinational Output Logic:
  count_1 = count
  count_2 = count2
  If flag1 == 1:
    For n in 1 .. out_neurons:
      output_feature[n] = neuron_hidden[n] + bias_full[n-1]
    ready_full = 1
  Else:
    output_feature = {0, ..., 0}
    ready_full = 0
```

Fig. 2.40 Module: First fully connected layer part 2

For each batch, the module instantiates multiple parallel processing elements (`full_pe`), each calculating the product of an input and its corresponding weight. The outputs from these units are summed and accumulated into registers (`neuron_hidden`) for each output neuron, utilizing the property of FC layers, where input neurons contribute to every output neuron through linear superposition. The accumulation is managed by internal counters (`count`, `count2`) that iterate over output neurons and input batches, respectively.

The `finish` signal is used to reset accumulators and counters when all FC computations for the current inference are complete. As new input data arrives (`ready` is high), the counters work and accumulation proceeds. When all inputs and outputs have been processed, the module asserts the output valid flag (`ready_full`) and outputs the final results for all neurons as the sum of the accumulated value and its corresponding bias.

In summary, the module's core logic is that each group of input features is processed in parallel; their weighted sums are accumulated into all output neurons. Once the computation is complete, each neuron's result is finalized by adding a bias.

The `full_layer2` module, as shown in Figs. [2.41](#) and [2.42](#), implements a parameterized fully connected (FC) output layer, where each output neuron accumulates weighted contributions from all input neurons. The input feature vector (`input_feature [IN_SIZE]`) is first latched into registers (`hidden_neurons [IN_SIZE]`). The output neurons (`output_neurons [OUT_SIZE]`) each have an accumulator to sum the results over all input neurons, and the per-neuron bias (`bias [OUT_SIZE]`) is loaded from file.

```

Module: full_layer2

Parameters:
- IN_SIZE      // Number of input neurons
- OUT_SIZE     // Number of output neurons

Inputs:
- input_feature[IN_SIZE] // Parallel input features
- weight_ful2           // Weight for current computation
- clk, n_reset          // Clock and active-low reset
- start, ready           // Control: start for PE, ready for new input
batch

Outputs:
- output_feature[OUT_SIZE] // Output values for all output neurons
- count1                   // Current output neuron index (1..OUT_SIZE)
- count2                   // Current input neuron index (1..IN_SIZE)
- finish                   // Flag: all computation done
- flag_ful2                // Indicates layer is active

Internal Variables:
- hidden_neurons[IN_SIZE] // Registers to store input features
- output_neurons[OUT_SIZE] // Accumulators for output neurons
- bias[OUT_SIZE]           // Bias for each output neuron
- count_1, count_1_1       // Output neuron counters
- count_2                   // Input neuron index
- flag                      // Indicates calculation in progress
- feature1                  // Output from full_pe (current input x
weight)

Initialization (off n_reset):
- Load bias[] from file (size = OUT_SIZE)
- All accumulators, counters, flags to zero

Instantiation:
- full_pe (
    .input_featuremap(hidden_neurons[count_2]),
    .weight(weight_ful2),
    .clk(clk), .n_reset(n_reset), .start(start),
    .output_featuremap(feature1)
)

Main Sequential Logic (on clock or off n_reset):
If n_reset == 0:
    count_1 = 1
    count_2 = 1
    count_1_1 = 0
    hidden_neurons = {0,...,0}
    output_neurons = {0,...,0}
    flag = 0
    finish = 0
Else:

```

Fig. 2.41 Module: Second fully connected layer part 1

```

If ready == 1:
    flag = 1
Else:
    If count_1 == OUT_SIZE && count_2 == IN_SIZE:
        flag = 0
    Else:
        flag = flag

If flag == 1:
    // Keep input; process output neurons sequentially
    hidden_neurons = hidden_neurons
    If count_1_1 == OUT_SIZE:
        count_1_1 = 1
    Else:
        count_1_1 = count_1_1 + 1

    If count_1 == OUT_SIZE:
        count_1 = 1
        If count_2 == IN_SIZE:
            count_2 = 1
            finish = 1
        Else:
            finish = 0
            count_2 = count_2 + 1
    Else:
        finish = 0
        count_1 = count_1 + 1

    // Accumulate product for current output neuron
    output_neurons[count_1_1] = output_neurons[count_1_1] + feature1
Else:
    // Latch new input and clear state
    count_1 = 1
    count_2 = 1
    hidden_neurons = input_feature
    finish = 0

Combinational Output Logic:
flag_ful2 = flag
count1 = count_1
count2 = count_2
If finish == 1:
    For i in 1..OUT_SIZE:
        output_feature[i] = output_neurons[i] + bias[i-1]
Else:
    output_feature = {0,...,0}

```

Fig. 2.42 Module: Second fully connected layer part 2

The module uses a single processing element (`full_pe`) to compute the product of the current input neuron and its corresponding weight

(`weight_ful2`), providing the result (`feature1`). Two counters, `count_1` and `count_2`, are used to iterate through output neurons and input neurons, respectively. Each time `flag` is high, the output neuron index (`count_1_1`) is incremented, and the current product is accumulated into the corresponding output neuron accumulator.

The sequential logic cycles through each output neuron for every input neuron, accumulating their weighted sums. When all output neurons have been updated for a given input, the module advances to the next input neuron. When all input and output neurons have been processed (`count_1 == OUT_SIZE && count_2 == IN_SIZE`), the `finish` flag is set, and the final outputs are computed by adding bias to each output neuron accumulator.

The outputs `flag_ful2`, `count1`, and `count2` provide control/status for external interfacing and weight management modules.

2.4.6.3 Classification Layer Module

The `classify` module, as shown in Fig. [2.43](#), implements the final decision logic for a neural network classifier. It receives as input a vector of scores (`input_feature[num_classes]`) for each category (e.g., ten classes for digit recognition), along with clock (`clk`), input ready (`ready`), and active-low reset (`n_reset`) signals.

```

Module classify

Inputs:
  - input_feature[num_classes]    // Input scores for all categories (e.g., 10
  classes)
  - clk, ready, n_reset           // Clock, input ready, and active-low reset
  signals

Outputs:
  - categories                    // The index of the category with the highest
  score
  - one_end                      // Flag: 1 if classification has completed
  for this sample

Parameters:
  - num_classes                  // Total number of classes (e.g., 10)

Internal Variables:
  - maximum                      // Tracks the maximum score found so far

Combinational Logic:
  // Find the maximum value among all input features
  maximum = input_feature[1]
  For i in 2 to num_classes:
    If input_feature[i] > maximum:
      maximum = input_feature[i]

Sequential Logic (on rising edge of clk or falling edge of n_reset):
  If n_reset == 0:
    categories = 0
    one_end = 0
  Else:
    If ready == 0:
      categories = 0
      one_end = 0
    Else:
      one_end = 1
      // Assign the category index corresponding to the maximum score
      For i in 1 to num_classes:
        If maximum == input_feature[i]:
          categories = i
          Break

```

Fig. 2.43 Module: Classify

Internally, the module performs a maximum search to determine the index of the class with the highest score. The combinational logic scans through all input scores, keeping track of the maximum value found. Sequential logic then updates the output registers on each clock edge or reset: if reset is triggered or input is not ready, the outputs (`categories`, `one_end`) are cleared. When valid input is

present (`ready` is high), the module sets the `one_end` flag and outputs the index (`categories`) of the class with the maximum score.

This module is typically used at the end of the neural network computation to convert raw class scores into a final classification result.

2.4.7 Top-Level Entity

The `gdp_net` module, as shown in Figs. [2.44](#), [2.45](#), and [2.46](#), describes the top-level architecture of a complete pipelined neural network accelerator, structured as a sequence of parameterizable hardware modules. The main input signals include the global system start (`start`), clock (`clk`), active-low reset (`n_reset`), an input pixel data stream (`input_data`), and a `store` flag for data buffering.

```

Module gdp_net

Inputs:
- start          // System start signal
- clk            // System clock
- n_reset        // Active-low reset
- input_data     // Input pixel data stream (e.g., from external source)
- store          // Store signal for input data buffering

Outputs:
- categories     // Classification result (category index)
- one_end        // Flag indicating completion of one inference pass

Internal Connections and Data Path:
// 1. Address calculation for reading input pixels
address_extract ae (
    .start(start), .clk(clk), .n_reset(n_reset),
    .address(address)
)

// 2. Input image buffering and supply for convolution
image_read ir (
    .read(start), .store(store), .input_data(input_data),
    .clk(clk), .n_reset(n_reset),
    .address(address),
    .input_feature(in),
    .ready_in(ready_in)
)

// 3. Kernel and bias loading for first convolutional layer
kernel_conv1 kc1 (
    .start(start), .clk(clk), .n_reset(n_reset),
    .weight_conv1(conv1_weight),
    .bias_conv1(bias_conv1)
)

// 4. First convolutional layer
conv_layer c1 (
    .input_feature(in), .weight(conv1_weight), .ready_in(ready_in),
    .clk(clk), .n_reset(n_reset), .start(start),
    .output_feature(output_conv1),
    .ready(ready_conv1), .bias_conv1(bias_conv1)
)

// 5. First activation layer
Relu6_layer act1 (
    .input_feature(output_conv1), .ready(ready_conv1),
    .clk(clk), .n_reset(n_reset), .start(start),
    .output_feature(output_act1),
    .ready_activation(ready_act1)
)

```

Fig. 2.44 Module: Top-level entity of accelerator part 1

```

// 6. First pooling layer
pool_layer p1 (
    .input_feature(output_act1), .ready(ready_act1),
    .clk(clk), .n_reset(n_reset), .start(start),
    .output_feature(output_pool1),
    .ready_pool(ready_pool1)
)

// 7. Kernel and bias loading for second convolutional layer
kernel_conv2 kc2 (
    .start(start), .clk(clk), .n_reset(n_reset), .ready(ready_pool1),
    .weight_conv2_1(conv2_weight[3:1]),
    .weight_conv2_2(conv2_weight[6:4]),
    .weight_conv2_3(conv2_weight[9:7]),
    .bias_conv2(bias_conv2)
)

// 8. Second convolutional layer
conv_layer2 c2 (
    .input_feature(output_pool1), .weight(conv2_weight),
    .bias_conv2(bias_conv2),
    .clk(clk), .n_reset(n_reset), .start(start), .ready_pool(ready_pool1),
    .output_feature(output_conv2), .ready(ready_conv2)
)

// 9. Second activation layer
activation_layer act2 (
    .input_feature(output_conv2), .ready(ready_conv2),
    .clk(clk), .n_reset(n_reset), .start(start),
    .output_feature(output_act2),
    .ready_activation(ready_act2)
)

// 10. Second pooling layer
pool_layer p2 (
    .input_feature(output_act2), .ready(ready_act2),
    .clk(clk), .n_reset(n_reset), .start(start),
    .output_feature(output_pool2),
    .ready_pool(ready_pool2)
)

// 11. First fully connected layer
full_layer1 full1 (
    .input_feature(output_pool2), .ready(ready_pool2),
    .weight_full(weight_full),
    .clk(clk), .n_reset(n_reset), .start(start),
    .count_finish1(count_finish), .finish(ready_full2),
    .output_feature(output_full1),
    .count_full(count_full), .ready_full(ready_full)
)
weight_full1 wf1 (
    .count_full(count_full), .count_finish(count_finish),
    .weight_full(weight_full)
)

```

Fig. 2.45 Module: Top-level entity of accelerator part 2

```

// 12. Activation after first FC layer
activation_layer_full actf2 (
    .input_feature(output_full1), .ready(ready_full1),
    .clk(clk), .n_reset(n_reset), .start(start),
    .output_feature(output_act3),
    .ready_activation(ready_act3)
)

// 13. Second fully connected layer
full_layer2 full2 (
    .input_feature(output_act3), .ready(ready_act3),
    .weight_full2(weight_full2), .flag_full2(flag_full2),
    .clk(clk), .n_reset(n_reset), .start(start), .count1(count_1),
    .count2(count_2),
    .output_feature(output_full2),
    .finish(ready_full2)
)
weight_full2 wf2 (
    .count_2(count_2), .count_1(count_1), .flag_full2(flag_full2),
    .weight_full2(weight_full2)
)

// 14. Final classifier
classify cl1 (
    .input_feature(output_full2),
    .clk(clk), .ready(ready_full2), .n_reset(n_reset),
    .categories(categories),
    .one_end(one_end)
)

```

Fig. 2.46 Module: Top-level entity of accelerator part 3

The inference datapath is organized as follows: pixel addresses are generated using an `address_extract` module; input data is buffered and synchronized by `image_read`. Two sets of kernel and bias loaders (`kernel_conv1`, `kernel_conv2`) initialize convolutional layer weights. Each convolutional layer (`conv_layer`, `conv_layer2`) is followed by an activation layer (`activation_layer`) and a pooling layer (`pool_layer`), maintaining the typical convolution-activation-pooling sequence.

The output of the second pooling layer is forwarded to a fully connected layer (`full_layer1`), with weights supplied by `weight_full1`. This is followed by a nonlinearity (`activation_layer_full`) and a final fully connected layer (`full_layer2`) with weights from `weight_full2`. The final classification result is produced by an `argmax` classifier (`classify`), which outputs the predicted category and a one-shot end flag (`one_end`).

Handshaking signals and ready flags ensure strict pipeline synchronization: each module waits for input-valid flags from the previous stage before beginning computation, enforcing a robust, data-driven hardware pipeline. All weight and bias memories are initialized from external files, enabling flexible deployment for various network architectures. Key structural parameters such as kernel and pooling sizes are fully configurable.

2.4.8 Testbench Design

The testbench, as shown in Fig. [2.47](#), for the `gdp_net` module verifies system-level functionality using a series of test images and expected category labels. The key parameters are the image size (`image_size`), the total number of test cases (`num_tests`), a 2D array of test images (`input_image [num_tests] [image_size* image_size]`), and the expected classification result for each image (`expected_category [num_tests]`).

```

Testbench for gdp_net

Parameters:
- image_size           // Image width/height
- num_tests            // Number of test cases
- input_image[num_tests][image_size*image_size] // Test images
- expected_category[num_tests] // Expected output for each
image

Internal Variables:
- clk                 // Clock
- n_reset             // Active-low reset
- start, store        // Control signals
- input_data          // Current pixel fed to DUT
- categories          // DUT output: predicted class
- one_end             // DUT output: inference done
- error_count         // Number of failed test cases

Main Testbench Logic:

For test_case in 1 .. num_tests:
    // 1. Apply reset and initialize control signals
    n_reset = 0
    Wait N cycles
    n_reset = 1
    start = 0
    store = 1
    pixel_index = 1

    // 2. Feed one pixel per clock when store=1
    While pixel_index <= image_size * image_size:
        input_data = input_image[test_case][pixel_index]
        Wait 1 clock cycle
        pixel_index = pixel_index + 1

    // 3. After all pixels loaded, disable store and enable start
    store = 0
    start = 1

    // 4. Wait for one_end=1, which indicates classification complete
    While one_end == 0:
        Wait 1 clock cycle

    // 5. Check and display result
    If categories == expected_category[test_case]:
        Print "Test case", test_case, "PASS"
    Else:
        Print "Test case", test_case, "FAIL: expected",
        expected_category[test_case], "but got", categories
        error_count = error_count + 1

    // 6. Prepare for next test (auto-loop to next iteration, which resets system
    at the beginning)

Print "Testing finished."
Print "Total:", num_tests, "Passed:", num_tests - error_count, "Failed:",
error_count

```

Fig. 2.47 Module: Testbench design

For each test case, the testbench logic is as follows:

1. System reset is applied by deasserting `n_reset`, followed by several clock cycles to ensure all internal states are cleared. The `start` and `store` signals are initialized appropriately.
 2. The test image is loaded into the system pixel by pixel: each clock cycle, a new pixel from `input_image` is presented on the input data line while `store` is high.
 3. Once the entire image is loaded, `store` is low and `start` is high to trigger inference.
 4. The testbench then waits until the `one_end` output from the DUT indicates that classification is complete.
 5. The predicted category (`categories`) is compared to the expected value (`expected_category`) for the current test case. Pass/fail status is reported for each case, and an error counter is incremented if any failures occur.
 6. The process repeats for all test cases. At the end of testing, the testbench prints a summary of the total number of tests, passes, and failures.
-

2.5 Conclusions

In this chapter, we presented a comprehensive approach to training CNNs at the software level using TensorFlow, as well as designing a simple neural network using System Verilog for hardware implementation. By integrating these methodologies, we aim to provide an accessible framework for researchers and developers to train neural networks and deploy them on FPGAs for performance analysis. This chapter serves as a foundational resource, supporting software-based neural network development and hardware-oriented implementation, thereby facilitating further exploration and innovation in FPGA-based deep learning acceleration.

Appendix: Pseudo Code of the Hardware-Level CNN Model Implementation

References

1. Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., Devin, M., Ghemawat, S., Irving, G., Isard, M., et al.: Tensorflow: a system for large-scale machine learning. In: 12th USENIX Symposium on

- Operating Systems Design and Implementation (OSDI 16), pp. 265–283 (2016)
2. Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., Chintala, S.: Pytorch: An imperative style, high-performance deep learning library. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 32. Curran Associates (2019)
 3. TensorFlow Team: Tensorflow blog. <https://blog.tensorflow.org/> (n.d.). Accessed: 1 June 2025
 4. PyTorch Team: Pytorch. <https://pytorch.org/>. Accessed: 1 June 2025
 5. Vahdat, S., Kamal, M., Afzali-Kusha, A., Pedram, M.: Tosam: an energy-efficient truncation-and rounding-based scalable approximate multiplier. IEEE Trans. Very Large Scale Integr. (VLSI) Syst. **27**(5), 1161–1173 (2019)
 6. Krizhevsky, A., Hinton, G., et al.: Learning multiple layers of features from tiny images. Technical Report (2009)
-

Footnotes

[1 https://github.com/HaosenYu/Security_of_AI_Accelerator_Book.git](https://github.com/HaosenYu/Security_of_AI_Accelerator_Book.git)

[2 https://github.com/HaosenYu/Security_of_AI_Accelerator_Book.git](https://github.com/HaosenYu/Security_of_AI_Accelerator_Book.git)

[3 https://github.com/HaosenYu/Security_of_AI_Accelerator_Book.git](https://github.com/HaosenYu/Security_of_AI_Accelerator_Book.git)

[OceanofPDF.com](https://oceanofpdf.com)

3. Performance Optimization of CNN Hardware

Peiyao Sun¹✉, Haosen Yu¹✉, Tom J. Kazmierski¹✉ and Basel Halak¹✉

(1) School of Electronics and Computer Science, University of Southampton,
Southampton, UK

✉ Peiyao Sun

Email: ps2m24@soton.ac.uk

✉ Haosen Yu

Email: Haosen.Yu@soton.ac.uk

✉ Tom J. Kazmierski

Email: T.J.Kazmierski@soton.ac.uk

✉ Basel Halak (Corresponding author)

Email: Basel.Halak@soton.ac.uk

Keywords Performance optimization – Approximate multipliers – Error injection – Error matrix – Computational efficiency – Accuracy trade-offs – Hardware acceleration – Data truncation – Error compensation – 4:2 compressors – Approximate arithmetic – Experimental validation – Optimization techniques – Resource utilization – Error analysis

3.1 Introduction

Approximate computing (AC) has become an increasingly attractive approach for enhancing the hardware efficiency of CNN accelerators, especially in the context of embedded systems where resource constraints are critical [1]. Among the many approximate computing techniques, approximate multipliers (AxMs), which can trade off between the hardware consumption and accuracy performance [2–4], have received significant attention due to the fact that multipliers are widely utilized in the CNN hardware accelerators [5].

Because of the wide variety of available AxMs [6–8], selecting the most appropriate type for a given CNN architecture is a fundamental step in designing an AxM-based CNN hardware accelerator. When addressing this task, designers need to carefully consider several key questions to ensure that approximate units are used effectively:

- How can approximate components be matched to the error tolerance levels of different CNN layers or operations?
- Which exact units in the hardware design can be replaced by approximate versions without introducing unacceptable loss in accuracy?

To address these design considerations, a systematic approach is required. This chapter introduces a structured methodology known as *error matrix-based error injection* (EMEI) [9]. EMEI serves as a practical tool for designers to evaluate and deploy existing AxMs within CNN accelerators. By using pre-characterized error rate matrices, the technique facilitates informed decision-making when integrating approximation into processing elements (PEs).

As a case study, the TOSAM approximate [6] multiplier is introduced and applied within a MobileNetV2-based CNN model trained on the CIFAR-10 dataset [10] and GTSRB [11]. This example demonstrates how AxMs with varying error profiles can be strategically assigned across different PEs, balancing efficiency and inference accuracy. Through this case-driven approach, the chapter provides a hands-on understanding of how approximation techniques can be systematically applied in modern CNN hardware design.

3.2 Chapter Overview

In this chapter, we begin by introducing several types of approximate multipliers that are commonly used in CNN hardware accelerators, as discussed in Sect. 3.3. Section 3.4 then describes the EMEI method, which provides a practical means for selecting suitable approximate multipliers with varying precision levels, while aiming to minimize hardware overhead. Finally, Sect. 3.5 presents a set of experimental results that demonstrate the effectiveness and practical benefits of applying the EMEI method in CNN accelerator design. The example code is available in the GitHub repository.¹

3.3 Approximate Multiplier

In the study referenced as [12], the authors introduce an innovative approach to enhancing the efficiency of CNN hardware accelerators through the adoption of AC technologies. Specifically, they propose the substitution of precise hardware components with approximate hardware components. Given that convolutional layers are responsible for most of the computational workload in CNN accelerators [13], the implementation of AxMs, instead of precise ones, significantly reduces the hardware costs of the CNN hardware accelerator. Emerging research in AxM technologies focuses on three methodologies: data truncation [6], error compensation [7], and the employment of 4:2 compressors [8]. This section will introduce works within each of these three domains.

3.3.1 Data Truncation

TOSAM AxM [6] is a type of energy-efficient, truncation, and rounding-based AxM that supports multiple precision levels. It reduces the computational workload of the multiplier by truncating input operands based on the position of their leading one-bit. This design significantly enhances energy consumption and area efficiency while maintaining acceptable accuracy levels. The methodology involves rounding input operands to the nearest odd number, which improves total accuracy and scalability, enabling higher precision with increased operand widths.

Compared to existing precise and approximate multipliers, the proposed TOSAM AxM achieves superior performance in terms of delay, area, and energy consumption. This makes it an attractive solution for error-resilient applications, including image processing, sharpening, and classification [6].

The functionality of the TOSAM AxM is defined in Eq. 3.1. Here, $(Y_A)_t$ and $(Y_B)_t$ denote the values obtained when A and B are truncated t bits after the first occurrence of a “1” in their binary representation. $(Y_A)_{APX}$ and $(Y_B)_{APX}$ represent the values when A and B are truncated h bits following the initial “1” bit, with an additional “1” appended to the right, resulting in $h + 1$ bits in total. k_A and k_B refer to the positions of the first “1” in A and B , respectively [6]:

$$A \times B \approx 2^{k_A+k_B} \times (1 + (Y_A)_t + (Y_B)_t + (Y_A)_{APX} \times (Y_B)_{APX}) \quad (3.1)$$

The structure of the TOSAM AxM is depicted in Fig. 3.1. This architecture comprises several key components:

- **Approximate Absolute Unit:** Calculates the absolute value of the inputs.

- **Sign or Zero Detector Unit:** Determines whether the result is zero and identifies the sign of the result.
- **Leading One Detector Unit:** Locates the position of the first occurrence of “1” in the inputs.
- **Truncation Unit:** Truncates the least significant bits of the operands.
- **Arithmetic Unit:** Performs the primary arithmetic computation based on the truncated inputs.
- **Shift Unit:** Scales the result from the arithmetic unit to recover the final value.
- **Compatible Module:** Selects the suitable accuracy level for the AxM based on input values.

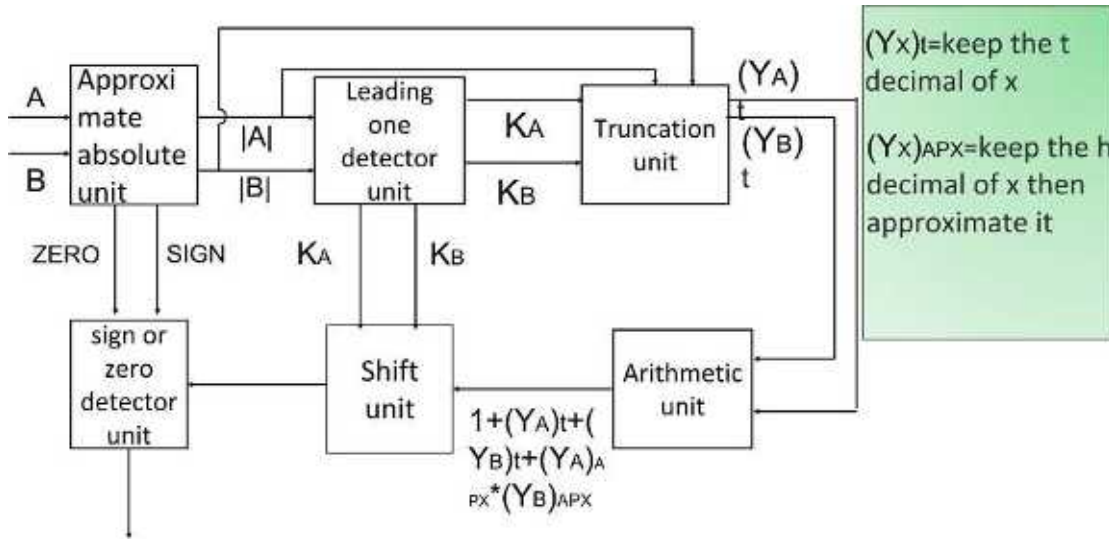


Fig. 3.1 The structure of the approximate TOSAM multiplier

3.3.2 Error Compensation

In work [7], a low-power, high-performance AxM with configurable partial error recovery was introduced. This design utilizes approximate adders that limit carry propagation to neighboring bits for fast partial product accumulation. By employing configurable error recovery, the design achieves varying levels of accuracy through the use of different numbers of most significant bits (MSBs) for error reduction.

Additionally, this approach allows for the adjustment of the number of MSBs involved in error recovery calculations to offer different levels of computational accuracy. The functionality of the approximate adder is defined by Eq. 3.2, which computes one bit of the sum result with inputs Am_i and Bm_i for each bit of the input operands [7]. The error recovery signal for each adder and the total error recovery signal for the multiplier are given by Eqs. 3.3 and 3.4, respectively.

Furthermore, an error recovery function (Eq. 3.5) adjusts the final output. In this context, Sm'_i represents the sum result's bit, Em_i denotes the error recovery signal's bit for each adder, E is the total error recovery signal, S is the corrected output, and E_{ap} is the approximated error recovery signal obtained after truncating less significant bits.

Figure 3.2 illustrates an 8-bit AxM based on this design, which requires the five most significant bits of the error recovery signal for effective functionality. Each circle in Fig. 3.2

represents a single bit in an 8-bit data value:

$$Sm'_i = (Am_i \oplus Bm_i) + Am_{i-1} \text{ } \text{ } Bm_{i-1} \quad (3.2)$$

$$Em_i = (Am_i \oplus Bm_i)Am_{i-1} \text{ } \text{ } Bm_{i-1} \quad (3.3)$$

$$E = E_1 \mid E_2 \mid \dots \mid E_m \quad (3.4)$$

$$S = S' + E_{ap} \quad (3.5)$$

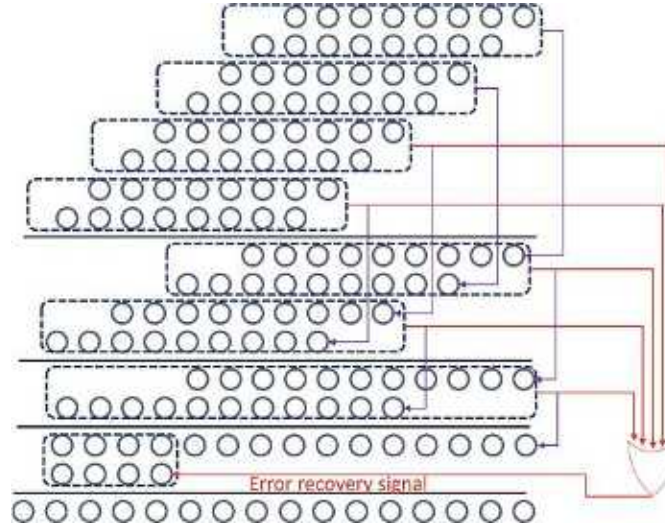


Fig. 3.2 The structure of the approximate multiplier based on the error compensation principle

3.3.3 Employment of 4:2 Compressors

Tianqi Kong and Shuguo Li proposed a design that utilizes high-accuracy approximate 4:2 compressors to enhance multiplier designs [8]. This approach introduces five novel high-accuracy approximate 4:2 compressor designs as substitutes for exact compressors or full adders. The functionality of these five approximate compressors is detailed in Eqs. 3.6–3.10.

In these equations, the term “*Compressor_i*” denotes the specific index of the proposed high-accuracy approximate compressors, where *i* represents different compressor designs. The variables *s*, *c*, *a*, *b*, and *d* refer to the sum and carry outputs and the four input operands, respectively. Additionally, Fig. 3.3 illustrates the 8-bit AxM design based on these approximate 4:2 compressors. Each circle in Fig. 3.2 represents a single bit in an 8-bit data value:

$$\text{Compressor1} - S = (a \oplus b)(c \oplus d) \mid (a \oplus b)(c \oplus d) \mid abcd \quad (3.6)$$

$$\text{Compressor1} - C = (a \mid b \text{ } \text{ } c \text{ } \text{ } d) \mid (c \mid d \text{ } \text{ } a \text{ } \text{ } b)$$

$$\text{Compressor2} - S = (a \oplus b)(c \oplus d) \mid (a \oplus b)(c \oplus d) \quad (3.7)$$

$$\text{Compressor2} - C = (a \mid b \text{ } \text{ } c \text{ } \text{ } d) \mid (c \mid d \text{ } \text{ } a \text{ } \text{ } b)$$

$$\begin{aligned} \text{Compressor3} - S &= (a \oplus b)(c \oplus d) \mid (a \oplus b)(c \mid d) \\ &= ((a \oplus b) \mid (c \oplus d)) \text{ } \text{ } ((a \oplus b)(c \mid d)) \end{aligned} \quad (3.8)$$

$$\text{Compressor3} - C = (a \mid b \text{ } \text{ } c \text{ } \text{ } d) \mid (c \mid d \text{ } \text{ } a \text{ } \text{ } b)$$

$$\begin{aligned}\text{Compressor4} - \bar{S} &= (a \oplus b)(c \oplus d) \mid (a \oplus b)(c \mid d) \\ \text{Compressor4} - S &= (c \mid d)(a \oplus b) \mid (a \oplus b)(c \oplus d)\end{aligned}\quad (3.9)$$

$$\begin{aligned}\text{Compressor4} - C &= (a \mid b \oplus \bar{c} \oplus \bar{d}) \mid (c \mid d \oplus a \oplus b) \\ \text{Compressor5} - C &= (a \mid b \mid c \mid d) \oplus (a \oplus b \oplus c \oplus d) \\ \text{Compressor5} - S &= ((a \mid b \mid c \mid d) \oplus (a \oplus b \oplus c \oplus d)) \oplus (a \mid b \oplus c \mid d)\end{aligned}\quad (3.10)$$

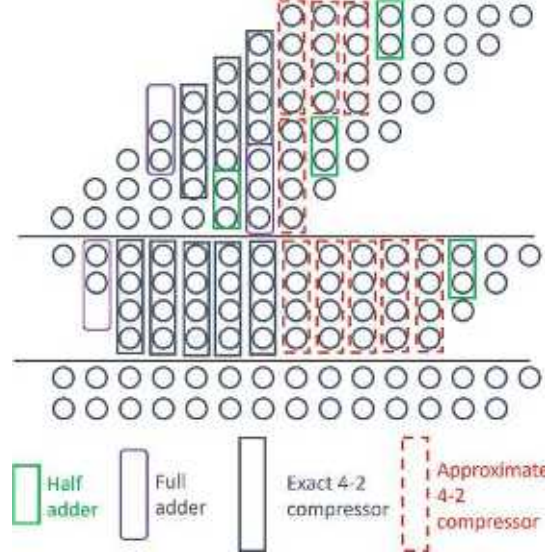


Fig. 3.3 The structure of the approximate multiplier based on the approximate 4:2 compressor

3.4 Error Matrix-Based Error Injection

3.4.1 Error Matrix of Approximate Multiplier Determination

The working principle of TOSAM was introduced in Sect. 3.3.1. The six accuracy levels are represented by bit retention post-truncation as (h, t) . These levels are $(0, 0)$, $(0, 2)$, $(1, 4)$, $(2, 6)$, $(3, 7)$, and $(5, 9)$. The error distribution matrix for each level is derived by comparing approximate outcomes with exact values, resulting in an error ratio. This comparison is expressed in Eq. 3.11.

In Eq. 3.11, N denotes the number of fractional bits in the binary fixed-point format. The variables A and B represent the values obtained by directly converting the binary representations of the multiplier and multiplicand into base-10 integers without considering the decimal point. $Ap(A, B)$ refers to the result of the approximate multiplication. As evident from Eq. 3.11, the length of fractional bits does not impact the error distribution. Therefore, for matrix calculations, both multipliers and multiplicands are treated as integers within the range 0 to $2^{15} - 1$.

The error rate distributions for these six levels of TOSAM AxMs are illustrated in Fig. 3.4:

$$E_r = \frac{(Ap(A, B) - A \times B) \cdot 2^{-2N}}{A \times B \cdot 2^{-2N}} = \frac{Ap(A, B) - A \times B}{A \times B} \quad (3.11)$$

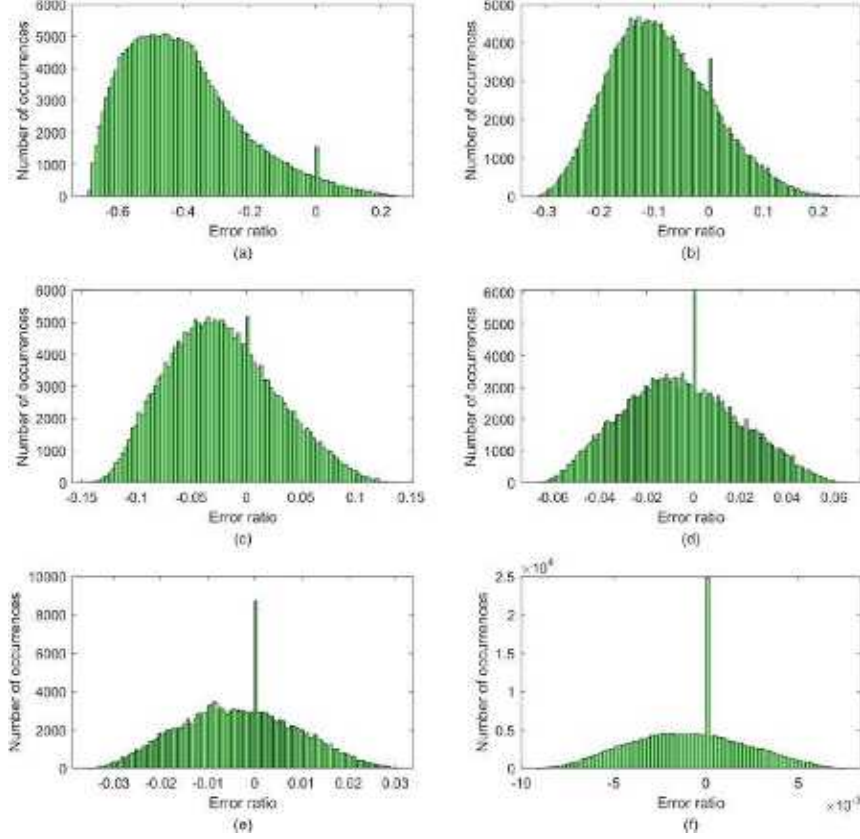


Fig. 3.4 Histogram depicting the error rate of TOSAM-based approximate multipliers. The error rates of TOSAM are shown for the following operation modes: (a) $(h, t) = (0, 0)$, (b) $(h, t) = (0, 2)$, (c) $(h, t) = (1, 4)$, (d) $(h, t) = (2, 6)$, (e) $(h, t) = (3, 7)$, (f) $(h, t) = (5, 9)$

3.4.2 Error Injection for Determination of Suitable Approximate Multiplier's Accuracy Level

The utilization of error distribution matrices is essential for selecting the optimal AxM and its precision across various conditions. A singular selection may not effectively balance hardware efficiency with the required accuracy level. Hence, employing a combination of AxMs with varying precision levels becomes necessary. Inspired by image processing techniques [14], where feature maps are divided into regions of high and low significance, we propose processing data within regions of high significance using high-precision PEs, while data in regions of low significance are processed by low-precision PEs. Each feature in feature maps is computed by the operations in the preceding layer, and the significance of these features determines the precision of multipliers for their production in the earlier layer.

To select suitable AxM precision levels and arrange them effectively, the EMEI framework includes two main steps: (1) The selection of two AxM types with high and low accuracy levels (2) The partitioning of importance-based segments within feature maps in CNNs

3.4.2.1 Selection of Suitable Approximate Multipliers' Accuracy Levels

To determine the most suitable AxMs and their accuracy levels, we start by assuming that AxMs replace all precise multipliers in CNN accelerators. This means that multiplications

within convolutional, normalization, and fully connected layers will include injected error, as specified in Eq. 3.12:

$$Ap(A, B) \approx A \times B \times M(\text{randi}(\text{size}(M))) \quad (3.12)$$

In this equation, M represents the error rate distribution matrix for the AxM at the evaluated accuracy level, and $M(A)$ refers to the data located at coordinate A within the matrix M .

The methodology for selecting the optimal AxMs and their precision levels is as follows:

1. Order the potential AxMs in descending order of their error rates.
2. Begin the process of injecting error rates using matrices with the lowest error rates.
3. Monitor the CNN's classification accuracy. Stop the error injection process if the classification accuracy falls below the predefined acceptable accuracy level.
4. Select the current precision level of the AxM as the low-precision option for lower-accuracy PEs in the CNN hardware accelerator. The adjacent higher-precision mode of the AxM is selected as the high-precision option for higher-accuracy PEs in the CNN hardware accelerator.

3.5 Experimental Setup and Experiment Result

To evaluate the effectiveness of EMEI, we developed two CNN models based on the MobileNetV2 architecture. The detailed structure of these CNNs is shown in Figs. 3.5 and 3.6. For training, we utilized a subset of the CIFAR-10 dataset containing high-clarity images, consisting of 37,074 images for the training set and 7,300 images for the testing set, trained using the CNN shown in Fig. 3.5. Additionally, we employed the GTSRB dataset, with 35,209 images for the training set and 12,630 images for the testing set, trained using the CNN in Fig. 3.6 to demonstrate the performance of EMEI.

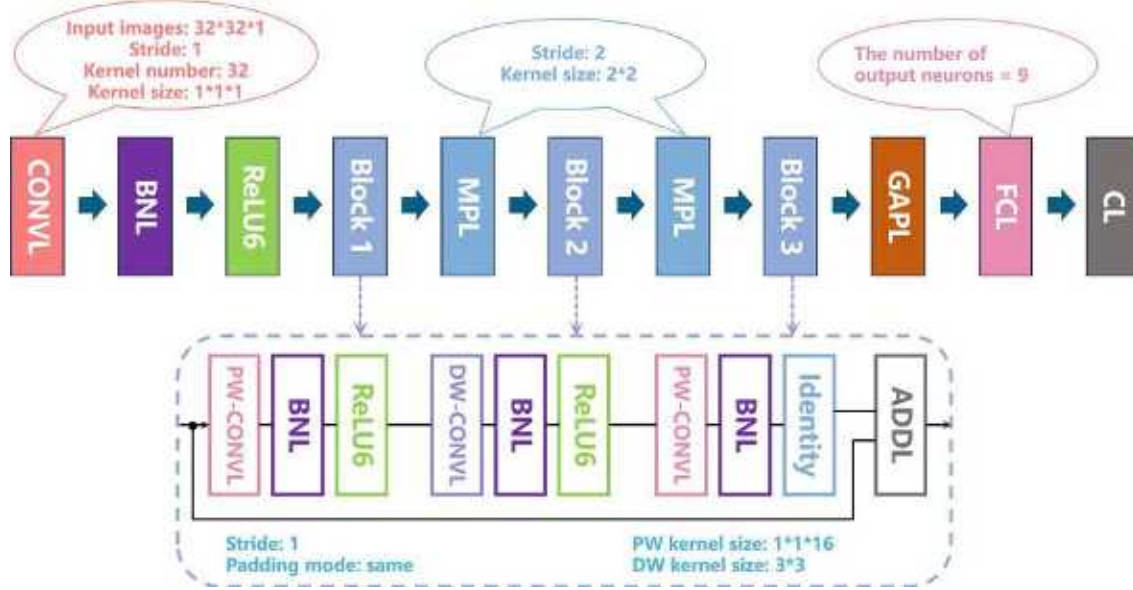


Fig. 3.5 The structure of the CNN for the CIFAR-10 dataset

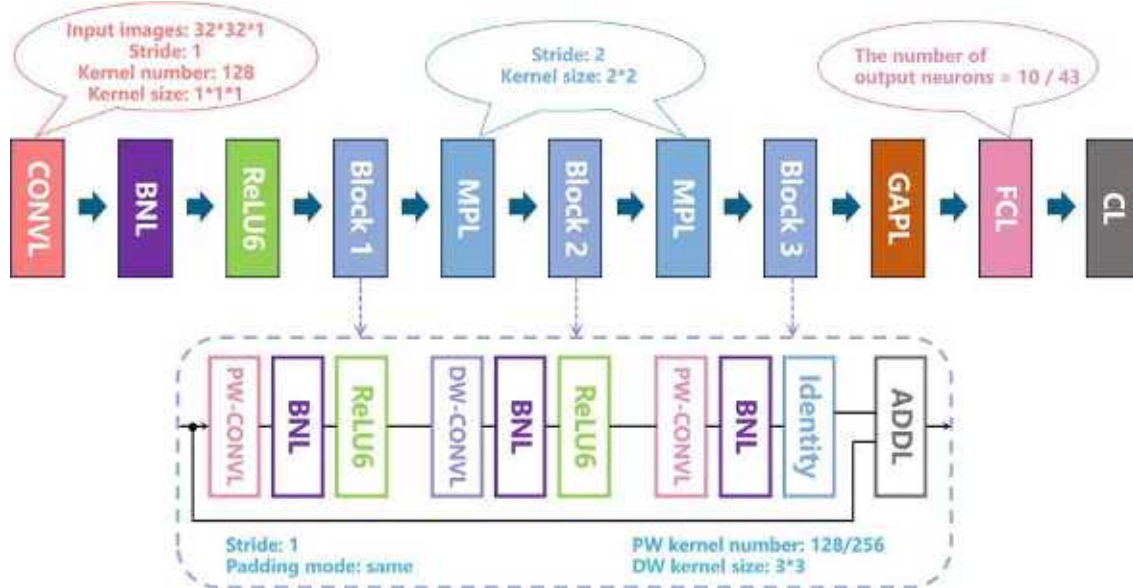


Fig. 3.6 The structure of the CNN for the GTSRB dataset

The architecture of the CNN hardware accelerator employed in our study is illustrated in Fig. 3.7. Within this architecture, the controller module controls the operational status of each layer's components. Each layer includes multiple PEs, and the controller ensures accurate memory addressing for the input and output of each PE. The input and output of each PE are not directly sent to the memory; instead, they are stored in a buffer and then transferred to memory through DMA. The data processed by this hardware accelerator is represented as signed 16-bit fixed-point numbers, which include one sign bit, six integer bits, and nine fractional bits.

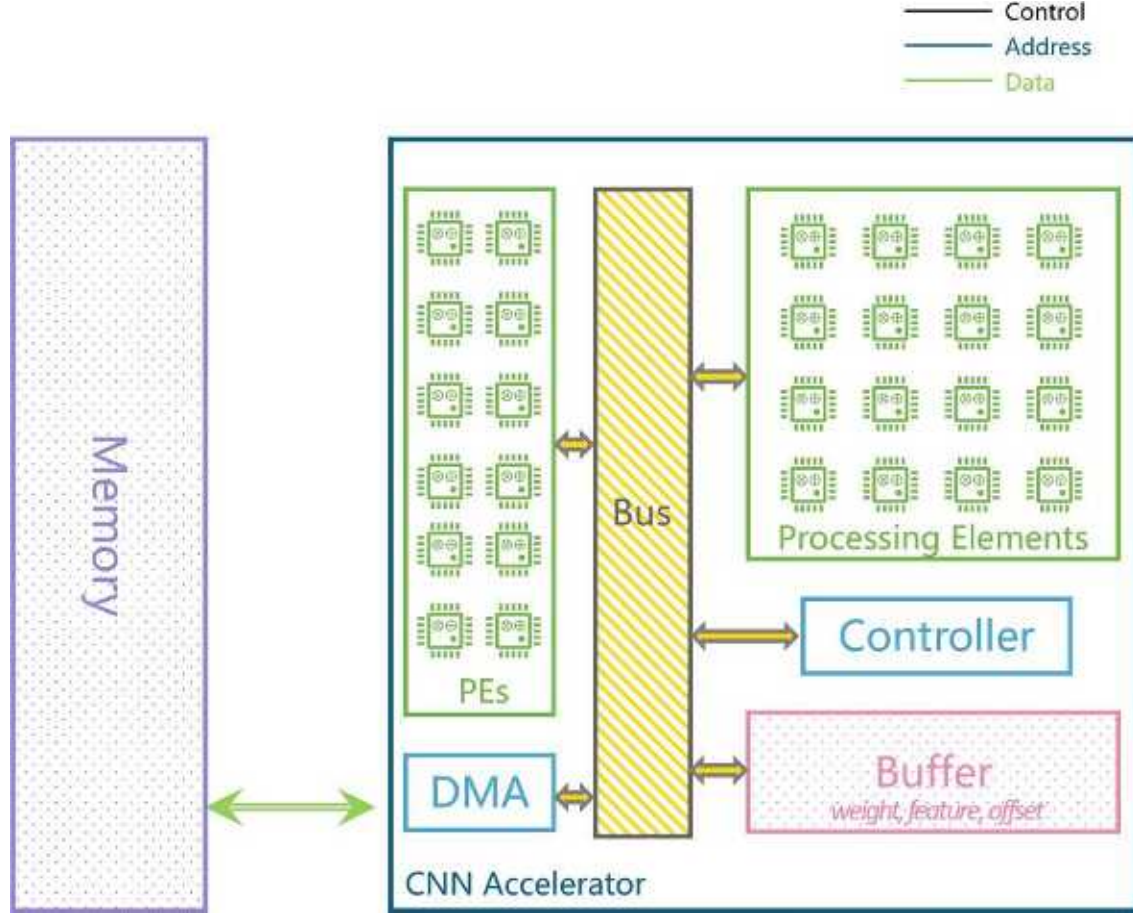


Fig. 3.7 The structure of the CNN hardware accelerator

Instead of employing precise multipliers, our system utilizes TOSAM AxMs. The selectable modes for the TOSAM AxMs are $(h, t) = (0, 0), (0, 2), (1, 4), (2, 6), (3, 7),$ and $(5, 9)$. Figure 3.4 presents the histogram of the error distribution matrix utilized in our experiments. For this investigation, we set a minimum acceptable classification accuracy threshold of 85% for the CIFAR-10 CNN and 93% for the GTSRB CNN.

3.5.1 Approximate Multiplier and Accuracy Determination

Based on the methodology described in Sect. 3.4, we injected errors into every multiplication operation within the CNN model, following the error distribution matrix of the AxM across various levels of accuracy, as shown in Fig. 3.4. The classification accuracies generated from the application of different error matrices are illustrated in Figs. 3.8 and 3.9. Given that the threshold for the lowest permissible classification accuracy was set at 85% and 93% for CIFAR-10 and GTSRB, respectively, the (h, t) mode selected for the high-accuracy AxM was determined to be $(2, 6)$, while the mode chosen for the low-accuracy AxM was $(1, 4)$. This selection was based on the observation that, at the accuracy level $(h, t) = (1, 4)$, the classification accuracy remained just below the threshold value. Following the instruction in Sect. 3.4, this accuracy level was selected as the low-accuracy multiplier, and the adjacent higher level, $(h, t) = (2, 6)$, was chosen as the high-accuracy multiplier.

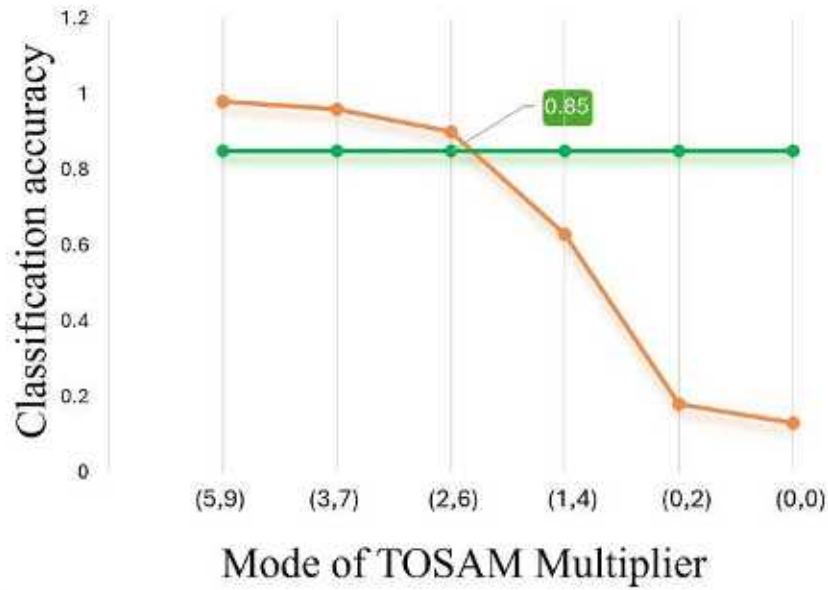


Fig. 3.8 Varied classification accuracies observed in CIFAR-10 systems employing approximate multipliers at distinct accuracy levels

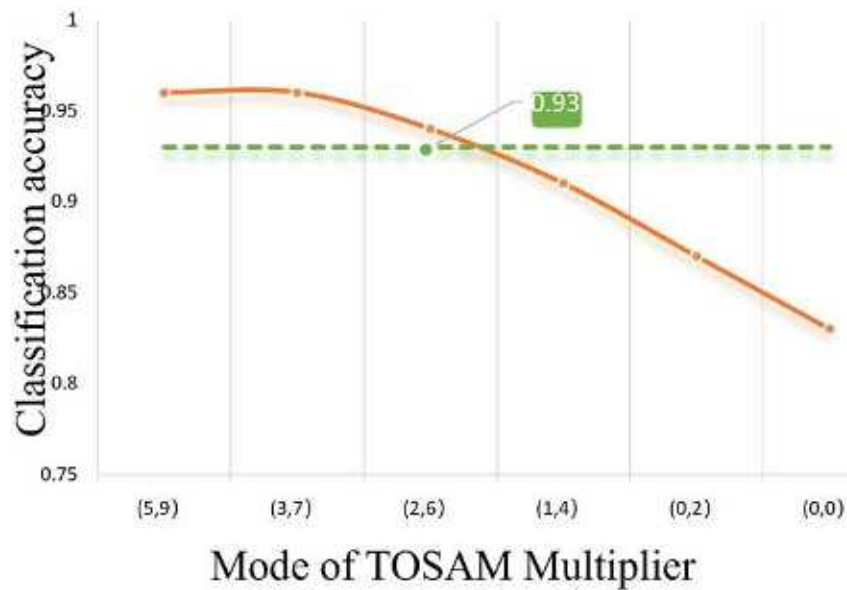


Fig. 3.9 Varied classification accuracies observed in GTSRB systems employing approximate multipliers at distinct accuracy levels

Following the instructions in Sect. 3.4, the expansion of the less important regions within the feature map was executed, initiating from the left toward the right, then from top to

bottom, followed by right to left, and finally from bottom to top. This expansion of nonessential regions generally leads to a decrease in classification accuracy. This trend in CIFAR-10 and GTSRB systems is represented in Tables 3.1 and 3.2. According to the guidelines specified in Sect. 3.4, when the classification accuracy drops below the predefined threshold, the expansion of the unimportant region is reversed by one grid in the recent direction of expansion.

Table 3.1 Classification accuracy varies with unimportant area size in CIFAR10

Number of iterations	Classification accuracy	Unimportant area size
Loop 1	89.50%	L = 1; T = 0; R = 0; B = 0
	89.00%	L = 1; T = 1; R = 0; B = 0
	89.30%	L = 1; T = 1; R = 1; B = 0
	88.90%	L = 1; T = 1; R = 1; B = 1
Loop 2	88.90%	L = 2; T = 1; R = 1; B = 1
	88.30%	L = 2; T = 2; R = 1; B = 1
	88.20%	L = 2; T = 2; R = 2; B = 1
	87.70%	L = 2; T = 2; R = 2; B = 2
Loop 3	87.50%	L = 3; T = 2; R = 2; B = 2
	87.50%	L = 3; T = 3; R = 2; B = 2
	87.10%	L = 3; T = 3; R = 3; B = 2
	86.30%	L = 3; T = 3; R = 3; B = 3
Loop 4	85.80%	L = 4; T = 3; R = 3; B = 3
	85.60%	L = 4; T = 4; R = 3; B = 3
	84.70%	L = 4; T = 4; R = 4; B = 3
	84.60%	L = 4; T = 4; R = 3; B = 4
Loop 5	85.50%	L = 5; T = 4; R = 3; B = 3
	84.90%	L = 5; T = 5; R = 3; B = 3
	×	×
	×	×
Loop 6	84.80%	L = 6; T = 4; R = 3; B = 3
	×	×
	×	×
	×	×

× Indicating expansion stopped as accuracy fell below the threshold

Table 3.2 Varied classification accuracies observed in the GTSRB system with different sizes of unimportant areas

Number of iterations	Classification accuracy	Unimportant area size
Loop 1	95.05%	L = 1; T = 0; R = 0; B = 0

Number of iterations	Classification accuracy	Unimportant area size
	95.04%	L = 1; T = 1; R = 0; B = 0
	94.99%	L = 1; T = 1; R = 1; B = 0
	94.95%	L = 1; T = 1; R = 1; B = 1
Loop 2	94.78%	L = 2; T = 1; R = 1; B = 1
	94.77%	L = 2; T = 2; R = 1; B = 1
	94.75%	L = 2; T = 2; R = 2; B = 1
	94.90%	L = 2; T = 2; R = 2; B = 2
Loop 3	94.85%	L = 3; T = 2; R = 2; B = 2
	94.70%	L = 3; T = 3; R = 2; B = 2
	94.63%	L = 3; T = 3; R = 3; B = 2
	94.77%	L = 3; T = 3; R = 3; B = 3
Loop 4	94.70%	L = 4; T = 3; R = 3; B = 3
	94.47%	L = 4; T = 4; R = 3; B = 3
	94.46%	L = 4; T = 4; R = 4; B = 3
	94.50%	L = 4; T = 4; R = 4; B = 4
Loop 5	94.55%	L = 5; T = 4; R = 4; B = 4
	94.33%	L = 5; T = 5; R = 4; B = 4
	94.51%	L = 5; T = 5; R = 5; B = 4
	94.28%	L = 5; T = 5; R = 5; B = 5
Loop 6	94.52%	L = 6; T = 5; R = 5; B = 5
	94.15%	L = 6; T = 6; R = 5; B = 5
	94.25%	L = 6; T = 6; R = 6; B = 5
	94.21%	L = 6; T = 6; R = 6; B = 6
Loop 7	94.24%	L = 7; T = 6; R = 6; B = 6
	94.14%	L = 7; T = 7; R = 6; B = 6
	94.05%	L = 7; T = 7; R = 7; B = 6
	93.90%	L = 7; T = 7; R = 7; B = 7
Loop 8	93.82%	L = 8; T = 7; R = 7; B = 7
	93.84%	L = 8; T = 8; R = 7; B = 7
	93.53%	L = 8; T = 8; R = 8; B = 7
	93.13%	L = 8; T = 8; R = 8; B = 8
Loop 9	92.85%	L = 9; T = 8; R = 8; B = 8
	92.76%	L = 8; T = 9; R = 8; B = 8
	92.71%	L = 8; T = 8; R = 9; B = 8
	92.64%	L = 8; T = 8; R = 8; B = 9

Based on the results shown in Table 3.1, the final boundary of the unimportant regions within the CIFAR-10 CNN is determined to include five columns on the left, four rows at the top, three columns on the right, and three rows at the bottom. Similarly, based on the results shown in Table 3.2, the final boundary of the unimportant regions within the GTSRB CNN

includes eight columns on the left, eight rows at the top, eight columns on the right, and eight rows at the bottom. Table 3.3 further reports the overheads of CNN hardware accelerators constructed under different mechanisms. Device A, B, C are arranged the same number of the TOSAM AxMs. Device A uses only the TOSAM AxM configuration with $(h, t) = (2, 6)$; Device B uses both TOSAM AxM settings $(h, t) = (2, 6)$ and $(h, t) = (1, 4)$, and the number of (2,6) AxMs versus (1,4) AxMs follows the important-to-unimportant pixel ratio based on results shown in Tables 3.1 and 3.2; and Device C uses only the TOSAM AxM configuration with $(h, t) = (1, 4)$. The results in Table 3.3 indicate that EMEI achieves a good balance between hardware overhead and classification accuracy: under the constraint of a minimum acceptable accuracy, EMEI incurs low hardware overhead, and the discrepancy between the predicted classification accuracy and the measured hardware-level accuracy is only about 3%.

Table 3.3 Classification and hardware overheads of the different CNN hardware accelerators

		Device A	Device B(EMEI)	Device C
Classification accuracy	Cifar10	93%	89%	68%
	GTSRB	95%	93%	89%
Hardware overheads (ALMs)	Cifar10	10,077	9,813	9,397
	GTSRB	15,638	16,182	15,502

3.6 Conclusions

In this chapter, we introduce an error injection methodology that leverages the error rate distribution matrix of AxMs. Before detailing the methodology itself, we first review the operational principles and structural designs of several commonly used AxMs, including TOSAM, error compensation-based designs, and multipliers built using approximate 4:2 compressors. These foundational concepts help to establish a clear understanding of how different AxMs function and why they are relevant for optimizing CNN hardware accelerators.

The error injection methodology offers a structured approach for assigning suitable precision levels of AxMs to individual PEs in a CNN accelerator. This approach enables designers to meet the minimum classification accuracy required by the target model while improving overall hardware efficiency through a balanced use of high- and low-precision multipliers. The advantages of this method become particularly apparent when the hardware cost difference between adjacent AxM precision levels is significant.

Experimental validation shows that this precision assignment strategy can maintain close alignment between hardware-level and software-level performance, with classification accuracy differences remaining within 3% in typical cases.

Appendix: Performance of Error Matrix-Based Error Injection

References

1. Abderrahmane, N., Lemaire, E., Miramond, B.: Design space exploration of hardware spiking neurons for embedded artificial intelligence. *Neural Netw.* **121**, 366–386 (2020)
[Crossref]
2. Venkataramani, S., Sun, X., Wang, N., Chen, C.Y., Choi, J., Kang, M., Agarwal, A., Oh, J., Jain, S., Babinsky, T., et al.: Efficient AI system design with cross-layer approximate computing. *Proc. IEEE* 108:2232–2250
3. Wang, Y., Li, H., Li, X.: Real-time meets approximate computing: an elastic cnn inference accelerator with adaptive trade-off between QOS and QOR. In: *Proceedings of the 54th Annual Design Automation Conference 2017*, pp. 1–6 (2017)
4. Liu, Z., Yazdanbakhsh, A., Park, T., Esmailzadeh, H., Kim, N.S.: Simul: an algorithm-driven approximate multiplier design for machine learning. *IEEE Micro*. **38**(4), 50–59 (2018)
[Crossref]
5. Chen, Y.H., Krishna, T., Emer, J.S., Sze, V.: Eyeriss: an energy-efficient reconfigurable accelerator for deep convolutional neural networks. *IEEE J. Solid-State Circ.* **52**(1), 127–138 (2016)
[Crossref]
6. Vahdat, S., Kamal, M., Afzali-Kusha, A., Pedram, M.: Tosam: an energy-efficient truncation-and rounding-based scalable approximate multiplier. *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* **27**(5), 1161–1173 (2019)
7. Liu, C., Han, J., Lombardi, F.: A low-power, high-performance approximate multiplier with configurable partial error recovery. In: *2014 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pp. 1–4. IEEE (2014)
8. Kong, T., Li, S.: Design and analysis of approximate 4–2 compressors for high-accuracy multipliers. *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* **29**(10), 1771–1781 (2021)
9. Sun, P., Yu, H., Halak, B., Kazmierski, T.: A method for swift selection of appropriate approximate multipliers for cnn hardware accelerators. In: *2024 IEEE International Symposium on Circuits and Systems (ISCAS)*, pp. 1–5. IEEE (2024)
10. Krizhevsky, A., Hinton, G., et al.: Learning multiple layers of features from tiny images (2009)
11. Stallkamp, J., Schlipsing, M., Salmen, J., Igel, C.: The german traffic sign recognition benchmark: a multi-class classification competition. In: *The 2011 International Joint Conference on Neural Networks*, pp. 1453–1460. IEEE (2011)
12. Gong, Y., Liu, B., Ge, W., Shi, L.: Ara: cross-layer approximate computing framework based reconfigurable architecture for CNNs. *Microelectron. J.* **87**, 33–44 (2019)
[Crossref]
13. Guo, C., Zhang, L., Zhou, X., Qian, W., Zhuo, C.: A reconfigurable approximate multiplier for quantized CNN applications. In: *2020 25th Asia and South Pacific Design Automation Conference (ASP-DAC)*, pp. 235–240. IEEE (2020)
14. Yellu, P., Buell, L., Xu, D., Yu, Q.: Blurring boundaries: a new way to secure approximate computing systems. In: *Proceedings of the 2020 on Great Lakes Symposium on VLSI*, pp. 327–332 (2020)

Footnotes

- 1 https://github.com/HaosenYu/Security_of_AI_Accelerator_Book.git

Part II

Applications of CNN in Hardware Security

OceanofPDF.com

4. A Gem5 Tutorial on Simulating DoS Attacks in Network-on-Chip

Haoyu Wang¹  and Basel Halak¹ 

(1) School of Electronics and Computer Science, University of Southampton, Southampton, UK

 **Haoyu Wang**

Email: Haoyu.Wang@soton.ac.uk

 **Basel Halak (Corresponding author)**

Email: Basel.Halak@soton.ac.uk

Keywords DoS attacks – Network-on-chip (NoC) – Gem5 simulator – Microarchitecture simulation – Security vulnerabilities – Flooding attacks – Simulation methodology – Network security – Performance evaluation – Hardware simulation – Garnet network model – Attack modeling – Traffic analysis – Attack simulation – Defense mechanisms

4.1 Introduction

This chapter serves as a hands-on tutorial focused on security in network-on-chip (NoC) systems, which are widely used communication fabrics in large-scale chips due to their scalability, lightweight design, and efficient communication. As NoCs handle a substantial portion of on-chip communication, they are increasingly exposed to diverse security threats. We begin by introducing the microarchitecture and design principles of NoC-based multiprocessor system-on-chip (MPSoC) platforms. Next, we provide an overview of NoC-specific security issues, including unauthorized access, data corruption, and denial-of-service (DoS) attacks. A

comprehensive literature review offers context on common attack vectors and current countermeasures. We then introduce a practical implementation of a DoS flooding attack model in the Gem5 Garnet simulator, enabling readers to reproduce and evaluate realistic threat scenarios. Finally, we summarize recent advances in NoC security architecture, highlighting promising directions for future research.

To better understand the urgency of securing NoC architectures, it is important to consider recent trends in hardware development. Machine learning models have demonstrated remarkable performance in classification and detection tasks, becoming vital tools in a wide range of applications. While their application in cybersecurity has been extensively explored, their use in hardware security remains relatively underdeveloped [1]. Meanwhile, the demand for large-scale computing chips is rapidly increasing to meet the needs of modern computational tasks, such as training and inference for large language models (LLMs) [2]. Companies like Nvidia and AMD have responded by designing high-performance data server chips. However, the significant data throughput and computational intensity of these workloads introduce new opportunities for adversaries to launch sophisticated and covert attacks [3, 4]. These threats emphasize the growing importance of developing secure hardware systems, particularly through the use of machine learning techniques to enhance resilience against attacks at the architectural level.

4.2 Chapter Overview

This chapter begins with an introduction to the microarchitecture of network-on-chip (NoC)-based multiprocessor system-on-chip (MPSoC) platforms in Sect. [4.3](#). We describe core components such as routers and network interfaces, common communication protocols, and typical architectural designs. This section also introduces tools and methodologies for modeling and simulating NoC systems, equipping readers with practical techniques for analyzing performance and functionality—an essential foundation for subsequent security analysis.

Section [4.4](#) focuses on the security challenges associated with NoCs. It starts with a classification of common attack types, including eavesdropping, spoofing, denial-of-service (DoS), and side-channel attacks. Section [4.4.2](#) provides a step-by-step tutorial on reproducing a DoS flooding attack using the Gem5 Garnet simulator. This hands-on guide demonstrates how to implement and evaluate custom threat models, offering valuable insights into real-world NoC vulnerabilities. Finally, Sect. [4.4.3](#) reviews current countermeasures for NoC security, highlighting their strengths and limitations, and paving the way for future research in secure NoC design.

4.3 Microarchitecture of Network-on-Chips

The concept of network-on-chip (NoC) as a key enabler for designing multiprocessor system-on-chips (MPSoCs) has evolved significantly, representing a major shift away from traditional bus-based and point-to-point communication architectures in SoCs. Initially conceived as a scalable solution for managing complex on-chip communication, NoCs gained prominence with the pioneering work of William Dally, who introduced the theoretical framework for packet-switched micro-networks. This innovation marked a paradigm shift in the way SoC designs were conceptualized [\[5\]](#). Building on this foundation, Luca Benini and Giovanni De Micheli further emphasized NoCs as a transformative “new paradigm” for SoC design, highlighting their critical role in addressing the communication demands of increasingly complex and heterogeneous SoCs [\[6\]](#).

Figure [4.1](#) illustrates a typical mesh NoC architecture. In this design, the NoC comprises nodes interconnected through links or channels. Each node,

often referred to as a tile in chip design, typically includes a router, a network interface, and various intellectual properties (IPs), such as CPUs, caches, or specialized accelerators. Communication between nodes involves packaging data, instructions, and information from one IP into packets, which are then transmitted across the network. Depending on the link width, these packets may be divided into smaller units called flits. Figure 4.1 also shows an example packet structure, derived from the Netrace dataset [7], which demonstrates the standardized format used for efficient packing and unpacking by routers and network interfaces based on a unified architecture.

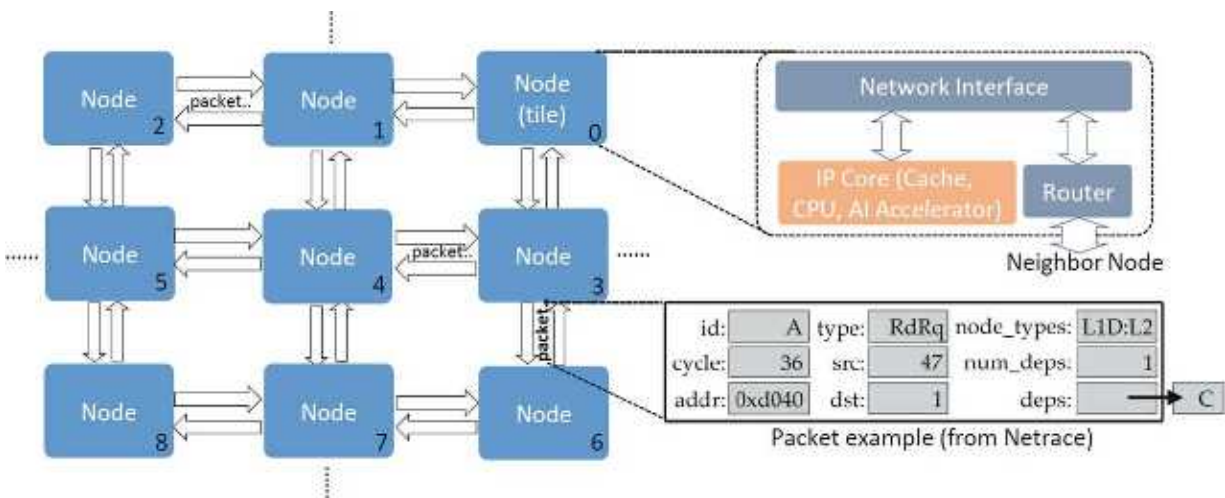


Fig. 4.1 A mesh-based NoC architecture

Different topologies significantly influence the structure and performance of a NoC [8]. Topology refers to the arrangement of nodes and channels within the network. Selecting an appropriate topology for chip development depends on various factors, including complexity, density, workload, and PPA (performance, power, and area). While the mesh topology is commonly employed in MPSoCs, many other topologies exist and can be broadly categorized as either direct or indirect. In direct topologies, each router acts as both a source and destination for traffic, as exemplified by ring, mesh, and torus topologies, which are prevalent in NoCs. Conversely, indirect topologies distinguish between routers and terminal nodes, with only the terminal nodes functioning as sources or sinks for traffic.

The router is a critical component in a NoC, facilitating information transfer, traffic arbitration, and control at each node. It is typically

responsible for three primary functions: routing, flow control, and switching. Consequently, its micro-architecture has a significant impact on the NoC's performance, particularly regarding per-hop delay and energy consumption. A standard router microarchitecture, shown in Fig. 4.2, includes essential components such as virtual channels (VCs), routing logic, allocators, a crossbar switch, and buffers. VC are not required for all NoC configurations; however, they play a crucial role in improving performance when traffic density becomes too high for fixed, non-virtual channels to handle. VCs enable NoCs to implement a dynamic and adaptive channel allocation strategy [9]. This design allows the NoC to efficiently and flexibly manage sudden spikes in high-volume traffic, ensuring smoother communication and reduced congestion. It also contributes to mitigate deadlock cases by cutting off the deadlock loop.

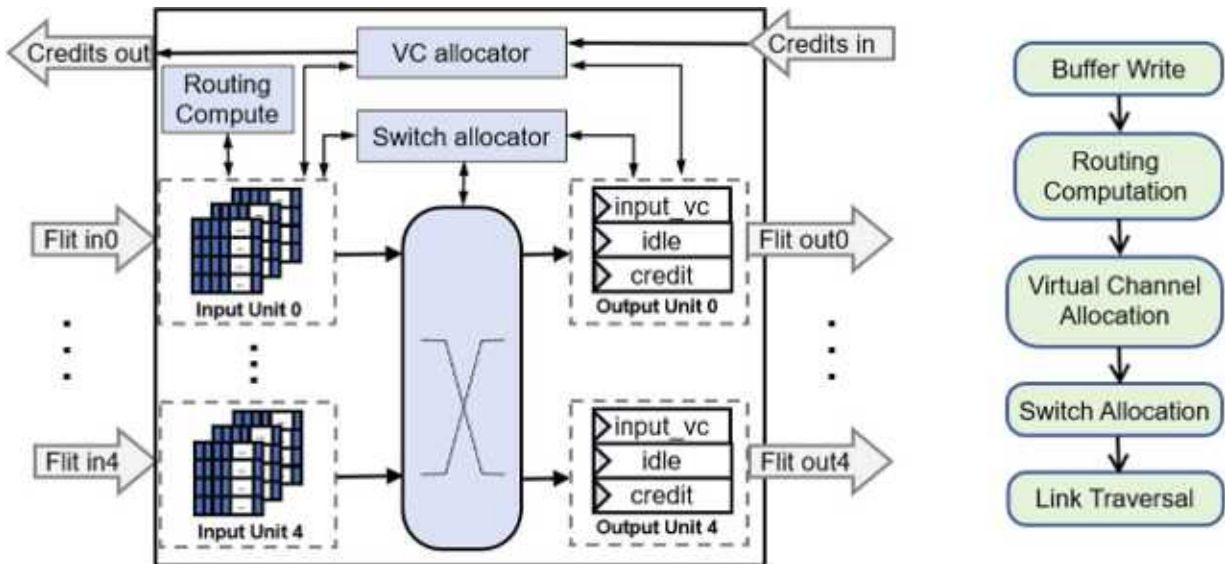


Fig. 4.2 Router microarchitecture and routing process

In operation, a neighboring router first verifies whether downstream routers have available credits. If credits are available, the neighboring router begins sending packets, which are written into the input buffers of the receiving router. The receiving router then computes the routing to determine the appropriate output port direction. Virtual channel allocation modules manage local communication resources by dynamically assigning VCs (buffers) to incoming packets. Following VC allocation and arbitration, the packet undergoes switch allocation and proceeds to the output stage via link traversal.

In addition to the router, the network interface is another critical component of NoC communication, serving as the initial point of interaction [10]. The NI is responsible for packetizing data from the host IPs connected to the local node and analyzing the communication requirements specified by these host IPs [11]. Depending on whether the packet needs to be sent via uni-casting, multi-casting, or broadcasting, the NI adjusts certain fields within the packets to indicate the appropriate communication mode for downstream nodes. Conversely, the NI is also tasked with depacketizing incoming data from other nodes, converting it into a format usable by the locally connected IPs. Due to the extensive customization required for different applications and IP types, there is no standard architecture diagram to represent the NI's design.

Recent developments in NoC designs are increasingly supporting specialized computing paradigms, particularly in machine learning as we mentioned in the introduction. José Flich and Davide Bertozzi highlighted the growing importance of NoCs in the era of nanoscale computing, emphasizing that architectural decisions made today will shape the next generation of NoCs [12]. One promising area of research is 3D network-on-chip architectures, with Miltos Grammatikakis leading studies on the fault-tolerant capabilities of 3D NoCs, which could revolutionize SoC design [13]. NoC architectures are increasingly being adopted not only in nanoscale computing but also in deep neural network (DNN) accelerators. Traditional platforms like CPUs and GPUs face significant challenges related to power consumption and performance when handling DNN workloads. Beyond general-purpose processors, NoCs are more commonly integrated into modern domain-specific accelerators (DSAs), such as DNN accelerators, due to their superior scalability and high communication efficiency. Chen et al. [14] proposes NoC-based DNN accelerators as an effective solution, emphasizing their ability to reduce off-chip memory accesses and enhance computational flexibility. By separating data computation and communication, NoC-based designs offer improved scalability and power efficiency (Fig. 4.3).

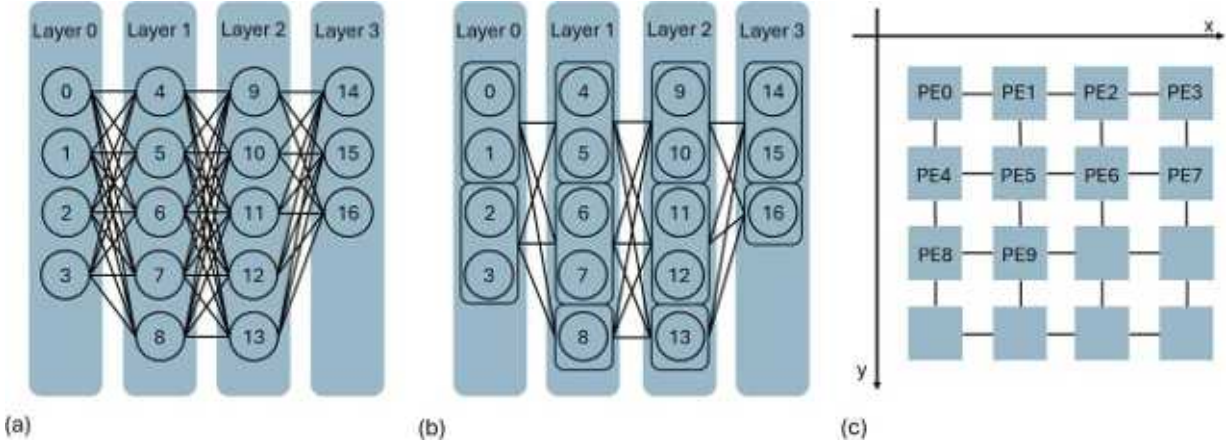


Fig. 4.3 (a) The flattened DNN model; (b) clustering neurons; (c) mapping the clustered ANN model to the target NoC platform

On the industrial front, several companies are actively incorporating academic research findings into their technology stacks, transforming NoCs from theoretical concepts into practical, real-world applications. These advancements aim to deliver technologies that are efficient, robust, and well suited to managing the computational complexities of modern systems.

To achieve the research objectives, a variety of modelling and generation tools for NoC systems can be utilized, many of which have been developed over the past decades. HORNET [15], Noxim [16], Gem5 Garnet [17], and BookSim [18] are command-based tools, primarily developed in C++ that provide comprehensive modelling capabilities for NoC systems. Among these, Gem5 Garnet is particularly notable for enabling full-system simulations, complete with a Linux OS, allowing researchers to assess realistic performance and functionality in diverse use cases.

In addition to modelling tools, several RTL (register transfer level) generators are available for NoCs, including ProNoC [19], RaveNoC [20], OpenPiton [21], ESP [22], and Hermes [23]. These tools are instrumental in rapidly implementing NoC-based SoCs or serving as design templates to integrate IPs and customize systems according to specific requirements. These RTL generators streamline the development process by providing preconfigured NoC frameworks, enabling efficient hardware design and validation. Most of these tools feature a GUI, providing users with a more intuitive and visualized configuration experience. Additionally, they often include early-stage simulation capabilities and brief performance evaluations, even without requiring RTL code. Together, these tools offer a robust suite of solutions for both simulating and building NoC-based

systems, making them invaluable resources for researchers and developers in the field.

4.4 Security in Network-on-Chips

4.4.1 Overview of Security Attacks in NoCs

NoC systems are essential for on-chip communication, making them an attractive target for a variety of attacks. While online threats like malware pose significant dangers, offline methods such as side-channel attacks also compromise the integrity and confidentiality of NoC systems. Due to the wide range and complexity of these threats, it is essential to understand their classification. Charles et al. [24] provide a detailed taxonomy, highlighting five primary types of attacks on NoCs: eavesdropping, spoofing/data integrity, denial-of-service (DoS), buffer overflow/memory extraction, and side-channel attacks.

NoC systems are vulnerable to various attacks that exploit on-chip communication. Eavesdropping involves adversaries intercepting sensitive data, often through hardware Trojans in routers along communication paths. Spoofing attacks occur when malicious nodes impersonate trusted ones, leading to data corruption, information leakage, or system collapse. DoS attacks disrupt access by overwhelming nodes with excessive traffic, often through flooding or tampering with packet destinations. Buffer overflow attacks inject malicious code, causing hardware buffer overflows that compromise shared memory in multicore systems, potentially impacting the entire system. Lastly, side-channel attacks exploit physical variations like power or electromagnetic signals to extract information, leveraging the high communication density in NoC systems.

In this work, our primary focus is on DoS attacks targeting NoC architectures. Among the various types of DoS attacks, traffic-based attacks stand out as particularly prevalent. These attacks are especially harmful because they compromise not only the security and integrity of NoC systems but also lead to significant degradation in overall system performance. This is particularly consequential in real-time and mission-critical applications, where even minor performance disruptions can have severe impacts. Our objective is to not only identify these attacks but also to model them effectively, enabling the development of more robust security solutions for NoCs. A more comprehensive discussion of DoS attacks,

including their various forms and how they can be modelled within the context of NoCs, will be provided in the following subsection.

DoS attacks, particularly flooding attacks, present significant challenges to NoC-based systems, leading to severe performance degradation, reduced quality of service, and increased power consumption. For instance, [25] thoroughly examines the impact of such attacks on MPSoC performance, identifying various attack paths, including overlaps and loop-based attacks. Additionally, [26] provides a detailed categorization of flooding attacks, introducing a conceptual framework that considers the number of attackers and victims. This framework illustrates attack patterns involving malicious IPs targeting victim IPs, offering valuable insights into how these attacks are executed, as depicted in Fig. 4.4.

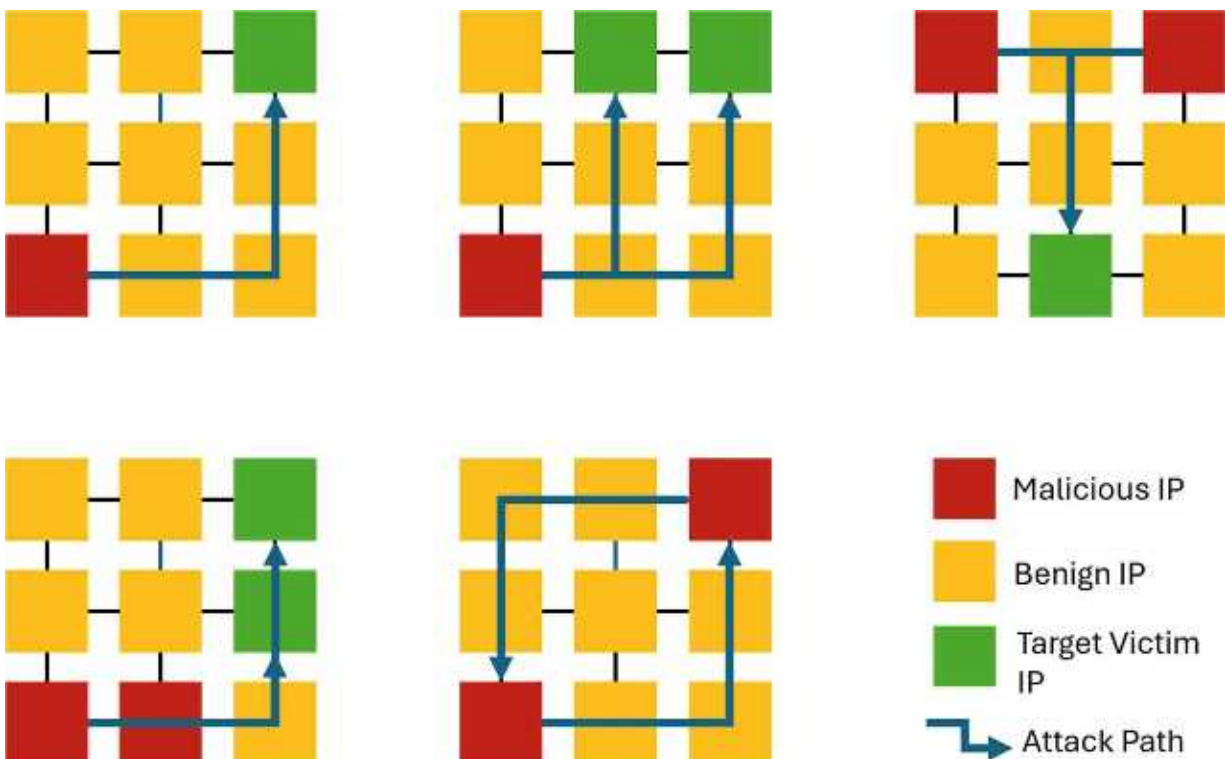


Fig. 4.4 Flooding attack simulation from Sinha et al.

Chaves et al. [27] introduced the term “Flooding DoS” to specifically describe flooding attacks within NoC environments. This study offers a unique perspective, highlighting that such attacks can be initiated not by hardware Trojans but through malicious software operating on a processing element. The research explores two FDoS implementation scenarios: one involving an increase in the packet injection rate and the other focusing on

extending the packet payload length. These findings highlight the complexity and variety of DoS attacks, particularly flooding attacks, and underscore the necessity for comprehensive and multilayered defense strategies.

In this case, it developed a refined flooding DoS threat model for NoCs by introducing an adjustable flooding injection rate. This innovation stems from the observation that varying injection rates can produce different impacts on NoC runtime performance and resource utilization. We integrated this functionality into the synthetic traffic pattern benchmarks in Gem5 Garnet, allowing users to control injection rates through newly introduced command options.

The threat model includes two attack scenarios: (1) a single attacker targeting a single victim and (2) two attackers simultaneously targeting a single victim. The injected packets contain random, useless data. A key novelty in this threat model is that normal communication across all nodes continues without interruption, albeit at a reduced efficiency due to the flooding attack. In this implementation, flooding traffic overlays normal communication, degrading its performance to represent an “ideal” flooding attack scenario, as illustrated in Fig. [4.5](#).

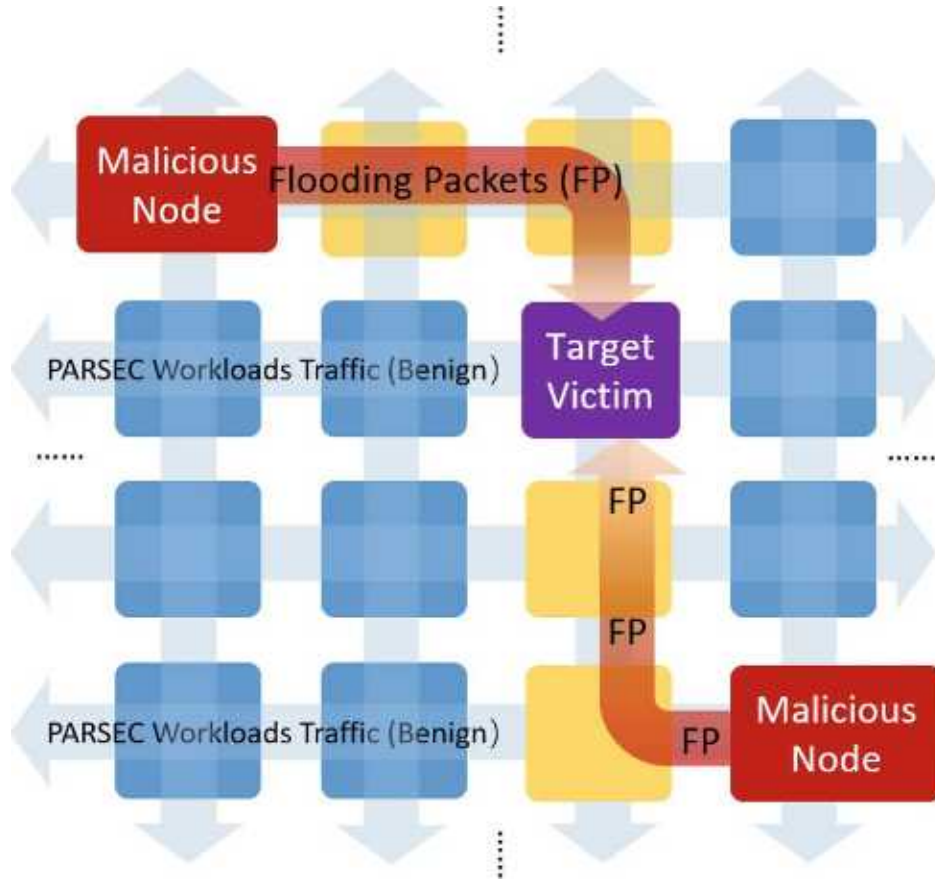


Fig. 4.5 “Ideal” flooding DoS threat modelling

Additionally, this model ensures that adversaries do not compromise network protocols or routing algorithms, maintaining the stealth of the attack. For instance, router communication credits remain balanced because all transactions, including those initiated by flooding attackers, are legitimate, and malicious packets adhere to the system’s default routing algorithm. Consequently, the NoC protocol permits the flooding attack to persist, overwhelming the system with a surge of malicious packets without violating protocol rules or engaging in unauthorized communications.

Consequently, system runtime characteristics, such as performance degradation or complete system failure due to traffic congestion, vary significantly based solely on the flooding injection rate (FIR). This functionality was implemented within workload applications as a malicious “Tick” function (in synthetic traffic pattern benchmark of gem5), with adjustable input parameters for flexibility. It will be explained more in Sect. [4.4.2](#) by a tutorial of reproducing this threat. The right-side plot in Fig. [4.6](#) illustrates the relationship between latency and FIR in our flooding model. As FIR increases, latency escalates dramatically, severely impacting system

performance. For instance, latency rises from 1.1 to 60 times the normal value as FIR increases from 0.1 to 0.9. At FIR=1, the system crashes entirely. Lower FIR values, while more covert, still exert a sustained negative impact on system performance.

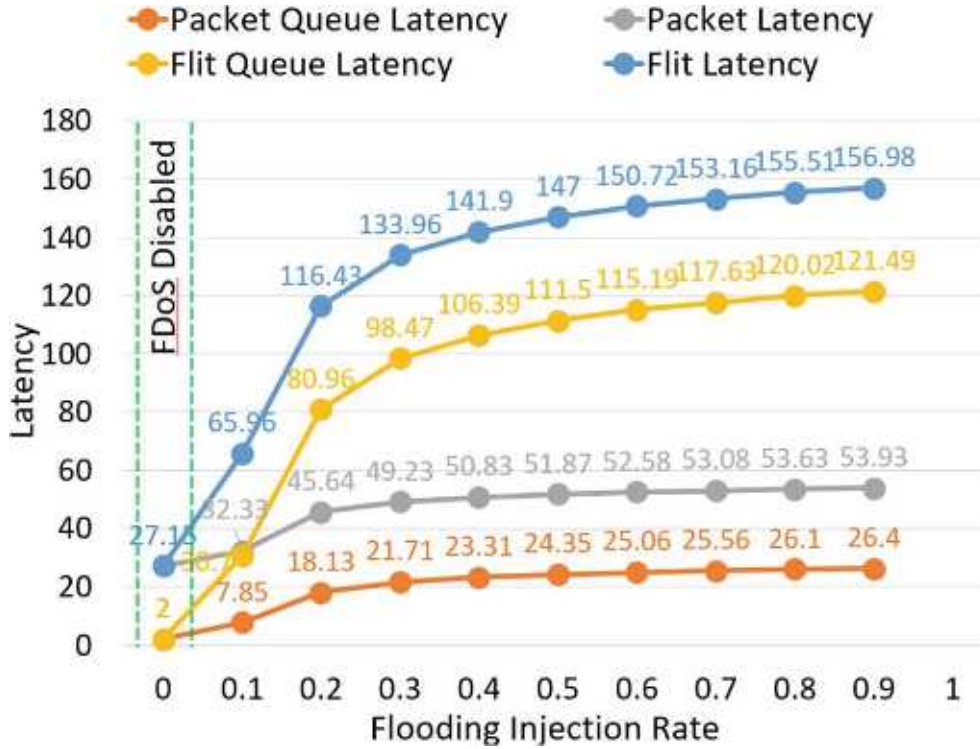


Fig. 4.6 The increasing system latency with increasing flooding injection rate

4.4.2 Tutorial: Fine-Tuning DoS Flooding Attack Reproducing in Gem5 Garnet

4.4.2.1 Overview of Gem5 and Garnet

Gem5 is a modular, open-source, cycle-accurate computer system simulator widely used in computer architecture research and education. It supports multiple instruction set architectures (ISAs) such as x86, ARM, RISC-V, and MIPS and offers various CPU models, including in-order, out-of-order, and atomic models. Gem5 can simulate both full-system and syscall emulation modes, making it versatile for different research needs [28].

Garnet is an on-chip network model within Gem5 that provides cycle-accurate microarchitectural implementation of network components. It allows detailed modeling of routers, links, and topologies, enabling

researchers to study network-on-chip (NoC) behaviors and performance [17].

We selected Gem5 with Garnet for our experiments due to its flexibility, extensibility, and detailed modeling capabilities, which are essential for accurately simulating and analyzing DoS attacks in NoC environments.

4.4.2.2 Setting Up the Gem5 Simulation Environment

To set up the Gem5 simulation environment with Garnet, follow these steps:

1. Clone the Gem5 Repository:

```
git clone https://gem5.googlesource.com/public/gem5
cd gem5
```

- 2. Install Dependencies:** Ensure that the necessary dependencies are installed. This typically includes Python 3, SCons, and other libraries.
- 3. Build Gem5 with Garnet Support:** Use the following command to build Gem5 with the Garnet standalone protocol:

```
scons build/NULL/gem5.debug

\ PROTOCOL=Garnet_standalone -j$(nproc)
```

This builds the Gem5 binary with the NULL ISA and Garnet standalone protocol, suitable for network-only simulations.

4.4.2.3 Configuring the Hardware Architecture

Our experimental setup uses a 4×4 mesh topology with 16 nodes. Each node is connected to a router, and the network operates with the following parameters:

The configuration parameters listed in Table 4.1 define the structural and functional setup of the simulated NoC environment:

- **Number of CPUs:** The simulation includes 16 processor cores, representing a typical multicore MPSoC platform. Each core is mapped

to a corresponding node in the mesh topology.

- **Number of Directories:** There are 16 directories used for cache coherence. This ensures one-to-one mapping between CPU nodes and directory nodes, simplifying coherence management.
- **Topology:** The Mesh_XY setting specifies a 2D mesh interconnect where routers are arranged in a grid. Communication is routed in the XY order, meaning X-direction traversal is completed before moving in the Y-direction.
- **Mesh Rows:** The mesh has four rows and four columns, forming a 4×4 grid of routers. This structure ensures uniform communication distances across the chip.
- **Simulation Cycles:** The simulation runs for 10,000 clock cycles, which is sufficient for observing traffic behavior and the impact of injected attacks without incurring long simulation times.
- **Synthetic Traffic Pattern:** The `uniform_random` pattern randomly selects destination nodes for packet injection, creating a balanced load across the network. It is commonly used for stress-testing NoC behavior.
- **Injection Rate:** Set to 0.05, this parameter determines the probability of packet injection per node per cycle. A moderate injection rate helps evaluate performance under typical workloads without immediate saturation.
- **Virtual Channels Per Virtual Network:** Each input port supports four virtual channels. Virtual channels are used to improve throughput, reduce congestion, and prevent deadlock by allowing multiple logical flows over the same physical link.

Table 4.1 Hardware architecture configuration

Parameter	Value
Number of CPUs	16
Number of directories	16
Topology	Mesh_XY
Mesh rows	4
Simulation cycles	10,000
Synthetic traffic pattern	uniform_random
Injection rate	0.05

Parameter	Value
Virtual channels per virtual network	4

These configuration options provide a well-balanced and realistic NoC simulation environment suitable for evaluating both performance and security behavior under synthetic workloads and adversarial conditions.

4.4.2.4 Building the Hardware Architecture and Running Traffic Pattern Tests

After configuring the hardware architecture, build the simulation environment and run the traffic pattern tests using the command provided. This will execute the simulation with the specified parameters, allowing observation of network behavior under uniform random traffic.

These parameters can be set using the following command:

```
./build/NULL/gem5.debug
\configs/example/garnet_synth_traffic.py
--num-cpus=16
--num-dirs=16
--network=garnet
--topology=Mesh_XY
--mesh-rows=4
--sim-cycles=10000
--synthetic=uniform_random
--injectionrate=0.05
--vcs-per-vnet=4
```

4.4.2.5 Implementing the DoS Flooding Attack

To model the DoS flooding attack, we introduced two custom functions: *tick_attack()* and *flooding_pkt_gen()*.

The *tick_attack()* function builds upon the built-in *tick()* function provided in the STP framework, serving as a parallel mechanism to introduce the flooding behavior. This function is triggered on every simulation cycle and acts as the primary controller for enabling or disabling the malicious packet injection process via the *flooding_pkt_gen()* function. The behavior is regulated by a user-defined flooding rate, which specifies the number of malicious packets injected into the NoC per 1000 cycles. This flooding rate is passed directly from the command line when

initializing the Gem5 simulation, allowing for flexible control over the attack intensity. Figure 4.7 presents the implementation of the *tick_attack()* function, showcasing its interaction with the overall benchmark logic.

```
//Flooding attack tick function
void
GarnetSyntheticTraffic::tick_attack()
{

    if(cnts<=1000){
        if(cnts<=1000*flooding_rate) //Flooding injection rate provided from command line
            flooding_pkt_gen();
        cnts+=1;
    }else{cnts=0;}

    if (curTick() >= simCycles)
        exitSimLoop("Network Tester completed simCycles");
    else {
        if (!tickEvent.scheduled())
            schedule(tickEvent, clockEdge(Cycles(1)));
    }
}
```

Fig. 4.7 “*tick_attack()*” function implementation

The *flooding_pkt_gen()* function performs the core tasks of generating and injecting malicious packets. Users can configure the attack by enabling it and specifying the attacker node(s) and victim node through command-line inputs. Once configured, this function is invoked during each cycle, injecting packets with the attacker node as the source ID and the victim node as the destination ID. The injected packets, designed to be useless or irrelevant to the regular system operations, flood the NoC at a predefined rate. At the conclusion of the packet generation process, the *sendPkt(packet)* function is called to successfully inject the customized packets into the NoC. The implementation of the *flooding_pkt_gen()* function is detailed in Fig. 4.8.

```

//Flooding packet generation function
bool
✓ GarnetSyntheticTraffic::flooding_pkt_gen()
{
    Request::Flags flags;
    if(enable_HT_TP && (id == attacker1 || id == attacker2)){
        MemCmd::Command requestType_att;
        RequestPtr req_att = nullptr;

        Addr paddr_vic = victim;
        paddr_vic <= blockSizeBits;
        requestType_att = MemCmd::WriteReq;
        // requestType_att = MemCmd::ReadReq;
        req_att = std::make_shared<Request>(paddr_vic, 1, flags,requestorId);

        req_att->setContext(attacker1);

        PacketPtr pkt_att = new Packet(req_att, requestType_att);
        pkt_att->dataDynamic(new uint8_t[req_att->getSize()]);
        pkt_att->senderState = NULL;

        sendPkt(pkt_att);
    }
    return false;
}

```

Fig. 4.8 “flooding_pkt_gen” function implementation

Both functions were integrated into the file *GarnetSyntheticTraffic.cc* within the Gem5 source code, ensuring their compatibility with the existing Garnet framework. The updated file, along with instructions for its usage, is available for public access on our GitHub repository.^{[1](#)}

By introducing these functions, we have provided a flexible and reusable mechanism for simulating DoS flooding attacks in NoCs, enabling researchers to further explore and evaluate security vulnerabilities and mitigation strategies in large-scale NoC designs.

4.4.3 Existing Countermeasures for NoC Security

Extensive research has been conducted to understand the security implications of DoS attacks in NoC systems, focusing on mechanisms for identifying malicious activities, particularly in third-party IPs. These security architectures typically adopt one of two design methodologies: in-router monitoring/detection schemes and global detection frameworks. The in-router approach involves implementing independent detection and localization mechanisms directly within each router, allowing for localized and real-time identification of anomalies. This method is well suited for modular systems where individual router autonomy is essential [29, 30]. On the other hand, global detection schemes operate by aggregating and analyzing data from all nodes or traffic components across a defined area, providing a comprehensive view of the system's behavior. This collective judgment approach enables the detection of larger, more distributed attack patterns but often requires greater computational resources and coordination [25, 31]. Both methodologies have their advantages and trade-offs, underscoring the need for context-specific solutions to address the diverse and evolving nature of DoS threats in NoC systems.

In the field of in-router security architectures, various innovative methods have been proposed to address flooding attacks in NoC systems. The Sniffer framework, introduced in [26], employs a lightweight distributed machine learning model (perceptron) integrated into each router node to accurately identify congestion caused by malicious traffic. These perceptrons analyze multiple network parameters, including buffer waiting time, inter-flit intervals, and virtual channel occupancy, to differentiate between attack and non-attack scenarios. A collective decision-making strategy further enhances accuracy by combining congestion data from neighboring routers. Additionally, a priority mechanism ensures the smooth traversal of probing packets during localization, minimizing disruption to legitimate network traffic. The architecture is scalable, exhibits minimal hardware and power overhead, and demonstrates high localization accuracy across varied attack scenarios, including coordinated attacks with multiple MIPs.

The AGAPE secure router architecture in [32] leverages a generative adversarial network (GAN)-based anomaly detection framework to identify and mitigate hardware Trojan (HT) attacks in NoC systems. The architecture incorporates dynamic error correction hardware for fault-injection HTs, bypass links for misrouting HTs, and channel buffers for

packet-drop HTs. Using a semi-supervised GAN, AGAPE models time-series data of various runtime NoC attributes, such as buffer utilization and latency, to detect anomalies and classify HT types with high precision. Instead of isolating infected routers entirely, AGAPE applies targeted mitigation strategies, reducing performance degradation. The architecture includes enhanced routing mechanisms, dynamic link bypassing, and state tracking to maintain network functionality even under attack, achieving improved detection accuracy (up to 91%) while reducing latency and power consumption by 39% and 30%, respectively.

The secure router architecture proposed by Chaves et al. [30]. focuses on mitigating low-and-slow denial-of-service (LSDoS) attacks in NoCs. This architecture integrates a lightweight LSDoS monitor within the router, positioned between the local FIFO buffer and subsequent router components. The monitor employs a finite state machine (FSM) to detect anomalies in packet inter-flit delays, triggering countermeasures when a predefined threshold is exceeded. When an LSDoS attack is detected, the monitor truncates malicious packets by injecting a tail flit, releasing communication resources. Any remaining flits of malicious packets are dropped, effectively neutralizing the attack. This approach ensures minimal disruption to legitimate traffic while maintaining negligible area and performance overheads. Experimental results demonstrate its robustness against both single and multisource LSDoS attacks, offering an effective solution for enhancing NoC security (Figs. 4.9, 4.10 and 4.11).

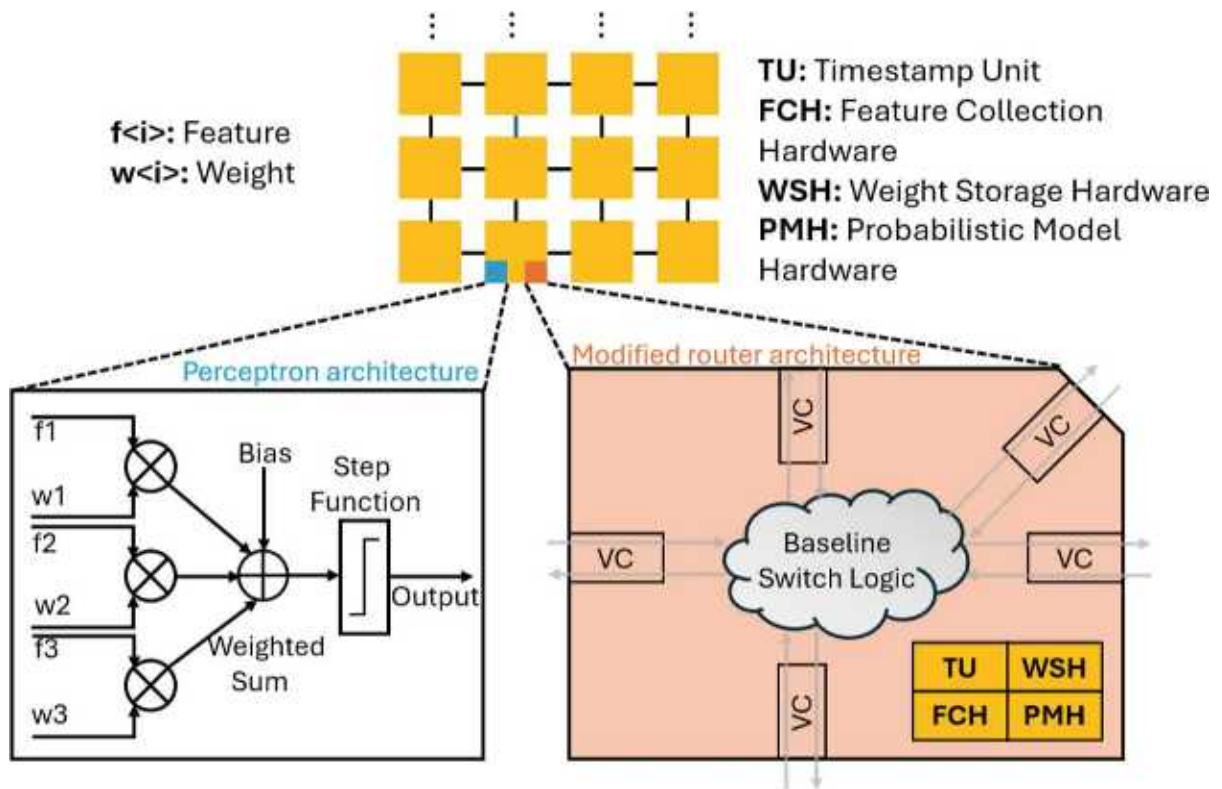


Fig. 4.9 Sniffer architecture

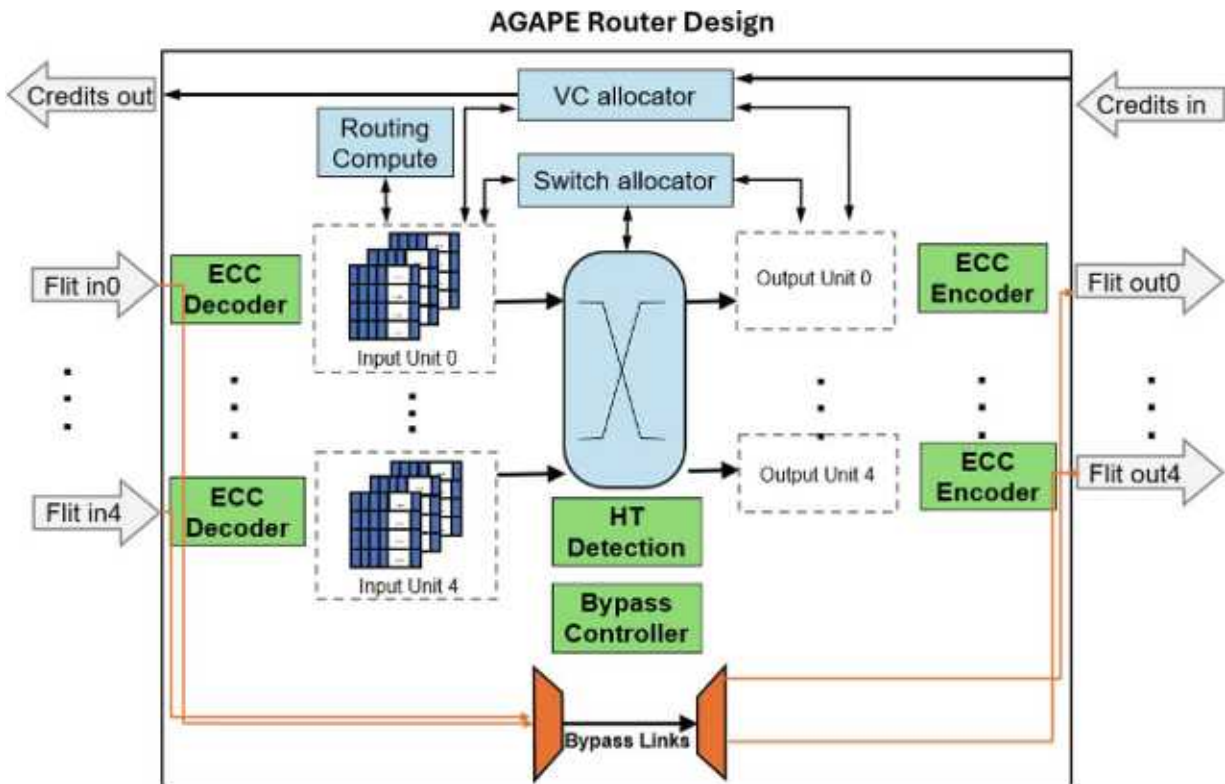


Fig. 4.10 AGAPE router example

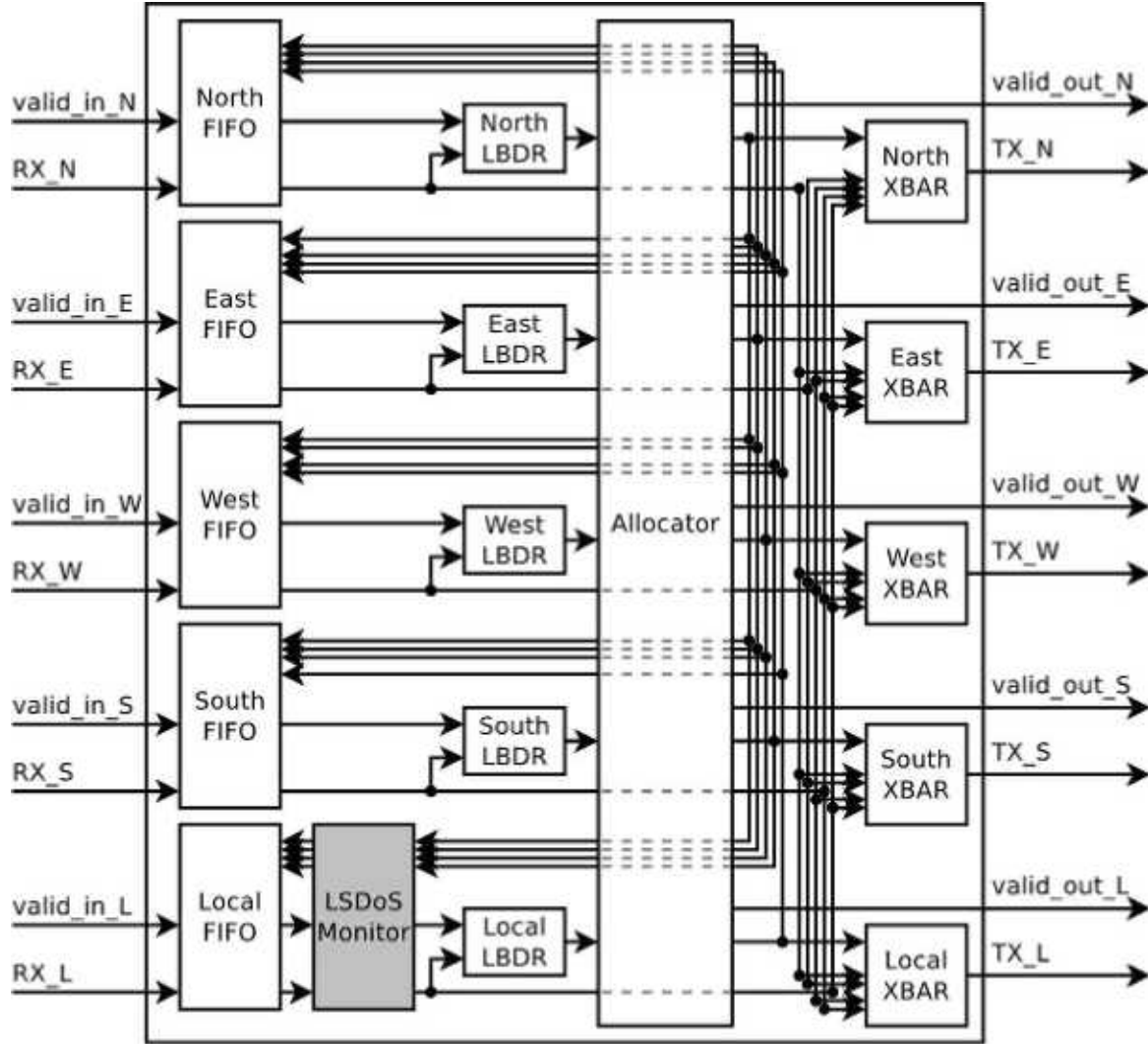


Fig. 4.11 Router architecture with LSDoS monitor

Numerous studies have explored global detection methodologies for addressing security threats in NoC systems, leveraging collective data analysis for comprehensive monitoring. The secure router architecture proposed in the study [33] introduces a lightweight and real-time framework for detecting and localizing DoS attacks in NoC-based SoCs. It relies on two key mechanisms: packet arrival curves (PACs) and destination latency curves (DLCs). Each router profiles traffic during the design phase to establish upper PAC bounds, which are used to monitor communication behavior and flag anomalies in real time. The DLCs, recorded at each destination IP, capture latency statistics and help identify potential malicious sources by analyzing deviations from expected values. The architecture integrates a leaky bucket algorithm to ensure efficient detection with minimal overhead. Upon detecting a threat, the routers collaborate

using diagnostic messages to localize the malicious IP, reducing false positives by evaluating congestion along the network path. Experimental results demonstrate the system's ability to achieve timely detection and localization with an area overhead of less than 6% and power overhead below 4%. This framework highlights a distributed, efficient, and scalable solution for improving NoC security against DoS attacks (Fig. 4.12).

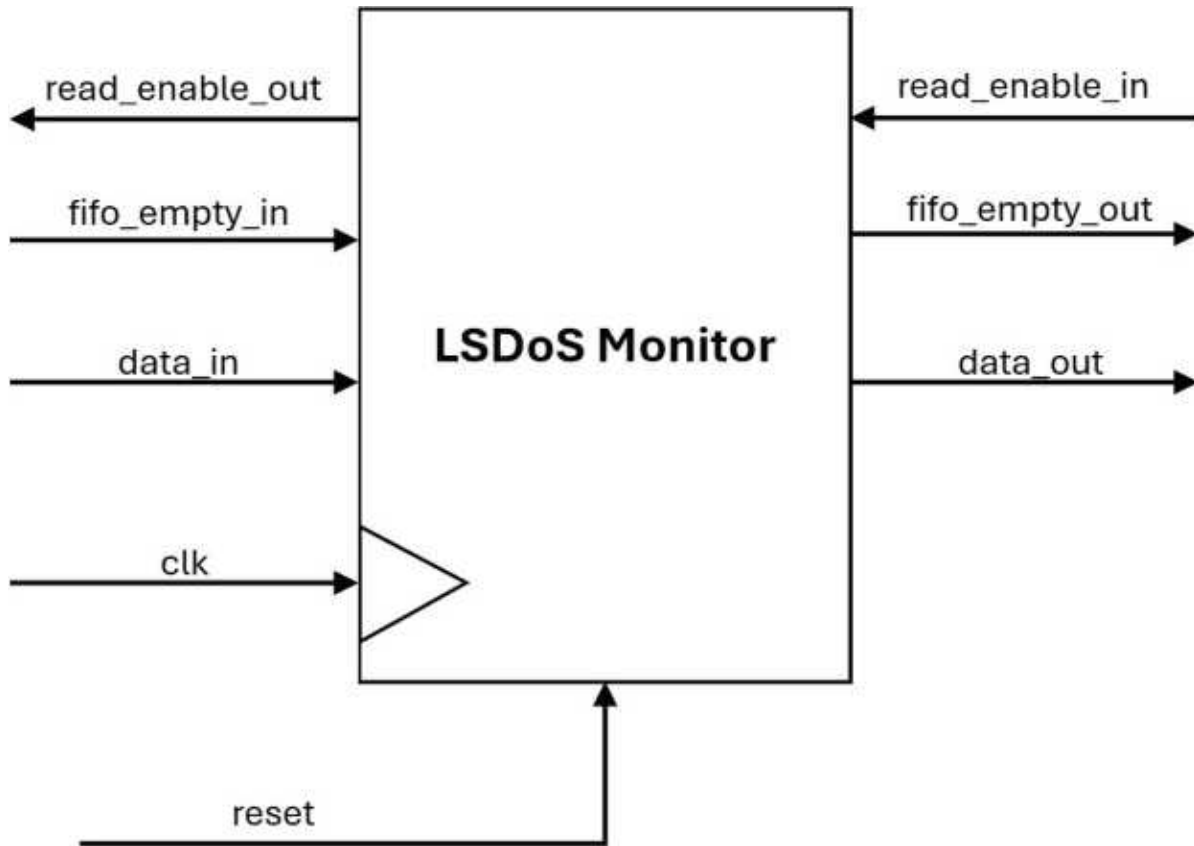


Fig. 4.12 LSDoS Monitor

Expanding on these approaches, Atul Prasad Deb Nath and colleagues proposed the ARK architecture in [34]. The ARK architecture introduces a robust framework for ensuring the security resilience of NoC-based SoCs in the presence of untrusted components. It consists of two primary components: satellite units deployed at each router and IP, and a centralized security policy engine (SPE). The satellite units monitor and detect critical interface events, generate standardized frame-based packets containing security metadata, and enforce real-time security policies by communicating with the SPE. The SPE, acting as the “security brain,” analyzes data received from satellite units, enforces fine-grained security policies, and updates the system’s security state to mitigate threats such as

packet misdirection, masquerading, and data tampering. This architecture is scalable and adaptable to various attack scenarios, providing dynamic on-the-fly detection and mitigation while incurring minimal area and power overhead. Experimental results demonstrate its efficiency in detecting and mitigating malicious NoC communications without significantly affecting performance (Fig. 4.13).

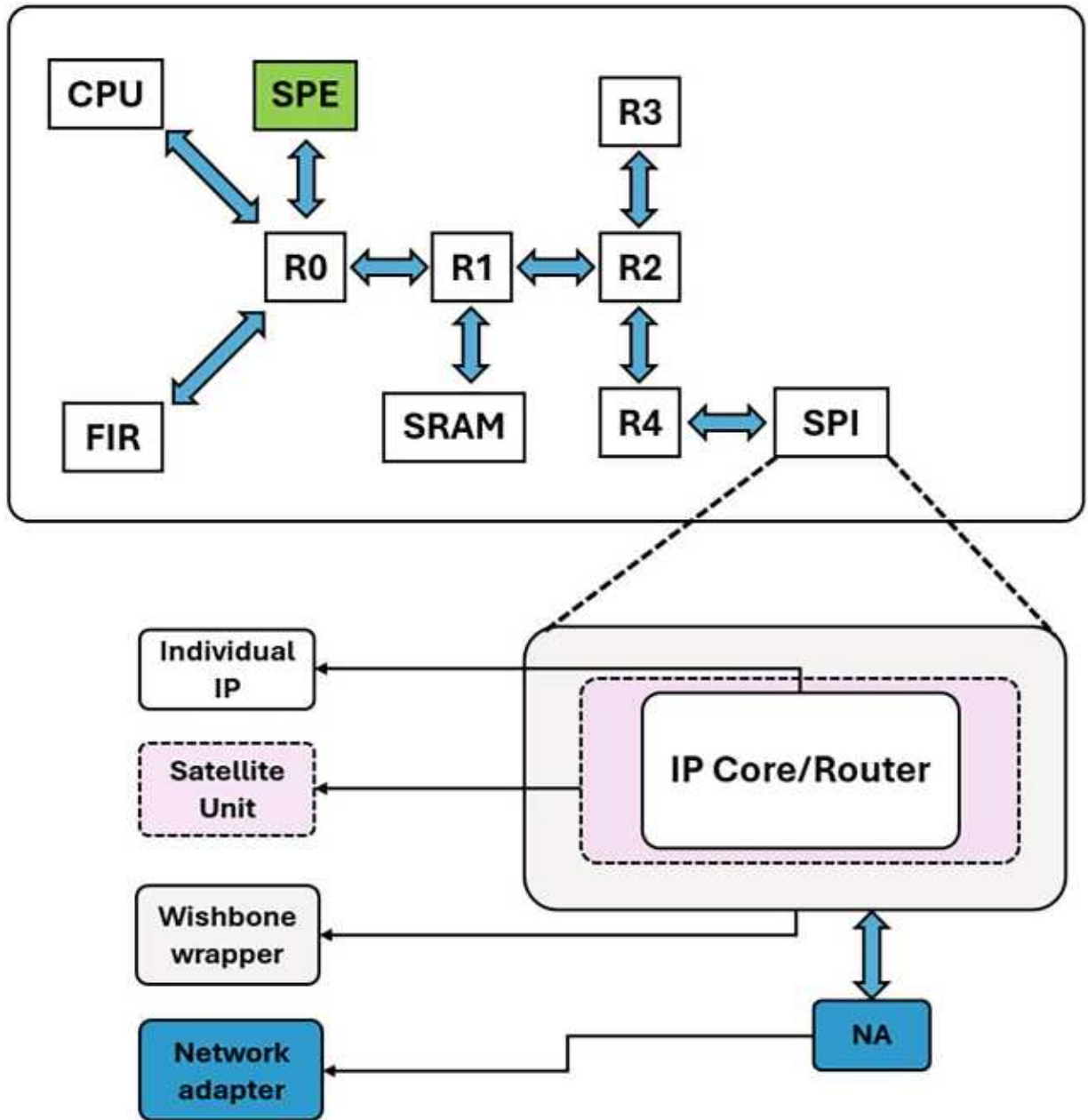


Fig. 4.13 Security ARK NoC architecture

4.5 Conclusions

This chapter provided a hands-on tutorial for simulating DoS attacks in NoC systems using Gem5. Starting with an overview of NoC architecture and security challenges, we focused on flooding-based DoS threats and introduced a practical approach to reproducing these attacks. Readers were guided through setting up the Gem5 simulation environment, configuring a mesh-based NoC, and modifying traffic generation code to inject attack packets at controlled rates. These steps enable systematic experimentation and help assess the performance degradation caused by varying attack intensities.

The goal of this tutorial is to equip readers with a reusable simulation framework for studying NoC security. By integrating custom threat models into Gem5 Garnet, users can replicate real-world vulnerabilities and evaluate system responses. This chapter also discussed key countermeasures from recent literature to offer broader context. We hope this resource supports researchers and engineers in exploring, testing, and advancing secure NoC design through accessible and replicable simulation practices.

References

1. Elnaggar, R., Chakrabarty, K.: Machine learning for hardware security: opportunities and risks. *J. Electr. Testing* **34**(2), 183–201 (2018) [[Crossref](#)]
2. Liu, Y., He, H., Han, T., Zhang, X., Liu, M., Tian, J., Zhang, Y., Wang, J., Gao, X., Zhong, T., et al.: Understanding llms: A comprehensive overview from training to inference. *arXiv preprint arXiv:2401.02038* (2024)
3. de Vries-Gao, A.: Ai is eating data center power demand—and it’s only getting worse. *WIRED* (2025). <https://www.wired.com/story/new-research-energy-electricity-artificial-intelligence-ai>
4. Micro, T.: Navigating the threat landscape for cloud-based gpus. *Trend Micro Security News* (2024). <https://www.trendmicro.com/vinfo/us/security/news/threat-landscape/navigating-the-threat-landscape-for-cloud-based-gpus>
5. Dally, W.J., Towles, B.: Route packets, not wires: on-chip interconnection networks. In: *Proceedings of the 38th Annual Design Automation Conference*, pp. 684–689 (2001)
6. Benini, L., De Micheli, G.: Networks on chips: a new soc paradigm. *Computer* **35**(1), 70–78 (2002) [[Crossref](#)]

7. Hestness, J., Grot, B., Keckler, S.W.: Netrace: dependency-driven trace-based network-on-chip simulation. In: Proceedings of the Third International Workshop on Network on Chip Architectures, pp. 31–36 (2010)
8. Sharma, R., Sharma, R., Sharma, S.: Comparative design and analysis of mesh, torus and ring noc. *Procedia Comput. Sci.* **57**, 610–619 (2015)
9. Yoon, Y.J., Concer, N., Petracca, M., Carloni, L.: Virtual channels vs. multiple physical networks: A comparative analysis. In: Proceedings of the 47th Design Automation Conference, pp. 162–165 (2010)
10. Bhojwani, R., Mahapatra, R.: Survey of network on chip (noc) architectures & contributions. *Int. J. Comput. Appl.* **43**(14), 1–7 (2012)
11. Gautier, L., Pétrot, F., Clément, B.: Design environment for the support of configurable network interfaces in noc platforms. In: Proceedings of the Conference on Design, Automation and Test in Europe, pp. 1–6. European Design and Automation Association (2010)
12. Flich, J., Bertozzi, D.: Designing Network On-Chip Architectures in the Nanoscale Era. CRC Press, Boca Raton (2010)
[\[Crossref\]](#)
13. Coppola, M., Grammatikakis, M.D., Locatelli, R., Maruccia, G., Pieralisi, L.: Design of Cost-Efficient Interconnect Processing Units: Spidergon STNoC. CRC Press, Boca Raton (2020)
[\[Crossref\]](#)
14. Chen, K.C., Ebrahimi, M., Wang, T.Y., Yang, Y.C.: Noc-based dnn accelerator: A future design paradigm. In: Proceedings of the 13th IEEE/ACM International Symposium on Networks-on-Chip, pp. 1–8 (2019)
15. Lis, M., Ren, P., Cho, M.H., Shim, K.S., Fletcher, C.W., Khan, O., Devadas, S.: Scalable, accurate multicore simulation in the 1000-core era. In: Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS), pp. 175–185 (2011).
<https://doi.org/10.1109/ISPASS.2011.5762734>. <https://ieeexplore.ieee.org/document/5762734>
16. Catania, V., Mineo, A., Monteleone, S., Palesi, M., Patti, D.: Noxim: An open, extensible and cycle-accurate network on chip simulator. In: Proceedings of the IEEE 26th International Conference on Application-specific Systems, Architectures and Processors (ASAP), pp. 162–163 (2015). <https://doi.org/10.1109/ASAP.2015.7245728>. <https://ieeexplore.ieee.org/document/7245728>
17. Agarwal, N., Krishna, T., Peh, L.S., Jha, N.K.: Garnet: A detailed on-chip network model inside a full-system simulator. In: Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS), pp. 33–42 (2009). <https://doi.org/10.1109/ISPASS.2009.4919636>. <https://ieeexplore.ieee.org/document/4919636>
18. Jiang, N., Becker, D.U., Michelogiannakis, G., Balfour, J., Towles, B., Shaw, D.E., Kim, J., Dally, W.J.: A detailed and flexible cycle-accurate network-on-chip simulator. In: Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS), pp. 86–96 (2013). <https://doi.org/10.1109/ISPASS.2013.6557149>. <https://ieeexplore.ieee.org/document/6557149>

19. Monemi, A., Tang, J.W., Palesi, M., Marsono, M.N.: Pronoc: a low-latency network-on-chip based many-core system-on-chip prototyping platform. *Microprocess Microsyst* **54**, 60–74 (2017). <https://doi.org/10.1016/j.micpro.2017.09.003>. <https://www.sciencedirect.com/science/article/pii/S0141933117301810>
20. Ignacio, A.: Ravenoc: Configurable network-on-chip (2022). <https://github.com/aignacio/ravenoc>
21. Balkind, J., McKeown, M., Fu, Y., Nguyen, T., Zhou, Y., Lavrov, A., Shahrad, M., Fuchs, A., Payne, S., Liang, X., Matl, M., Wentzlaff, D.: Openpiton: an open source many-core research framework. *ACM SIGPLAN Not.* **51**(4), 217–232 (2016). <https://doi.org/10.1145/2872362.2872414>. <https://dl.acm.org/doi/10.1145/2872362.2872414>
22. Mantovani, P., Giri, D., Guglielmo, G.D., Piccolboni, L., Zuckerman, J., Cota, E.G., Petracca, M., Pilato, C., Carloni, L.P.: Agile soc development with open esp. In: *Proceedings of the 39th International Conference on Computer-Aided Design (ICCAD)*, pp. 1–9 (2020). <https://doi.org/10.1145/3400302.3415753>. <https://dl.acm.org/doi/10.1145/3400302.3415753>
23. Moraes, F., Calazans, N., Mello, A., Möller, L., Ost, L.: Hermes: an infrastructure for low area overhead packet-switching networks on chip. *Integration* **38**(1), 69–93 (2004). <https://doi.org/10.1016/j.vlsi.2004.03.001>. <https://www.sciencedirect.com/science/article/pii/S016792600400028X>
24. Charles, S., Mishra, P.: A survey of network-on-chip security attacks and countermeasures. *ACM Comput. Surv.* **54**(5), 1–36 (2021). <https://doi.org/10.1145/3450964>. <https://dl.acm.org/doi/10.1145/3450964>
25. Charles, S., Lyu, Y., Mishra, P.: Real-time detection and localization of distributed dos attacks in noc-based socs. *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* **39**(12), 4510–4523 (2020). <https://doi.org/10.1109/TCAD.2020.3016820>. <https://ieeexplore.ieee.org/document/8989809>
26. Sinha, M., Gupta, S., Rout, S.S., Deb, S.: Sniffer: a machine learning approach for dos attack localization in noc-based socs. *IEEE J. Emerg. Sel. Top. Circuits Syst.* **11**(2), 278–291 (2021). <https://doi.org/10.1109/JETCAS.2021.3082180>. <https://ieeexplore.ieee.org/document/9439909>
27. Chaves, C.G., Sepúlveda, J., Hollstein, T.: Lightweight monitoring scheme for flooding dos attack detection in multi-tenant mpsoes. In: *Proceedings of the IEEE International Symposium on Circuits and Systems (ISCAS)*, pp. 1–5 (2021). <https://doi.org/10.1109/ISCAS51556.2021.9401600>. <https://ieeexplore.ieee.org/document/9401600>
28. Lowe-Power, J., Ahmad, A.M., Akram, A., Alian, M., Amslinger, R., Andreozzi, M., Armejach, A., Asmussen, N., Beckmann, B., Bharadwaj, S., et al.: The gem5 simulator: Version 20.0+. *arXiv preprint arXiv:2007.03152* (2020)
29. Charles, S., Lyu, Y., Mishra, P.: Real-time detection and localization of dos attacks in noc based socs. In: *Proceedings of the Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pp. 1–6. IEEE (2019)
30. Chaves, C.G., Azad, S.P., Sepúlveda, J., Hollstein, T.: Detecting and mitigating low-and-slow DoS attacks in NoC-based MPSoCs. In: *Proceedings of the 14th International Symposium on*

- Reconfigurable Communication-centric Systems-on-Chip (ReCoSoC), pp. 82–89 (2019). <https://doi.org/10.1109/ReCoSoC.2019.8705300>. <https://ieeexplore.ieee.org/document/8705300>
31. Weerasena, H., Jia, X., Mishra, P.: Topology-aware detection and localization of distributed denial-of-service attacks in network-on-chips. *ACM Trans. Embed. Comput. Syst.* **24**(1), 1–25 (2025)
 32. Wang, K., Zheng, H., Li, Y., Li, J., Louri, A.: Agape: Anomaly detection with generative adversarial network for improved performance, energy, and security in manycore systems. In: *Proceedings of the Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pp. 849–854 (2022). <https://doi.org/10.23919/DATe54114.2022.9774720>. <https://ieeexplore.ieee.org/document/9774720>
 33. Mishra, P.K., Charles, S.T., Lyu, Y.: Real-time detection and localization of dos attacks in noc-based soc architectures (2023). <https://patents.google.com/patent/US11797667B2/en>. U.S. Patent 11,797,667
 34. Nath, A.P.D., Boddupalli, S., Bhunia, S., Ray, S.: Ark: Architecture for security resiliency in soc designs with network-on-chip (NoC) fabrics. Technical Report (2019). <https://arxiv.org/abs/1903.11822>
-

Footnotes

¹ https://github.com/HaosenYu/Security_of_AI_Accelerator_Book.git

5. A Case Study of CNN Applications in Network-on-Chip Security

Haoyu Wang¹ , Ahmad Atamli¹  and Basel Halak¹ 

(1) School of Electronics and Computer Science, University of Southampton, Southampton, UK

 **Haoyu Wang**

Email: Haoyu.Wang@soton.ac.uk

 **Ahmad Atamli**

Email: A.Atamli@soton.ac.uk

 **Basel Halak (Corresponding author)**

Email: Basel.Halak@soton.ac.uk

Keywords CNN applications – Security monitoring – DoS attack detection – DoS attack localization – Network-on-Chip security – CNN classifiers – Feature extraction – CNN segmentation – Security analytics – Attack path detection – Traffic monitoring – Victim localization – Attacker identification – Real-time monitoring – Performance metrics

5.1 Introduction

In the previous chapter, we discussed the background and general architecture of network-on-chip (NoC). With its highly efficient communication structure and scalable design methodology, NoC has emerged as one of the most promising solutions for on-chip interconnection design. However, it is increasingly facing significant security challenges, such as malicious third-party intellectual property (IP) introduction and

malicious packet injection. These threats can lead to serious consequences, including resource exhaustion, additional unwanted power consumption, and even denial-of-service (DoS) attacks [1, 2]. As such, addressing NoC security is no longer optional in the era of large-scale computing hardware.

To mitigate these risks, researchers and integrated circuit (IC) designers have been actively proposing a variety of security countermeasures. These solutions aim to monitor runtime performance, detect malicious data or behaviors, localize malicious nodes or IPs, isolate attackers, and safeguard critical communications within the network. For instance, [3] introduced a reconfigurable NoC security architecture that dynamically adapts to different threat scenarios. Weerasena et al. [2] proposed a topology-aware detection and localization framework for distributed denial-of-service (DDoS) attacks using graph neural networks. Additionally, [4] discussed lightweight and trust-aware routing mechanisms to bypass malicious IPs, enhancing the security of NoC-based systems.

This chapter focuses on applying deep learning models, specifically convolutional neural networks (CNNs), to NoC security research. Deep learning techniques offer unique advantages over conventional methods, such as their ability to analyze complex patterns and adapt to dynamic environments. The target of this chapter is threefold:

- **Highlighting Deep Learning in Hardware Security:** We will explore various applications of deep learning models in hardware security, emphasizing why these models, particularly CNNs, are better suited for this scenario compared to conventional methods.
- **Introducing the Framework in the Case Study:** This chapter details CNN-based attack detection and localization framework for NoC security. We will provide a comprehensive explanation of the framework and its implementation, including a step-by-step tutorial on reproducing the DoS flooding attack scenario.
- **Performance Evaluation and Comparison:** Finally, we will present the performance and hardware overhead of the method in this chapter. This includes a comparison with other state-of-the-art works, highlighting the effectiveness and efficiency of the approach in addressing NoC security challenges.

Through this chapter, we aim to demonstrate the potential of deep learning techniques, particularly CNNs, in enhancing NoC security,

providing both a theoretical foundation and practical insights for future advancements in this critical field.

5.2 Chapter Overview

This chapter introduces the application of CNN accelerators in addressing security challenges in NoCs. It begins with an introduction to the broader role of deep learning in hardware security, setting the stage for the subsequent focus on CNN-based techniques. Section [5.3](#) provides a comprehensive overview of deep learning applications in hardware security, covering foundational concepts and highlighting notable examples of real-world applications.

Section [5.4](#) introduces the CNN-based method for detecting and localizing flooding denial-of-service (FDoS) attacks in NoCs. This includes an overview of the case study, which integrates CNN models for classification and segmentation, as well as techniques like multi-frame fusion and victim completing enhancement for precise attack path and victim/attacker localization.

Feature selection and model design considerations are addressed in Sects. [5.5](#) and [5.6](#), focusing on the rationale behind choosing VCO (virtual channel occupancy) and BOC (buffer operation counts) features and the process of selecting an optimal CNN architecture for detection and localization tasks.

In Sect. [5.7](#), the experimental setup and results evaluation are presented. This includes a detailed discussion of detection and localization performance under various benchmarks, alongside comparisons with related works. Hardware overhead and scalability analyses are also provided.

Finally, Sect. [5.8](#) concludes the chapter by summarizing the contributions and insights gained, reinforcing the significance of CNN-based solutions for addressing NoC security challenges.

5.3 Deep Learning Model Application in Hardware Security

5.3.1 Background

Since 2006, when Geoffrey Hinton coined the term “deep learning,” [\[5\]](#) this transformative technology has been applied across a wide range of fields, extending beyond scientific research to profoundly impact people’s daily lives [\[6\]](#). Scientists leverage deep learning to unravel complex, previously

unknown principles of nature and science. At the same time, robots equipped with deep learning models assist individuals with disabilities, enabling them to lead more independent and barrier-free lives. In the field of engineering, deep learning has demonstrated its potential to address challenges in innovative ways. For instance, aerodynamics engineers employ deep learning models to optimize designs for motors or airplanes, creating more efficient and effective shapes.

However, there remains a critical engineering challenge in the field of computer hardware that demands nonconventional solutions: hardware security. Unlike traditional engineering problems, hardware security spans the entire hardware supply chain [Z], including every stage from circuit design and functionality verification to manufacturing, packaging, testing, and vending. The unpredictable nature of hardware vulnerabilities means that no single stage can be deemed entirely secure. Engineers must maintain equally high levels of vigilance throughout the hardware life cycle. The growing globalization of the hardware supply chain has further heightened the urgency of addressing these security concerns [Z]. With components often sourced, manufactured, and distributed globally, the risks of malicious modifications, counterfeits, and other security threats have escalated. Introducing smarter, automated, and less human-dependent security mechanisms is critical to safeguarding hardware at every stage of its life cycle. Deep learning, with its ability to identify patterns, analyze anomalies, and adapt to dynamic environments, offers a promising approach to enhancing hardware security in this interconnected and globalized era [8].

5.3.2 Deep Learning Application Examples in Hardware Security

There are relatively few studies applying deep learning models to hardware security, with most focusing on detecting hardware Trojans at various levels of the IC design flow, including the register-transfer level (RTL), netlist, gate, and layout levels. These works predominantly utilize deep learning for static security detection, identifying malicious modifications or vulnerabilities embedded during the design and manufacturing stages of the IC supply chain. In contrast, other studies apply deep learning to detect anomalies in runtime performance data or behaviors, addressing dynamic security threats that emerge during the operational phase of IC products. This distinction highlights the dual application of deep learning in hardware

security: static detection focuses on ensuring integrity during development and production, while dynamic detection safeguards IC products during their deployment and usage, offering a comprehensive approach to addressing security threats throughout the IC life cycle.

Li et al. [9] proposes a hardware Trojan detection method based on a back-propagation (BP) neural network, leveraging its strong nonlinear mapping and adaptive learning capabilities to enhance the detection process. The approach focuses on extracting and analyzing side-channel power consumption data from chips under test. A BP neural network with a three-layer architecture (input, hidden, and output layers) is used to model and classify subtle variations in power consumption data, distinguishing between Trojan-free and Trojan-infected chips. The system was validated on an AES encryption circuit implemented on an FPGA, where power consumption data were collected and processed using MATLAB. The BP neural network effectively retained key features of the power consumption data, offering high classification accuracy. The experimental results demonstrated that larger Trojans were easier to detect, while the effectiveness decreased for smaller Trojans due to noise and waveform alignment issues, highlighting areas for future improvement. This work underscores the potential of neural networks for robust hardware security applications.

Hasegawa et al. [10] presents a novel approach to hardware Trojan classification in gate-level netlists using multilayer neural networks (MLNNs). The proposed method extracts 11 distinctive Trojan-net features from each net in a netlist and uses these features as input to the neural network. The architecture employs multiple hidden layers in the MLNN to classify nets as either Trojan affected or normal. By leveraging supervised learning and applying a weighted cross-entropy loss function, the method addresses the imbalance in Trojan and normal net instances. This approach achieves high classification accuracy, maximizing both true positive rate (TPR) and true negative rate (TNR), demonstrating superiority over existing methods like SVM-based models in benchmarks. This contribution highlights the potential of deep learning for automated and precise hardware Trojan detection, significantly improving the reliability of IC designs.

The paper [11] presents a post-deployment trust evaluation architecture for wire-less cryptographic ICs, designed to detect dormant hardware

Trojans that remain undetected during pre-deployment testing. The architecture integrates an on-chip neural network-based classifier, parametric measurement sensors, and a programmable stimulus generator to assess trustworthiness dynamically. The deep learning component, implemented through a neural network, analyzes side-channel parameters, such as current measurements, to differentiate between trusted and compromised behavior. By leveraging the adaptability of deep learning, the classifier is capable of detecting anomalous patterns associated with Trojans without requiring prior knowledge of their specific design or behavior. The approach was validated using a wireless cryptographic IC and a reconfigurable analogue neural network chip, demonstrating high detection accuracy and low rates of false positives and negatives. The architecture is efficient, introducing minimal overhead, and emphasizes robustness and flexibility for real-world operational environments, making it a significant advancement in post-deployment hardware security.

The paper [12] introduces a novel anomaly detection framework for power-grid controllers using an enhanced temporal deep learning model. It leverages long short-term memory (LSTM) networks to predict the normal behavior of controllers based on hardware performance counter (HPC) data. The system computes a reconstruction error distribution (RED), capturing the deviations between predicted and observed controller behaviors. Using statistical tests on RED, the method detects anomalies with high accuracy, including zero-day attacks. This approach requires no prior knowledge of attack data, making it effective against unknown threats. The experimental results demonstrate nearly perfect detection rates and low latency, emphasizing its capability to safeguard critical cyber-physical infrastructure like power grids.

The paper [13] introduces RATAFIA, a ransomware detection framework leveraging deep learning to identify and differentiate malicious encryption processes in modern computing systems. RATAFIA employs two sequential long short-term memory (LSTM) autoencoders: the first autoencoder operates on time-domain data gathered from hardware performance counters (HPCs) to model normal system behavior and detect anomalies, while the second utilizes Fast Fourier Transformation (FFT) to analyze the frequency domain characteristics of HPC data. This dual-layer approach allows the framework to distinguish ransomware from benign high-computation processes, such as disk encryption programs, with

enhanced precision. The system achieves this by training the autoencoders on normal system behaviors and correlating reconstruction errors for anomaly detection. Furthermore, RATAFIA introduces a correlation module that uses prestored templates of benign disk encryption processes to minimize false positives, ensuring reliable ransomware detection without requiring kernel modifications. This methodology showcases the effective application of LSTM-based deep learning models for real-time, lightweight, and robust security monitoring (Figs. 5.1 and 5.2).

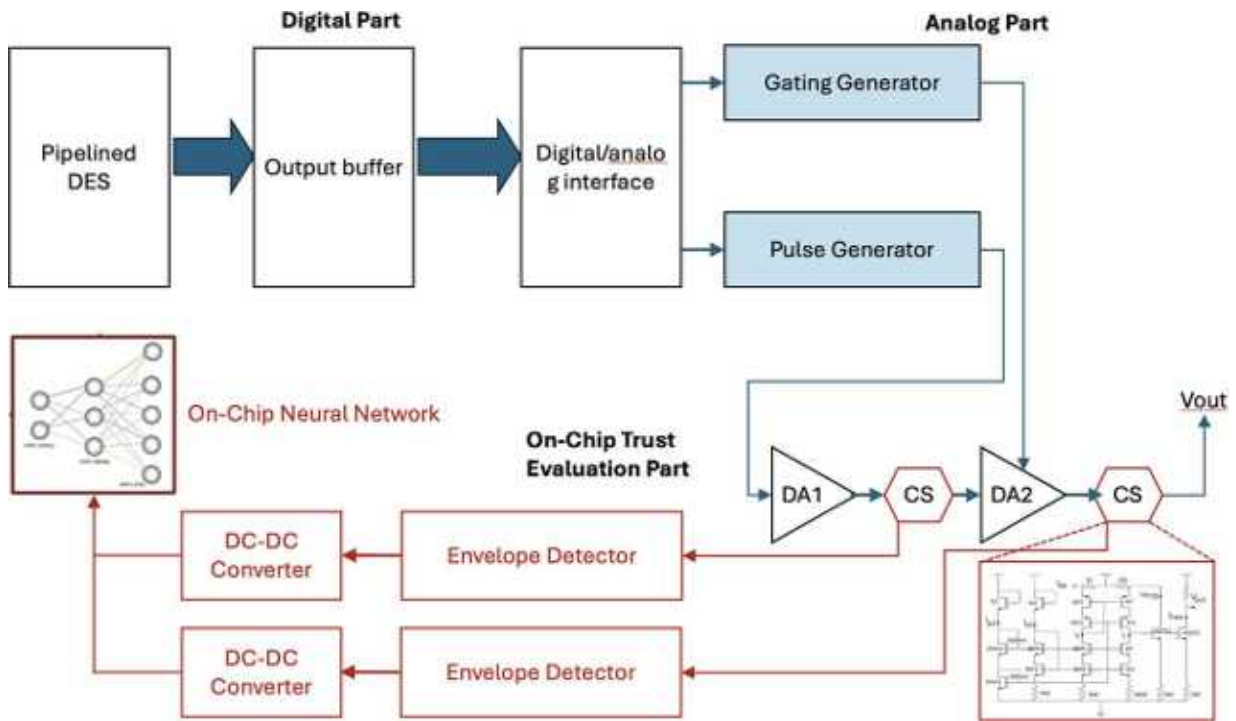


Fig. 5.1 Architecture of IC integrating an on-chip trust evaluation module

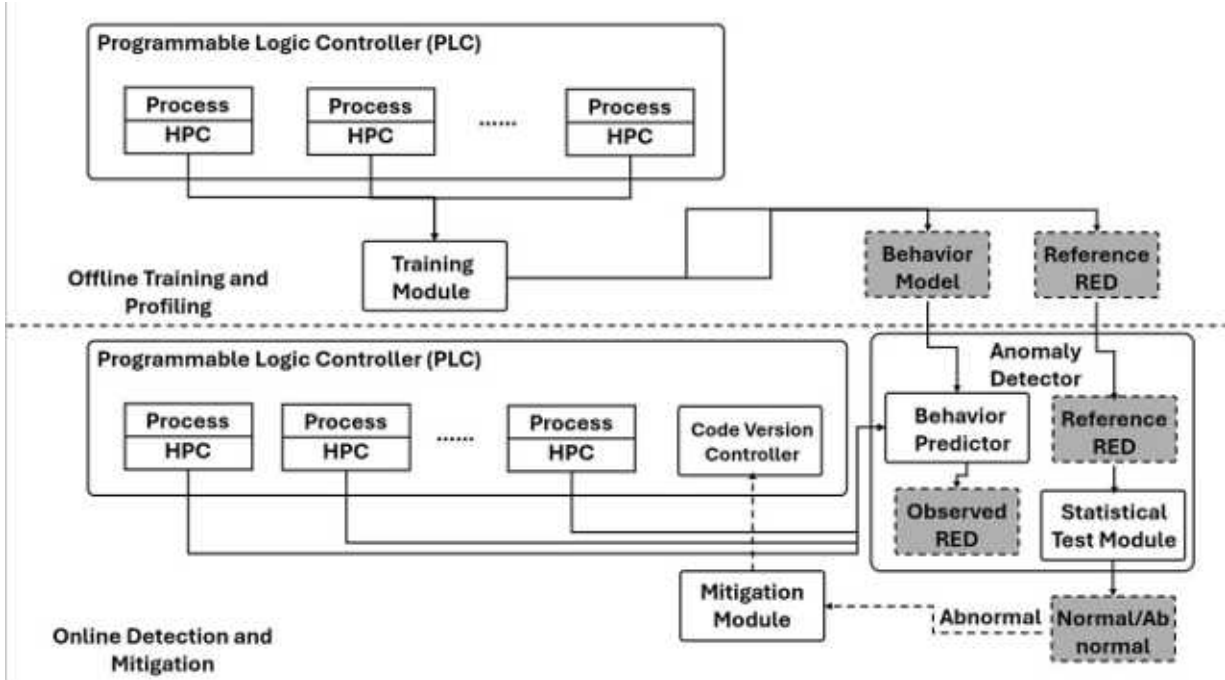


Fig. 5.2 A deep temporal model

The paper discussed in this section explores the application of various deep learning models to address a range of hardware security issues using innovative approaches. Compared to traditional detection methods, which often rely on predefined algorithms or rule-based anomaly detection driven by expert predictions, deep learning techniques offer significant advantages. Deep learning models can automatically learn and extract meaningful features directly related to security concerns, minimizing the need for extensive human intervention. This capability allows for more accurate, adaptable, and efficient detection of threats.

By leveraging deep learning, hardware security systems can swiftly and intelligently respond to potential vulnerabilities or attacks, enhancing their ability to safeguard target hardware systems. This automated and data-driven approach not only improves detection accuracy but also reduces the time and resources required for manual analysis, making it a powerful tool in the fight against evolving hardware security threats.

5.4 CNN-Based Flooding DoS Detection and Localization in NoCs

This section demonstrates a deep learning-based framework for detecting security issues in NoCs, with a particular focus on denial-of-service (DoS) attacks, while the concept of DoS attacks was detailed in the previous chapter, which emphasizes flooding attacks as one of the most pervasive threats in NoCs. As such, this method prioritizes not only detecting these attacks but also localizing their sources within the NoC.

Despite the growing interest in machine learning for hardware security, the use of convolutional neural networks (CNNs) in this context remains rare. To the date, CNNs have not been previously applied to NoC security. This case study leverages CNN models for detecting and localizing flooding attacks in NoCs, marking a significant advancement in the use of deep learning for hardware security. By addressing this critical gap, we aim to contribute a novel approach that enhances the robustness and reliability of NoC systems against malicious threats.

5.4.1 Overview of the Case Study

Inspired by computer vision techniques such as pattern recognition and semantic segmentation, a novel approach was designed to analyze runtime global features of all routers in NoCs by visualizing them as image frames. This approach transforms the problem of anomaly detection and localization into an object detection and segmentation task, which is particularly suitable for NoCs with regular topologies like 2D meshes or even extended to 3D mesh configurations. We used the Gem5 simulator to extract virtual channel occupancy (VCO) and buffer operation counts (BOC) data from the input ports of routers in a 16×16 NoC at specific runtime instances. The presence of a DoS flooding attack is clearly marked by anomalously high local feature values in these frames. Additionally, the BOC matrix frames reveal abnormal routing patterns, providing critical insights into attack pathways. These findings validate the potential of employing CNN models to effectively detect and localize anomalies in NoCs. This framework consists of three core stages:

- 1. DoS Detector (CNN Classification)**
- 2. DoS Profile Localizer (CNN Segmentation)**

3. Victim and Attacker Localization (multi-frame fusion (MFF) combined with table-like method (TLM))

The framework operates as follows:

1. Periodically, the detector processes sampled VCO feature frames from 2–4 directions of each router, provided no previous attack was detected.
2. If the detector identifies any anomalous frames, the corresponding BOC feature frames are collected and sent to the segmentation model-based localizer for detailed analysis. The input data at this stage can be 1–4 frames which represent the feature for different directions, aiming to a higher-accurate flooding profiling.
3. The localizer reconstructs the attack routes by merging segmented frames, identifying all victim routers. The TLM-based algorithm then accurately determines the source of the attacks by pinpointing the malicious nodes. This algorithm has the minimum computation and latency cost to achieve a fast response.
4. The system promptly resumes the next round of VCO sampling and detection/localization, ensuring rapid identification of any remaining anomalies until no abnormal frames persist.

This framework ensures efficient and systematic detection and localization of anomalies in NoCs, leveraging the power of CNNs for high accuracy and scalability in managing complex network security challenges.

5.4.2 DoS Detector: CNN Classification

To detect DoS flooding attacks in NoCs, a CNN classification model was trained using VCO features. The reasoning behind selecting VCO as a key indicator is elaborated in Sect. 5.5. The model requires input frames representing the four possible directional ports of routers in NoCs: east (E), north (N), west (W), and south (S), as illustrated in Fig. 5.3. Central routers in a 2D mesh NoC possess four directional ports, edge routers have three, and corner routers are limited to two. Additionally, edge routers lack input ports connected to external traffic. As a result, the input frame for the system is structured as an $R \times (R-1)$ matrix, where R represents the row

count of the 2D mesh NoC. The reason for having $R-1$ rows in the input image is that routers at the edges of the NoC have at most three ports, as there are no external connections beyond the NoC boundary. Since the VCO value corresponds to each port, the correct size for the input frames is $R \times (R-1)$. VCO values dynamically adjust to runtime traffic loads, increasing when virtual channels are heavily utilized, making them suitable for detecting anomalous traffic patterns.

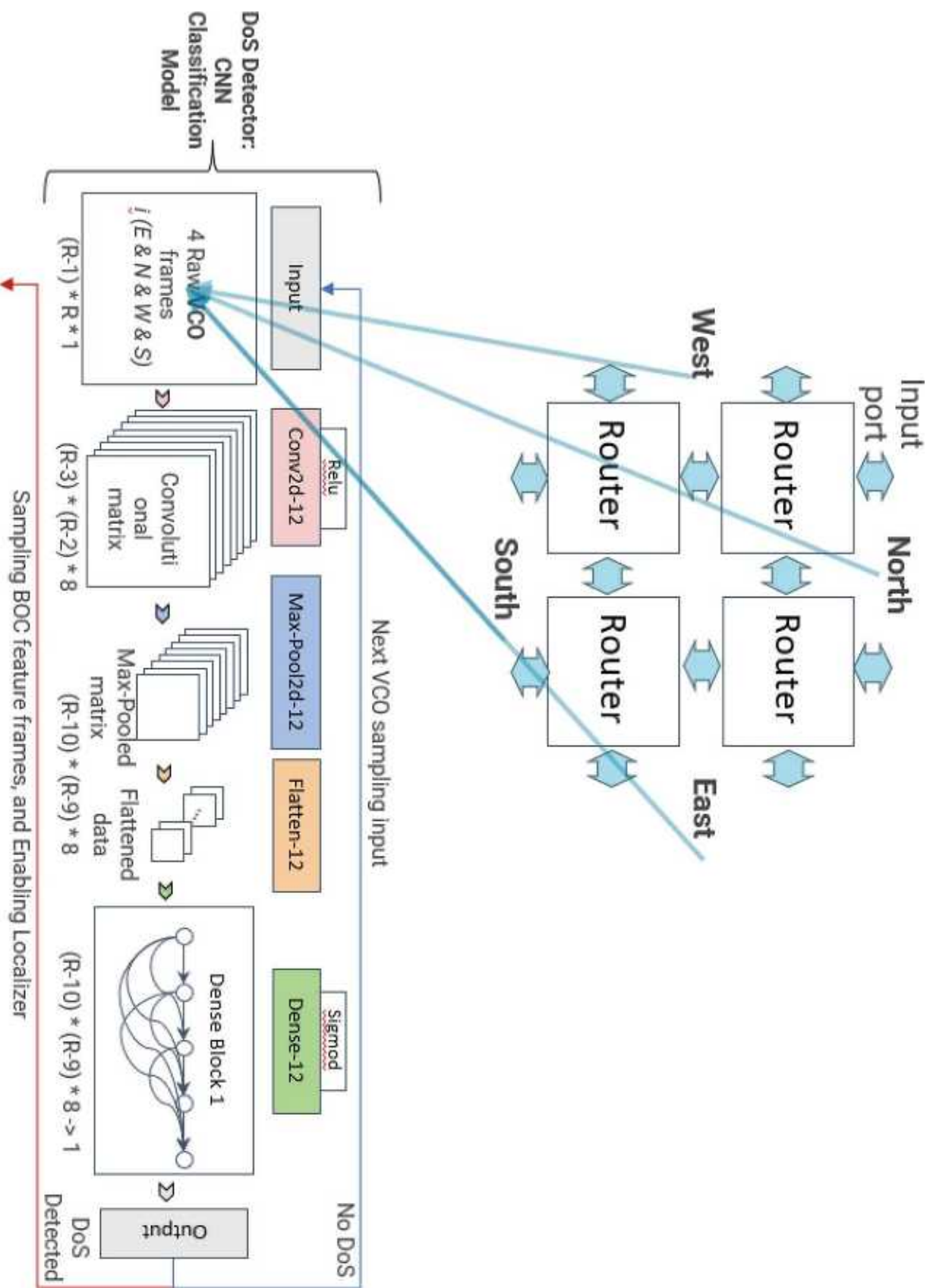


Fig. 5.3 CNN classification model

To achieve a balance between hardware resource efficiency and detection precision, a lightweight CNN architecture was developed, as depicted in Fig. 5.3. The architecture comprises an input layer (gray block), a convolutional layer with ReLU activation (pink block), a max-pooling layer to reduce dimensionality (blue block), a flatten layer (orange block) to transform the data, and a dense output layer (green block). During the training phase, the model processes directional input frames, learning critical features by optimizing its weights and biases. These features allow the model to generalize and make accurate predictions when it encounters similar traffic patterns during operation. By maintaining a compact design, this model ensures effective real-time attack detection while minimizing hardware overhead, making it a practical solution for NoC security.

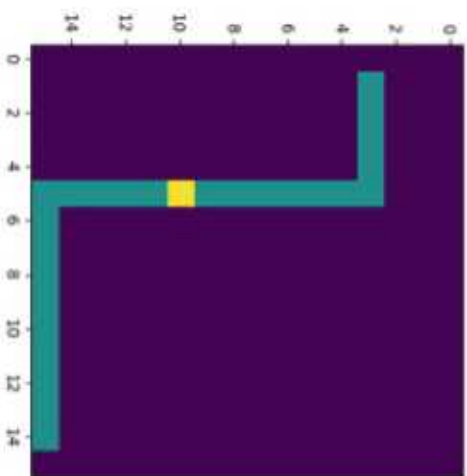
5.4.3 DoS Localizer: CNN Segmentation

Another CNN model, specifically designed for segmentation tasks, has been trained to process BOC frames collected alongside VCO frames when abnormalities are detected. Leveraging deep learning-based semantic segmentation techniques, such as those popularized by the U-Net model in medical imaging, this segmentation model aims to localize flooding attacks within the NoC and reconstruct the entire attacking route. The high precision offered by CNN segmentation models makes them ideal for this task.

The input layer of the DoS localizer, as illustrated in Fig. 5.4, receives abnormal “E || N || W || S” frames, representing input from the four possible directions of the routers. This selective input strategy minimizes computational overhead while maintaining effectiveness. The feature frames, representing buffer operation counts, capture the number of reads and writes at each input port of the routers, providing a detailed snapshot of network activity.

760	1637	1420	1354	2147	2404	2702	2836	2887	2717	2488	2129	1762	1291	999	677
1316	1637	1864	2203	2699	2867	3222	3273	3234	3041	2540	2486	2219	1866	1332	1230
1167	2003	1421	2699	3094	3293	3068	3671	3707	3632	3443	3095	2834	2291	2067	1719
2182	3294	3627	3946	6332	6604	4122	4176	4333	4127	3880	3265	3275	2805	2501	2164
2024	2848	3273	3333	2931	7022	4467	4375	4710	4316	4233	3953	3933	3146	2874	2478
2730	3129	3643	3083	4706	5094	4846	4976	5095	4819	4623	4781	5036	3307	3109	2754
2923	3299	3719	4032	4384	4284	4234	6983	5153	5284	5046	4891	4513	4135	3773	3430
3011	3413	3846	4210	4566	7743	3297	5291	5425	5065	4867	4693	4366	3829	3484	3041
3018	3403	3839	4209	4540	7710	5124	5277	5347	5090	4917	4647	4120	3778	3486	3040
2886	3314	3250	4108	4437	7547	5922	5153	5160	4925	4726	4433	4041	3696	3306	2944
2737	3093	3512	3843	4241	5651	4821	4971	4953	4721	4580	4283	3896	3508	3243	2796
2005	2917	3333	3638	4099	7110	4540	4666	4693	4427	4327	4043	3696	3277	2998	2613
2150	2554	2992	3269	3562	6206	4163	4282	4277	4079	3995	3685	3316	2889	2602	2190
1713	2104	2517	2852	3112	6246	3668	3764	3703	3560	3467	3312	2828	2446	2149	1722
1212	1576	1961	2286	2635	3793	3142	3118	3144	3119	2923	2674	2302	1928	1596	1242
698	1049	1326	1664	2299	5483	3773	3673	3709	5442	3099	4645	4246	3910	3520	3099

Raw data: Buffer Write Number



Segmentation result

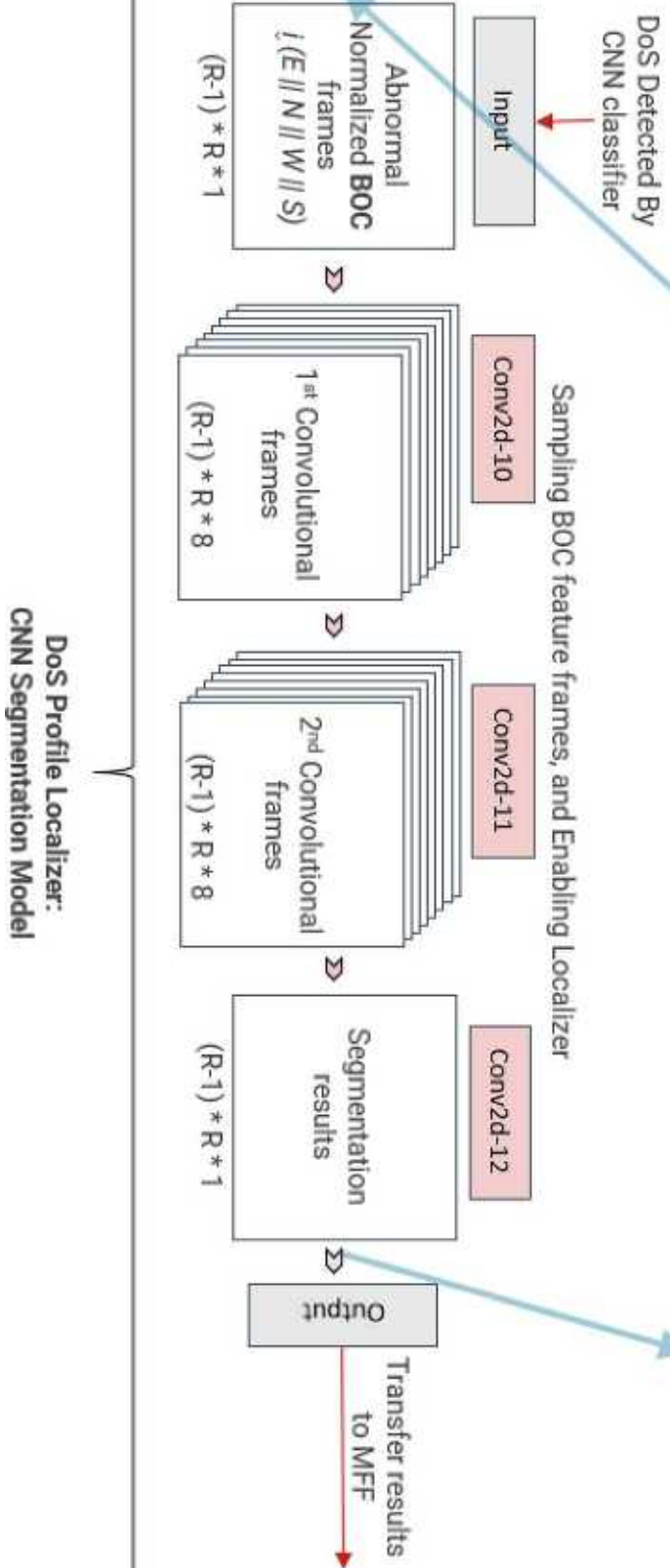


Fig. 5.4 CNN segmentation model

The segmentation model, shown in the Fig. 5.4, consists of a compact yet efficient architecture: an input layer, two convolutional layers, and an output layer. The first and second convolutional layers progressively abstract key spatial and operational features from the input frames. These layers are followed by the output layer, which generates segmentation maps, highlighting regions of abnormal behavior. The model's training process is guided by dice accuracy, which iteratively adjusts parameters to minimize discrepancies between the predicted segmentation results and the ground truth.

While additional convolutional layers could further enhance segmentation precision, they would also significantly increase the hardware overhead. To balance accuracy and hardware resource constraints, the model architecture can be dynamically tuned, ensuring adaptability to different application scenarios. Ultimately, the segmentation results are passed to subsequent components of the detection framework, enabling precise localization of attackers and victims in the NoC.

5.4.4 DoS Attack Path, Victims, and Attacker Localization

The first critical step in attacker localization is reconstructing the flooding attack routes, known as routing-path victims (RPV). This process is guided by Algorithm Fig. 5.5 and visually detailed in Fig. 5.6. Victim localization integrates the multi-frame fusion (MFF) technique, enhanced by an optional victim complementing enhancement (VCE) function. The VCE function refines and completes RPV localization under specific conditions, making it highly effective when initial detection accuracy is sufficient. Before applying the MFF technique, segmentation results are preprocessed into a standard binary 16×16 matrix format through binarization and padding. In the final MFF result frame, any node with a pixel value of 1 is identified as a victim. Drawing inspiration from the RPV deduction strategy in [14], the VCE function employs a pseudo-source ID, adjacent to the attacker, and a specific destination ID under the reverse XY-routing algorithm to deduce all RPVs between these points. This ensures a more accurate and complete identification of attack paths.

Algorithm 1: Victim (attacking route) Localization

Data: Segmented frames: *seg_result_d* (d: e, n, s, w)

Result: All victim node IDs including Routing-Path Victims (RPV) and Target Victims (TV)

```
1 while Getting all seg_result_d do
2   seg_bin_d = Binarization(seg_result_d)
3   full_frame_d( $16 \times 16$ )  $\leftarrow$ 
     Zero_Pad_R/L/T/B(seg_bin_d)
   /* Multi-Frame Fusion */
4   while  $i \leq$  Number of (malicious)seg_result_d do
5     |  $MFF+ = full\_frame\_d[i]$ 
6   end
7   victim_x, victim_y  $\leftarrow$  Where(MEF(pixel_value == 1))
8   victim_id  $\leftarrow$  Get_Node_ID(victim_x, victim_y)
   /* VCE by xy-routing deducing */
9   if Enable Victim Completing Enhancement then
10    | pseudo_src  $\leftarrow$  Get_SRC(vic_id)
11    |  $TV \leftarrow dst \leftarrow$  Get_DST(vic_id)
12    |  $RPV \leftarrow XY\_Routing(pseudo\_src, dst)$ 
13  end
14 end
```

Fig. 5.5 Localization algorithm

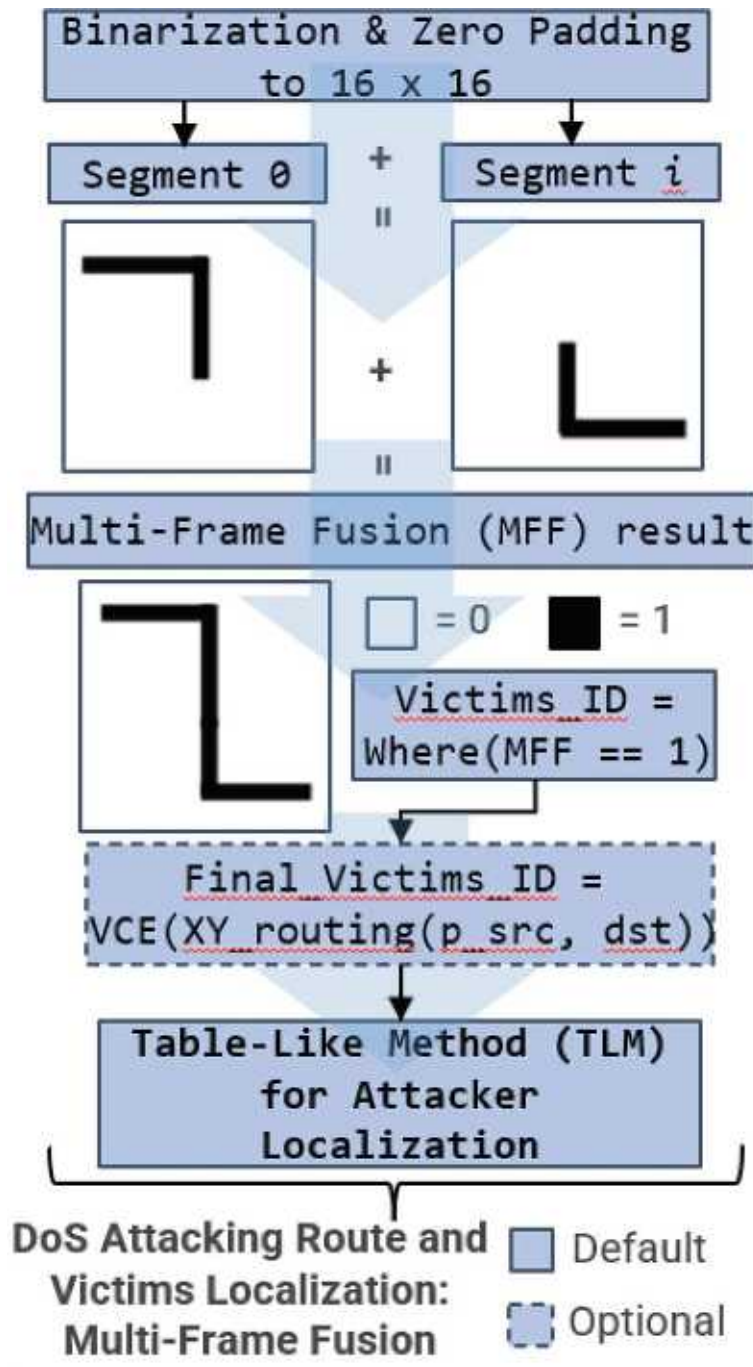


Fig. 5.6 DoS attacking router and victim localization: Multi-frame fusion

The final stage in the framework is attacker localization. By leveraging the XY-routing algorithm, followed by both attackers and regular traffic, the framework identifies specific attack patterns. Using the directional insights from feature frames, a table-like method (TLM) was developed to enumerate single or multi-attacker scenarios for each victim. The TLM structure, detailed in Fig. 5.7, systematically lists all detected abnormalities,

attack patterns, and directional frame combinations. Each row outlines identified attack routes, predicted attacker numbers, and directional abnormalities (e.g., **E=1** signifies anomalies in data from **east input ports**). The results, located between rows 4 and 7, include attacker IDs calculated through a combination of abnormal input port directions and the XY-routing algorithm.

Frame Combination Dir Frame	One Abnormal Frame	Two Abnormal Frames				Three Abnormal Frames				Four Abnormal Frames
		E & N/S or W & N/S $(\text{Max}(\text{N/S}) - \text{Min}(\text{N/S})) \% R = 0$								
		$\text{Max}(\text{E/W}) - \text{Min}(\text{E/W}) < R-1$ (2R-1, 3R-1,...) $> \text{Max}(\text{E/W}) - \text{Min}(\text{E/W}) > R$								
Attack Path Example (Frame Fusion Results)										
	Attacker Number	1	1	≥ 2	≥ 2	≥ 2	≥ 2	≥ 2	≥ 2	≥ 2
	E=1	Max(E) + 1	Max(E) + 1	Max(E) + 1 by multiple samples	Max(E) + 1 by multiple samples	Min(W) - 1, Max(E) + 1	Max(E) + 1	Max(E) + 1	Max(E) + 1	Max(E) + 1, Min(W)
	N=1	Max(N) + R	Max(E) + 1	Max(E) + 1 by multiple samples	Max(E) + 1 by multiple samples	Min(W) - 1, Max(E) + 1	Max(E) + 1	Max(E) + 1	Max(E) + 1	Max(E) + 1, Min(W)
W=1	Min(W) - 1	Min(W) - 1	Min(W) - 1 by multiple samples	Min(W) - 1 by multiple samples	Min(S) - R & Max(N) + R	Min(W) - 1	Min(W) - 1	Min(S) - R	Min(W) - 1	Max(E) + 1, Min(W) - 1
S=1	Min(S) - R	Min(W) - 1	Min(W) - 1 by multiple samples	Min(W) - 1 by multiple samples	Min(S) - R & Max(N) + R	Min(W) - 1	Min(W) - 1	Min(S) - R	Min(W) - 1	Max(E) + 1, Min(W) - 1

Fig. 5.7 TLM algorithm

For example, if a single attacker floods from the east, the framework would detect an abnormal east VCO feature frame. With the directional information from “**east**,” the attacker is pinpointed as being to the **right** of the victim. Their ID is then computed as the maximum RPV ID in the east direction plus 1. The TLM framework optimizes the localization process to minimize detection rounds, efficiently localizing all attackers in as few cycles as possible. For simple attack patterns, such as single- or dual-attacker scenarios, the framework handles localization in a single round. However, more complex scenarios involving three or more attackers may require multiple rounds. The approach ensures robustness by systematically accounting for all potential attacker configurations while maintaining high efficiency and accuracy.

5.5 Feature Selection

Earlier work, such as [15], utilized average link utilization to train models sensitive to DoS attacks. While effective in detecting anomalies, this approach lacked granularity and directional detail, making it insufficient for localized identification of attack routes. To overcome these limitations, this approach incorporates VCO as a dynamic feature from the input ports of routers. VCO captures real-time information about the occupancy ratio of virtual channels, expressed as a floating-point value between 0 and 1. To provide a more cumulative perspective, we also include BOC, which record the total number of buffer write and read operations, offering deeper insights into sustained traffic behavior.

The suitability of VCO and BOC for different tasks—detection and localization—is evaluated through design considerations and empirical results. VCO proves to be highly effective for detection tasks due to its instantaneous nature and the ability to process it without normalization. Its floating-point format allows for seamless identification of anomalies even in high-traffic scenarios. However, VCO has limitations in attack localization because it does not provide a full view of the attacking routes, as illustrated in Fig. 5.8. On the other hand, BOC offers a complete and detailed perspective, excelling in localizing attacks with higher accuracy. Yet, its integer-based format necessitates normalization to ensure consistency across different workloads and configurations.

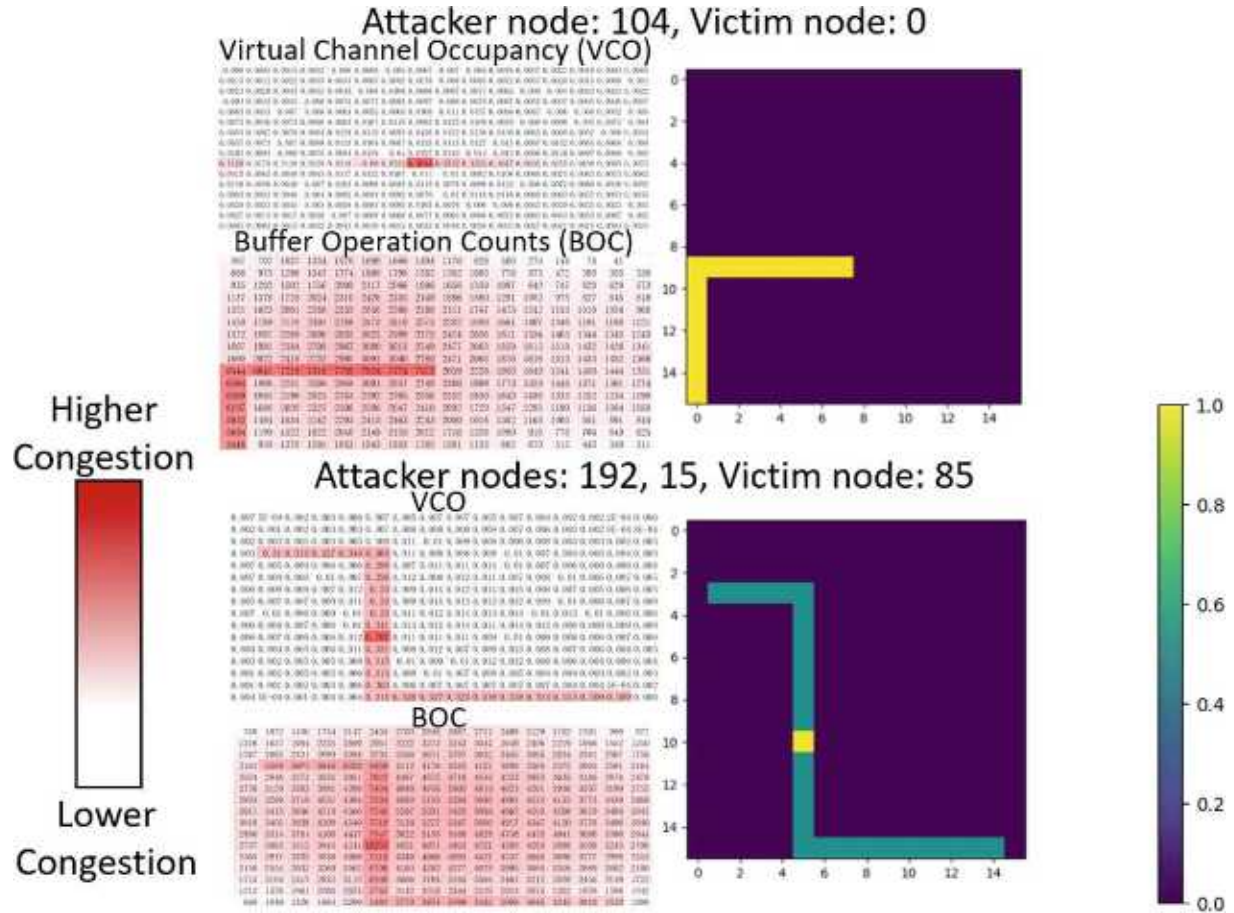


Fig. 5.8 Localization examples (on STP benchmark)

To strike a balance between computational efficiency and performance, the framework adopts a hybrid approach: VCO is used for the detection stage, leveraging its real-time insights without requiring normalization, thereby minimizing hardware complexity. For the localization phase, BOC is employed after normalization, ensuring accurate reconstruction of attack routes. This dual-feature strategy optimizes accuracy for both tasks while maintaining a manageable computational load. Additionally, for lighter workloads, such as those from PARSEC, VCO can effectively handle both detection and localization, reducing the need for feature switching. The figure highlights how VCO effectively pinpoints congestion areas, while BOC maps the full attack path, ensuring reliable and precise detection and localization.

To tackle the detection and localization tasks, we initially selected a straightforward CNN architecture due to its widespread use in hardware accelerators and its demonstrated effectiveness in similar applications. While binarized neural network (BNN) models are known for their low

computational overhead and strong performance in specific domains [16], we encountered challenges in training BNN models on the VCO and BOC feature sets. The training process proved less stable compared to CNNs, resulting in inconsistent outcomes.

5.6 CNN Model Design and Selection

We further explored various CNN architectures, experimenting with adjustments such as increasing the number of convolutional filters and adding layers to enhance model capacity. Although these modifications yielded slight improvements in accuracy, the additional computational and power costs far exceeded the performance gains, making these approaches impractical for hardware implementation. Based on this evaluation, we adopted two lightweight CNN models, each designed with minimal complexity to balance performance and efficiency. Specifically, these models feature only eight convolutional kernels per layer, a design choice that ensures reliable accuracy without imposing significant hardware overhead. This architecture, as illustrated in Figs. 5.3 and 5.4, effectively supports both detection and localization tasks while maintaining a low resource footprint, aligning with the requirements of NoC-based systems. For larger NoC scales like 32×32 , this case study is using the MobileNet architecture as the CNN model structure to improve localization accuracy. However, for smaller sizes, such as 8×8 to 16×16 , the current CNN structure offers a better balance between accuracy and hardware overhead.

5.7 Experiment Setup and Result Evaluation

Effectiveness is verified through quantitative analysis based on experimental results. This section presents the experimental setup, including the simulation platform, hardware configuration, dataset preparation, evaluation metrics, detection and localization performance, and assessment of hardware resource overhead.

5.7.1 Experiment Setup

The threat model and framework were initially implemented using the Garnet [17] interconnection model integrated within the Gem5 simulator [18]. A 16×16 Mesh-XY NoC topology was constructed, featuring one

VNET and default virtual channels, representing the largest scale explored in this research area to date. To evaluate the framework and the flooding DoS model, six synthetic traffic pattern (STP) benchmarks and three PARSEC workloads were employed. However, due to limitations in the Gem5 simulator, PARSEC workloads were only executed on a smaller 8×8 NoC. The system was clocked at 2 GHz, with feature sampling intervals set at every 1000 cycles for STP and every 100,000 cycles for PARSEC. It is worth noting that shorter monitoring cycles could be implemented in higher-frequency systems, depending on the specific security sensitivity of the SoC.

A global performance monitoring system was designed to collect datasets, simulating 18 unique attack scenarios under a flooding injection rate (FIR) of 0.8 across the six STP and three PARSEC benchmarks, resulting in a total of 162 datasets. From these, 12,960 feature frames were extracted from the four directional input ports of the routers. With each frame comprising 256 pixels, the final dataset amounted to an impressive 3.3 million pixel values. To train the machine learning models, the TensorFlow 2.0 framework was used for offline training of lightweight CNN-based detection and localization models.

To evaluate the hardware overhead of the method, lightweight CNN detection and localization accelerators were synthesized, and ProNoC-generated NoC architectures [19] were used. The framework was tested for both DoS detection and localization tasks using each of the two feature sets: VCO and BOC (as detailed in Tables 5.1, 5.2, 5.3 and 5.4). These experiments were conducted to identify the most effective feature combination. Ultimately, the results of using the combined VCO+BOC feature set are summarized in Tables 5.5 and 5.6, showcasing the robustness and scalability of the solution.

Table 5.1 DoS Detection (Left) and Localization (Right) Results for **virtual channel occupancy** Feature (**without normalization**)—Synthetic Traffic Pattern (STP)

	Uniform				Bit	Bit	
Metric	random	Tornado	Shuffle	Neighbor	Rotation	Complement	Average
Accuracy	1—0.74	0.96—0.43	0.93—0.31	0.97—0.61	1—0.58	1—0.52	0.98 —0.53
Precision	1—0.89	1—0.63	0.97—0.42	0.98—0.84	1—0.68	1—0.65	0.99 —0.69
Recall	1—0.8	0.92—0.52	0.92—0.56	0.97—0.65	1—0.79	1—0.68	0.96 —0.67

	Uniform				Bit	Bit	
Metric	random	Tornado	Shuffle	Neighbor	Rotation	Complement	Average
F1 Score	1—0.84	0.95—0.54	0.93—0.44	0.97—0.71	1—0.72	1—0.65	0.97—0.65

Table 5.2 DoS Detection (Left) and Localization (Right)
Results for **virtual channel occupancy** Feature (**without normalization**)—PARSEC

Metric	Blackscholes	Bodytrack	X264	Average
Accuracy	0.95—0.98	0.92—0.99	0.94—0.99	0.93—0.98
Precision	1—0.99	1—0.99	0.89—1	0.96—0.99
Recall	0.92—0.99	0.87—0.99	0.98—1	0.92—0.99
F1 Score	0.96—0.99	0.93—0.99	0.93—0.99	0.94—0.99

Table 5.3 DoS detection (left) and localization (right) results for **buffer operation counts** feature (**with normalization**)—synthetic traffic pattern (STP)

	Uniform				Bit	Bit	
Metric	random	Tornado	Shuffle	Neighbor	rotation	complement	Average
Accuracy	1—0.97	1—0.98	1—0.97	1—0.98	0.99—0.97	0.99—0.97	0.997—0.973
Precision	1—1	1—1	1—1	1—1	1—1	1—1	1.000—1.000
Recall	1—0.97	1—0.98	1—0.97	1—0.98	0.97—0.97	0.97—0.97	0.990—0.973
F1 score	1—0.99	1—0.99	1—0.98	1—0.99	0.98—0.98	0.98—0.98	0.993—0.985

Table 5.4 DoS detection and localization results for **buffer operation counts** feature (**with normalization**)—PARSEC

Metric	Blackscholes	Bodytrack	X264	Average
Accuracy	0.93—0.94	0.99—0.99	0.92—1	0.94—0.97
Precision	1—0.99	1—1	0.84—1	0.94—0.99
Recall	0.85—0.99	0.96—1	1—1	0.93—0.99
F1 score	0.92—0.99	0.98—1	0.91—1	0.93—0.99

Table 5.5 DoS detection and localization results for **virtual channel occupancy + buffer operation counts** features—synthetic traffic pattern (STP)

	Uniform				Bit	Bit	
Metric	random	Tornado	Shuffle	Neighbor	rotation	complement	Average

	Uniform				Bit	Bit	
Metric	random	Tornado	Shuffle	Neighbor	rotation	complement	Average
Accuracy	0.99— 0.96	0.93— 0.84	0.93— 0.86	0.9—0.9	1—0.97	1—0.97	0.958— 0.917
Precision	1—1	1—1	0.97— 0.96	0.94—1	1—1	1—1	0.985— 0.993
Recall	0.98— 0.96	0.86— 0.84	0.92— 0.89	0.85—0.9	0.99— 0.97	0.99—0.97	0.932— 0.922
F1 score	0.99— 0.98	0.9—0.89	0.92—0.9	0.88— 0.93	1—0.98	1—0.99	0.948— 0.945

Table 5.6 DoS detection and localization results for **virtual channel occupancy + buffer operation counts** features—PARSEC

Metric	Blackscholes	Bodytrack	X264	Average
Accuracy	0.95—0.89	0.92—0.91	0.94—0.94	0.933—0.913
Precision	1—0.99	1—1	0.89—0.89	0.963—0.960
Recall	0.92—0.91	0.87—0.87	0.98—0.98	0.923—0.920
F1 score	0.96—0.94	0.93—0.93	0.93—0.93	0.940—0.933

5.7.2 Evaluation Metrics

To assess the performance of the model employed, four widely used evaluation metrics are employed: precision, recall, accuracy, and F1 score. These metrics provide a comprehensive understanding of the model’s ability to correctly classify both positive and negative samples.

Precision measures the proportion of correctly predicted positive samples among all samples that are predicted as positive. It reflects how accurate the model is when it predicts a positive outcome. The formula is given in Eq. [5.1](#):

$$\text{Precision} = \frac{TP}{TP+FP} \quad (5.1)$$

Here, TP (true positives) refers to the number of correctly predicted positive samples, and FP (false positives) refers to the number of negative samples that are incorrectly predicted as positive.

Recall represents the proportion of actual positive samples that are correctly predicted by the model. It indicates the model’s ability to identify all relevant positive cases. The formula is shown in Eq. [5.2](#):

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (5.2)$$

Here, FN (false negatives) refers to the number of positive samples that were incorrectly predicted as negative.

Accuracy evaluates the overall correctness of the model. It calculates the proportion of all correctly predicted samples, including both positive and negative classes, over the total number of samples. The formula is provided in Eq. 5.3:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \quad (5.3)$$

Here, TN (true negatives) represents the number of correctly predicted negative samples. Although accuracy gives a general overview of performance, it may be misleading in imbalanced datasets.

F1 score is the harmonic mean of precision and recall. It is particularly useful when both false positives and false negatives carry significant consequences. The formula is given in Eq. 5.4:

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (5.4)$$

A higher F1 score indicates that the model achieves a good balance between precision and recall, which is important for applications where identifying true positives while minimizing errors is essential.

5.7.3 Detection Performance

The detection capabilities of the CNN models using the VCO feature are summarized in the left portion of each column in Table 5.1. For synthetic traffic pattern (STP) benchmarks, the CNN achieves an average detection accuracy of 0.98 and precision of 0.99, as highlighted in the last column. In contrast, detection performance for PARSEC benchmarks shows slightly reduced metrics, with an average accuracy of 0.93 and precision of 0.96. The BOC feature demonstrates stronger detection performance, as presented in Table 5.3, with average accuracies consistently reaching or exceeding 0.99 and precision of 1 across both STP and PARSEC workloads. Although the VCO feature underperforms slightly compared to BOC, it remains the preferred choice for detection tasks due to its instantaneous values, which eliminate the need for normalization, and its lower memory overhead compared to the accumulated nature of BOC values.

5.7.4 Localization Performance

Localization tasks present notable differences in performance between the VCO and BOC features. For STP benchmarks, the VCO feature underperforms in localization, with an average accuracy gap of 0.45 compared to BOC, as indicated in the right side of each column in Tables [5.1](#) and [5.2](#). The BOC feature, on the other hand, proves highly effective for localization tasks, achieving average accuracies of 0.973 and 0.917 in STP and PARSEC workloads, respectively, as shown in Tables [5.3](#), [5.4](#), [5.5](#) and [5.6](#). The enhanced clarity of attacking routes in BOC data is exemplified in Fig. [5.8](#), where BOC matrices highlight abnormal routes with greater precision.

Localization performance on the PARSEC benchmarks benefits from the lower traffic density during the region of interest (ROI) period, where computational activity is higher and network congestion is reduced. This scenario enables the framework to effectively detect and localize flooding denial-of-service (FDoS) attacks within the ROI. Notably, for PARSEC workloads, the potential to utilize VCO for simultaneous detection and localization emerges as a viable scheme, particularly for less traffic-intensive scenarios, where its computational efficiency aligns well with workload demands. This adaptive feature selection ensures the framework remains flexible and resource efficient across diverse workloads.

5.7.5 Hardware Overhead and Related Work Comparison

Two lightweight CNN accelerators were meticulously designed with a pipeline architecture using three convolutional kernels, prioritizing minimal logic utilization. These accelerators were evaluated for hardware overhead on a standalone NoC, comprising routers, network interfaces, and links, to eliminate inaccuracies from SoC tiles. As shown in the hardware overhead trend (Fig. [5.9](#)), the models in this case study exhibit overheads of 1.9% on 8x8 NoCs and just 0.45% on 16x16 NoCs, indicating a significant 76.3% reduction as the NoC scale increases. This decreasing trend highlights approach's efficiency and scalability, making it adaptable for various SoC demands.



Fig. 5.9 Hardware overhead decreasing with larger NoCs

In comparison to previous works, such as the perceptron model in [20] and the SVM-based model in [21], this case study demonstrates superior scalability and efficiency, achieving 42.4% less overhead than [20] while maintaining comparable detection accuracy. Table 5.7 summarizes the comparative performance of DoS detection and localization techniques across varying NoC scales, with the framework achieving detection precision of 98.5% and localization precision of 99.3% for 16x16 NoCs. This significantly outperforms other models that either lack scalability or incur higher hardware overhead.

Table 5.7 Comparison to related works

Works	[2]	[13]	[8]	[Our work]
ML model(s)	Perceptron	SVM	XGB	CNN classifier + segmentor
FDoS	✓	Non-FDoS	✓	✓
Scalability	✓	✓	✓	✓
Hardware overhead	3.3%	9%	N/A	0.45%/16 × 16NoC 1.9%/8 × 8NoC
<i>DoS detection and localization performance</i>				
NoC scale	8 × 8	4 × 4	4 × 4	16 × 16 ★
D: accuracy	97.6%	95.5%	~96%	95.8%
D: precision	N/A	~94.5%	94.8%	98.5%
L: accuracy	96.7%	N/A	N/A	91.7%
L: precision	N/A	N/A	N/A	99.3%

D detection, L localization

The framework’s ability to adapt to realistic workloads is further highlighted by its exceptional performance in PARSEC benchmarks. Unlike other works that primarily evaluate under synthetic traffic patterns, the

approach demonstrates a robust capability to detect and localize flooding attacks in computationally intensive workloads, where the communication bottlenecks are more prominent. By focusing on both detection precision and efficient hardware implementation, the framework establishes itself as a state-of-the-art solution, particularly excelling in scenarios demanding stringent NoC communication performance.

5.8 Conclusions

This chapter presents a comprehensive security framework in NoC, a robust deep learning-based framework designed for flooding denial-of-service (FDoS) detection and localization, incorporating a finely tuned FDoS model. The framework leverages two CNN models, one for feature frame classification (DoS detector) and another for segmentation (DoS profile localizer), ensuring effective and accurate anomaly detection. Additionally, it introduces advanced techniques such as multi-frame fusion (MFF), victim completing enhancement (VCE), and a table-like method (TLM) to achieve precise attacker and victim localization.

It demonstrates exceptional performance, achieving a detection accuracy of 95.8% and a localization accuracy of 91.7% on a 16×16 mesh NoC, with nearly 99% precision in both tasks, rivalling state-of-the-art approaches. A notable advantage of scheme in this case study is its scalability and efficiency, reducing hardware overhead by 76.3% when scaling from 8×8 to 16×16 NoCs. For larger NoCs, such as those with over 32×32 nodes, we will incorporate a MobileNet-based CNN model to enhance localization accuracy while maintaining an estimated hardware overhead of less than 2.5%. This scalable and efficient design ensures the framework's applicability to modern, high-performance, and large-scale NoC systems.

References

1. Sudusinghe, C., Charles, S., Mishra, P.: Denial-of-service attack detection using machine learning in network-on-chip architectures. In: Proceedings of the 15th IEEE/ACM International Symposium on Networks-on-Chip, pp. 35–40 (2021)
2. Weerasena, H., Jia, X., Mishra, P.: Topology-aware detection and localization of distributed denial-of-service attacks in network-on-chips. arXiv preprint arXiv:2505.14898 (2025)

3. Charles, S., Lyu, Y., Mishra, P.: Real-time detection and localization of distributed dos attacks in NoC-based SoCs. *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* **39**(12), 4510–4523 (2020). <https://doi.org/10.1109/TCAD.2020.3016820>. <https://ieeexplore.ieee.org/document/8989809>
4. Mishra, P., Charles, S.: *Network-on-Chip Security and Privacy*. Springer, Berlin (2021) [[Crossref](#)]
5. Hinton, G.E., Osindero, S., Teh, Y.W.: A fast learning algorithm for deep belief nets. *Neural Comput.* **18**(7), 1527–1554 (2006) [[MathSciNet](#)][[Crossref](#)]
6. Shinde, P.P., Shah, S.: A review of machine learning and deep learning applications. In: 2018 Fourth International Conference on Computing Communication Control and Automation (ICCUBE), pp. 1–6. IEEE (2018)
7. Halak, B.: Cist: A threat modelling approach for hardware supply chain security. In: *Hardware Supply Chain Security: Threat Modelling, Emerging Attacks and Countermeasures*, pp. 3–65. Springer, Cham (2021)
8. Regazzoni, F., Bhasin, S., Pour, A.A., Alshaer, I., Aydin, F., Aysu, A., Beroulle, V., Di Natale, G., Franzon, P., Hely, D., et al.: Machine learning and hardware security: Challenges and opportunities. In: *Proceedings of the 39th International Conference on Computer-Aided Design*, pp. 1–6 (2020)
9. Li, J., Ni, L., Chen, J., Zhou, E.: A novel hardware trojan detection based on bp neural network. In: 2016 2nd IEEE International Conference on Computer and Communications (ICCC), pp. 2790–2794. IEEE (2016)
10. Hasegawa, K., Yanagisawa, M., Togawa, N.: Hardware trojans classification for gate-level netlists using multi-layer neural net-works. In: 2017 IEEE 23rd International Symposium on On-Line Testing and Robust Sys-tem Design (IOLTS), pp. 227–232. IEEE (2017)
11. Jin, Y., Maliuk, D., Makris, Y.: Post-deployment trust evaluation in wireless cryptographic ics. In: 2012 Design, Automation & Test in Europe Conference & Exhibition (DATE), pp. 965–970. IEEE (2012)
12. He, Z., Raghavan, A., Hu, G., Chai, S., Lee, R.: Power-grid controller anomaly detection with enhanced temporal deep learning. In: 2019 18th IEEE International Conference On Trust, Security And Privacy In Computing And Communications/13th IEEE International Conference On Big Data Sci-ence And Engineering (TrustCom/BigDataSE), pp. 160–167. IEEE (2019)
13. Alam, M., Bhattacharya, S., Dutta, S., Sinha, S., Mukhopadhyay, D., Chattopadhyay, A.: Ratafia: Ransomware analysis using time and frequency informed autoencoders. In: 2019 IEEE International Symposium on Hardware Oriented Security and Trust (HOST), pp. 218–227. IEEE (2019)
14. Wang, H., Halak, B.: Hardware trojan detection and high-precision localization in noc-based mp soc using machine learning. In: *Proceedings of the 28th Asia and South Pacific Design Automation Conference*, pp. 516–521 (2023)

15. Hu, S., Wang, H., Halak, B.: Cascaded machine learning model based dos attacks detection and classification in noc. In: 2023 IEEE International Symposium on Circuits and Systems (ISCAS), pp. 1–5. IEEE (2023)
16. Rasoulinezhad, S., Fox, S., Zhou, H., Wang, L., Boland, D., Leong, P.H.: Majoritynets: Bnns utilising approximate popcount for improved efficiency. In: 2019 International Conference on Field-Programmable Technology (ICFPT), pp. 339–342. IEEE (2019)
17. Agarwal, N., Krishna, T., Peh, L.S., Jha, N.K.: Garnet: A detailed on-chip network model inside a full-system simulator. In: 2009 IEEE International Symposium on Performance Analysis of Systems and Software, pp. 33–42. IEEE (2009)
18. Lowe-Power, J., Ahmad, A.M., Akram, A., Alian, M., Amslinger, R., Andreozzi, M., Armejach, A., Asmussen, N., Beckmann, B., Bharadwaj, S., et al.: The gem5 simulator: Version 20.0+. arXiv preprint arXiv:2007.03152 (2020)
19. Monemi, A., Tang, J.W., Palesi, M., Marsono, M.N.: Pronoc: a low latency network-on-chip based many-core system-on-chip prototyping platform. *Microprocess. Microsyst.* **54**, 60–74 (2017)
[\[Crossref\]](#)
20. Sinha, M., Gupta, S., Rout, S.S., Deb, S.: Sniffer: A machine learning approach for dos attack localization in noc-based socs. *IEEE J. Emerg. Sel. Top. Circuits Syst.* **11**(2), 278–291 (2021)
[\[Crossref\]](#)
21. Kulkarni, A., Pino, Y., French, M., Mohsenin, T.: Real-time anomaly detection framework for many-core router through machine-learning techniques. *ACM J. Emerg. Technol. Comput. Syst.* **13**(1), 1–22 (2016)
[\[Crossref\]](#)

6. Comparative Analysis on Machine Learning for DoS Detection in Network-on-Chip

Haoyu Wang¹✉, Jianjie Ren¹✉, Shengkai Hu¹✉, Ahmad Atamli¹✉ and Basel Halak¹✉

(1) School of Electronics and Computer Science, University of Southampton, Southampton, UK

✉ Haoyu Wang

Email: Haoyu.Wang@soton.ac.uk

✉ Jianjie Ren

Email: jr1u22@southamptonalumni.ac.uk

✉ Shengkai Hu

Email: Shengkai.Hu@soton.ac.uk

✉ Ahmad Atamli

Email: A.Atamli@soton.ac.uk

✉ Basel Halak (Corresponding author)

Email: Basel.Halak@soton.ac.uk

Keywords Machine learning – DoS detection – Cascading models – Graph-based methods – Network-on-Chip security – Traffic correlation – Attribute selection – Graph theory – Malicious traffic detection – Deep learning – Graph Neural Networks (GNN) – Long Short-Term Memory (LSTM) – Model evaluation – Anomaly detection – Comparative analysis

6.1 Introduction

We have extensively discussed the CNN-based security countermeasures for NoC security in Chap. 5, demonstrating their significant potential in detecting abnormal data while achieving a commendable balance between security performance and hardware overhead. These insights underscore CNN's suitability for addressing specific challenges in NoC security. However, CNN models represent just one facet of the machine learning landscape. Many other machine learning models, with their unique strengths and capabilities, remain to be explored for conducting security tasks in NoC contexts.

In this chapter, we highlight two representative works that effectively employ diverse machine learning models in NoC security. These examples illustrate how combining different models can outperform single-model approaches by leveraging their complementary strengths. Furthermore, the strategic utilization of different models at various stages of data processing can ensure swift and efficient responses once an attack is detected. Such dynamic approaches not only enhance detection accuracy but also optimize resource utilization.

Building upon these insights, we discuss a case study that shows a novel secure NoC microarchitecture designed to integrate more advanced machine learning models, particularly those excelling in time-series analysis. By incorporating these models, the architecture aims to improve anomaly detection capabilities, enabling real-time responses and enhanced adaptability to evolving security threats. This chapter not only showcases the versatility of machine learning models in NoC security but also sets the stage for further innovations in secure hardware design.

6.2 Chapter Overview

In Sect. [6.3](#) Scheme 1: Cascaded Machine Learning Model for NoC Security, we introduced a cascaded machine learning approach designed to mitigate general DoS attacks in NoC systems. In Sect. [6.4](#): Graph-Based Early Defense Scheme for NoCs, an advanced graph-based defense mechanism is introduced for real-time detection and mitigation of refined flooding DoS attacks in NoCs. In Sect. [6.5](#), it concludes with a summary of insights into the integration of machine learning models for NoC security and mentions directions for future research.

6.3 Scheme 1: Cascaded Machine Learning Model for NoC Security

6.3.1 Extensive DoS Attacks

DoS attacks are common attack methods in NoC systems, aimed at exploiting shared resources to block the transmission of legitimate traffic [\[1\]](#). Attackers exhaust the target system's resources by injecting a large number of request packets or malicious packets, thereby preventing legitimate users from receiving service. In NoC environments, DoS attacks are typically achieved by injecting unnecessary traffic [\[2\]](#), occupying communication bandwidth [\[1\]](#), or blocking routers.

Detecting and preventing DoS attacks in NoC systems is crucial, especially in real-time and safety-critical applications. Research has shown that DoS attacks can have severe consequences for network transmission, primarily by causing significant performance degradation [\[3\]](#). Malicious third-party IPs (M3PIPs) can send a large volume of packets to the network, interfering with the availability of on-chip resources, which may lead to a severe drop in system performance. Additionally, DoS attacks can also involve multiple infected IPs attacking one or more components simultaneously, exacerbating the impact of the attack [\[1\]](#). In multicore SoC systems based on NoCs, DoS attacks caused by malicious IP cores may increase packet transmission delays, leading to task deadline violations [\[1\]](#).

As the communication backbone of multiprocessor SoCs (MPSoCs), NoCs are particularly vulnerable to DoS attacks because multiple cores and IP blocks share critical resources such as communication links, router buffers, and arbitration units [\[4\]](#). Attackers can exploit these shared resources to

prevent task completion or degrade system performance. Common implementations of DoS attacks on NoC systems include traffic diversion attacks, traffic loop attacks, and flooding attacks.

(a) Traffic Diversion (TD) Attack

When a NoC router is subjected to a traffic diversion (TD) attack and is located along the packet transmission path, the destination address of the packet may change, resulting in packet loss or misdelivery. For example, Fig. 6.1a shows a 6×6 Mesh_XY router topology, where the source router is R0 and the destination is R34. If router 22 (R22), which lies along the path between the source and the destination, is attacked by a TD attack, the packet will first reach router R22. Following the NoC's routing rules, the packet will then be incorrectly forwarded to router R23.

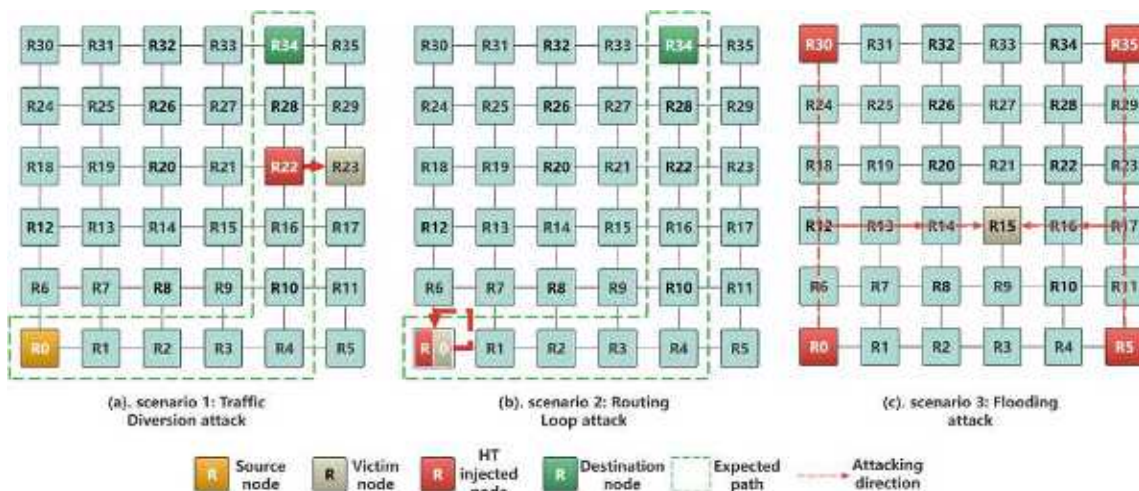


Fig. 6.1 Three DoS attack scenarios

(b) Routing Loop (RL) Attack

When a NoC router is subjected to a routing loop (RL) attack, the attacked router will send the packets back to the source router, resulting in packet loss and resource consumption in the NoC network. Additionally, the attacked router will lose the ability to transmit packets to other routers. For example, Fig. 6.1b shows a 6×6 Mesh_XY router topology with a uniform random traffic pattern. When router R0 is attacked, all packets injected from router R0 will be redirected back to R0.

(c) **Flooding Attack**

In a NoC, one or more routers may be simultaneously attacked and transmit packets to a designated router, which is known as a flooding attack. A flooding attack causes redundant packets to appear along the transmission path, leading to NoC congestion and packet loss or misdelivery. For example, as shown in Fig. [6.1](#) 2(c), when routers R0, R5, R30, and R35 are simultaneously attacked, their traffic pattern will no longer follow the predefined uniform random but will instead uniformly send packets to the designated router (e.g., R15).

6.3.2 Correlation Analysis and Attribute Selection

Selecting the appropriate NoC hardware performance attributes as the dataset is the first crucial step in developing a supervised machine learning model. We selected attributes closely related to NoC performance from the Gem5 computer architecture simulator, including packets injected, packets received, average packet queueing latency, average packet network latency, average packet latency, average link utilization, average hops, and average power. To reduce redundant attributes and consequently lower model complexity while improving computational efficiency, we performed correlation analysis using Pearson's correlation coefficient and generated a correlation heatmap. Figure [6.2](#) shows the results of the correlation analysis for the selected features. We observed that packets injected and packets received have a high correlation, so we decided to keep only one of them. Similarly, average packet queueing latency, average packet network latency, and average packet latency are highly correlated, and we retained one of these. Furthermore, average link utilization and average hops are strongly correlated, so we chose to keep one of them. Additionally, due to the high correlation between average link utilization and both packets injected and packets received, we ultimately selected average link utilization and average packet latency as the input features for the machine learning model (Fig. [6.3](#)).

6.3.3 Cascaded Model Scheme

For the three classic DoS attacks mentioned above, we introduce a case study that shows a machine learning-based cascaded detection-classification model for their detection and classification. Inspired by the field of computer vision, we adopt the basic idea of first detection [5] and then classification for NoC attacks. This approach not only improves detection efficiency and reduces computational load but also gradually enhances classification accuracy.

As shown in Fig. 6.4, the cascaded model scheme employs support vector machine (SVM) for the detection phase. SVM is effective at finding hyperplanes for classification in high-dimensional data, particularly well suited for complex, nonlinear datasets. This makes it an ideal choice for NoC attack detection. For attack classification, we choose K-nearest neighbors (KNN), which is advantageous due to its simplicity, intuitive computation, and the absence of a training process. KNN classifies data by calculating the distance between test samples and training samples, making it suitable for handling high-dimensional and complex datasets. Since KNN does not rely on assumptions about data distribution, it can flexibly handle various types of data and quickly process new samples. In real-time NoC attack classification applications, KNN provides fast classification and adapts to different attack patterns, especially when dealing with small-scale datasets or situations where boundaries between features are not clearly defined.

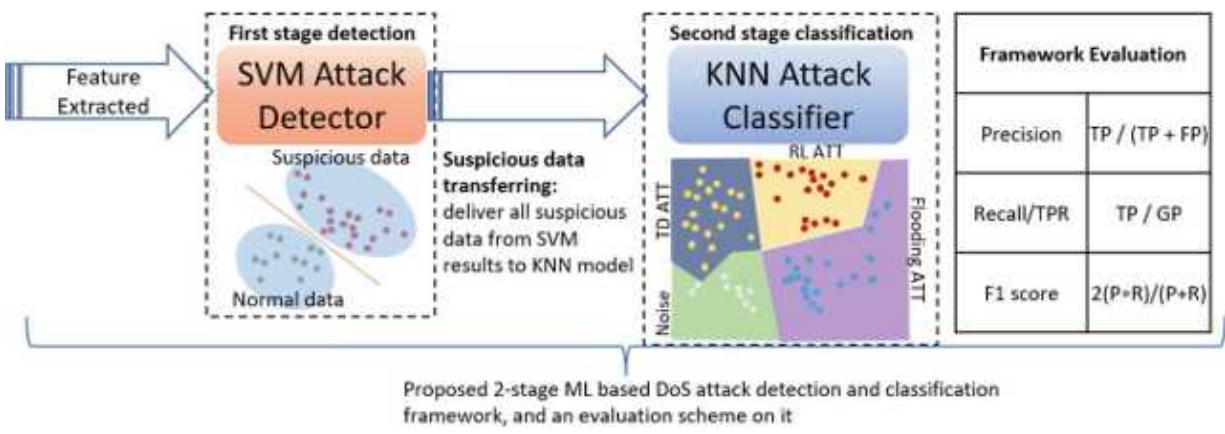


Fig. 6.4 Cascaded ML models

Through this cascaded model, KNN offers higher classification accuracy and real-time performance in the classification phase, ensuring the system's efficiency and robustness when facing multiple DoS attacks.

6.3.4 Experiment Results and Evaluation

In this experiment, we used the Gem5 architecture simulator to construct a NoC system consisting of 64 routers and 64 cores, with a Mesh_XY topology that offers high scalability and balance. To ensure the comprehensiveness and realism of the experiment, we set the injection rate to 0.02 and selected four typical traffic patterns (uniform random, tornado, shuffle, and bit reverse) as benchmarks. The simulation ran for 100,000 cycles, and data was collected every 100 cycles to obtain the average link utilization and average packet latency for different traffic patterns. Additionally, three types of DoS attacks including traffic diversion, traffic loop, and flooding were injected, which were implemented by modifying the “*RoutingUnit.cc*” file in the Garnet module, in order to test the performance of the NoC under different attack scenarios.

During the dataset preparation process, we selected average link utilization and average packet latency under different traffic patterns as training attributes and applied normalization to reduce the impact of dimensional differences. Next, we constructed training and test datasets for the SVM detector, with two labels, “normal” and “attacked,” and for the KNN classifier, we created four labels, “normal,” “TD,” “RL,” and “flooding.” To enhance the realism of the simulation and reduce assumptions, we built a synthetic mixed dataset that combines different types of attacks and normal data. The purpose of this approach is to make the simulation closer to the complex scenarios found in the real world. In reality, network attacks may occur in multiple types simultaneously, so using only a single attack pattern or normal traffic pattern in the simulation might not fully reflect actual situations. By mixing different attack types and traffic patterns, we can better simulate possible attack scenarios, reducing the reliance on idealized assumptions. This improves the diversity and representativeness of the dataset, making the model training more realistic. Finally, we validated the framework using the synthetic mixed dataset, which consisted of HT-infected and normal data, across all benchmarks (a total of 16,000 data points) to account for any potential attack scenarios. The validation results were evaluated using a unified and formal performance matrix.

We first evaluated the performance of the SVM detector. As shown in Fig. [6.5](#), the SVM detector achieved a precision between 85 and 91% across all five traffic patterns, with recall fluctuating between 85 and 88%. This indicates a good balance between precision and recall, and it also overcomes

the traditional limitation of requiring both high precision and high recall in a single machine learning model.

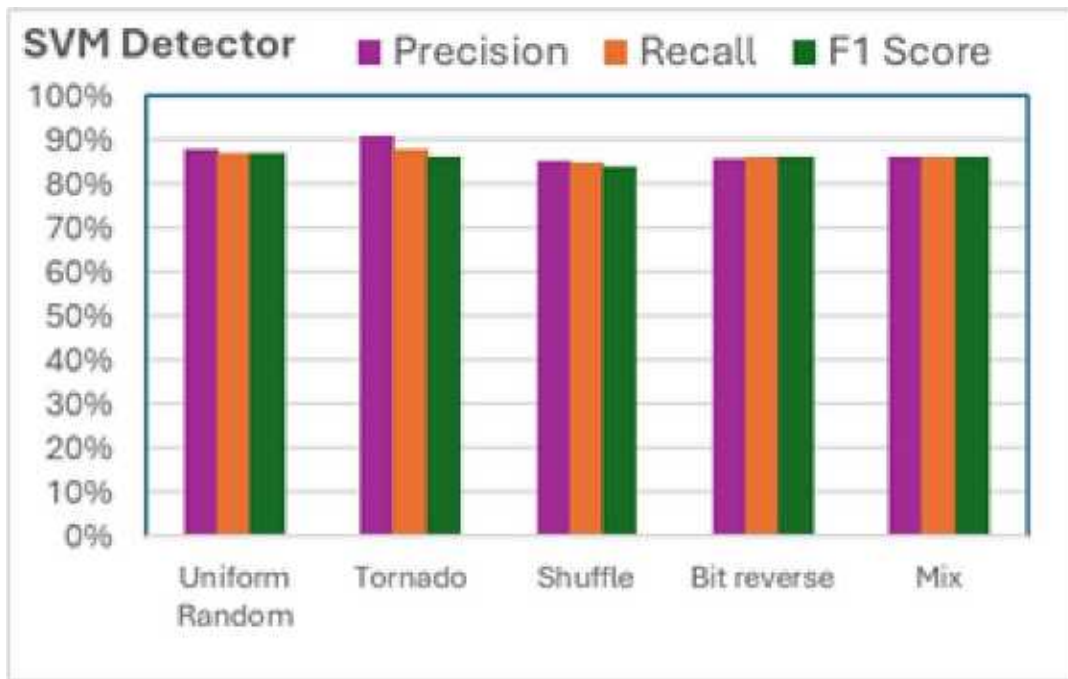


Fig. 6.5 SVM detection results

Next, we evaluated the performance of the KNN classifier. As shown in Fig. 6.6, the KNN classifier achieved high precision and recall across all five traffic patterns, with excellent balance. The performance evaluation of the uniform random pattern is particularly noteworthy, as it most closely resembles the behavior of actual hardware NoCs transmitting random packets, triggering all three types of attacks. As observed, the KNN classifier performed best under the uniform random traffic pattern, achieving the expected precision and excellent recall.

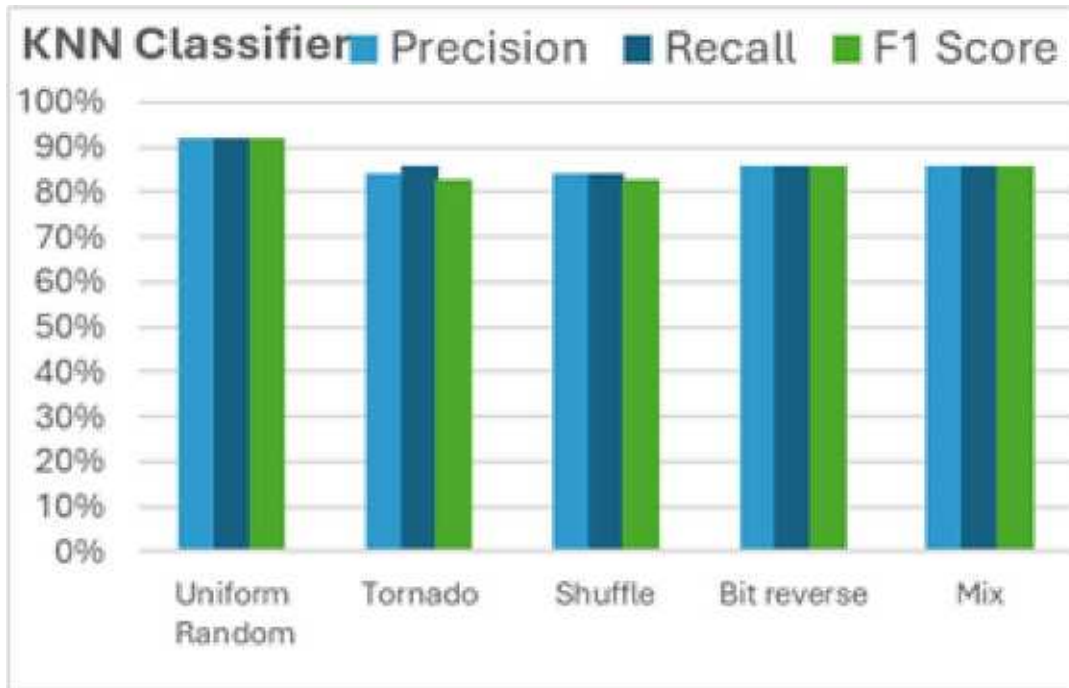


Fig. 6.6 KNN classification results

Finally, we tested the precision of the three attacks under different traffic pattern benchmark cases, as shown in Fig. 6.7. It can be observed that the performance of the KNN model varies under different traffic patterns.

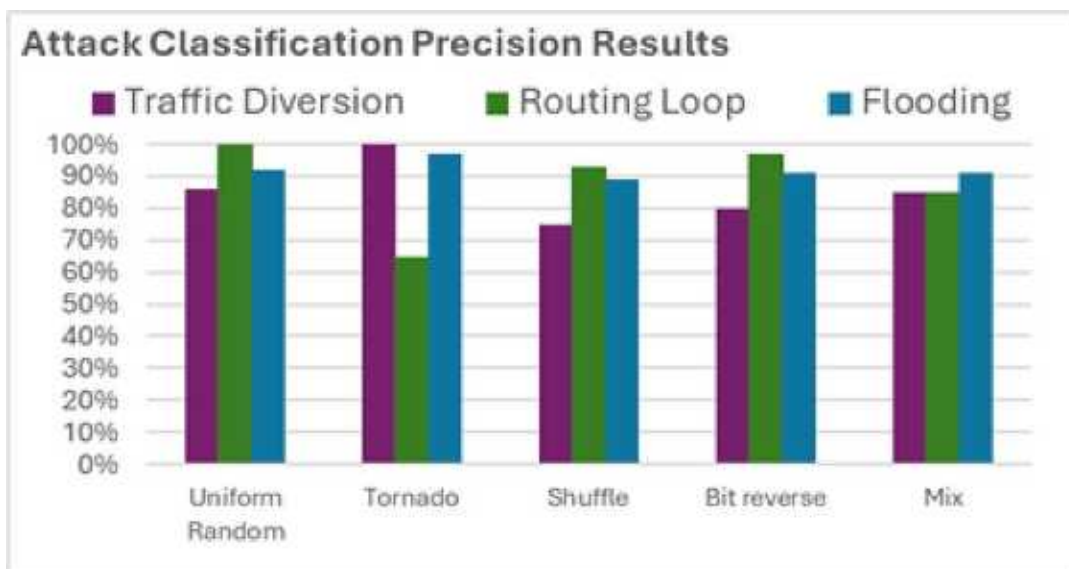


Fig. 6.7 Precision results of attack classification

For example, the precision of the RL attack is highest at 100% in uniform random, but around 65% in Tornado. This result indicates that the ML model exhibits varying adaptability across different application scenarios. However,

we found that the trained ML model shows strong applicability in uniform random and the fully mixed dataset, which closely simulate hardware NoC applications, achieving high detection accuracy.

To validate the effectiveness and innovation of the cascaded model designed in this case study, several related classic studies were compared, as shown in Table 6.1.

Table 6.1 Result comparison

Works	DoS attack	ML model	Dataset	Detection and	Precision
				classification	
Kulkarni et al. [6]	TD, RL, spoofing	SVM	Split	SVM for classification	94% on spoofing, 95% on RL, 97% on TD
Sinha et al. [7]	Flooding	ANN	Integrated	Sanity check and ANN for classification	89% on flooding
Sudusinghe et al. [8]	Flooding	XGB	Integrated	XGB for classification	~96% on Flooding
This case study	Flooding, TD, RL	SVM + KNN	Mixed	SVM for detection + KNN for classification	Up to 91% on detection, 89–97% on flooding, up to 100% on RL, up to 100% on TD

In terms of detection and classification methods, compared to other single machine learning models, we cascaded SVM and KNN models to achieve accurate attack detection and classification. Additionally, we created a hybrid dataset that includes all attack types, rather than handling each attack individually. The approach achieved 91% accuracy in the detection phase, with 100% accuracy in classifying RL and TD attacks, and an average accuracy of 93% for flooding attacks.

6.4 Scheme 2: Graph-Based Early Defense Scheme for NoCs

In this section, we place a strong emphasis on flooding denial-of-service (FDoS) attacks, as they are among the most prevalent and stealthy threats targeting NoC systems. FDoS attacks have significant implications, as they

can disrupt critical communication paths and degrade the overall performance of NoC architectures.

To address these concerns, we begin by introducing a more refined FDoS threat model, building upon the foundational concepts detailed in Sect. [4.4.2](#). This advanced model incorporates more realistic and dynamic attack scenarios, enabling a deeper understanding of the vulnerabilities in NoC systems.

Following this, we deliver the motivation for leveraging graph theory in the context of FDoS detection. Graph theory provides a robust framework for analyzing NoC topologies and communication patterns, making it an ideal tool for identifying anomalies caused by malicious activities. We also outline some of the preliminary works that laid the foundation for this approach, showcasing how these studies informed the design of solution in this case study.

The comprehensive security detection architecture we developed is detailed in Sect. [6.4.3](#). This architecture integrates advanced techniques for identifying and mitigating FDoS attacks, leveraging both graph-based methodologies and time-series related machine learning models to achieve high detection accuracy and scalability.

Finally, we present the results and performance evaluations of the novel architecture. These results demonstrate its effectiveness in detecting and localizing FDoS attacks under various scenarios, highlighting its potential for enhancing NoC security in both academic and industrial applications.

6.4.1 Threat Modelling for Refined Flooding DoS

This study centers on the detection and mitigation of FDoS attacks initiated by one or more malicious third-party intellectual properties (M3PIPs). These malicious entities are integrated into the NoC through network interfaces and routers, as illustrated in Fig. [6.8](#). The primary attack vector involves the malicious IP leveraging various methods, such as hardware Trojan (HT) insertion or malware execution, to launch the FDoS attacks. Consequently, our focus narrows to the node housing the malicious IP, aiming to achieve security by identifying and isolating the compromised node.

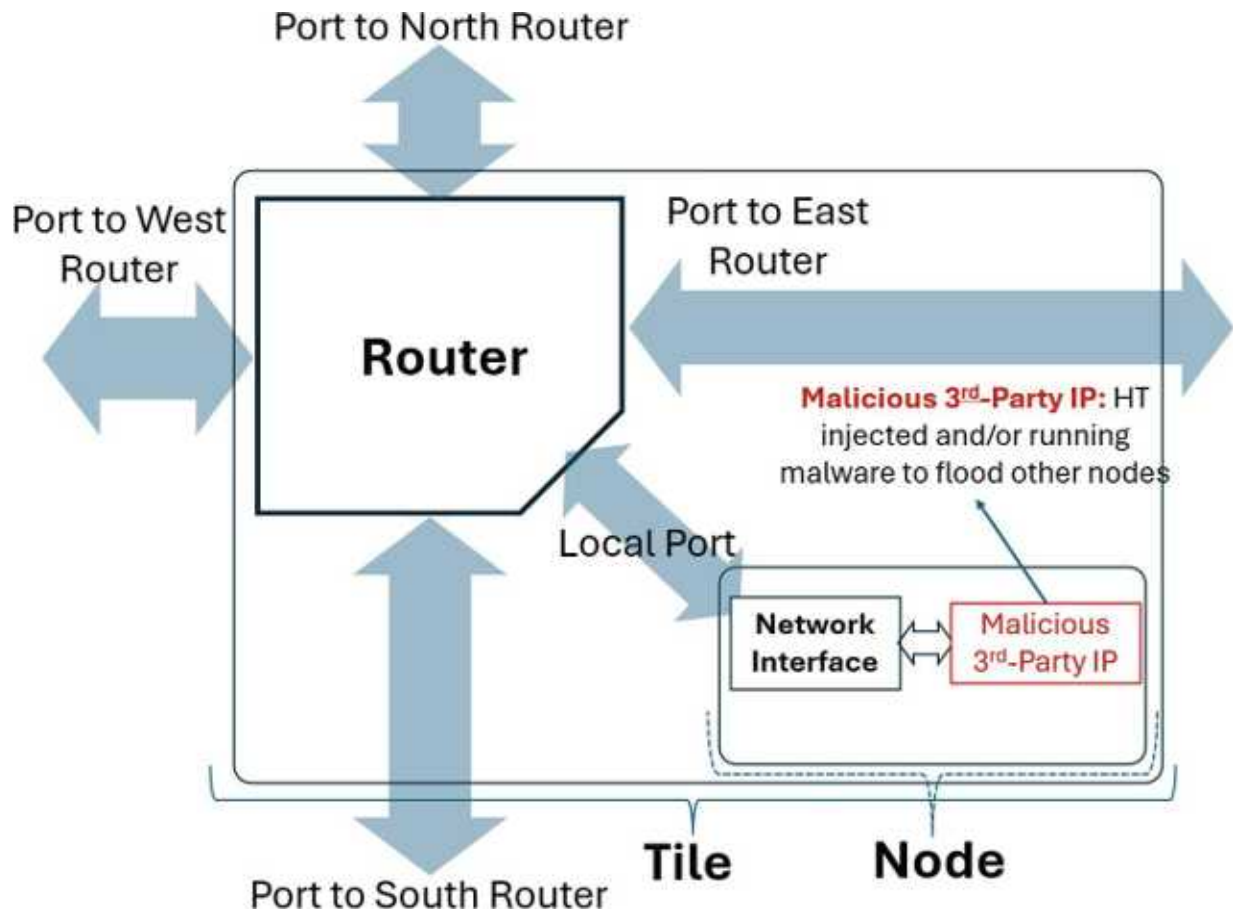


Fig. 6.8 The location of malicious third party IP in NoC system

HTs represent a significant mechanism for realizing malicious IP functionality. HTs are covert modifications introduced into integrated circuits during either the design or manufacturing phase. These modifications are particularly insidious as they can evade traditional testing methods and remain dormant until triggered. HTs can infiltrate various NoC components, such as third-party IP cores, network interfaces, or even the ports and buffers of routers. Once activated, these HTs are capable of flooding the NoC with an overwhelming volume of data packets, disrupting the normal flow of communication, creating bottlenecks, and ultimately leading to a DoS scenario.

By focusing on detecting and isolating nodes compromised by M3PIPs or HTs, this study aims to ensure the reliability and security of NoC-based systems. This approach not only addresses the immediate threat of FDoS attacks but also highlights the broader implications of integrating third-party IPs into critical hardware infrastructures, emphasizing the need for robust and proactive security measures in the design and operation of modern NoC architectures.

These flooding attacks inundate the victim with an extraordinarily high number of continuous packets, leading to severe network congestion, potential service denial, and the risk of prolonged disruptions impacting multiple nodes within a NoC system [9]. The assumption is that the flooding behavior persists for hundreds of cycles at an abnormally high injection rate; sporadic or fewer flooding packets can typically be managed by the system and do not result in significant consequences. It is crucial to highlight that both primary and collateral victims are affected—the primary victim being the intended target, while collateral victims are routers inadvertently impacted along the attack path. Our threat model thoroughly examines diverse DoS scenarios, focusing on those triggered by flooding attacks. Several definitions and clarifications are outlined below:

1. **M3PIP Attackers and Victims:** M3PIPs can target a single victim or, in cases involving multiple M3PIPs, target various victims either independently or through coordinated efforts.
2. **Attack Path Profiling:** The attack path is characterized by congestion levels observed at the router's input units. Multiple routing-path victims can exist along a single attack path if the attacker is not directly adjacent to the victim. The framework in this case study is specifically designed to localize these attack paths and identify all affected victims.
3. **Attack Strategies:** Flooding attack strategies fall into two categories: collaborative and non-collaborative. Collaborative strategies involve multiple M3PIPs synchronizing their efforts to attack a single victim, resembling a distributed DoS attack. Non-collaborative strategies consist of individual M3PIPs targeting separate victims independently.

The patterns of flooding attacks detectable and localizable by the method are visualized in Fig. 6.9. In scenario (a), a single M3PIP launches an attack on a single victim. When multiple M3PIPs are involved, they may target either a single victim or several victims, as depicted in scenarios (b), (c), and (d). Specifically, scenario (d) demonstrates overlapping attack paths, where multiple M3PIPs simultaneously target multiple victims, with their routes partially or completely intersecting. Scenario (c) showcases a coordinated attack, where two attackers from opposite corners converge on a victim in another corner. To provide further clarity, Fig. 6.10 illustrates specific examples of scenarios (a) and (c). Both scenarios begin with the NoC system

operating under normal conditions, where each pixel represents the buffer occupancy for an input port of a router in one direction. Once the flooding attack is initiated, buffer occupancy values shift dramatically from normal to highly skewed, even when averaged over a monitoring window of 100 cycles. The sequential frames display the mean occupancy values over the first, second, and third 100-cycle intervals, respectively. These examples highlight that not only the intended target suffers but also the intermediate routers along the attack path experience substantial performance degradation.

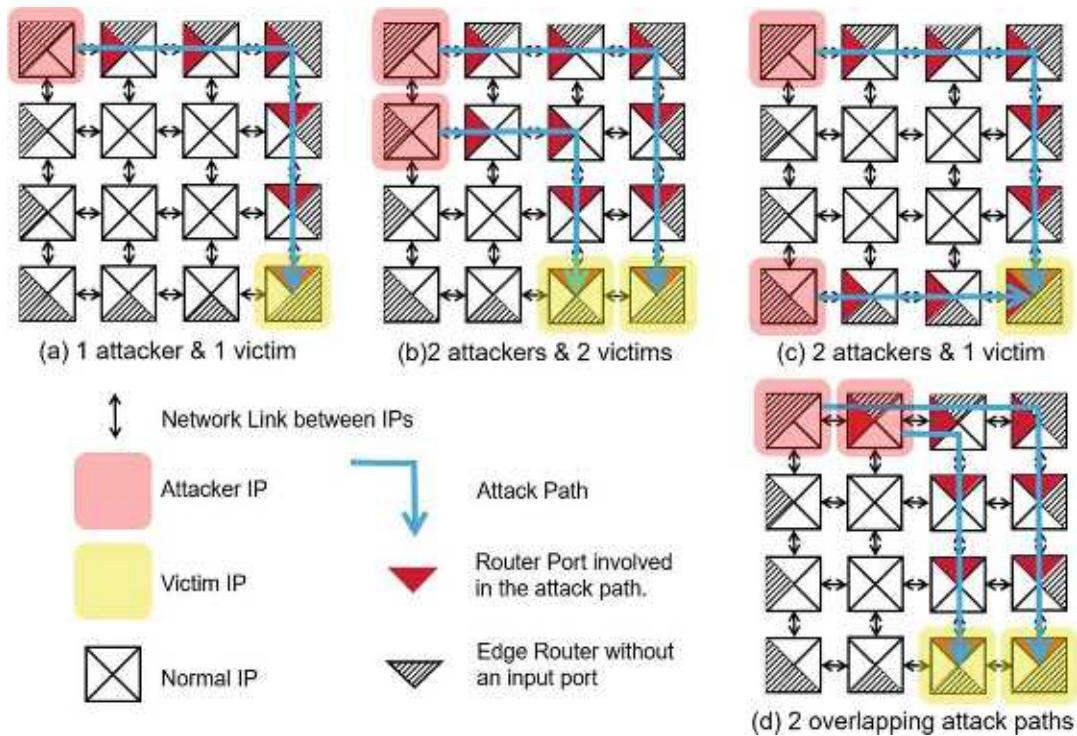


Fig. 6.9 Four flooding DoS attack scenarios

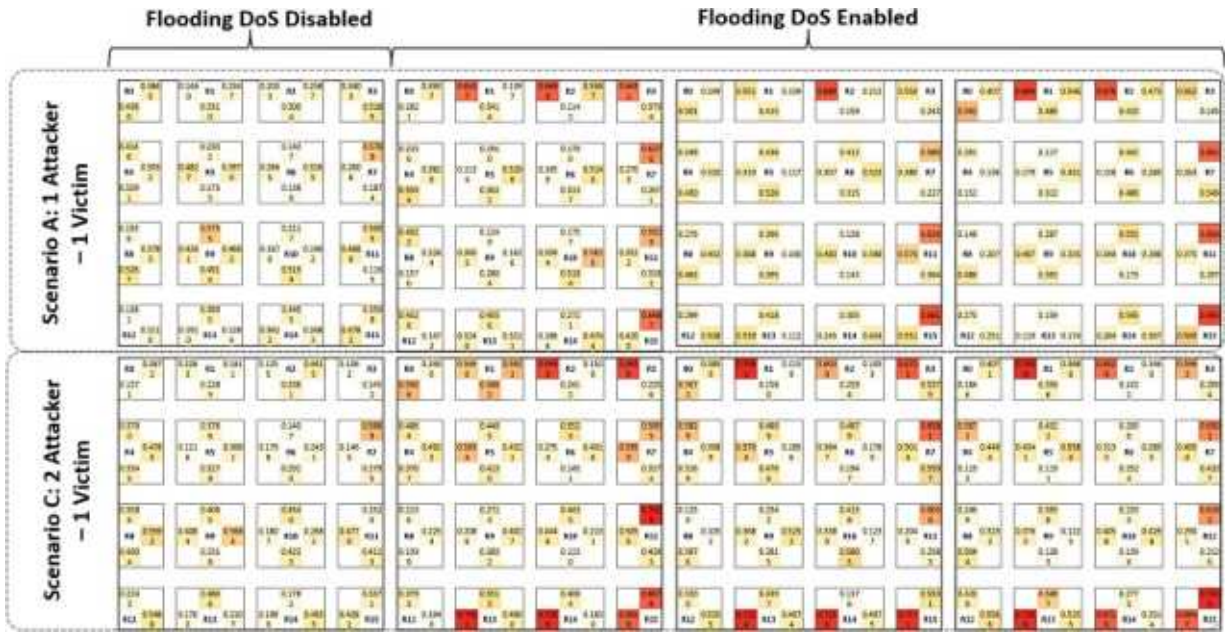


Fig. 6.10 Two real attack examples on buffer occupancy: scenarios a and c in the Fig. 6.9

As shown in Fig. 6.11, the latency of packets increases drastically following the activation of the flooding attack, reaching up to 13 times the normal level under regular workloads. While this specific example involves a single attacker targeting a single victim, it is clear that simultaneous attacks involving multiple M3PIPs would result in even more severe traffic congestion and system instability.

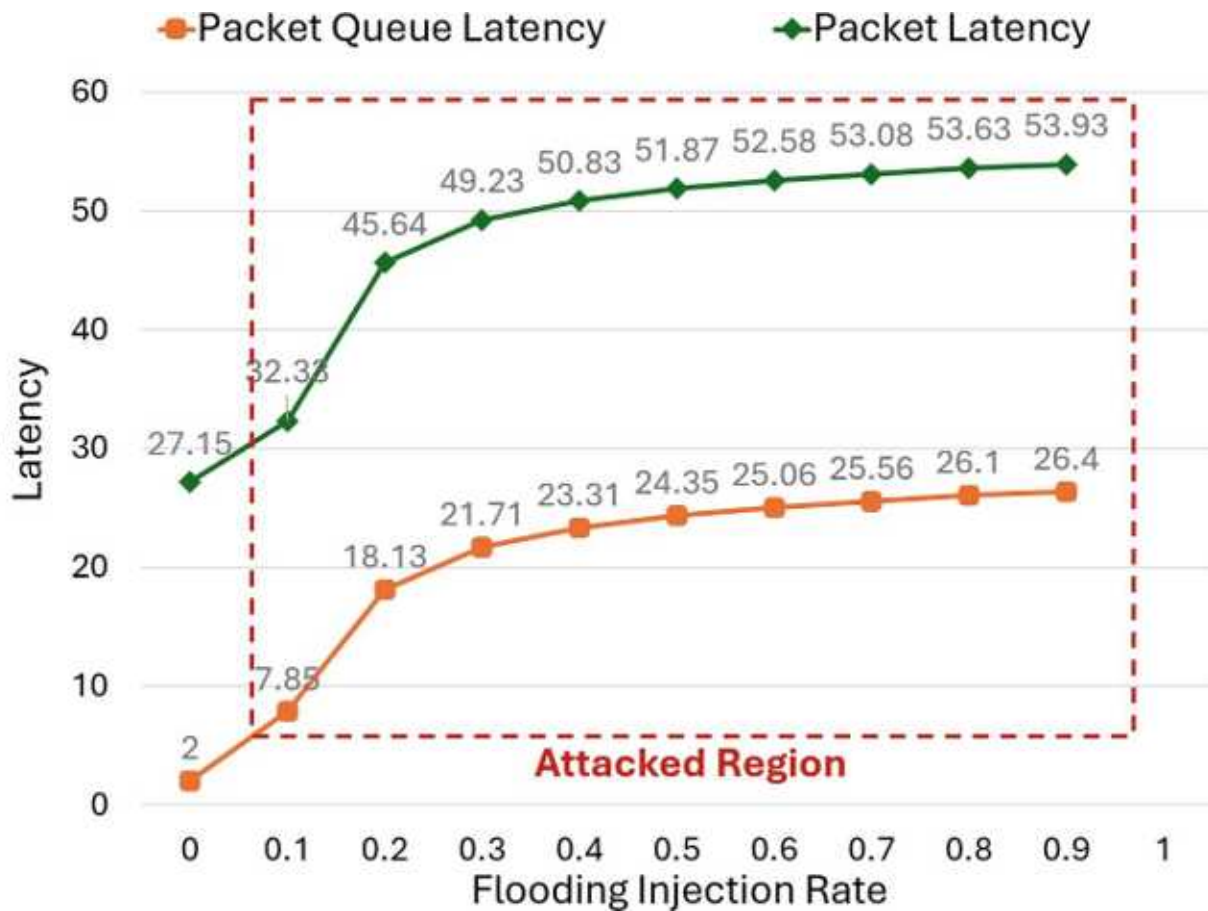


Fig. 6.11 The traffic congestion caused by flooding DoS attacks at different packet injection rates, as reflected by the latency

6.4.2 Graph Theory for Malicious Traffic Detection

Researchers have extensively studied and modelled DoS attacks, focusing primarily on traffic analysis at the network level to identify anomalies and potential threats. For instance, the work referenced in [10] explores a novel approach by utilizing graph-based techniques to evaluate node behavior across the network, moving beyond the conventional focus on detecting individual malicious connections. By integrating advanced neural network and graph convolutional network techniques, this method leverages spatial feature extraction, temporal profiling, and data preprocessing to classify network node behaviors with greater precision. This approach aims to uncover more nuanced insights into security threats, offering a richer understanding of anomalous traffic patterns and network vulnerabilities.

Graph neural networks (GNNs) are widely recognized for their ability to effectively represent spatial features by aggregating information from neighboring nodes and edges, thereby providing a holistic view of message

passing and data aggregation within a network. Building on these capabilities, Zhilong Lu et al. [11] introduced an innovative framework called graph long short-term memory (graph LSTM), which combines the spatial proficiency of GNNs with the temporal processing strengths of LSTM networks. This integrated model is particularly adept at capturing intricate topological relationships in road networks while simultaneously accounting for the temporal variations in traffic patterns. The authors highlight the novelty of their approach in merging GNNs and LSTMs for the specific task of traffic speed prediction. Utilizing an unweighted directed graph representation of road networks, the model incorporates encoder-decoder architecture and message-passing strategies during training. Experimental results demonstrate that the GNN+LSTM model outperforms conventional techniques, achieving significantly improved accuracy in traffic prediction.

6.4.3 Traffic Flow and Routing Dependencies Graph in NoCs

NoC architecture is essential for enabling accurate and efficient data communication in system-on-chip (SoC) environments, utilizing routing protocols such as the static Mesh-XY algorithm to guide packet movement. Figure 6.12 illustrates how a 16-tile NoC traffic system can be mapped to a routing dependency graph, emphasizing the influence of upstream data flow on downstream channels due to the algorithm's static characteristics. In this graph representation, nodes correspond to communication endpoints, while directed edges signify the routing dependencies between them, establishing an orderly sequence for packet movement. By framing traffic flow as directed links within a network, this approach abstracts the complexity of bidirectional communication into a more streamlined model, allowing for a detailed analysis of traffic dependencies without overwhelming the monitoring process.

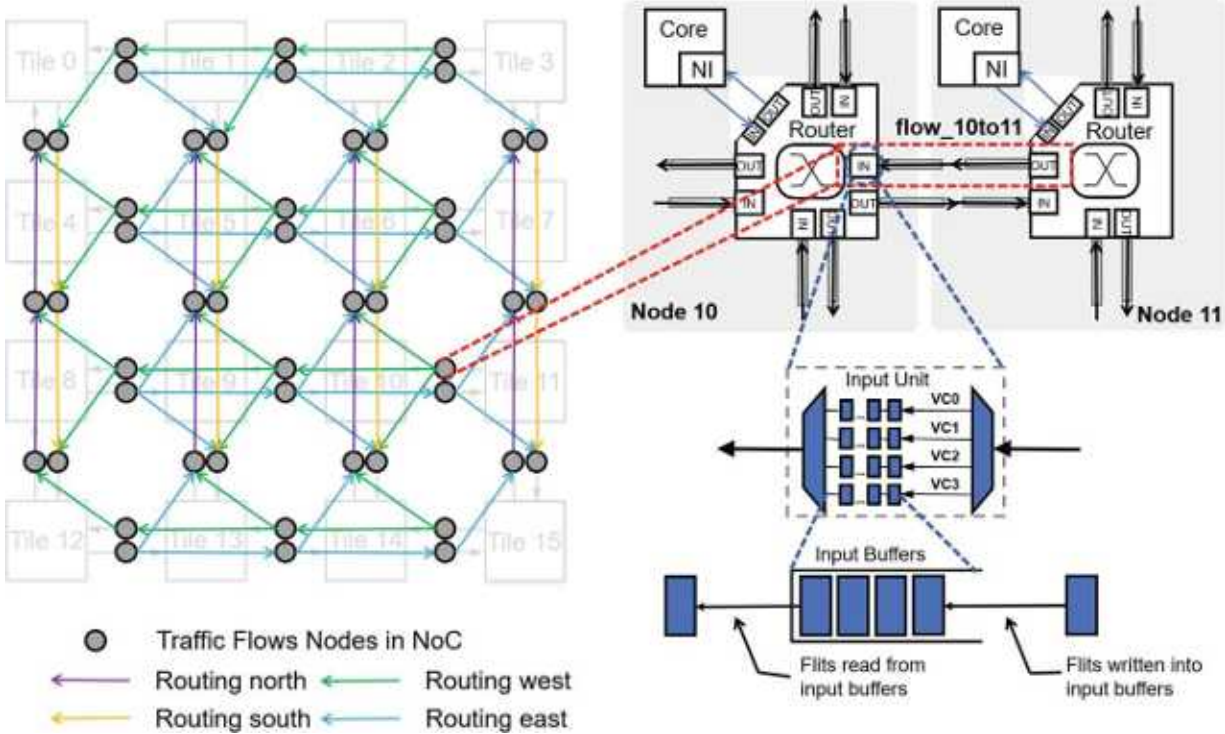


Fig. 6.12 Traffic flow and routing dependencies graph in NoC

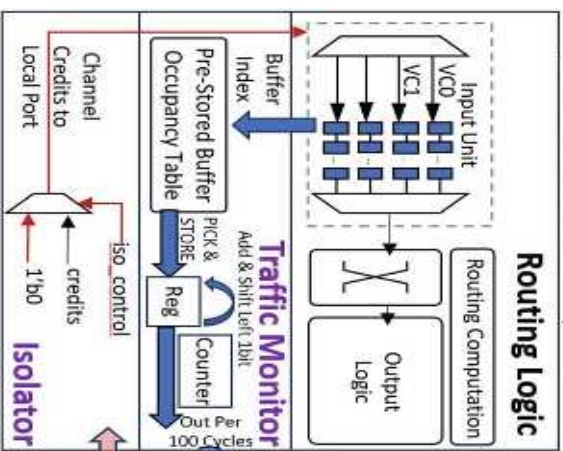
To evaluate traffic congestion within a NoC, metrics such as buffer occupancy (BO) and virtual channel occupancy (VCO) play a pivotal role. As depicted in Fig. 6.12, these metrics are dynamically determined during runtime by measuring the proportion of used buffers and virtual channels relative to their preconfigured capacities. Both metrics exhibit comparable patterns in reflecting congestion levels, making them interchangeable for monitoring purposes in subsequent discussions. A noticeable increase in utilization or packet buildup at a specific router signals significant forwarding delays, serving as a clear indicator of congestion hotspots within the network. This approach not only provides an intuitive way to visualize and understand data flow bottlenecks but also exemplifies a proactive method for identifying and addressing potential transmission inefficiencies in NoC systems.

6.4.4 Graph-Based Method

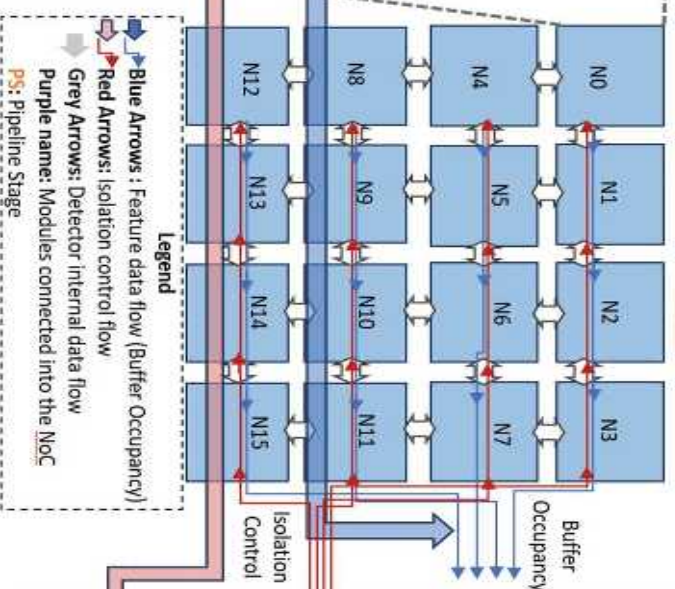
The novel secure NoC system architecture integrates three essential components: the traffic monitor, isolator, and global detector, with the traffic monitor and isolator embedded within custom routers. This architecture is specifically designed to detect and neutralize malicious activities originating from M3PIPs by isolating compromised nodes and attack paths involving harmful third-party IPs. Operating as an independent security layer, the

global detector oversees chip-wide security threats without interfering with the primary NoC-based MPSoC functionality. Figure [6.13](#) provides an architectural overview of the graph-based secure NoC, modelled on a 4×4 Mesh NoC system, showcasing its hierarchical structure. This includes router-level traffic monitors, a standalone global detection unit equipped with GNN and LSTM accelerators, and router-integrated isolators. The diagram also illustrates the workflow between components: the traffic monitors extract localized traffic data and send it to the global detector, which analyzes it for threats and relays this information to the isolators for targeted security actions. Employing a centralized global detection unit offers significant area efficiency compared to decentralized, in-router models. The subsequent sections explain the design details, microarchitectures, and algorithms that power these modules.

Proposed Router Arch (of Node 0)



4*4 Mesh NoC



Global Detector: GNN + LSTM

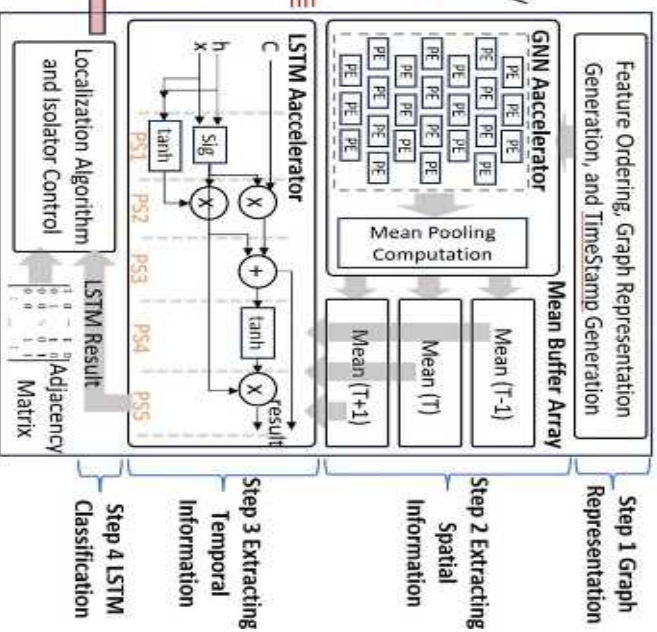


Fig. 6.13 Graph-based secure NoC architecture: A custom router consisting of traffic monitor and isolator, and global detector consisting of GNN accelerator and LSTM accelerator

6.4.4.1 Traffic Monitor

We employed employs buffer occupancy at each router's input port as its primary monitored feature, chosen for two key reasons. First, as an immediate metric, it enables rapid identification of anomalies or attacks. Second, its association with input buffers makes it simple to measure and compute. To enhance the analysis with spatial-temporal context, timestamps are assigned to each frame of feature data. These timestamps, generated by an integrated counter within the detector, capture runtime system dynamics while minimizing communication bandwidth between traffic monitors and the global detector. Only buffer occupancy data is transmitted via 16 bus groups, streamlining the process.

Each router incorporates a lightweight traffic monitor to track buffer occupancy, leveraging a low-overhead design. A look-up table is employed to store and update occupancy values based on buffer indices, eliminating the need for complex calculations. Since buffer configurations are predefined, occupancy values can be determined efficiently by referencing the buffer index. This look-up table approach offers the quickest method for retrieving values, ensuring efficient operation.

Additionally, calculating the mean value of two buffer occupancy readings is simplified through a bitwise left shift operation, avoiding the computational cost of division. The final buffer occupancy values are output every 100 cycles, as regulated by a local counter, providing a timely and resource-efficient mechanism for monitoring traffic in the NoC system.

6.4.4.2 Global Detector: GNN+LSTM

The integrated GNN+LSTM architecture was designed for detecting and classifying traffic congestion in NoC systems by harnessing spatial and temporal data insights. This section outlines both the conceptual design and the corresponding hardware implementation. As visualized in Fig. [6.14](#), the process involves four key stages:

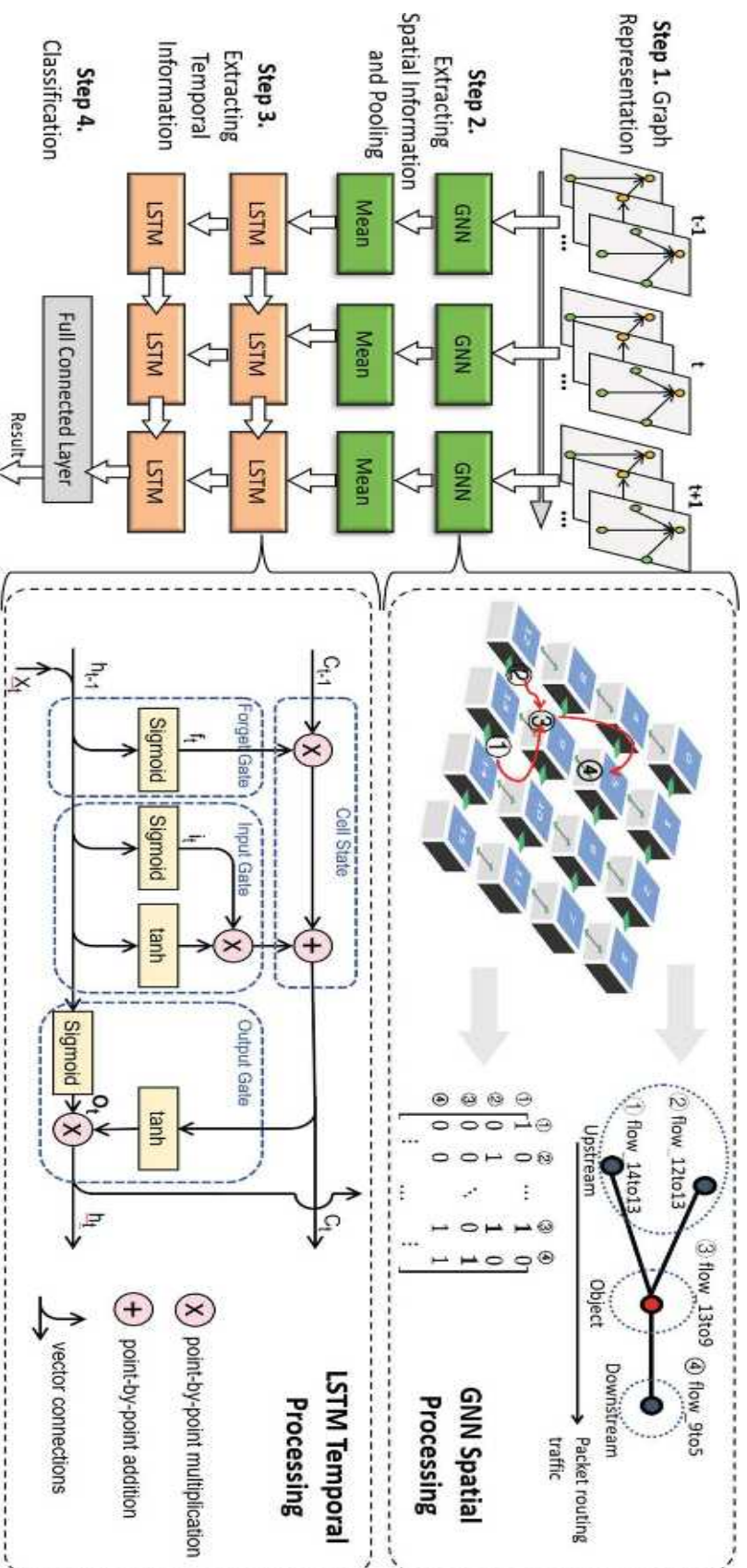


Fig. 6.14 The algorithm structure of GNN and LSTM detection model in the global detector

1. **Graph Representation:** The NoC is abstracted as a graph where nodes symbolize traffic links and edges denote spatial dependencies. An adjacency matrix is constructed to map these dependencies, reflecting the routing hierarchy and directional relationships of data flow.
2. **Spatial Analysis:** Through GNNs, spatial features are extracted from nodes across consecutive time intervals. Mean pooling aggregates data from each node and its connected neighbors, providing a snapshot of localized traffic behavior within the network.
3. **Temporal Analysis:** The aggregated spatial features are processed by LSTM layers to capture the evolution of traffic patterns over time. This temporal analysis identifies trends and recurring anomalies in the data flow, enabling dynamic behavioral modelling.
4. **Traffic Classification:** Temporal outputs from the LSTM layers are passed to a fully connected layer that categorizes the traffic. The classifier discerns between normal network operation and congestion-induced anomalies.

This hybrid GNN+LSTM architecture effectively combines localized spatial insights and temporal flow analysis, making it a robust solution for identifying traffic anomalies in NoC systems. Its modular design ensures adaptability to varying NoC configurations and traffic complexities.

The hardware implementation of the framework aligns closely with the algorithmic stages, as illustrated on the right side of Fig. [6.13](#). The GNN module processes the NoC's graph representation, where nodes symbolize traffic links and edges denote inter-router dependencies and data flow pathways (as shown in the top-right of Fig. [6.14](#)). By employing "embedding," the GNN reduces multidimensional features of network nodes into simplified vectors. This embedding process incorporates average pooling, which merges a node's intrinsic features with those of its upstream neighbors, capturing both localized and spatial characteristics. The resulting adjacency matrix encapsulates the hierarchical relationships between upstream and downstream nodes, essential for NoC routing mechanisms like the deterministic Mesh-XY algorithm. This structured representation, as visualized in Fig. [6.14](#), provides a clear depiction of traffic flows, enabling

efficient analysis of communication patterns and the identification of anomalous or malicious activity.

- (a) **GNN Algorithm:** The impact of applying GNN processing to virtual channel occupancy (VCO) data is demonstrated in Fig. 6.15, showcasing two distinct trends. The blue curve, representing raw VCO data, exhibits sharp fluctuations with numerous peaks that could mislead conventional machine learning models relying on static thresholds, resulting in false positives. On the other hand, the red curve illustrates VCO data after GNN processing, presenting a smoother trajectory with reduced peaks while still preserving the crucial upward trend signaling an attack initiation. Red markers on both curves highlight checkpoints identified by GNN+LSTM framework and a baseline model lacking smoothing and temporal analysis. For instance, checkpoint P0 on the blue curve is mistakenly flagged as abnormal by models using fixed thresholds, despite being a typical peak unrelated to any attack. In contrast, the same point on the red curve, refined by GNN processing, avoids misclassification, demonstrating the GNN's ability to eliminate noise and enhance feature clarity for improved detection accuracy. This comparison underscores the advantages of GNN in preprocessing NoC data, enabling more reliable anomaly identification.

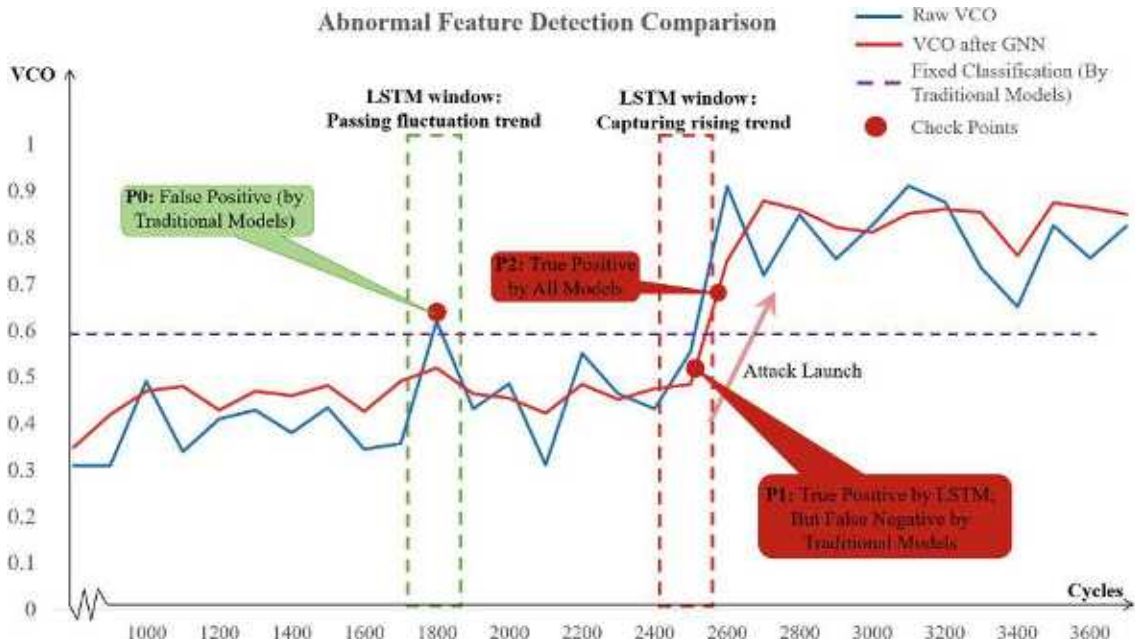


Fig. 6.15 Demonstration for early detection mechanism

- (b) **GNN Hardware Implementation:** To implement the GNN model efficiently, we utilized a systolic processing element (PE) array architecture [12], as depicted in Fig. 6.13. The interconnect structure of the PE array was tailored to align with the abstracted graph of traffic flow nodes presented in Fig. 6.12, ensuring optimized data processing and transfer efficiency. The GNN accelerator activates upon receiving a complete graph representation frame, leveraging mean pooling kernels within the PE array to process the data. The resulting computations are then stored across three buffer arrays, ready to be accessed by the LSTM module for subsequent temporal analysis.
- (c) **LSTM Algorithm:** The LSTM module is composed of a cell state and three gates: input, forget, and output. Each gate performs specific tasks, such as integrating relevant information, discarding unnecessary data, and generating final predictions. To ensure compactness and efficiency, three LSTM units share parameters across their hidden layers. These units play a vital role in analyzing complex temporal sequences, such as patterns in network traffic, to effectively identify anomalies. The internal structure and connectivity of these gates, illustrated in the lower right corner of Fig. 6.14, demonstrate their ability to dynamically process sequential feature data. Optimized using Python and the PyTorch framework and implemented through the Gem5 [13] simulator, the LSTM model employs two sets of weights and biases to handle input vectors and recurrent hidden states. The design refines 12 neural network parameters associated with the gates, enabling precise and adaptive temporal analysis.
- (d) **LSTM Hardware Implementation:** The LSTM classification is executed every 100 cycles, adhering to a predefined schedule, allowing sufficient time for data preparation in the mean buffer arrays. Consequently, its operational latency is less critical compared to the GNN accelerator. To optimize resource efficiency, a five-stage pipeline structure was implemented for the LSTM module, enabling it to handle one node from the GNN's output frame at a time. This design effectively reduces hardware complexity while maintaining functionality. The identified anomalies are subsequently fed into the localization algorithm, which, in tandem with the adjacency matrix, pinpoints the attacker and informs subsequent actions.

When applied to real-time feature data, the LSTM model operates continuously, resembling the functionality of a moving average filter. It dynamically captures new data for analysis while selectively discarding outdated information, enabling a seamless and adaptive evaluation of incoming traffic patterns. Figure [6.15](#) illustrates LSTM's performance on feature data, highlighting its advantages over traditional machine learning approaches such as SVM, random forest, logistic regression, and decision trees. Unlike conventional models, which rely on fixed threshold classifications and lack temporal analysis, LSTM incorporates an advanced mechanism through its input, forget, and output gates. These gates allow LSTM to analyze feature data dynamically within a defined temporal window, minimizing false positives caused by transient peaks that exceed preset thresholds.

The green and red vertical bars in Fig. [6.15](#) indicate the detection periods identified by LSTM. For instance, point P0 on the blue curve within the green bar, previously misclassified as abnormal by traditional models, is accurately identified by LSTM as a benign fluctuation. Point P1, missed entirely by conventional methods, is effectively detected by LSTM, showcasing its superior sensitivity. Point P2, a significant peak detected by all models, further demonstrates LSTM's ability to distinguish genuine threats from routine traffic variations. Notably, LSTM identifies P1 approximately 100 cycles earlier than P2, a feat unachievable by other models, emphasizing its early-detection capabilities. This aligns with the concept of predictive anomaly detection, as discussed in [\[14\]](#), where one-step-ahead prediction proved essential. In the secure NoC architecture, LSTM extends this approach by functioning as a multi-step-ahead prediction tool, significantly enhancing its ability to identify and respond to emerging threats in real time.

6.4.4.3 Localization Algorithm

The final stage in the operation of the global detector is the localization algorithm module, activated only when the LSTM model flags an abnormal classification. Algorithm Fig. [6.16](#) describes a process involving three key steps to identify malicious nodes within the NoC traffic graph. First, the algorithm identifies victim nodes by analyzing signs of upstream congestion. Next, it applies a breadth-first search (BFS) approach to trace attack paths originating from these victim nodes, prioritizing routes with significant traffic

anomalies. Finally, it pinpoints the source nodes responsible for the attack by locating the origins of the congested paths.

Algorithm 1 Localizing M3PIPs

Input: *upstream_adjacency*, *LSTM_result*, *features*
Output: *m3pips*, *attack_paths*

function MAIN
 VictimIDs $\leftarrow$ FindRootNodes_Victims()
 AttPaths $\leftarrow$ IdentifyAttPaths(*VictimIDs*)
 Src_attacker $\leftarrow$ IdentifySrcID_attacker(*AttPaths*)
 Output *Src_attacker* and *AttPaths*
end function

function FINDROOTNODES_VICTIMS
 Traverse *upstream_adjacency* to find nodes without
 downstream nodes in congestion, based on *LSTM_result*
 return *VictimIDs*
end function

function IDENTIFYATTPATHS(*VictimIDs*)
 for each victim in *VictimIDs* **do**
 Use BFS to traverse upstream nodes from each
 victim
 Identify the path with maximum feature values as a
 potential attack path
 Add the identified path to *AttPaths*
 end for
 return *AttPaths*
end function

function IDENTIFYSRCID_ATTACKER(*AttPaths*)
 For each *AttackPath*, return the first node
 Mark these nodes as *Src_attacker*
 return *Src_attacker*
end function

Fig. 6.16 Algorithm of localizing M3PIPs

Figure [6.17](#) provides a visual representation of the localization process, which begins with the detection of congestion in specific traffic nodes. This detection triggers the localization algorithm to analyze the attack path systematically. By examining the adjacency matrix that outlines the NoC's network topology, the algorithm tracks congestion upstream from the affected nodes. Using a reverse traversal with BFS, it evaluates traffic loads at each upstream node to identify those with the highest congestion levels. This process helps delineate the paths impacted by malicious traffic and attributes the traffic flow to the flooding attack.

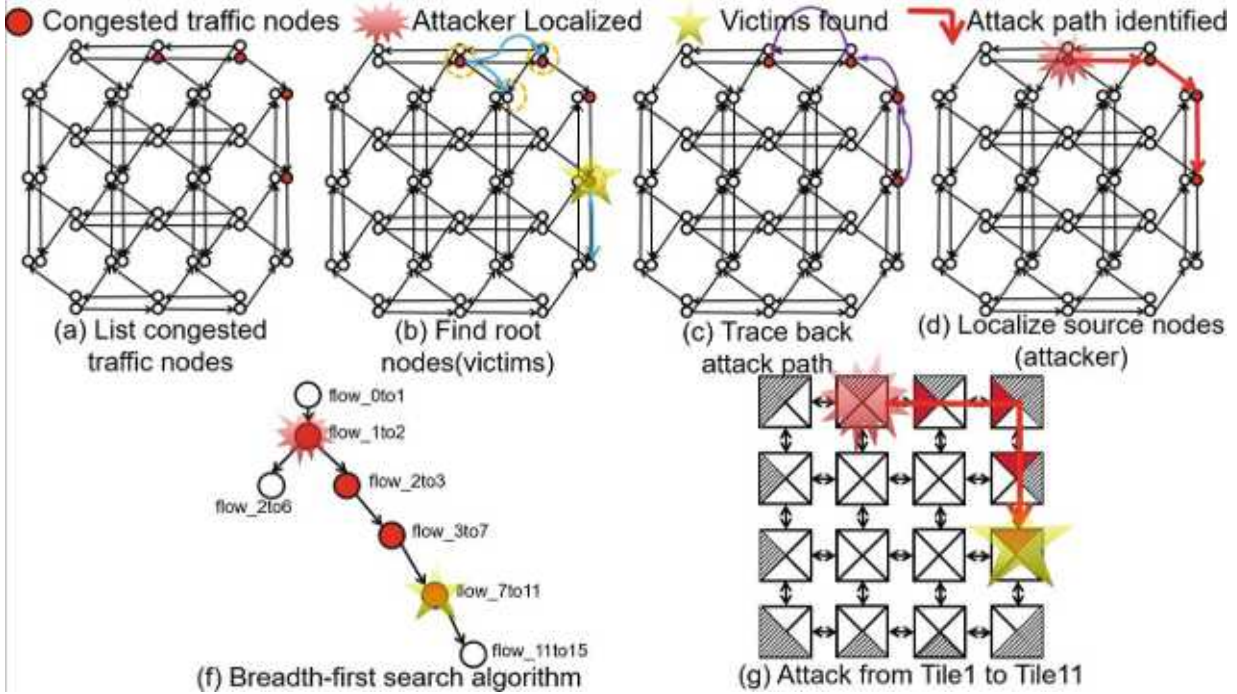


Fig. 6.17 M3PIP localization demonstration in 4×4 NoC: tile 1 is an attacker; tile 11 is a target victim

Nodes are classified based on their role in the attack path. Victim nodes, for example, are characterized by congestion in their own traffic without corresponding downstream congestion. Conversely, source nodes initiating the attack are identified by locating the first congested node upstream of a non-congested area.

Step 1: As shown in Fig. 6.17a, the algorithm starts by detecting traffic nodes experiencing congestion, which indicates the likelihood of malicious activity within the network.

Step 2: It then identifies “root nodes,” defined as nodes without any upstream congestion, which are potential sources of the traffic anomalies. From these root nodes, the algorithm tracks the propagation of congestion downstream to uncover the impacted paths. The process concludes when a node without downstream congestion is encountered, designating it as the attack’s target, as depicted in Fig. 6.17b.

Step 3: Utilizing the BFS traversal method, the algorithm maps out attack paths by working upstream from victim nodes, thereby outlining the entire attack route. Figure 6.17c illustrates this process. BFS evaluates adjacent nodes at each level, sequentially queuing them for further exploration. This

systematic traversal, depicted in Fig. [6.17f](#), ensures accurate path tracing, even in scenarios involving multiple simultaneous attacks.

Result The source IPs of the attackers are identified by pinpointing the origins of these attack paths, effectively tracing the roots of the DoS attack, as illustrated in Fig. [6.17d](#).

Once attackers and their corresponding victims are identified, the secure NoC system activates a credit manipulation mechanism to isolate the attackers from the NoC. This mechanism blocks malicious packet injections by cutting off the attackers' communication pathways. Additionally, the secure NoC system continuously monitors the network load and can extend isolation to additional attackers sharing the same path, ensuring comprehensive protection against DoS attacks and safeguarding the NoC's overall performance.

6.4.4.4 Defense: Credits Interposing Manipulation

In the secure NoC router architecture introduced in Sect. [6.4.4](#), a credit interposition mechanism is introduced to isolate potentially malicious third-party IPs and regulate traffic during DoS attacks. Traditionally, credits in routers are used to indicate buffer availability and manage data flow, but here, they are repurposed to enhance security. By assigning a zero-credit value to untrusted data sources, the system effectively blocks their ability to transmit packets. This approach not only signals a loss of trust but also halts data reception from the offending source. Importantly, this enhancement is seamlessly integrated into the router's operation, preserving its primary function of forwarding legitimate data. Furthermore, the isolation mechanism operates locally, targeting only the port connected to the suspicious IP while keeping other ports functional, ensuring the uninterrupted routing of normal traffic.

Figure [6.18](#) illustrates the isolation circuit embedded within the router, a key component of the secure NoC security framework. Controlled by an "iso_control" signal from the global detector, these circuits distinguish between authorized data and malicious traffic. Upon detecting suspicious activity, it triggers the isolation mechanism via a single-bit control signal to the affected router's isolator. This swift intervention disconnects the problematic IP from the network, mitigating potential damage with minimal delay. The design's simplicity avoids the need for complex modifications to

routing algorithms or additional hardware, thereby reducing overhead. This streamlined approach ensures rapid and efficient network defense while maintaining the overall integrity and performance of the NoC.

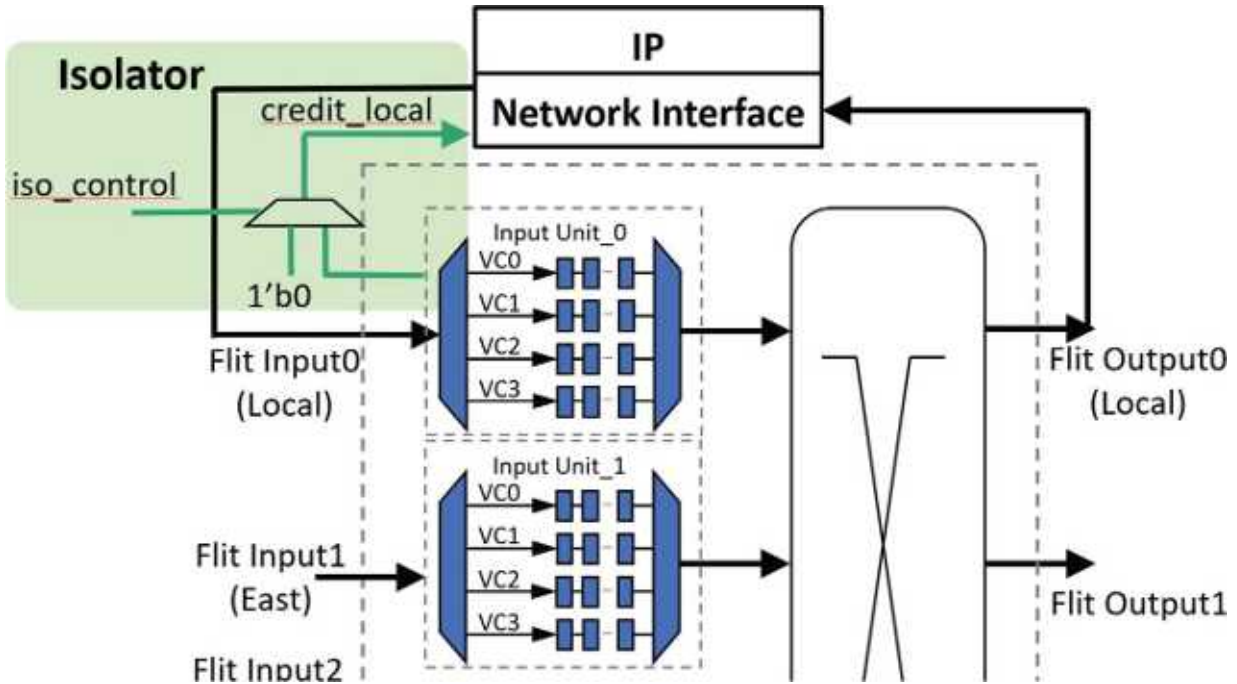


Fig. 6.18 The credits interposing manipulation in routers for M3PIPs isolation

6.4.5 Experiment and Evaluation

To evaluate the effectiveness of the secure NoC system in the case study, we employed the Garnet NoC simulator, integrated within the Gem5 framework [13], to simulate network activities over time. The experimental setup, detailed in Table 6.2, involved generating flooding DoS attacks by a malicious node that injected excessive, nonessential packets into the NoC, while synthetic traffic pattern (STP) and PARSEC benchmarks were running. Each experiment spanned 5 million clock cycles, and the collected data was split into 70% for training, 20% for validation, and 10% for testing. This data preparation pipeline is illustrated in the top section of Fig. 6.19.

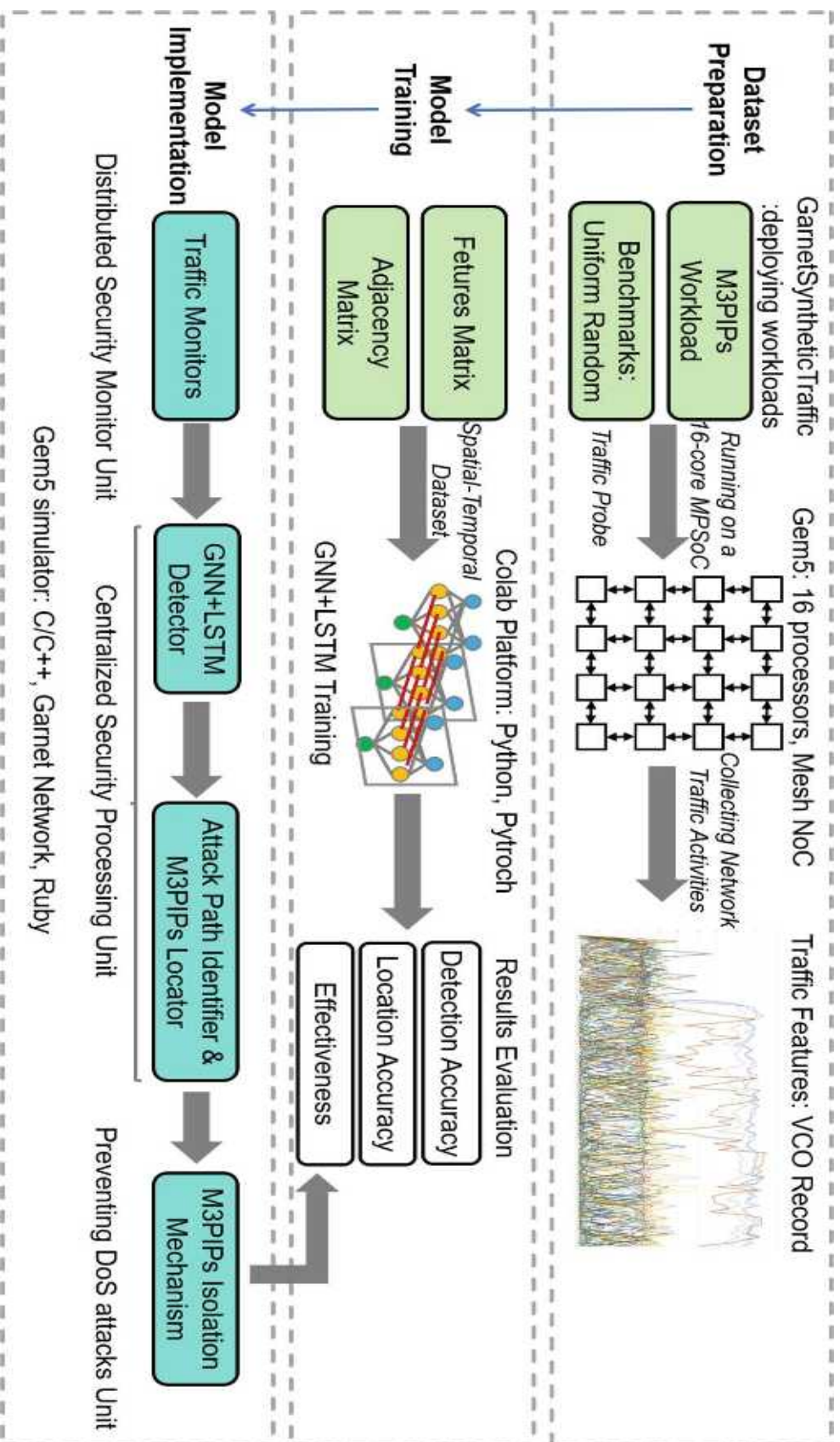


Fig. 6.19 Experiment flow

Table 6.2 Experiment configuration

Configuration	Parameter
NoC topology and size	4×4 mesh
Router I/O ports	5 I/O ports, 4 VCs per port
Router microarchitecture	Wormhole switching, three-stage routing
CPU	16 X86 cores (for PARSEC simulation in Gem5) or 16 RISC cores (Mor1kx for overhead evaluation)
Cache coherence model	Ruby
Cache configuration	L1 and L2
Benchmarks	Synthetic Traffic Pattern, PARSEC

The figure's middle section outlines the process for compiling, training, validating, and evaluating the GNN+LSTM model, which was embedded into the Garnet framework for runtime detection evaluation. For hardware-level assessments, ProNoC [15] was utilized as a versatile simulation platform tailored for MPSoC architectures. This allowed us to configure and analyze a 16-core system with a mesh topology, enabling detailed simulations of traffic patterns. The experiments focused on multiple aspects: the classification accuracy of the detection model, the precision of localization efforts, and the associated hardware overhead.

The operational workflow and implementation details of the secure NoC system are depicted in the bottom section of Fig. 6.19. This comprehensive evaluation framework provides insights into the system's robustness in detecting and localizing malicious activities while also assessing its hardware efficiency in practical MPSoC environments.

6.4.5.1 Flooding DoS Detection Evaluation

Accurately detecting network traffic congestion and evaluating binary classification models are critical for securing NoC systems. This study introduces the GNN+LSTM framework while also comparing its performance against several traditional machine learning methods, including SVM, random forest, logistic regression, decision tree, Gaussian NB, and perceptron. Key metrics such as test accuracy, precision for **Class 0** (representing normal, non-attack states) and **Class 1** (indicating attacks), recall, and F1 score were used to assess the effectiveness of each model.

The experimental setup involved testing 100 scenarios with varying combinations of attackers and victims, allowing for diverse evaluation conditions. A time-averaging window of 100 clock cycles was applied to extract traffic behaviors, using the mean buffer occupancy over these cycles as a representative feature of the network's status. This approach ensures detection is performed every 100 cycles rather than at each individual cycle, balancing security demands with computational efficiency. Table 6.3 provides a detailed comparison, highlighting the performance of the detection algorithm alongside the evaluated machine learning models.

Table 6.3 Detection performance comparison between different models on DoS detection (STP benchmark)

Model	Accuracy	Class 0			Class 1		
		Precision	Recall	F1 score	Precision	Recall	F1 score
SVM	0.95	0.94	1.00	0.97	0.99	0.86	0.92
RF	0.96	0.96	0.98	0.97	0.96	0.92	0.94
LR	0.93	0.90	0.99	0.95	0.99	0.81	0.88
DT	0.95	0.94	0.98	0.96	0.97	0.88	0.92
GNB	0.97	0.96	0.99	0.97	0.98	0.92	0.95
PERCEP	0.94	0.92	0.99	0.96	0.99	0.83	0.90
GNN+LSTM	0.98	0.98	0.99	0.99	0.97	0.98	0.98

SVM support vector machine, *RF* random forest, *LR* logistic regression, *DT* decision tree, *GNB* Gaussian NB, *PERCEP* perceptron, *GNN* graph neural network, *LSTM* long short-term memory

The results highlight the secure NoC system's strong capability to detect congestion and accurately identify attackers, even in challenging situations involving multiple attackers and intersecting attack paths. In scenarios where several sources contribute to network congestion, the system prioritizes isolating the node causing the most significant disruption. During subsequent detection cycles, the system continues to identify and disable previously concealed malicious nodes, effectively reducing the overall impact of the attack.

In a comparative analysis of detection models, the secure NoC system, utilizing a GNN+LSTM algorithm, demonstrated superior performance with a test accuracy of 0.98. Unlike traditional models such as SVMs and random

forests, this approach effectively captures spatial relationships and temporal patterns, making it particularly adept at rapid anomaly detection. This capability is critical for addressing traffic congestion caused by flooding attacks, which evolve from normal to congested states. The model excels in identifying these transitions early, allowing for timely mitigation of potential disruptions and preventing severe congestion. When benchmarked against other advanced ML models, including Gaussian NB and random forest, the GNN+LSTM consistently outperforms in accuracy and other metrics, highlighting its reliability and robustness. This architecture sets a new benchmark for analyzing data with complex spatial and temporal dependencies, offering a valuable framework for future research.

The detection capabilities of the system were further validated using real workload PARSEC benchmarks during periods of simulated flooding attacks. As shown in Table 6.4, the system achieved an average accuracy and precision of 0.99 for class 0 cases but a slightly reduced precision of 0.97 for class 1 cases. The x264 workload contributed to this reduction due to its pipelined media processing, which exhibits high data dependencies distinct from workloads like Black-Scholes and Bodytrack. These differences, characterized by both high data sharing and extensive data exchange, present unique challenges for detection accuracy.

Table 6.4 Detection performance on PARSEC benchmark

Parsec	Class 0				Class 1		
	Accuracy	Precision	Recall	F1 score	Precision	Recall	F1 score
Black-Scholes	0.98	0.98	0.99	0.98	0.99	0.97	0.98
Bodytrack	0.99	0.99	0.99	0.99	0.99	0.99	0.99
x264	0.99	1.00	0.99	0.98	0.92	1.00	0.96
Ferret	0.99	0.99	0.98	0.99	0.98	0.99	0.99
Swaptions	0.99	0.99	0.97	0.98	0.97	0.99	0.98
Average	0.99	0.99	0.98	0.98	0.97	0.99	0.98

6.4.5.2 Malicious Node Localization Performance

We carried out an extensive analysis to evaluate the potential impact of DoS attacks across all nodes within a network. In a 16-core multiprocessor system, each node was considered capable of launching up to 15 distinct malicious activities, targeting each of the remaining nodes as victims. This

configuration resulted in the identification of 240 unique attack behavior patterns for testing. The results demonstrated that the secure NoC system achieved perfect localization accuracy (100%) for flooding-based DoS attacks, contingent on the accurate detection of malicious traffic to facilitate source tracing. These findings validate the effectiveness and reliability of the framework in identifying and mitigating such security threats.

6.4.5.3 *Timing and Hardware Overhead Analysis*

Table 6.5 summarizes the average results from detection and localization tests across 240 distinct attack scenarios. For standard attack patterns without overlapping, the localization of attackers was completed within an average of 29.1 cycles. In cases where multiple attackers initiated overlapping attack patterns, the secure NoC system demonstrated enhanced effectiveness. When faced with multiple attackers, the system prioritized identifying and isolating the most disruptive attacker first. After isolating the primary source of the attack, an additional sampling period was employed to detect and neutralize other hidden malicious nodes along the attack path. As a result, handling overlapping attack patterns required an average of 53.3 cycles. The overall mean for detection and localization across all scenarios was 35.1 cycles. This structured approach ensures thorough identification and isolation of all attackers, significantly bolstering the network's security.

Table 6.5 Localization performance and timing latency

Scenario	Timing latency (average)	Localization accuracy
1 ATT and 1 VIC	13.3	100%
2 ATT and 1 VIC	29.9	100%
2 ATT and 2 VIC	43.9	100%
OL attack path (2 ATT)	53.3	100%
Average	29.1 (non-OL)/35.1 (OL)	100%

ATT attacker, *VIC* victim, *OL* overlapping, *non-OL* nonoverlapping

The hardware overhead for key components including the router, traffic monitor, the complete 4×4 MPSoC [15] with 16 RISC cores, the GNN processing element array, and the LSTM module is detailed in Table 6.6. For synthesis, Quartus Prime software and an Altera Cyclone V Series FPGA were utilized, highlighting the system's practical hardware implementation.

Table 6.6 Hardware overhead: logic synthesizer Results

Hardware	Router	Traffic	Traffic monitor	4 × 4 MPSoC	GNN PE	LSTM
		Monitor	overhead (%)	(mor1kx_tile)	array	
Logic elements	1806	21	1.1%	37284	4680	1764
Registers	676	19	2.7%	13427	480	415
Memory bits	1360	9	0.66%	3384544	7032	0

6.4.5.4 Comparison with Related Works

Table 6.7 provides a comparative analysis of our introduced secure NoC system against other solutions that integrate detection and localization capabilities. Our framework offers a holistic approach to managing DoS attacks by detecting malicious traffic paths, pinpointing M3PIP attackers, and isolating them effectively. This comprehensive design allows our system to outperform related works in both detection and localization accuracy while maintaining the lowest hardware overhead among the compared approaches.

Table 6.7 Comparison with related works

Related works	Threat model	Detection	Machine	Localization	Localization	Isolation and	Hardware overhead
		accuracy	learning model	accuracy	time	defense layer	(per router)
Sniffer [16]	M3PIPs	97.6% (STP)	PERCEP	96.7%	84 cycles	×	3.3%
AGAPE [17]	MNoC	91%	GAN	×	×	Hardware	7.2%
[1]	M3PIPs	×	LUT	×	0.75 μ s	×	5.9%
[18]	MNoC	93.8%	ANN	100%	×	×	×
GNN+LSTM scheme	M3PIPs	98%	GNN+LSTM	100%	35.1 cycles	Hardware	1.8%

Although our timing overhead is moderate in comparison, with localization times slightly exceeding those in [16], our framework compensates with its robust early detection capabilities. As shown in Fig. 6.15, the system can detect potential attacks hundreds or even thousands of cycles in advance. This early detection capability minimizes the impact of extended localization time by ensuring timely responses to security threats. Such foresight allows for preemptive action, preserving the overall system

responsiveness and reliability. Future improvements could focus on further optimizing localization speed to enhance the system's overall performance.

6.5 Conclusions

This chapter explored the transformative potential of machine learning in enhancing the security of NoCs. Building upon CNN-based approaches discussed earlier, this chapter highlighted two advanced schemes that integrate diverse machine learning models to address a broader range of NoC security challenges. The first scheme demonstrated how cascading machine learning models can improve detection and response to general DoS attacks, emphasizing the power of combining different algorithms for improved accuracy and efficiency. The second scheme introduced a graph-based method, leveraging GNNs and LSTMs, to proactively detect and localize refined flooding DoS attacks with high precision.

By systematically analyzing threat modelling, proposing innovative architectures, and rigorously evaluating performance, this chapter demonstrated the versatility and adaptability of machine learning techniques in tackling complex NoC vulnerabilities. The integration of graph theory and advanced time-series models provided a robust framework for early detection and real-time response to dynamic security threats.

The findings in this chapter underscore the importance of adopting multilayered machine learning strategies to safeguard NoCs in an era of increasing cyber threats. These insights pave the way for future research, emphasizing the need for scalable, efficient, and adaptive security solutions to meet the growing demands of large-scale NoC systems. Through these efforts, the foundation is set for designing more resilient hardware architectures that can preemptively address evolving security challenges.

References

1. Charles, S., Lyu, Y., Mishra, P.: Real-time detection and localization of distributed dos attacks in noc-based socs. *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* **39**(12), 4510–4523 (2020) [[Crossref](#)]
2. Chaves, C.G., Azad, S.P., Hollstein, T., Sepúlveda, J.: A distributed DoS detection scheme for NoC-based MPSoCs. In: 2018 IEEE Nordic Circuits and Systems Conference (NORCAS): NORCHIP and International Symposium of System-on-Chip (SoC), pp. 1–6. IEEE (2018)
3. Charles, S., Lyu, Y., Mishra, P.: Real-time detection and localization of DoS attacks in NoC based SoCs. In: 2019 Design, Automation & Test in Europe Conference & Exhibition (DATE), pp. 1160–

1165. IEEE (2019)
4. Sharma, G., Bousdras, G., Ellinidou, S., Markowitch, O., Dricot, J.M., Milojevic, D.: Exploring the security landscape: NoC-based mpsoC to cloud-of-chips. *Microprocess. Microsyst.* **84**, 103963 (2021)
[\[Crossref\]](#)
 5. Redmon, J.: You only look once: Unified, real-time object detection. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (2016)
 6. Kulkarni, A., Pino, Y., French, M., Mohsenin, T.: Real-time anomaly detection framework for many-core router through machine-learning techniques. *ACM J. Emerg. Technol. Comput. Syst.* **13**(1), 1–22 (2016)
[\[Crossref\]](#)
 7. Sinha, M., Bhattacharyya, P., Rout, S.S., Prakriya, N.B., Deb, S.: Securing an accelerator-rich system from flooding-based denial-of-service attacks. *IEEE Trans. Emerg. Top. Comput.* **10**(2), 855–869 (2021)
 8. Sudusinghe, C., Charles, S., Mishra, P.: Denial-of-service attack detection using machine learning in network-on-chip architectures. In: *Proceedings of the 15th IEEE/ACM International Symposium on Networks-on-Chip*, pp. 35–40 (2021)
 9. Wang, H., Halak, B., Ren, J., Atamli, A.: DI2fence: Integrating deep learning and frame fusion for enhanced detection and localization of refined denial-of-service in large-scale NoCs. In: *Proceedings of the 61st ACM/IEEE Design Automation Conference*, pp. 1–6 (2024)
 10. Zola, F., Seguro-Gil, L., Bruse, J.L., Galar, M., Orduna-Urrutia, R.: Network traffic analysis through node behaviour classification: a graph-based approach with temporal dissection and data-level preprocessing. *Comput. Secur.* **115**, 102632 (2022)
[\[Crossref\]](#)
 11. Lu, Z., Lv, W., Xie, Z., Du, B., Huang, R.: Leveraging graph neural network with lstm for traffic speed prediction. In: *2019 IEEE SmartWorld, Ubiquitous Intelligence & Computing, Advanced & Trusted Computing, Scalable Computing & Communications, Cloud & Big Data Computing, Internet of People and Smart City Innovation (SmartWorld/SCALCOM/UIC/ATC/CBDCOM/IOP/SCI)*, pp. 74–81. IEEE (2019)
 12. Abadal, S., Jain, A., Guirado, R., López-Alonso, J., Alarcón, E.: Computing graph neural networks: A survey from algorithms to accelerators. *ACM Comput. Surv.* **54**(9), 1–38 (2021)
[\[Crossref\]](#)
 13. Lowe-Power, J., Ahmad, A.M., Akram, A., Alian, M., Amslinger, R., Andreozzi, M., Armejach, A., Asmussen, N., Beckmann, B., Bharadwaj, S., et al.: The gem5 simulator: Version 20.0+. *arXiv preprint arXiv:2007.03152* (2020)
 14. Woo, L.L., Zwolinski, M., Halak, B.: Early detection of system-level anomalous behaviour using hardware performance counters. In: *2018 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pp. 485–490. IEEE (2018)
 15. Monemi, A., Tang, J.W., Palesi, M., Marsono, M.N.: Pronoc: a low latency network-on-chip based many-core system-on-chip prototyping platform. *Microprocess. Microsyst.* **54**, 60–74 (2017)
[\[Crossref\]](#)

16. Sinha, M., Gupta, S., Rout, S.S., Deb, S.: Sniffer: a machine learning approach for DoS attack localization in NoC-based SoCs. *IEEE J. Emerg. Sel. Top. Circuits Syst.* **11**(2), 278–291 (2021) [[Crossref](#)]
17. Wang, K., Zheng, H., Li, Y., Li, J., Louri, A.: Agape: Anomaly detection with generative adversarial network for improved performance, energy, and security in manycore systems. In: 2022 Design, Automation & Test in Europe Conference & Exhibition (DATE), pp. 849–854. IEEE (2022)
18. Wang, H., Halak, B.: Hardware trojan detection and high-precision localization in NoC-based MPSoC using machine learning. In: Proceedings of the 28th Asia and South Pacific Design Automation Conference, pp. 516–521 (2023)

[OceanofPDF.com](https://www.oceanofpdf.com)

Part III

Threat Analysis of AI Production Chain

[OceanofPDF.com](https://oceanofpdf.com)

7. AI Production Process Security Attacks and Mitigation Techniques

Haosen Yu¹✉ and Basel Halak¹✉

(1) School of Electronics and Computer Science, University of Southampton, Southampton, UK

✉ Haosen Yu

Email: Haosen.Yu@soton.ac.uk

✉ Basel Halak (Corresponding author)

Email: Basel.Halak@soton.ac.uk

Keywords Security threats – AI production process – Data acquisition risks – Model training vulnerabilities – Software implementation risks – Hardware implementation risks – Cloud service vulnerabilities – Edge computing risks – Inference accelerators – PCB assembly threats – Functional testing risks – Post-deployment threats – Security analysis – Risk assessment – Model archiving security

7.1 Introduction

The deployment of artificial intelligence (AI) models into production environments marks a critical transition from a theoretical construct to a functional application, interacting with real-world data and users. This stage, however, is fraught with a unique set of security vulnerabilities that can be exploited by malicious actors [1]. As AI systems become more integrated into critical infrastructure, finance, and autonomous systems, the consequences of a security breach extend far beyond data loss, posing risks to physical safety and economic stability [2]. The choices made regarding

the software implementation or hardware accelerator implementation each introduce distinct threat vectors that must be meticulously addressed [3]. This chapter provides a comprehensive review of the security threats inherent to each stage of the AI production process, from initial data acquisition to post-deployment archiving. By dissecting both the software and hardware implementation pipelines, we will explore the multifaceted attack surface, supported by recent academic literature, and examine a range of proposed mitigation strategies designed to build more secure and resilient AI systems.

7.2 Chapter Overview

This chapter provides a comprehensive examination of the security threat landscape across the entire AI system production life cycle, beginning with a high-level overview of the distinct software and hardware implementation pathways and their inherent vulnerabilities in Sect. [7.3](#). The discussion then proceeds chronologically in Sect. [7.4](#), first focusing on the AI model development stage, detailing threats such as data poisoning, backdoor insertion during model training, and evasion attacks during evaluation. Subsequently, Sect. [7.5](#) analyzes the software implementation stage, exploring risks associated with deploying models on cloud services, dedicated inference accelerators, and edge devices, including model extraction, side-channel attacks, and privacy leakage. Attention then shifts to the complex supply chain threats within the hardware accelerator implementation pipeline in Sect. [7.6](#), outlining vulnerabilities that arise during IP core licensing, SoC integration, fabrication, PCB assembly, and functional testing, with a particular focus on hardware Trojans. Section [7.7](#) addresses the post-deployment stage, covering both interactive attacks during the consumption phase and latent risks, such as intellectual property theft and vulnerability propagation that persist during model archiving and recycling. Finally, Sect. [7.8](#) provides a summary of this chapter.

7.3 Threat Landscape Across the AI System Production Process

7.3.1 AI Production Process Overview

An analysis of the security vulnerabilities in the AI system production process landscape reveals a complex and multifaceted threat surface. The provided diagram Fig. [7.1](#), which illustrates both software and hardware implementation pathways for AI systems, serves as a critical roadmap for identifying potential points of compromise. From data acquisition to final deployment and consumption, each stage presents unique security challenges that can be exploited by malicious actors, potentially leading to model theft, data poisoning, system failure, or the execution of unseen, harmful functionalities. This chapter will discuss these vulnerabilities

within the two primary implementation pipelines: software implementation and hardware accelerator implementation.

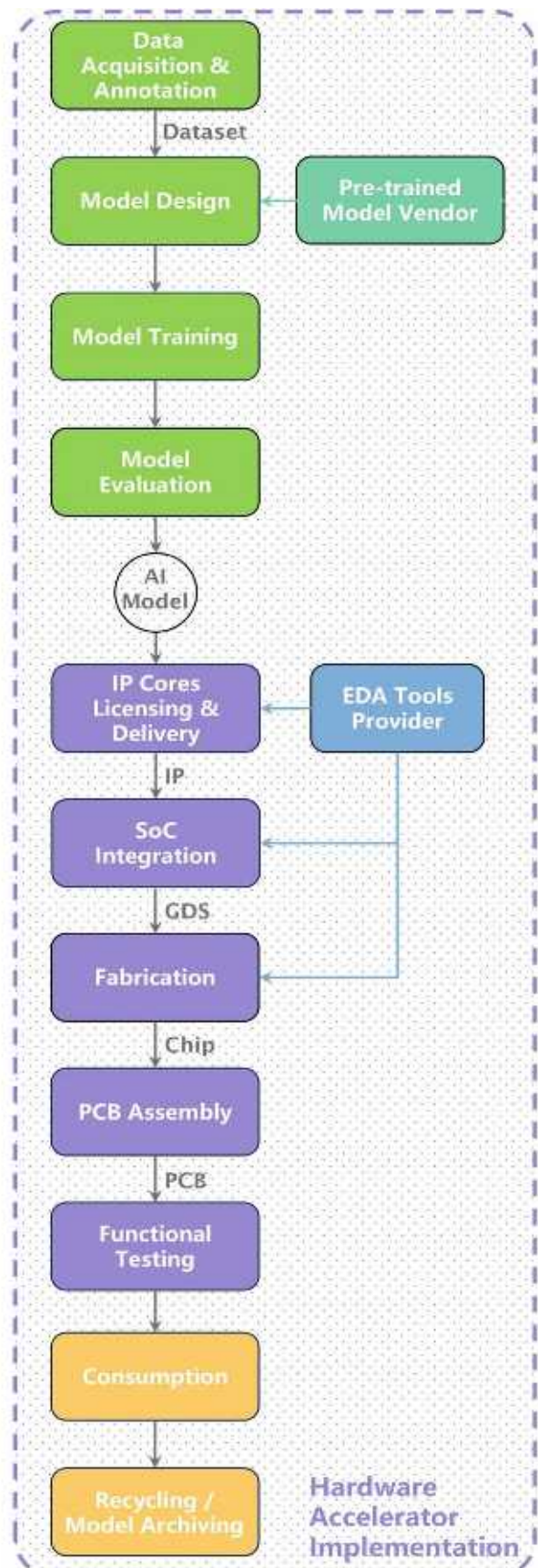
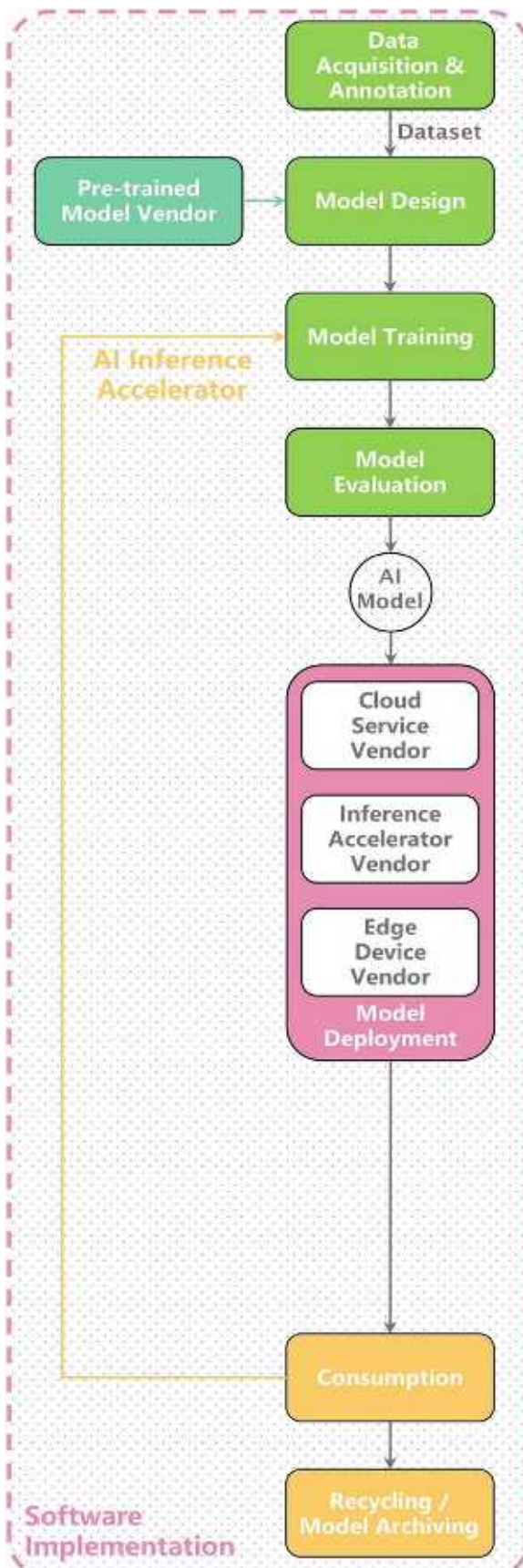


Fig. 7.1 AI system production process landscape

The “software implementation” pipeline, while seemingly more straightforward, is fraught with peril. The process begins with “data acquisition and annotation,” a stage critically vulnerable to data poisoning attacks [4]. An adversary could intentionally introduce mislabeled or malicious data into the training dataset. If undetected, this compromised dataset can lead to the training of a model with embedded biases or backdoors. For instance, a model trained on poisoned data might misclassify specific inputs, leading to denial-of-service or targeted attacks once deployed. The reliance on a “pretrained model vendor” introduces another significant risk. These models, often sourced from third-party repositories, can harbor hidden backdoors or Trojan attacks [5]. An attacker could publish a seemingly high-performing pretrained model that contains a trigger; when the deployed model encounters a specific, unusual input, it executes a malicious payload. The subsequent stages, including “model training” and “model evaluation,” are also susceptible. The integrity of the training process itself can be compromised through attacks on the underlying software frameworks and libraries.

Following the model’s creation, the “model deployment” phase presents another set of critical vulnerabilities. When deploying to a “cloud service vendor,” the security of the cloud infrastructure itself becomes paramount. Misconfigurations, insecure APIs, or insider threats within the cloud provider can expose the AI model and its associated data [6]. The “inference accelerator vendor” for cloud services also adds a layer of abstraction and potential risk; vulnerabilities in their hardware or firmware could be exploited [7]. Similarly, deploying to an “edge device vendor” introduces risks associated with the physical security of the device and the integrity of its operating system and software stack. An attacker with physical access to an edge device could potentially extract the model, reverse-engineer it, or tamper with its functionality [8]. The “consumption” and “recycling/model archiving” phases are not exempt from security concerns. Insecure communication channels during consumption can lead to data interception, while improper model archiving or disposal can result in the leakage of sensitive intellectual property [9].

The “hardware accelerator implementation” pipeline, while offering performance advantages, introduces a more complex and opaque supply chain, significantly expanding the attack surface. This pathway inherits all

the vulnerabilities of the software implementation’s initial stages (“data acquisition” to “AI model”). However, the subsequent steps, beginning with “IP core licensing and delivery,” introduce profound hardware-level threats. Malicious actors could insert hardware Trojans into the licensed Intellectual Property (IP) cores. These Trojans can be incredibly difficult to detect through standard testing, remaining dormant until activated by a specific condition, at which point they could leak sensitive data, degrade performance, or cause a complete system failure [10].

The integration of these potentially compromised IP cores during the “SoC integration” phase, facilitated by “EDA tool providers,” presents another vector of attack. The electronic design automation tools themselves can be compromised, leading to the insertion of malicious logic during the chip design process without the knowledge of the designers [11]. The “fabrication” stage, often outsourced to external foundries, is a well-documented source of security risks, including chip counterfeiting, overproduction, and the physical insertion of malicious circuitry [12]. Even after a chip is fabricated, the “PCB assembly” and “functional testing” stages can be targeted. A compromised component could be introduced during assembly, or testing procedures might be manipulated to overlook certain hardware-level vulnerabilities [13]. By the time the hardware accelerator is deployed and reaches the “consumption” stage, it may contain deeply embedded threats that are virtually impossible to detect through software-based security measures alone, making the entire AI system susceptible to manipulation or failure [14]. The security of the AI system is, therefore, only as strong as the weakest link in this intricate production and deployment chain.

7.4 Security Analysis of AI Model Development Stage

During the AI model development stage, significant security threats arise, primarily categorized into data acquisition and annotation, model design and training, and model evaluation.

7.4.1 Data Acquisition and Annotation

One of the main threats during data acquisition and annotation is data poisoning, where adversaries manipulate training data to influence the model's predictions adversely. For instance, Shafahi et al. demonstrated clean-label poisoning attacks, which subtly alter data without changing labels, making detection difficult [15]. Chen et al. showed that attackers could inject manipulated images into datasets, drastically degrading the performance of facial recognition systems [16]. Additionally, Schwarzschild et al. presented recent methods of data poisoning in federated learning environments, showing increased sophistication in distributed settings [17].

Mitigation strategies for data poisoning include using robust training methods such as outlier detection, filtering suspicious data points, and anomaly detection techniques. For example, Steinhardt et al. proposed methods to systematically detect poisoned data by identifying statistical anomalies [18].

Another significant threat is biased annotation, either intentional or inadvertent, causing model bias. For example, Mor Geva et al. analyzed how annotation bias could severely affect the fairness and accuracy of object detection models [19].

To mitigate biased annotations, auditing and correcting annotated datasets using fairness-aware annotation guidelines and diverse annotator teams is recommended. Buolamwini and Gebru demonstrated the necessity of diverse datasets in reducing bias in facial recognition systems [20]. Mehrabi et al. recommended standardized bias assessment frameworks to systematically reduce bias in data annotation [21].

7.4.2 Model Design and Training

During model design and training, adversarial attacks and backdoor insertion pose significant threats. Goodfellow et al. identified adversarial examples as crafted inputs that mislead machine learning models despite minor perturbations invisible to human observers [22]. Additionally, Gu et al. demonstrated that attackers could insert backdoors into neural networks during training, triggering malicious behaviors when specific conditions are met [23].

Adversarial training, which incorporates adversarial examples during the training phase, is a popular mitigation technique. Madry et al. demonstrated that adversarial training significantly improves model robustness against adversarial examples [24]. To mitigate backdoor

insertion, Wang et al. proposed neural cleanse, a technique to detect and mitigate backdoors by reverse engineering potential triggers [25].

Furthermore, model theft or extraction attacks, where adversaries attempt to replicate proprietary AI models, have grown prominent. Tramer et al. highlighted techniques for model extraction via query attacks, showing the vulnerability of models to unauthorized duplication [26]. Krishna et al. explored model extraction attacks in cloud-based machine learning deployments [27].

Mitigation strategies for model theft include employing model watermarking and output perturbation. Uchida et al. introduced watermarking techniques to securely embed identification markers within models, allowing detection and legal protection against theft [28].

7.4.3 Model Evaluation

In the evaluation phase, threats such as evaluation metric manipulation and evasion attacks become critical. Carlini et al. illustrated how adversarial attacks could specifically target evaluation metrics, misleading assessments of model robustness [29].

Additionally, evasion attacks, wherein attackers manipulate inputs during evaluation to bypass security checks, are a significant concern. Papernot et al. provided evidence of evasion attacks capable of degrading the effectiveness of malware detection systems by subtly altering malware features to evade detection [30].

These threats underscore the critical need for robust security mechanisms at each stage of AI model development to mitigate potential risks effectively.

7.5 Security Analysis of Software Implementation Stage

The deployment stage is when a trained AI model is made available for use in a production environment. At the deployment stage, AI systems encounter numerous security threats that can compromise their functionality, integrity, and privacy. The security of this stage is paramount, as a compromised model can lead to erroneous outputs, service disruption, and data breaches, with potentially severe consequences. This stage

presents a unique set of security vulnerabilities, as the model is exposed to various inputs and potential attackers. The specific threats and viable solutions often depend on the deployment environment, which can broadly be categorized into cloud services, inference accelerators, and edge devices.

7.5.1 Deploying Models on Cloud Service

Deploying AI models on cloud platforms offers scalability and cost-effectiveness but also introduces a broad attack surface. The multi-tenant nature of the cloud and the reliance on third-party infrastructure create unique security challenges.

7.5.1.1 Threats

One of the most significant threats in a cloud environment is the model extraction attack. In this scenario, an attacker with query access to the deployed model can effectively steal the model's intellectual property by creating a functionally equivalent replica. For instance, a malicious user could repeatedly query a deployed machine learning-as-a-service (MLaaS) API to observe the input-output behavior and then use this information to train a substitute model. This was demonstrated by Tramèr et al., who successfully extracted models from commercial cloud ML platforms [26]. The economic and intellectual property losses from such an attack can be substantial.

Another prevalent threat is the evasion attack, where an adversary intentionally crafts inputs to deceive the deployed model into making incorrect predictions. These “adversarial examples” are often imperceptible to humans but can cause the model to misclassify data with high confidence. For example, a carefully perturbed image of a stop sign might be classified as a speed limit sign, with catastrophic implications for autonomous driving systems deployed in the cloud. Papernot et al. explored the practical aspects of crafting such attacks against models hosted on cloud platforms, highlighting their real-world viability [31].

7.5.1.2 Mitigation Strategies

To counter model extraction attacks, several defensive strategies have been proposed. One approach is watermarking, where a unique identifier is embedded into the model during training. This watermark can later be used to prove ownership if a suspect model is discovered. Adi et al. proposed a

method for watermarking deep neural networks that is resilient to various model modification attacks, providing a means of recourse in cases of model theft [32]. Another mitigation is to limit the information provided in the model's output. For instance, instead of providing detailed confidence scores for each class, the API could only return the top predicted label. This reduces the amount of information an attacker can glean from each query, making model extraction more difficult [26].

For mitigating evasion attacks, adversarial training has emerged as a robust defense. This technique involves augmenting the training dataset with adversarial examples, thereby teaching the model to be resilient to such perturbations. Madry et al. demonstrated that models trained against a strong adversary are more robust to a wide range of adversarial attacks [24]. Another effective technique is input sanitization, where potentially malicious inputs are preprocessed to remove adversarial perturbations before they are fed to the model. For example, an image could be slightly blurred or compressed to smooth out adversarial noise. Guo et al. showed that simple image transformations can effectively defend against a variety of adversarial attacks [33].

7.5.2 Deploying Models on Inference Accelerator

To meet the high computational demands of modern AI models, specialized hardware accelerators like GPUs and TPUs are often used for inference. While these accelerators boost performance, they also introduce a new layer of hardware-specific vulnerabilities, such as side-channel attacks and hardware Trojan insertion attacks.

7.5.2.1 Threats

Side-channel attacks pose a significant threat to inference accelerators. By observing physical characteristics such as power consumption, electromagnetic emissions, or memory access patterns, an attacker can infer sensitive information about the model, including its architecture and parameters. Adversaries can analyze an accelerator's side-channel emissions to reconstruct a proprietary neural network. For instance, Zhan et al. demonstrated how attackers could exploit electromagnetic side-channel information from NVIDIA and AMD GPUs to infer sensitive user activities, such as identifying visited webpages or inferring keystroke timings [34].

This allows for direct intellectual property theft without any software access.

Another concern involves hardware Trojan insertion, where malicious actors embed covert circuitry into accelerators to degrade performance or leak confidential data. While not exclusively focused on GPUs, the principles of hardware Trojan attacks on neural networks, as demonstrated by Clements et al., are directly applicable to GPU-accelerated deep learning systems [35]. Yang et al. demonstrated a realistic scenario in which such Trojans were used to induce undetected computational errors in hardware accelerators [36]. A related but distinct threat is targeted misclassification, or backdoors, where Trojans are designed to remain inactive until triggered by a specific condition—such as a small, invisible patch on an input image. Once triggered, the Trojan forces the accelerator to produce an incorrect output predetermined by the attacker. Clements et al. showed how a hardware Trojan could be inserted to create such a backdoor, causing specific misclassifications while the system otherwise appears to function normally [35].

7.5.2.2 *Mitigation Strategies*

Defending against side-channel attacks on inference accelerators often involves a combination of hardware and software techniques. At the hardware level, hardware isolation techniques can securely partition GPU resources and establish strong security boundaries between different users or applications. Secure memory partitioning and hardware random number generators also help reduce the attack surface [37]. Lungu et al. proposed a classification of side-channel attacks targeting the NVIDIA GeForce RTX 3090 GPU and outlined mitigation measures, including these hardware and software techniques, to effectively reduce power, timing, and electromagnetic side-channel leakage [37]. Constant-time execution, a software-based approach, ensures that the time taken for a computation is independent of the input data, thereby thwarting timing-based side-channel attacks. Maji et al. propose a neural network accelerator design based on threshold implementation, which incorporates power and electromagnetic side-channel countermeasures to enhance hardware security [38].

Mitigating hardware Trojan attacks in AI accelerators requires a multilayered defense that combines reactive detection with proactive, secure design principles. While initial security layers include pre-

deployment screening via side-channel analysis to find physical anomalies and runtime monitoring to catch real-time faults, the most robust strategies focus on making the hardware inherently resilient to tampering. This proactive approach is exemplified by recent work in tamper-resistant design, which makes the accelerator’s internal structure unpredictable to an adversary in the supply chain. For instance, by systematically applying techniques like hardware component shuffling and substitution during the design phase of a CNN accelerator, as demonstrated by Yu et al., the fixed, predictable layout that an attacker relies upon is fundamentally disrupted. This makes it extremely difficult to insert a functional and stealthy hardware Trojan that can reliably trigger a backdoor, thus providing a foundational layer of security against malicious modifications [12]. Additionally, GPUs enhance protection for models and data through hardware-assisted security features like memory encryption and secure boot [39].

7.5.3 Deploying Models on Edge Device

Deploying AI models on edge devices, such as smartphones, IoT sensors, and embedded systems, offers advantages in terms of low latency and data privacy. However, the resource-constrained nature of these devices and their physical accessibility make them susceptible to a different set of security threats.

7.5.3.1 Threats

When deployed in the real world, models on edge devices face a diverse set of potent threats that exploit their accessibility. A primary concern is the integrity of the model’s predictions, which can be undermined by adversarial attacks. These attacks use carefully crafted, subtle perturbations to an input to cause the model to make an incorrect prediction. For instance, Su et al. demonstrated a “one-pixel attack” where changing a single pixel in an image tricked advanced neural networks into misclassifying it, highlighting a simple yet effective method for bypassing AI-based visual recognition systems [40]. Beyond manipulating inputs, adversaries also seek to compromise the confidentiality of the training data through techniques like model inversion attacks. These attacks aim to reconstruct sensitive data from the original training set by repeatedly querying the model; Fredrikson et al. successfully reconstructed recognizable facial

images from a facial recognition model using only API access, thereby leaking private biometric data [41]. Similarly, membership inference attacks pose another significant privacy risk by allowing an adversary to determine if a specific data record was part of the model's training set. Shokri et al. showed that an attacker could train “shadow models” based on a target model's outputs to infer membership with high accuracy [42]. In addition to privacy breaches, the intellectual property of the model itself is at risk from model stealing attacks, where an attacker systematically queries a proprietary model to train a functional replica. Tramèr et al. demonstrated that even black-box models from major cloud providers could be stolen with a limited number of queries, allowing for the illicit replication of valuable intellectual property [26].

7.5.3.2 Mitigation Strategies

To counter the threats faced by edge-deployed models, researchers have developed several defense mechanisms. To protect model integrity against adversarial attacks, the most prominent defense is adversarial training. This technique makes a model more robust by augmenting its training dataset with adversarial examples, thereby teaching it to ignore malicious perturbations, a method shown by Madry et al. to be highly effective [24]. To address the privacy breaches from inversion and inference attacks, statistical methods are employed, with differential privacy being a leading approach. By adding carefully calibrated noise during training, it becomes computationally infeasible for an attacker to reconstruct training data or confirm an individual's participation, as demonstrated by Abadi et al. who developed a method to train deep learning models with these formal privacy guarantees [43]. Finally, to protect the model as intellectual property against theft, model watermarking has been proposed. This technique embeds a unique, hidden signature into a model that can be used to prove ownership. Adi et al. introduced a method to “backdoor” a model with a specific trigger—a set of inputs that produces a unique, predefined output—which serves as an undeniable watermark if the model is stolen [32].

In conclusion, the deployment phase of the AI life cycle is a critical juncture where the security of the model is put to the test. A thorough understanding of the unique threats posed by different deployment environments, coupled with the implementation of robust and multilayered

defense mechanisms, is essential for ensuring the safe and reliable operation of AI systems in the real world.

7.6 Security Analysis of Hardware Accelerator Implementation Stage

7.6.1 IP Core Licensing and Delivery

IP core vendors play a critical role in the semiconductor supply chain by providing reusable design blocks, such as processors, memory controllers, and interfaces, which are licensed by chip manufacturers to accelerate development and reduce costs [1]. Prominent examples include Arm cores, Synopsys IP, and Cadence libraries. Despite their importance, IP vendors face several risks and threats that can undermine the integrity and security of the supply chain [44].

Key risks include counterfeit or unauthorized IP, which may introduce reliability or security vulnerabilities, and Trojaned IP, where malicious modifications can result in data leakage, backdoor access, or operational malfunctions [10]. Additionally, IP theft or cloning through reverse engineering poses a significant threat, as does the obsolescence of vendor-supported IP, leaving designs susceptible to unaddressed vulnerabilities [45]. Mitigation strategies to address these challenges involve sourcing certified IP from trusted vendors, conducting static and dynamic analyses of IP prior to integration, leveraging hardware assurance tools for integrity verification, and employing hardware obfuscation techniques to protect against cloning and reverse engineering [1]. These practices collectively enhance the security and reliability of IP within the supply chain.

7.6.2 SoC Integration

SoC integration entails combining multiple IP blocks, often sourced from various vendors, into a single chip. Given its complexity, this process is frequently outsourced to design houses or third-party electronic design automation (EDA) tools, which play a critical role in chip design and verification. However, the integration process is not without risks and threats that can compromise the security and functionality of the final product [46].

Notable risks include integration errors, such as insecure interconnects or race conditions, which may introduce unintentional vulnerabilities [47]. The use of third-party EDA tools further raises concerns, as compromised toolchains could embed backdoors or Trojans during design [11]. Incorporating IP from untrusted vendors amplifies the likelihood of vulnerabilities, while improper design practices can expose side-channel vulnerabilities, allowing sensitive data leakage through power analysis or electromagnetic emissions [48]. Mitigation strategies to address these issues include applying zero-trust principles by validating and isolating IP components, employing formal verification and runtime monitoring to detect anomalies, and securing the EDA toolchain by using licensed and updated software while verifying tool outputs [49]. Additionally, designing SoCs with side-channel resistance, such as through noise injection and balanced data paths, helps enhance their resilience to data leakage [50].

7.6.3 Fabrication

In the production life cycle of an AI hardware accelerator, the fabrication stage represents one of the most critical and vulnerable points in the entire hardware supply chain. During this phase, the entity that designed the chip relinquishes direct physical control over its intellectual property, sending the final design file—typically in GDSII format—to a semiconductor foundry for mass production. This handoff to a potentially untrusted third party exposes the sensitive design to a range of sophisticated hardware-level attacks. A primary threat is the insertion of a hardware Trojan, which is a malicious and stealthy modification to the circuit design. An adversary within the foundry can introduce logic that remains dormant during standard testing but activates under specific trigger conditions, at which point it can leak secret data, degrade performance, or cause a catastrophic system failure [10]. Furthermore, IC counterfeiting and overproduction pose a severe economic and security threat. A malicious foundry can produce more chips than authorized (overproduction) or sell defective, out-of-spec chips as genuine articles on the gray market. This not only leads to significant revenue loss for the original design house but also risks introducing unreliable or insecure hardware into critical AI systems [51]. Finally, the design itself is vulnerable to theft through reverse engineering. The GDSII file provides a complete blueprint of the chip, but even without it, an adversary can acquire a finished chip and use advanced imaging and

de-layering techniques to reconstruct the full transistor-level layout, thereby stealing valuable and hard-won intellectual property [52].

To counter these profound threats at the fabrication level, several innovative “design-for-trust” countermeasures have been developed. One effective physical security technique is split manufacturing, which divides the fabrication process across two separate foundries. The front-end-of-line (FEOL) processes, which create the core transistors, are performed at a state-of-the-art but potentially untrusted facility. The wafers are then shipped to a trusted, albeit lower-tech, facility to add the final back-end-of-line (BEOL) metal layers that connect the transistors into a functional circuit. Without the secret BEOL design, the FEOL wafers are useless, thus obscuring the chip’s functionality and preventing both IP theft and Trojan insertion [53]. Another powerful cryptographic approach is logic locking, which transforms the original circuit by adding “key” inputs so that the chip will only function correctly if a secret digital key is provided. This renders any chips overproduced by the foundry useless, as the adversary does not possess the secret key required for activation [54]. To provide a robust and unique identity for each chip, physically unclonable functions (PUFs) are integrated into the design. A PUF leverages the random, uncontrollable variations inherent in the chip manufacturing process to generate a unique and unclonable device “fingerprint.” This fingerprint can be used as a root of trust for cryptographic operations or as a unique identifier to authenticate the device, making it impossible for even the original manufacturer to create a clone and thereby preventing counterfeit chips from being used in secure systems [55].

7.6.4 PCB Assembly

Printed circuit board (PCB) assembly is the stage where individual electronic components, including the newly fabricated AI accelerator SoC, are mounted onto a PCB to create a functional electronic system. This process involves sourcing numerous components from a global supply chain, soldering them to the board, and programming the necessary firmware. The distributed and complex nature of this stage exposes the final product to significant security risks, particularly from counterfeit components and supply chain compromises [56].

A primary threat at this stage is the infiltration of counterfeit electronic components. These can range from recycled and remarked components to

out-of-spec or cloned parts that do not meet quality and reliability standards [57]. Such components can introduce unpredictable behavior, degrade performance, or create exploitable vulnerabilities in the AI hardware. A more malicious threat is the introduction of modified components or circuitry at the board level. An attacker with access to the assembly process could substitute a legitimate component with a malicious one containing a hardware Trojan or discreetly add, remove, or alter components on the PCB to create a backdoor or a kill switch [58]. For instance, a compromised microcontroller on the board could be used to manipulate the AI accelerator's operations or exfiltrate data. The supply chain for PCB assembly is often opaque, making it difficult to verify the integrity of every component and process, a vulnerability that attackers can exploit to inject malicious hardware without the manufacturer's knowledge [13].

To counter these threats, a multilayered defense strategy is essential. Rigorous component authentication and screening is the first line of defense. Techniques such as X-ray inspection, decapsulation, and statistical analysis of physical and electrical characteristics can help identify counterfeit and tampered components before they are mounted on the board [59]. For securing the supply chain itself, blockchain technology has been proposed as a means to create a transparent and immutable ledger for tracking components from their origin to the assembly line, ensuring provenance and preventing unauthorized substitutions [60]. To protect against unauthorized modifications during and after assembly, physical inspection and imaging of the assembled PCB can be compared against a golden reference design to detect discrepancies [61]. Furthermore, techniques like conformal coating can be applied to PCBs, which makes physical tampering difficult without leaving obvious evidence [62]. Finally, employing secure programming protocols at trusted facilities ensures that the firmware and software loaded onto the hardware have not been tampered with and originate from a legitimate source [63].

7.6.5 Functional Testing

Functional testing is a critical quality assurance step where the fully assembled hardware accelerator is tested to verify that it operates according to its specifications. This process, often conducted by a third-party test facility, involves running a suite of test vectors and checking for correct

outputs. While designed to detect manufacturing defects, the testing stage can itself be a vector for attack or fail to detect sophisticated threats.

A fundamental threat is hardware Trojan evasion. Malicious circuits are often designed to be stealthy and are only activated by rare trigger conditions (e.g., a specific input sequence or an internal state) that are unlikely to be covered by standard functional test patterns [10]. Consequently, a trojaned accelerator can pass all functional tests yet contain a potent backdoor. Another threat involves the test infrastructure itself. A malicious test facility could leverage its access to inject faults, clone devices, or, more subtly, use high-precision probes to conduct side-channel attacks. By analyzing power consumption or electromagnetic emissions during testing, an adversary can extract sensitive on-chip assets like cryptographic keys or even elements of the proprietary AI model being executed [64]. The test process can also be exploited to insert faults that cause accelerated aging, degrading the accelerator’s performance and reliability over time in a difficult-to-attribute manner [65].

Mitigation strategies for this stage extend beyond simple pass/fail functional checks. To detect stealthy Trojans, side-channel analysis, often enhanced with machine learning algorithms to improve accuracy and reduce false positives, is used as a powerful detection method. By generating a power or EM signature from a trusted “golden” device, deviations in the signatures of devices under test can reveal the presence of extra, malicious circuitry not caught by logic tests [66]. Another approach is the use of Trojan-aware test pattern generation, which uses static analysis of the netlist to identify suspicious rare-net switching that could correspond to a Trojan trigger and then generates vectors specifically to activate it [67]. To counter test-based information leakage and tampering, runtime monitoring and built-in self-test (BIST) circuits are crucial. These on-chip monitors can continuously check for anomalous behavior (e.g., unusual power draws, invalid computations) during operation and can be designed to attest to the integrity of the device, providing a trusted execution environment that is secure even if the external test infrastructure is compromised [68].

7.7 Security Analysis of Post-Deployment Stage

The security posture of an artificial intelligence (AI) system is not solidified upon its creation; rather, it remains a critical concern throughout its entire

life cycle. While significant attention is often directed toward vulnerabilities in the data acquisition and model training phases, the post-deployment stages, namely, “consumption” and “recycling/model archiving,” present a distinct and equally perilous attack surface. During consumption, the model transitions from a static, internal asset to a dynamic, interactive service, exposing it to external probing. In the archiving and recycling phase, the model becomes a latent risk, a stored representation of intellectual property and potential vulnerabilities that can be stolen directly or propagated into future systems. Compromises during these final stages can lead to intellectual property theft, severe privacy breaches, systemic erosion of trust, and the propagation of deeply embedded flaws. This section provides a detailed analysis of the security threats endemic to these stages and discusses corresponding mitigation strategies, supported by recent literature.

7.7.1 Consumption Stage

The consumption stage, where a deployed model serves live predictions via an application programming interface (API) or on an edge device, is the primary interface between the AI system and its users, including potential adversaries. This interactive nature is its greatest strength and its most significant weakness. Unlike attacks on the training pipeline, which are often difficult to mount without internal access, attacks on the consumption endpoint can be performed by any user with query access. A primary threat in this stage is the model extraction attack, where adversaries effectively steal a proprietary model by repeatedly querying it and using the resulting input-output pairs to train a functionally equivalent substitute. This constitutes a direct theft of intellectual property, allowing attackers to bypass expensive API fees or reverse-engineer a competitor’s technology, with research demonstrating the feasibility of creating clones with over 99% fidelity to the original [26]. Furthermore, a profound privacy risk emerges in the form of membership inference attacks. These attacks aim to determine if a specific data record was part of the model’s training set by exploiting the subtle differences in how a model responds to “seen” versus “unseen” data. A successful attack in a healthcare context, for instance, could leak a patient’s sensitive medical status [69]. The most widely discussed threat at inference is the adversarial attack, where an adversary introduces carefully crafted, often human-imperceptible perturbations to an

input to cause a confident misclassification. The physical manifestations of these attacks are particularly alarming, such as placing small stickers on a “stop” sign to fool an autonomous vehicle’s perception system, highlighting a critical risk to safety-relevant applications [70].

In response to this multifaceted threat landscape, a range of defensive strategies has been developed to harden models at the point of inference. Differential privacy stands as the formal, mathematical countermeasure to membership inference, providing a guarantee of privacy by injecting calibrated noise either during model training or into the model’s outputs at inference time, making it computationally infeasible for an attacker to glean information about any single training record [71]. To deter model theft, model watermarking provides a method for proving ownership. This is often achieved by embedding a secret backdoor into the model, causing it to produce a specific, unique output in response to a secret set of inputs, a behavior that would be statistically impossible to replicate by chance and can be used as forensic evidence of theft [32]. To combat adversarial evasion, the most proven defense is adversarial training, a process that enhances model robustness by augmenting the training data with adversarial examples and explicitly teaching the model to classify them correctly [24]. This is often supplemented by complementary defenses such as input sanitization, which aims to filter out perturbations before the input reaches the model [72], and certified defenses, which provide mathematical guarantees of robustness against certain classes of attack [73]. Together, these strategies form a layered defense to protect the active, deployed model during its consumption phase.

7.7.2 Recycling/Model Archiving Stage

The final phase of the AI life cycle, involving the archiving of models for compliance and the recycling of models for new tasks, is characterized by latent risks. Here, the model is an inactive, stored digital asset, yet it represents a concentration of intellectual property and potential vulnerabilities. The threats in this stage are not interactive but are instead focused on asset theft and the propagation of hidden flaws through an insecure model supply chain. The most direct threat is unauthorized access and leakage, where the model files themselves are exfiltrated from storage due to misconfigured cloud services, compromised credentials, or insider threats. The stolen asset is not merely a file but a complete blueprint of a

research and development effort, including the model’s architecture, weights, and embedded knowledge, the loss of which can be devastating to a company’s competitive advantage [9]. A more subtle but equally pernicious threat is vulnerability injection via improper recycling. When an archived model is used as a foundation for a new project via transfer learning, it inherits the security posture of its predecessor. If the original model contained a hidden backdoor or hardware Trojan, instilled through data poisoning during its initial training, this vulnerability will be carried silently into the new system, ready to be activated by an adversary [74]. Beyond malicious backdoors, other flaws like inherited bias from the original training data can be propagated, creating significant ethical and legal risks in the new application domain [75].

Mitigating these latent threats requires a dual focus on robust asset security and the establishment of a trustworthy model supply chain. The foundational layer of defense is the application of strong, end-to-end cryptography for model files at rest, coupled with a zero-trust access control policy. This means that access to encrypted models and their corresponding decryption keys is never trusted by default and is governed by the principle of least privilege, with all actions subject to strict authentication and auditing [74]. Beyond securing the asset, ensuring its integrity before reuse is paramount. This requires a proactive process of model integrity verification and sanitization. Before an archived model is recycled, it should be subjected to model scanning using tools designed to detect hidden backdoors by reverse-engineering their potential triggers [76]. If a vulnerability is found, sanitization techniques such as fine-pruning or unlearning can be employed to attempt to remove the malicious behavior from the model [77]. To operationalize this security-conscious approach to the model life cycle, organizations should adopt rigorous governance practices, chief among them the use of Model Cards and an AI Bill of Materials (AI BOM) [78]. These documents provide essential, transparent information about a model’s provenance, training data, architecture, and known limitations, creating a chain of trust and enabling developers to make informed risk assessments before reusing an archived asset [79].

7.8 Conclusions

The journey of an AI system from conception to consumption is paved with a diverse and evolving landscape of security threats. This chapter has illuminated the critical vulnerabilities present at every stage of the production process, spanning both software and hardware implementation pathways. From the initial phase of data acquisition, where poisoning and bias can corrupt a model at its source, to the final stages of consumption and archiving, where intellectual property theft and latent vulnerabilities pose significant risks, no part of the life cycle is immune to attack.

The analysis underscores a fundamental principle: AI security is not a feature to be added post hoc but a continuous and holistic process. The software pipeline is vulnerable to threats like model extraction, evasion attacks, and privacy breaches through membership inference, which can be mitigated by defenses such as adversarial training, differential privacy, and model watermarking. Simultaneously, the hardware implementation pipeline introduces a more opaque and challenging set of threats, including hardware Trojans embedded in IP cores, compromised EDA tools, and supply chain attacks during fabrication and assembly. Countering these requires a different class of solutions, from trusted hardware verification and split manufacturing to physical inspection and tamper-resistant designs.

Ultimately, the security of an AI system is only as strong as its weakest link. A vulnerability introduced by an untrusted IP vendor, a subtle data poisoning attack, or an insecure cloud deployment can undermine the entire system, regardless of the strength of other defenses. Therefore, creating truly robust AI requires a multilayered, defense-in-depth strategy that combines technical solutions with rigorous governance, such as the adoption of model cards and secure supply chain management. As adversaries continue to develop more sophisticated attack methods, ongoing vigilance, research, and a security-first mindset are paramount to ensuring these powerful technologies are deployed safely, securely, and ethically.

References

1. Hu, W., Chang, C.H., Sengupta, A., Bhunia, S., Kastner, R., Li, H.: An overview of hardware security and trust: threats, countermeasures, and design tools. *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* **40**(6), 1010–1038 (2020)
[\[Crossref\]](#)
2. Hu, Y., Kuang, W., Qin, Z., Li, K., Zhang, J., Gao, Y., Li, W., Li, K.: Artificial intelligence security: threats and countermeasures. *ACM Comput. Surv.* **55**(1), 1–36 (2021)
[\[Crossref\]](#)

3. Oseni, A., Moustafa, N., Janicke, H., Liu, P., Tari, Z., Vasilakos, A.: Security and privacy for artificial intelligence: Opportunities and challenges. arXiv preprint arXiv:2102.04661 (2021)
4. Yerlikaya, F.A., Bahtiyar, Ş.: Data poisoning attacks against machine learning algorithms. *Expert Syst. Appl.* **208**, 118101 (2022)
[\[Crossref\]](#)
5. Guo, W., Tondi, B., Barni, M.: An overview of backdoor attacks against deep neural networks and possible defences. *IEEE Open J. Signal Process.* **3**, 261–287 (2022)
[\[Crossref\]](#)
6. Luqman, A., Mahesh, R., Chattopadhyay, A.: Privacy and security implications of cloud-based ai services: A survey. arXiv preprint arXiv:2402.00896 (2024)
7. Gupta, N., Jati, A., Chattopadhyay, A.: AI attacks AI: Recovering neural network architecture from NVDLA using AI-assisted side channel attack. *ACM Trans. Embed. Comput. Syst.* **24**(5), 1–29 (2025)
8. Waguie, F.T., Al-Turjman, F.: Artificial intelligence for edge computing security: A survey. In: 2022 International Conference on Artificial Intelligence in Everything (AIE), pp. 446–450. IEEE (2022)
9. Xia, B., Lu, Q., Zhu, L., Xing, Z.: An AI system evaluation framework for advancing AI safety: Terminology, taxonomy, lifecycle mapping. In: Proceedings of the 1st ACM International Conference on AI-Powered Software, pp. 74–78 (2024)
10. Xue, M., Gu, C., Liu, W., Yu, S., O’Neill, M.: Ten years of hardware trojans: a survey from the attacker’s perspective. *IET Comput. Digit. Tech.* **14**(6), 231–246 (2020)
[\[Crossref\]](#)
11. Qiu, Y., Li, H., Wang, T., Liu, B., Gao, Y., Li, X.: Property coverage analysis based trustworthiness verification for potential threats from eda tools. In: 2016 IEEE 25th Asian Test Symposium (ATS), pp. 43–48. IEEE (2016)
12. Yu, H., Sun, P., Halak, B., Shanthakumar, K., Kazmierski, T.: Tamper resistant design of convolutional neural network hardware accelerator. In: 2023 Asian Hardware Oriented Security and Trust Symposium (AsianHOST), pp. 1–5 (2023). <https://doi.org/10.1109/AsianHOST59942.2023.10409319>
13. Harrison, J., Asadizanjani, N., Tehranipoor, M.: On malicious implants in PCBs throughout the supply chain. *Integration* **79**, 12–22 (2021)
[\[Crossref\]](#)
14. Mittal, S., Gupta, H., Srivastava, S.: A survey on hardware security of DNN models and accelerators. *J. Syst. Archit.* **117**, 102163 (2021)
[\[Crossref\]](#)
15. Shafahi, A., Huang, W.R., Najibi, M., Suciu, O., Studer, C., Dumitras, T., Goldstein, T.: Poison frogs! targeted clean-label poisoning attacks on neural networks. In: Advances in Neural Information Processing Systems, 31 (2018)

16. Chen, X., Liu, C., Li, B., Lu, K., Song, D.: Targeted backdoor attacks on deep learning systems using data poisoning. arXiv preprint arXiv:1712.05526 (2017)
17. Schwarzschild, A., Goldblum, M., Gupta, A., Dickerson, J.P., Goldstein, T.: Just how toxic is data poisoning? A unified benchmark for backdoor and data poisoning attacks. In: International Conference on Machine Learning, pp. 9389–9398. PMLR (2021)
18. Steinhardt, J., Koh, P.W.W., Liang, P.S.: Certified defenses for data poisoning attacks. In: Advances in Neural Information Processing Systems, 30 (2017)
19. Geva, M., Goldberg, Y., Berant, J.: Are we modeling the task or the annotator? An investigation of annotator bias in natural language understanding datasets. arXiv preprint arXiv:1908.07898 (2019)
20. Buolamwini, J., Gebru, T.: Gender shades: Intersectional accuracy disparities in commercial gender classification. In: Conference on Fairness, Accountability and Transparency, pp. 77–91. PMLR (2018)
21. Mehrabi, N., Morstatter, F., Saxena, N., Lerman, K., Galstyan, A.: A survey on bias and fairness in machine learning. ACM Comput. Surv. **54**(6), 1–35 (2021) [[Crossref](#)]
22. Goodfellow, I.J., Shlens, J., Szegedy, C.: Explaining and harnessing adversarial examples. arXiv preprint arXiv:1412.6572 (2014)
23. Gu, T., Dolan-Gavitt, B., Garg, S.: Badnets: Identifying vulnerabilities in the machine learning model supply chain. arXiv preprint arXiv:1708.06733 (2017)
24. Madry, A., Makelov, A., Schmidt, L., Tsipras, D., Vladu, A.: Towards deep learning models resistant to adversarial attacks. arXiv preprint arXiv:1706.06083 (2017)
25. Wang, B., Yao, Y., Shan, S., Li, H., Viswanath, B., Zheng, H., Zhao, B.Y.: Neural cleanse: Identifying and mitigating backdoor attacks in neural networks. In: 2019 IEEE Symposium on Security and Privacy (SP), pp. 707–723. IEEE (2019)
26. Tramèr, F., Zhang, F., Juels, A., Reiter, M.K., Ristenpart, T.: Stealing machine learning models via prediction {APIs}. In: 25th USENIX Security Symposium (USENIX Security 16), pp. 601–618 (2016)
27. Krishna, K., Tomar, G.S., Parikh, A.P., Papernot, N., Iyyer, M.: Thieves on sesame street! Model extraction attacks in cloud-based ml systems. In: International Conference on Learning Representations 2020 (2020)
28. Uchida, Y., Nagai, Y., Sakazawa, S., Satoh, S.: Embedding watermarks into deep neural networks. In: Proceedings of the 2017 ACM on International Conference on Multimedia Retrieval, pp. 269–277 (2017)
29. Carlini, N., Wagner, D.: Towards evaluating the robustness of neural networks. In: 2017 IEEE Symposium on Security and Privacy (SP), pp. 39–57. IEEE (2017)

30. Papernot, N., McDaniel, P., Goodfellow, I.: Transferability in machine learning: From phenomena to black-box attacks using adversarial samples. arXiv preprint arXiv:1605.07277 (2016)
31. Papernot, N., McDaniel, P., Goodfellow, I., Jha, S., Celik, Z.B., Swami, A.: Practical black-box attacks against machine learning. In: Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security, pp. 506–519 (2017)
32. Adi, Y., Baum, C., Cisse, M., Pinkas, B., Keshet, J.: Turning your weakness into a strength: Watermarking deep neural networks by backdooring. In: 27th USENIX Security Symposium (USENIX Security 18), pp. 1615–1631 (2018)
33. Guo, C., Rana, M., Cisse, M., Van Der Maaten, L.: Countering adversarial images using input transformations. arXiv preprint arXiv:1711.00117 (2017)
34. Zhan, Z., Zhang, Z., Liang, S., Yao, F., Koutsoukos, X.: Graphics peeping unit: Exploiting em side-channel information of gpus to eavesdrop on your neighbors. In: 2022 IEEE Symposium on Security and Privacy (SP), pp. 1440–1457. IEEE (2022)
35. Clements, J., Lao, Y.: Hardware trojan design on neural networks. In: 2019 IEEE International Symposium on Circuits and Systems (ISCAS), pp. 1–5. IEEE (2019)
36. Yang, C., Hou, J., Wu, M., Mei, K., Geng, L.: Hardware trojan attacks on the reconfigurable interconnections of convolutional neural networks accelerators. In: 2020 IEEE 15th International Conference on Solid-State & Integrated Circuit Technology (ICSICT), pp. 1–3. IEEE (2020)
37. Lungu, N., Patra, S.S., Mishra, M.R., Dash, B.B., Sasmal, G.C., Pattanayak, P., Singh, S., Gourisaria, M.K.: GPU side-channel attack classification for targeted secure shader mitigation. SN Comput. Sci. 5(8), 1–23 (2024)
[\[Crossref\]](#)
38. Maji, S., Banerjee, U., Fuller, S.H., Chandrakasan, A.P.: A threshold implementation-based neural network accelerator with power and electromagnetic side-channel countermeasures. IEEE J. Solid-State Circuits 58(1), 141–154 (2022)
[\[Crossref\]](#)
39. Na, S., Lee, S., Kim, Y., Park, J., Huh, J.: Common counters: Compressed encryption counters for secure GPU memory. In: 2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA), pp. 1–13. IEEE (2021)
40. Su, J., Vargas, D.V., Sakurai, K.: One pixel attack for fooling deep neural networks. IEEE Trans. Evol. Comput. 23(5), 828–841 (2019)
[\[Crossref\]](#)
41. Fredrikson, M., Jha, S., Ristenpart, T.: Model inversion attacks that exploit confidence information and basic countermeasures. In: Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security, pp. 1322–1333 (2015)
42. Shokri, R., Stronati, M., Song, C., Shmatikov, V.: Membership inference attacks against machine learning models. In: 2017 IEEE Symposium on Security and Privacy (SP), pp. 3–18. IEEE (2017)

43. Abadi, M., Chu, A., Goodfellow, I., McMahan, H.B., Mironov, I., Talwar, K., Zhang, L.: Deep learning with differential privacy. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, pp. 308–318 (2016)
44. Halak, B.: Cist: A threat modelling approach for hardware supply chain security. In: Hardware Supply Chain Security, pp. 3–65. Springer, Berlin (2021)
45. Tauhid, A., Xu, L., Rahman, M., Tomai, E.: A survey on security analysis of machine learning-oriented hardware and software intellectual property. *High-Confidence Comput.* **3**(2), 100114 (2023)
[\[Crossref\]](#)
46. Basak, A., Bhunia, S., Tkacik, T., Ray, S.: Security assurance for system-on-chip designs with untrusted ips. *IEEE Trans. Inf. Forensics Secur.* **12**(7), 1515–1528 (2017)
[\[Crossref\]](#)
47. Kibria, R., Farahmandi, F., Tehranipoor, M.: A survey on soc security verification methods at the pre-silicon stage. *Cryptology ePrint Archive* (2024)
48. Mishra, J., Sahay, S.K.: Modern hardware security: A review of attacks and countermeasures. *arXiv preprint arXiv:2501.04394* (2025)
49. Tan, B., Elnaggar, R., Fung, J.M., Karri, R., Chakrabarty, K.: Toward hardware-based ip vulnerability detection and post-deployment patching in systems-on-chip. *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* **40**(6), 1158–1171 (2020)
[\[Crossref\]](#)
50. Das, D., Maity, S., Nasir, S.B., Ghosh, S., Raychowdhury, A., Sen, S.: ASNI: attenuated signature noise injection for low-overhead power side-channel attack immunity. *IEEE Trans. Circuits Syst. I Regular Papers* **65**(10), 3300–3311 (2018)
[\[Crossref\]](#)
51. Guin, U., Shi, Q., Forte, D., Tehranipoor, M.M.: Fortis: a comprehensive solution for establishing forward trust for protecting ips and ics. *ACM Trans. Des. Autom. Electr. Syst.* **21**(4), 1–20 (2016)
[\[Crossref\]](#)
52. Quadir, S.E., Chen, J., Forte, D., Asadizanjani, N., Shahbazmohamadi, S., Wang, L., Chandy, J., Tehranipoor, M.: A survey on chip to system reverse engineering. *ACM J. Emerg. Technol. Comput. Syst.* **13**(1), 1–34 (2016)
[\[Crossref\]](#)
53. Hammi, B., Zeadally, S., Nebhen, J.: Security threats, countermeasures, and challenges of digital supply chains. *ACM Comput. Surv.* **55**(14s), 1–40 (2023)
[\[Crossref\]](#)
54. Xie, Y., Srivastava, A.: Delay locking: Security enhancement of logic locking against ic counterfeiting and overproduction. In: Proceedings of the 54th Annual Design Automation Conference 2017, pp. 1–6 (2017)

55. Shamsoshoara, A., Korenda, A., Afghah, F., Zeadally, S.: A survey on physical unclonable function (PUF)-based security solutions for internet of things. *Comput. Netw.* **183**, 107593 (2020)
[\[Crossref\]](#)
56. Tehranipoor, M., Pundir, N., Vashistha, N., Farahmandi, F.: Counterfeit and recycled ic detection. In: *Hardware Security Primitives*, pp. 281–300. Springer, Berlin (2022)
57. Guin, U., Huang, K., DiMase, D., Carulli, J.M., Tehranipoor, M., Makris, Y.: Counterfeit integrated circuits: a rising threat in the global semiconductor supply chain. *Proc. IEEE* **102**(8), 1207–1228 (2014)
[\[Crossref\]](#)
58. McGuire, M., Ogras, U., Ozev, S.: PCB hardware trojans: Attack modes and detection strategies. In: *2019 IEEE 37th VLSI Test Symposium (VTS)*, pp. 1–6. IEEE (2019)
59. Roy, A., Al Hasan, M.M., Ghosh, S., Varshney, N., Julia, J., Forghani, R., Asadizanjani, N.: Applications and challenges of ai in PCB x-ray inspection: a comprehensive study. *ACM J. Emerg. Technol. Comput. Syst.* **21**(1), 1–28 (2025)
[\[Crossref\]](#)
60. Hossain, M.M., Vashistha, N., Allen, J., Allen, M., Farahmandi, F., Rahman, F., Tehranipoor, M.: Thwarting counterfeit electronics by blockchain. *IEEE Blockchain* (2022)
61. Kaji, S., Fujimoto, D., Hayashi, Y.: Simulation-based approach to generating golden data for PCB-level hardware trojan detection using capacitive sensor. In: *2023 IEEE Physical Assurance and Inspection of Electronics (PAINE)*, pp. 1–7. IEEE (2023)
62. Shen, H., Rahman, M.T., Asadizanjani, N., Tehranipoor, M., Bhunia, S.: Coating-based PCB protection against tampering, snooping, em attack, and x-ray reverse engineering. In: *International Symposium for Testing and Failure Analysis*, vol. 81009, pp. 290–294. ASM International (2018)
63. Marchand, A., Imine, Y., Ouarnoughi, H., Tarridec, T., Gallais, A.: Firmware integrity protection: a survey. *IEEE Access* **11**, 77952–77979 (2023)
[\[Crossref\]](#)
64. Meschkov, S., Gnad, D.R., Krautter, J., Tahoori, M.B.: New approaches of side-channel attacks based on chip testing methods. *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* **42**(5), 1411–1424 (2022)
[\[Crossref\]](#)
65. Karimi, N., Kanuparthi, A.K., Wang, X., Sinanoglu, O., Karri, R.: Magic: malicious aging in circuits/cores. *ACM Trans. Archit. Code Optim.* **12**(1), 1–25 (2015)
[\[Crossref\]](#)
66. Köylü, T.Ç., Wedig Reinbrecht, C.R., Gebregiorgis, A., Hamdioui, S., Taouil, M.: A survey on machine learning in hardware security. *ACM J. Emerg. Technol. Comput. Syst.* **19**(2), 1–37 (2023)
[\[Crossref\]](#)

67. Bhunia, S., Hsiao, M.S., Banga, M., Narasimhan, S.: Hardware trojan attacks: threat analysis and countermeasures. *Proc. IEEE* **102**(8), 1229–1247 (2014)
[Crossref]
68. Bellec, N., Rokicki, S., Puaut, I.: Attack detection through monitoring of timing deviations in embedded real-time systems. In: *ECRTS 2020–32nd Euromicro Conference on Real-Time Systems*, pp. 1–22 (2020)
69. Liu, L., Wang, Y., Liu, G., Peng, K., Wang, C.: Membership inference attacks against machine learning models via prediction sensitivity. *IEEE Trans. Dependable Secure Comput.* **20**(3), 2341–2347 (2022)
70. Gu, T., Liu, K., Dolan-Gavitt, B., Garg, S.: Badnets: Evaluating backdooring attacks on deep neural networks. *IEEE Access* **7**, 47230–47244 (2019)
[Crossref]
71. Ziller, A., Usynin, D., Braren, R., Makowski, M., Rueckert, D., Kaissis, G.: Medical imaging deep learning with differential privacy. *Sci. Rep.* **11**(1), 13524 (2021)
[Crossref]
72. Xu, W., Evans, D., Qi, Y.: Feature squeezing: Detecting adversarial examples in deep neural networks. *arXiv preprint arXiv:1704.01155* (2017)
73. Cohen, J., Rosenfeld, E., Kolter, Z.: Certified adversarial robustness via randomized smoothing. In: *International Conference on Machine Learning*, pp. 1310–1320. PMLR (2019)
74. Shahriar, S., Allana, S., Hazratifard, S.M., Dara, R.: A survey of privacy risks and mitigation strategies in the artificial intelligence life cycle. *IEEE Access* **11**, 61829–61854 (2023)
[Crossref]
75. Zhang, D.: Protecting privacy and copyright throughout the data lifecycle of AI systems. Ph.D. Thesis, The Australian National University (Australia) (2024)
76. Botero, U.J., Wilson, R., Lu, H., Rahman, M.T., Mallaiyan, M.A., Ganji, F., Asadizanjani, N., Tehranipoor, M.M., Woodard, D.L., Forte, D.: *Hardware Trust and Assurance Through Reverse Engineering*. Association for Computing Machinery, New York, NY (2020)
77. Liu, K., Dolan-Gavitt, B., Garg, S.: Fine-pruning: Defending against backdooring attacks on deep neural networks. In: *International Symposium on Research in Attacks, Intrusions, and Defenses*, pp. 273–294. Springer (2018)
78. Santos, O., Radanliev, P.: Toward trustworthy AI: An analysis of artificial intelligence (AI) bill of materials (AI BOMs) (2023). <https://doi.org/10.13140/RG.2.2.18893.61929>
79. Xia, B., Zhang, D., Liu, Y., Lu, Q., Xing, Z., Zhu, L.: Trust in software supply chains: Blockchain-enabled SBOM and the AIBOM future. In: *Proceedings of the 2024 ACM/IEEE 4th International Workshop on Engineering and Cybersecurity of Critical Systems (EnCyCriS) and 2024 IEEE/ACM Second International Workshop on Software Vulnerability*, pp. 12–19 (2024)

8. A Case Study on Shuffle and Substitution-Based Defense Mechanism

Haosen Yu¹✉, Peiyao Sun¹✉ and Basel Halak¹✉

(1) School of Electronics and Computer Science, University of Southampton, Southampton, UK

✉ Haosen Yu

Email: Haosen.Yu@soton.ac.uk

✉ Peiyao Sun

Email: ps2m24@soton.ac.uk

✉ Basel Halak (Corresponding author)

Email: Basel.Halak@soton.ac.uk

Keywords Hardware Trojans – CNN accelerators – Shuffle mechanisms – Substitution mechanisms – Defense strategies – Triggering methods – Hardware security – Trojan detection – CNN security – Pixel-level shuffle – Neuron-level shuffle – Hardware implementation – Experimental evaluation – Backdoor attacks – Defense performance

8.1 Introduction

In hardware security research, numerous HT attacks and their corresponding countermeasures have been delineated. However, a significant portion of these countermeasures are tailored to specific segments of the supply chain or particular attack modalities. This specialization has engendered an important challenge: the absence of a

holistic countermeasure capable of concurrently defending against diverse Trojan attacks, thereby maximizing hardware protection [1].

In response to this challenge, an innovative countermeasure has been proposed in our previous work [2]. This approach preprocesses input image data using shuffling and substitution techniques and reorders the sequence of output neurons at the hardware level. Termed the “shuffle and substitution-based defense mechanism (SSDM),” this dual-layered countermeasure is meticulously architected to counter threats emanating from both the third-party dataset provisioning phase and the fabrication phase. Consequently, it offers robust resistance against a wide range of CNN HT insertion attacks by adversaries and backdoor attacks by third-party dataset providers.

This chapter presents the design, principles, and implementation of the shuffle and substitution-based defense mechanism (SSDM), attack experiments are also set up to verify the defense effectiveness of SSDM, and the performance is discussed.

8.2 Chapter Overview

This chapter introduces the shuffle and substitution-based defense mechanism (SSDM), a novel countermeasure designed to enhance the security of convolutional neural network (CNN) accelerators against sophisticated hardware Trojan (HT) attacks and backdoor vulnerabilities. Section [8.3](#) begins by thoroughly examining the threat landscape posed by HT attacks in CNN accelerators, detailing various attack vectors and classification methods based on their triggering mechanisms. Building upon this understanding, Sect. [8.4](#) delineates the comprehensive design of the SSDM, elucidating its core components: pixel-level shuffling, neuron-level shuffling, and the substitution mechanism. The subsequent Sect. [8.5](#) elaborates on the practical hardware implementation aspects of the SSDM. Finally, Sects. [8.6](#) and [8.7](#) present the experimental configuration and an exhaustive evaluation of the SSDM's performance against diverse HT insertion attacks and patch-based backdoor attacks, demonstrating its efficacy in improving the robustness and security of CNN accelerators, followed by Sect. [8.8](#) concluding remarks that summarize the key findings and emphasize the importance of integrated security measures in CNN accelerators.

8.3 Threat Modelling for HT Attacks in CNN Accelerators

8.3.1 Hardware Trojan Security Threats in CNN Accelerators

CNN accelerators are specialized hardware designed to enhance the computational efficiency of convolutional neural networks (CNNs). Given the critical applications of CNNs, such as autonomous driving, medical imaging, and defense, ensuring the security of these accelerators is paramount.

In recent years, various hardware Trojans (HTs) have been deliberately engineered to exploit vulnerabilities in CNN accelerators [[3–10](#)]. These attacks primarily target critical components, including processing units (PEs) [[3, 4](#)], memory subsystems [[5, 6](#)], prediction outputs [[7](#)], bias buffers [[8](#)], activation parameters [[9](#)], and reconfigurable interconnection networks [[10](#)].

The first study to explore hardware Trojan threats in CNN-based image classification involved embedding a Trojan into the Multiplier and Adder Tree (MAT) module of an FPGA accelerator [3]. Experimental results revealed precise manipulation of CNN classification outcomes with minimal system overhead and modifications. Another approach introduced a Trojan into the rectified linear unit (ReLU) function block, successfully achieving adversarial objectives [4]. This study highlighted both the stealth and effectiveness of such hardware Trojans. Building on these methods, a memory Trojan was designed to attack a neural network accelerator, departing from techniques requiring detailed knowledge of the model and toolchain. This Trojan executed an accuracy degradation attack by accessing data on the memory bus and was triggered by specific input images [5].

Further, a sequence-triggered hardware Trojan was developed to manipulate the prediction outcomes of neural network systems. Unlike pixel-based triggers, this Trojan relied on a sequence of input images, making it resistant to image preprocessing techniques and undetectable by human observers. Experiments using MNIST, CIFAR100, and ISLVRD datasets demonstrated the Trojan's ability to activate selectively with minimal hardware overhead [7].

Additionally, Int-Monitor, a hardware Trojan targeting DNN accelerators, was specifically designed to exploit the global bias buffer. By embedding an interrupt monitor between the host processor and the DNN accelerator, this Trojan disrupted neuron activation, invalidated forward propagation, and rendered the accelerator nonfunctional [8].

A hardware Trojan attacks targeting deep neural network (DNN) hardware accelerators implemented on FPGAs have been introduced [9]. These Trojans are designed to manipulate activation parameters within DNN accelerators. Experimental results demonstrate that these Trojans achieve high levels of stealth, making detection difficult, while significantly degrading DNN inference accuracy when activated.

To mitigate such threats, various countermeasures have been proposed, including hardware attestation, Trojan detection techniques, and secure chip manufacturing processes [1].

Hardware Trojans in CNNs can be broadly classified based on their activation mechanisms into always-on and trigger-required types. Always-on Trojans remain perpetually active, making them particularly stealthy due

to the absence of additional circuits. This lack of extra hardware makes them challenging to detect using bitstream reverse engineering or pre-silicon analysis methods [11]. However, their continuous activation necessitates careful design of malicious functionality; overt changes to outputs can make them detectable during testing.

In contrast, trigger-required Trojans activate only upon receiving specific low-probability signals [12, 13]. While this activation mechanism allows them to remain undetected during standard testing, their reliance on additional circuitry, such as a trigger recognition model (TRM) for detecting signals and processing payloads, introduces anomalies in the circuit network table. These discrepancies increase their susceptibility to detection through reverse engineering and pre-silicon analysis [11].

8.3.2 Classification of Triggering Methods for the HTs in CNN Accelerators

According to the different trigger methods of hardware Trojans in CNN accelerators, they can be categorized into four types: position-specific and value-specific triggered HTs [3, 9, 14], neuron-specific triggered HTs [15, 16], pattern-specific triggered HTs [3, 5, 6], and sequence-specific triggered HTs [7]. More details are introduced next (Fig. 8.1).



Fig. 8.1 Classification of triggering methods for the HTs in CNN accelerators

8.3.2.1 Position-Specific and Value-Specific Triggered HTs

This type of hardware Trojan (HT) operates by embedding malicious information at specific positions within the input image. When the CNN accelerator detects that the pixel values at these designated positions match

predefined values, the HT is triggered [3]. The working process of this attack is illustrated in Fig. 8.2.

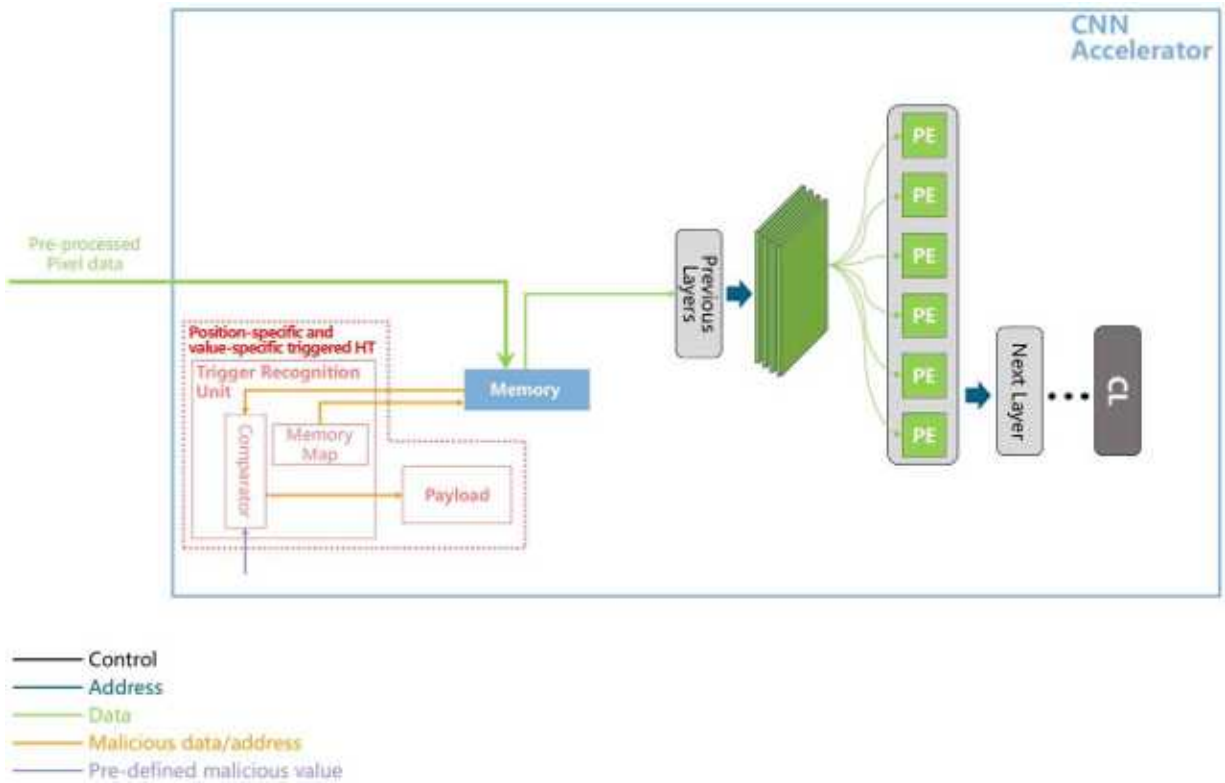


Fig. 8.2 Position-specific and value-specific triggered HTs

8.3.2.2 Neuron-Specific Triggered HTs

This type of hardware Trojan (HT) operates by monitoring the output value of a processing element (PE) in a specific layer of the CNN accelerator. The HT is triggered when the output value exceeds a threshold k , predefined by the adversary, and the cumulative number of exceedances surpasses a specified value m [16]. The working process of this attack is illustrated in Fig. 8.3.

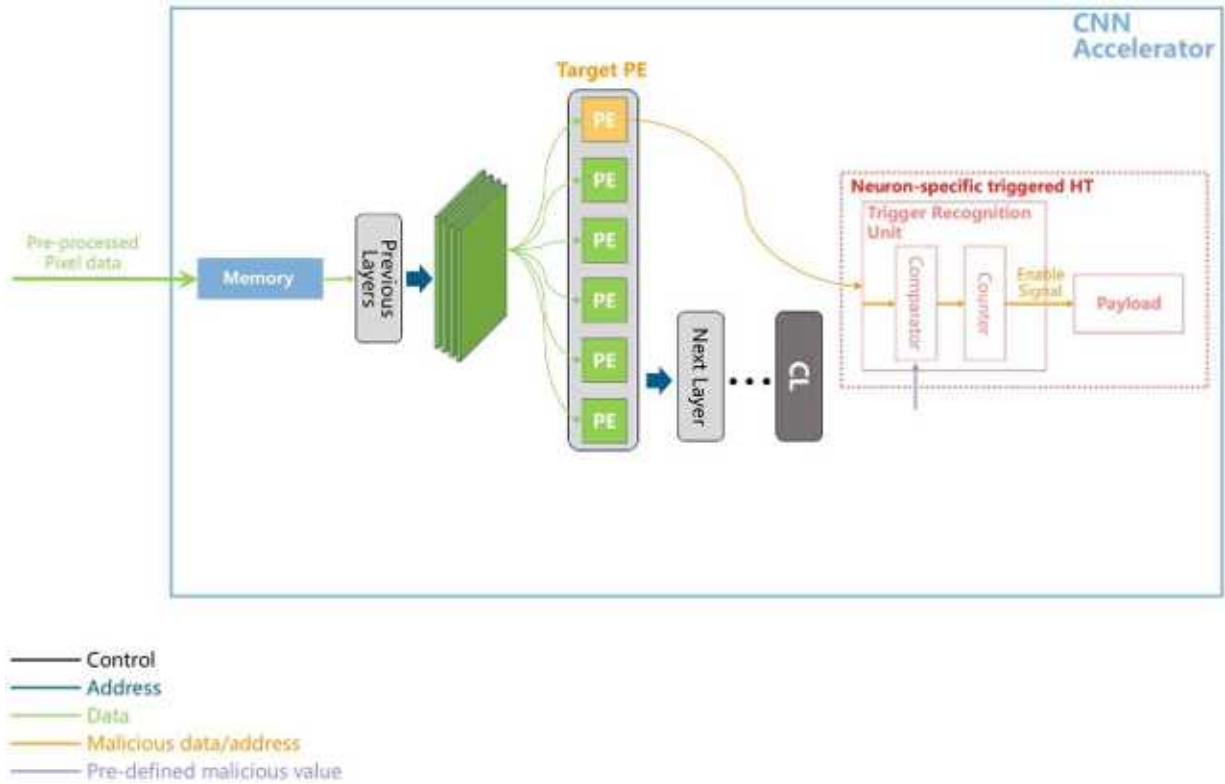


Fig. 8.3 Neuron-specific triggered HTs

8.3.2.3 Pattern-Specific Triggered HTs

This type of hardware Trojan (HT) employs a trigger recognition unit, typically implemented as a tiny malicious CNN. The HT is triggered when this miniature CNN detects a specific pattern, predefined by the adversary, within the designated area of the input image [6]. The working process of this attack is illustrated in Fig. 8.4.

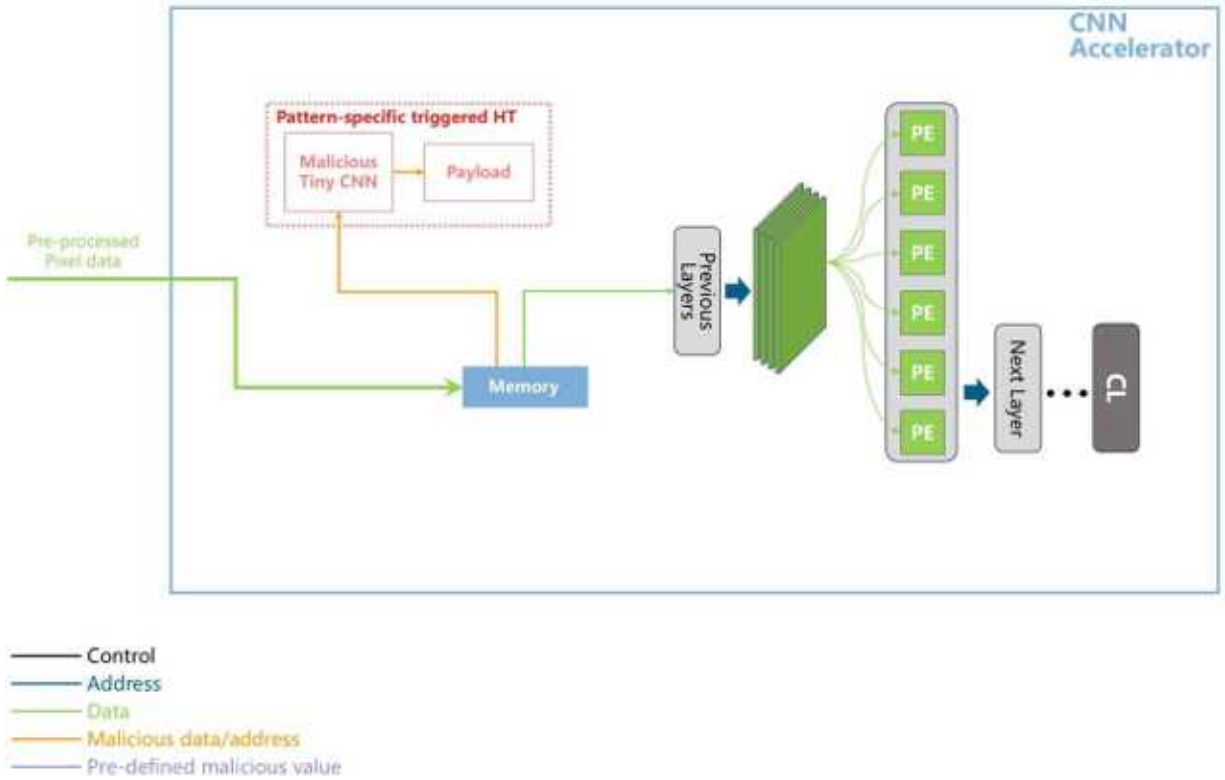


Fig. 8.4 Pattern-specific triggered HTs

8.3.2.4 Sequence-Specific Triggered HTs

This type of hardware Trojan (HT) is triggered by a specific sequence of input images fed into the CNN accelerator. The accelerator processes these images sequentially, outputting classification results as index values corresponding to the output neurons associated with each image's class. By analyzing the sequence of these index values, the HT is triggered if the observed sequence matches the adversary's predefined sequence [7]. The working process of this attack is illustrated in Fig. 8.5.

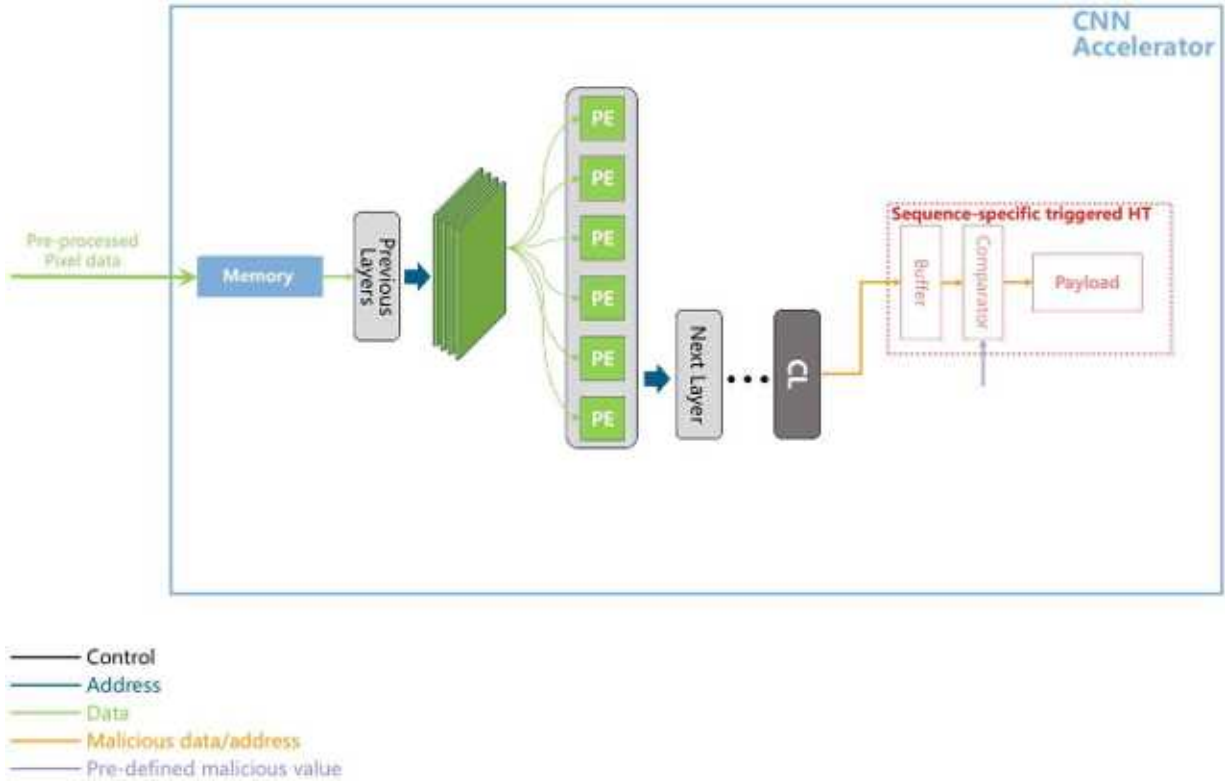


Fig. 8.5 Sequence-specific triggered HTs

8.4 Design of Defense Mechanism

Hardware Trojan (HT) insertion represents a significant hardware-level threat, predicated on an adversary's capacity to maliciously alter design files. Such attacks are primarily executed during the intellectual property (IP) vending, system-on-chip (SoC) integration, and fabrication stages of the integrated circuit (IC) supply chain [17]. It is posited that adversarial actors are situated within the fabrication stage, where they can introduce illicit modifications to the design files. These alterations are intended to embed malicious functionalities, thereby compromising the integrity of classification algorithms executed on the affected hardware.

In the domain of convolutional neural networks (CNNs), the procurement of training data from third-party providers is a prevalent practice. Although this approach serves to augment the available data pool, it concurrently introduces a significant risk of backdoor vulnerabilities. The sheer volume of contemporary datasets renders exhaustive image-by-image inspection infeasible, thus creating opportunities for the surreptitious embedding of such vulnerabilities [18, 19].

In light of these distinct but related threats, this research broadens the conventional threat model. The expanded framework, as depicted in Fig. 8.6, is augmented to encompass not only adversaries at the fabrication stage but also those operating as third-party image providers. This holistic approach allows for a more comprehensive analysis of the security landscape for CNNs, from the underlying hardware to the training data.

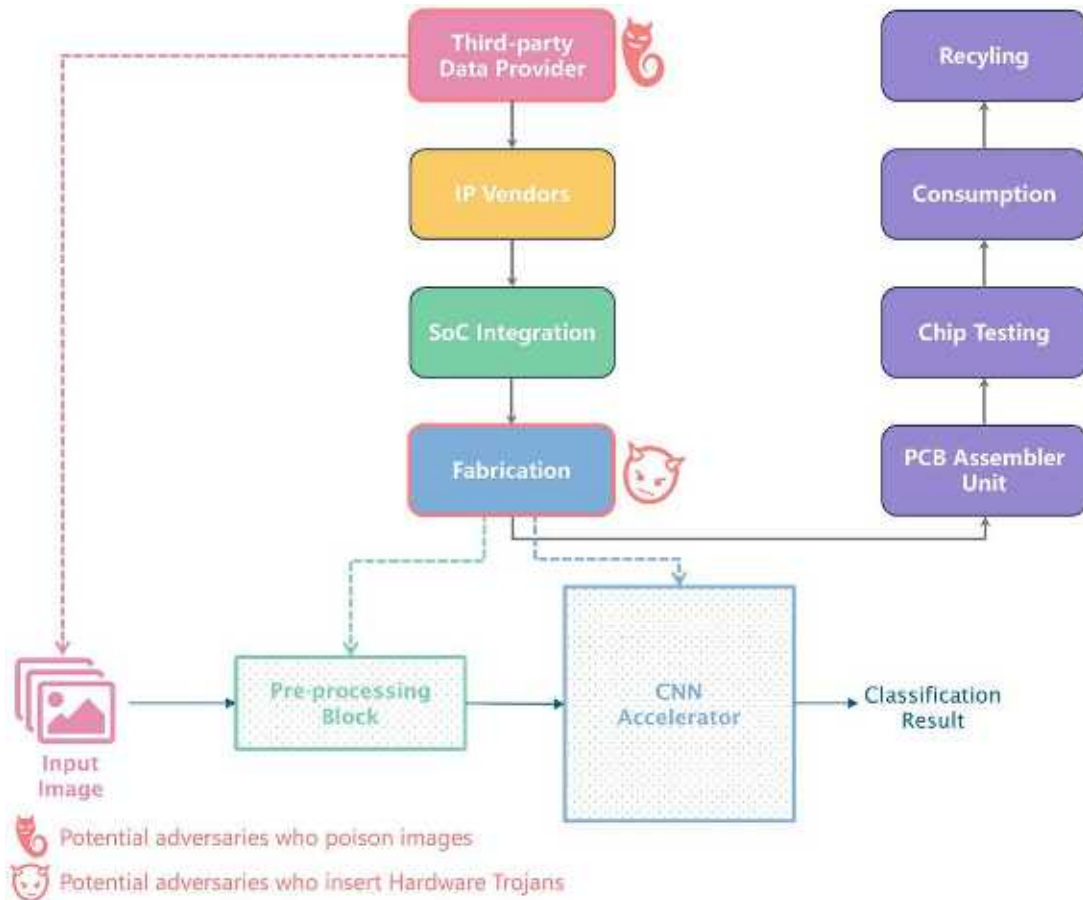


Fig. 8.6 Threat modelling

Contemporary CNN systems are vulnerable to HTs that activate by leveraging embedded activation cues within input images. These cues often correlate with the malicious data’s location, sequence, and value, as substantiated by a plethora of studies [3, 5–7, 9, 14–16]. In response to this vulnerability, the “shuffle-substitution defense mechanism” (SSDM) is developed [2]. SSDM is strategically formulated to curtail an adversary’s capability to manipulate malicious data, thereby impeding HT activation and neutralizing potential threats. The defense strategy involves shuffling

training images and labels to implant a backdoor during the CNN's training phase [19–22]. By perturbing the spatial attributes of these altered images, the SSDM mechanism aims to render the covertly introduced backdoors ineffectual.

8.4.1 Pixel-Level Shuffle Mechanism

The shuffle mechanism enhances the security of CNN accelerators against both hardware Trojans (HTs) and backdoor attacks by disrupting the spatial arrangement of input data. For HTs, this mechanism obfuscates the location of malicious information, preventing the precise trigger conditions required for activation, as adversaries at the fabrication stage cannot predict the runtime shuffle patterns. In the case of backdoor attacks, altering the shuffle order for noncritical regions of images disrupts the spatial coherence that backdoor mechanisms rely on. This disruption effectively mitigates the threat by preventing the CNN from detecting malicious patterns during the inference stage when the arrangement of training and testing data is inconsistent.

The pixel-level shuffle mechanism, inspired by approximate computing concepts in image processing [23], divides training and test images into important and unimportant regions, as illustrated in Fig. 8.9. This segmentation is consistent across all images within a dataset due to the similar data distribution within the dataset. Shuffling is applied with varying granularity: at the pixel level within unimportant regions and at the region level within important regions. Additionally, within unimportant regions, pixel-level shuffle strategies remain consistent within a single dataset but differ between datasets (i.e., between training and testing datasets). For important regions, the region-level shuffling sequences are identical across both training and test datasets.

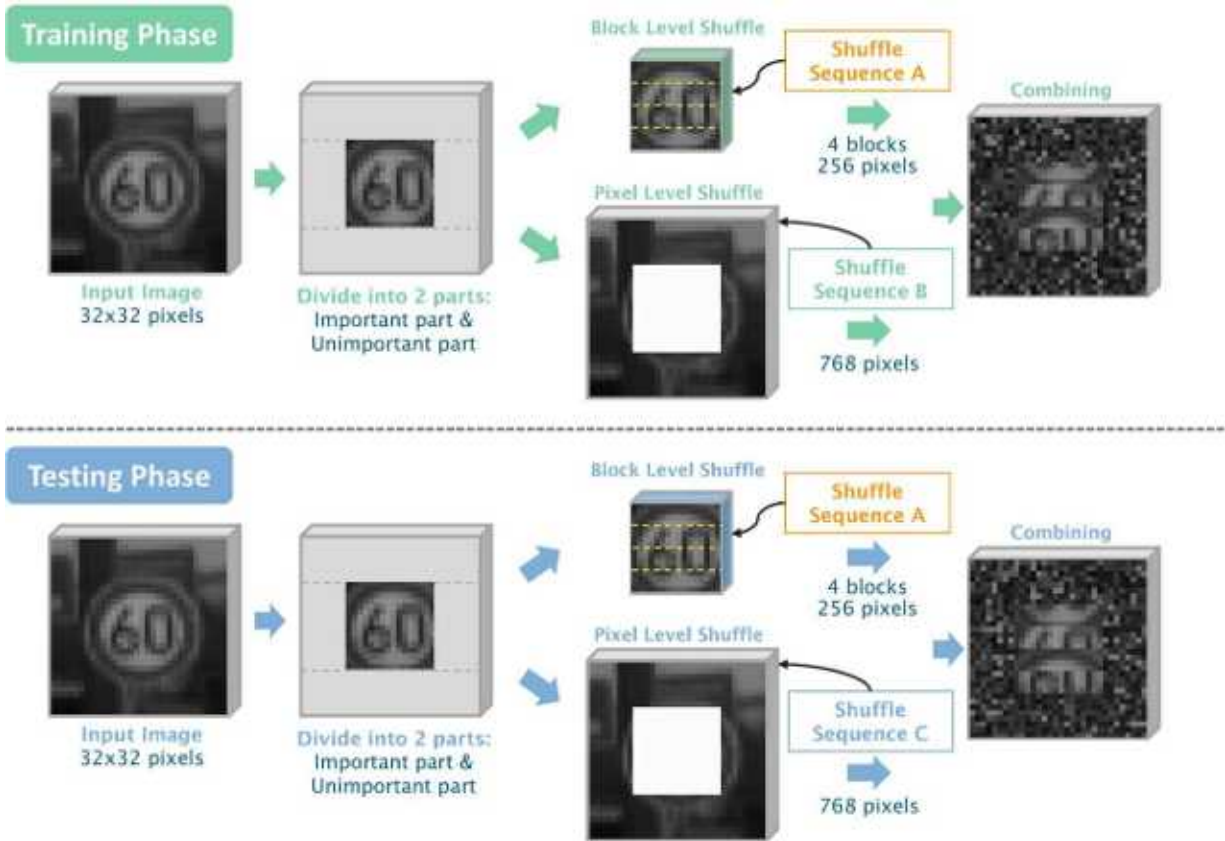


Fig. 8.9 Pixel-level shuffle mechanism

These strategies maintain the consistency of information distribution within important regions across training and test datasets, ensuring minimal degradation of classification accuracy. Simultaneously, the disorder introduced in unimportant regions effectively mitigates the potential impact of embedded malicious information. This approach enhances the model's resilience against adversarial attacks triggered by malicious information in the input images, thereby improving the overall security of CNN accelerators.

8.4.2 Neuron-Level Shuffle Mechanism

Neuron-level shuffling prevents the activation of neuron-specific triggered hardware Trojans (HTs). In CNN accelerators, classification results are represented as output neuron indices, which require a decoder for category mapping. By modifying the sequence of these output neurons at the hardware level, the accelerator's output is altered, thereby confusing adversaries who lack knowledge of the new order. This approach effectively

defends against HTs that rely on predetermined output sequences for activation.

To counter sequence-specific triggered HTs, it is essential to reorder the output neurons. In convolutional neural networks (CNNs), the sequence of output neurons can be altered by modifying the arrangement of weights in the final fully connected layer. For example, in the GTSRB dataset [24], which includes 43 output neurons corresponding to 43 unique sets of weights in the final fully connected layer, the computation within this layer can be expressed as shown in Eq. 8.1:

$$[X] \cdot \begin{matrix} W_1 \\ W_2 \\ \dots \\ W_{43} \end{matrix} = \begin{matrix} o_1 \\ o_2 \\ \dots \\ o_{43} \end{matrix} \quad (8.1)$$

In Eq. 8.1, X represents the input array of the final fully connected layer, W_i denotes an array of one weight set, and o_i refers to the value of a single output neuron. The equation illustrates that the sequence of the 43 weight sets directly corresponds to the sequence of the output neurons. By reordering these weights, as shown in Eq. 8.2, the sequence of the output neurons is similarly reordered. With the implementation of an appropriate decoding mechanism at the software level, it becomes feasible to modify the order of output neurons at the hardware level while still achieving accurate classification outcomes:

$$[X] \cdot \begin{matrix} W_2 \\ W_{43} \\ \dots \\ W_1 \end{matrix} = \begin{matrix} o_2 \\ o_{43} \\ \dots \\ o_1 \end{matrix} \quad (8.2)$$

8.4.3 Substitution Mechanism

The substitution mechanism effectively prevents the activation of hardware Trojans (HTs), particularly those triggered by solid-colored images. It achieves this by executing pixel-level substitutions using distinct substitution maps for different regions of an image.

In this strategy, substitution maps are generated to provide a unique mapping for each pixel value, ensuring that every pixel in a map is assigned a distinct substitute while maintaining the overall relationship among pixel

values. The process begins by segmenting the pixel value range (0–255) into equivalent partitions. These segments are then randomly paired, with elements from one segment mapped to elements in the paired segment. Furthermore, the input image is divided into smaller regions, with each region assigned a unique substitution map. The substitution process is illustrated in Fig. 8.10. This approach modifies pixel values in such a way that solid-colored images are eliminated, thereby preventing the activation of HTs, while ensuring that the CNN processes the image data accurately and effectively.

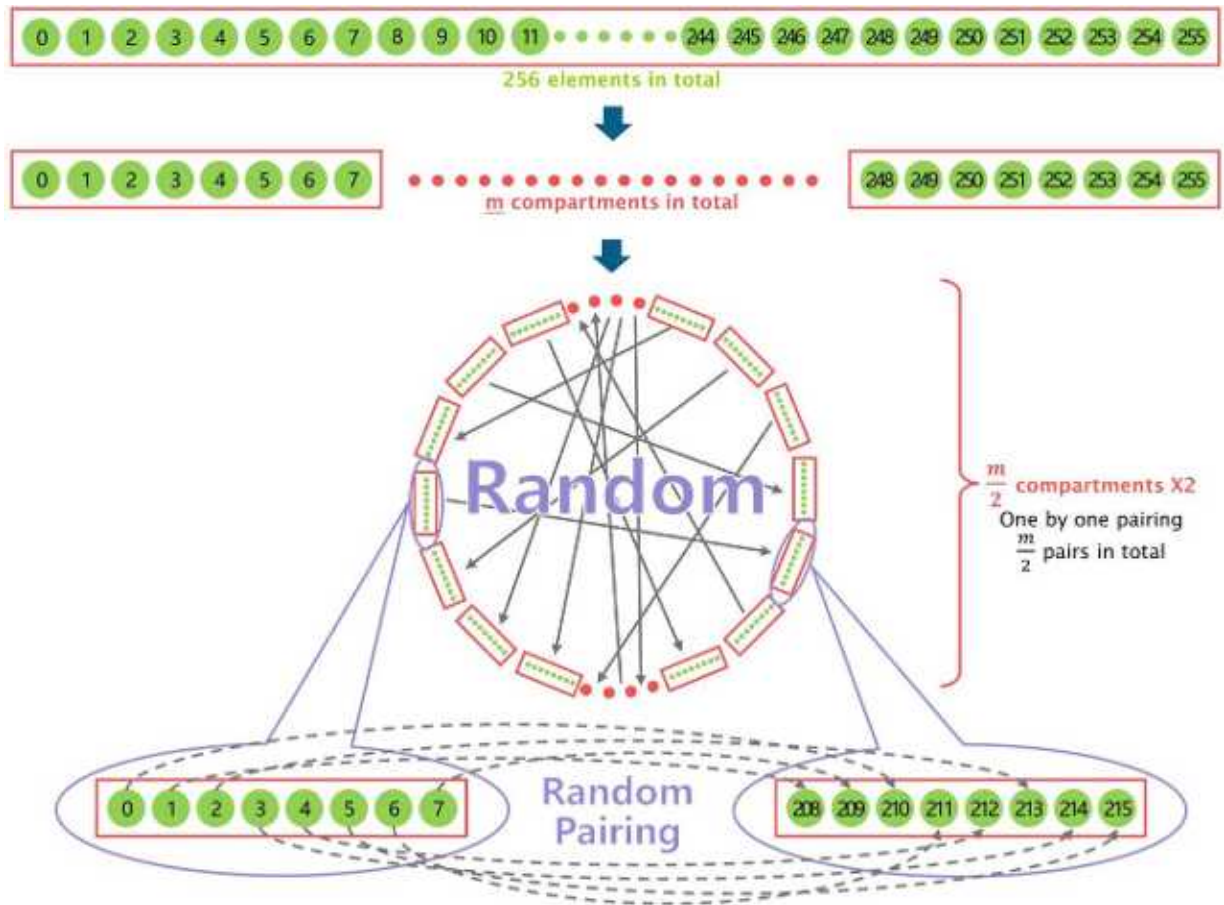


Fig. 8.10 Sectionalized pixel substitution strategy

For generating substitution maps for the non-important regions of the training dataset, the pixel value range (0–255) is divided into fewer blocks, with each block containing a substantial number of elements. Multiple substitution maps are required for these regions to introduce sufficient randomness. Conversely, when creating substitution maps for the important regions of the training dataset and for the test dataset, the range is divided

into numerous smaller segments, each containing fewer elements. In these cases, the pairing among segments is not random but instead follows a symmetrical mapping scheme to preserve critical information. Additionally, only a few substitution maps are needed for these areas to maintain the integrity of the important regions.

This differentiation ensures that important regions retain their detail and relevance while the non-important regions are sufficiently obfuscated to enhance randomness and disrupt potential patterns. Furthermore, using fewer substitution maps for the test dataset reduces complexity and facilitates efficient hardware implementation.

8.5 Hardware Implementation

The implementation of the SSDM within hardware is divided into three principal components: substitution, pixel-level shuffle, and neuron-level shuffle, as shown in Fig. 8.11.

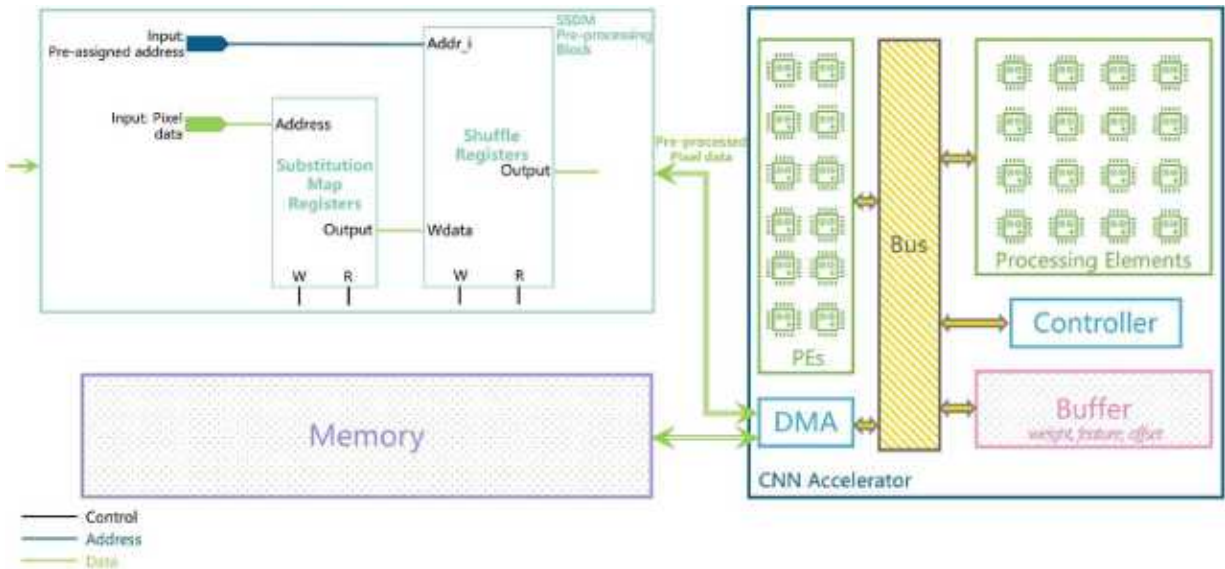


Fig. 8.11 Hardware implementation of SSDM

The substitution phase is facilitated using a lookup table (LUT) methodology. In this approach, each pixel value from the original image serves as an index to retrieve a corresponding value from registers containing substitution maps. This retrieved value then replaces the original pixel value. To accommodate the use of multiple substitution maps for different regions of the image, a multiplexer (MUX) is employed. The

MUX allows for dynamic selection among different substitution maps, ensuring adaptability and flexibility in substitution across varied image regions.

The pixel-level shuffle mechanism requires pre-computation of a shuffle order, which maps each pixel's original position to its new shuffled position. After the substitution phase, the pixel data is stored in memory addresses according to this predetermined shuffle sequence, effectively executing the shuffling process. This pre-computation ensures a seamless and efficient pixel rearrangement while maintaining data consistency throughout the process.

The neuron-level shuffle involves reordering the sets of weights in the final fully connected layer. This reordering does not necessitate changes to the memory address of the weights. Instead, the weight reading sequence is altered, which effectively changes the order in which weights are stored in the buffer. This method achieves the same performance as modifying memory addresses without increasing complexity. Simultaneously, the decoder is updated to correctly interpret the reordered outputs. This implementation supports online shuffling of weight sets while preserving memory access capabilities and overall system efficiency.

8.6 Evaluation Experiment Configuration for the Approach

In this section, four different types of HTs will be introduced and evaluated by inserting them into the CNN hardware accelerator. Afterward, the same HTs are inserted into a CNN accelerator with a preprocessing module to be tested and evaluated. A typical patch-based backdoor attack by third-party dataset providers is then introduced, and the success rates of the backdoor attack before and after the application of the SSDM countermeasures are calculated. This validates the effectiveness of the new countermeasures in preventing HT insertion attacks and backdoor attacks by third-party dataset providers.

8.6.1 Hardware Trojan Insertion Attacks

The testing of hardware Trojan (HT) insertion attacks involves designing various HTs based on known attack mechanisms and embedding them into

CNN accelerators. This approach allows for a comprehensive evaluation of the vulnerabilities and effectiveness of defensive strategies. A controlled datasets of 18207 and 4320 gray-scale speed-limit traffic sign images from GTSRB [24] are utilized for experimentation. The primary objective is to assess the probability of successful HT activation under different scenarios, providing insights into the robustness of CNN accelerators against such attacks.

1. **Position-specific and value-specific triggered HT:** This type of hardware Trojan (HT) operates by embedding malicious information at specific positions within the input image. Activation is triggered when the CNN accelerator detects pixel values at these positions matching predefined criteria. In the experiments, the malicious data is positioned at coordinates (1,1), (1,2), (2,1), and (2,2) in the input image, with corresponding activation pixel values of 132, 121, 144, and 73. A comparison of the images before and after the injection of the trigger signal for position-specific and value-specific triggered HTs by adversaries is illustrated in Fig. 8.12.

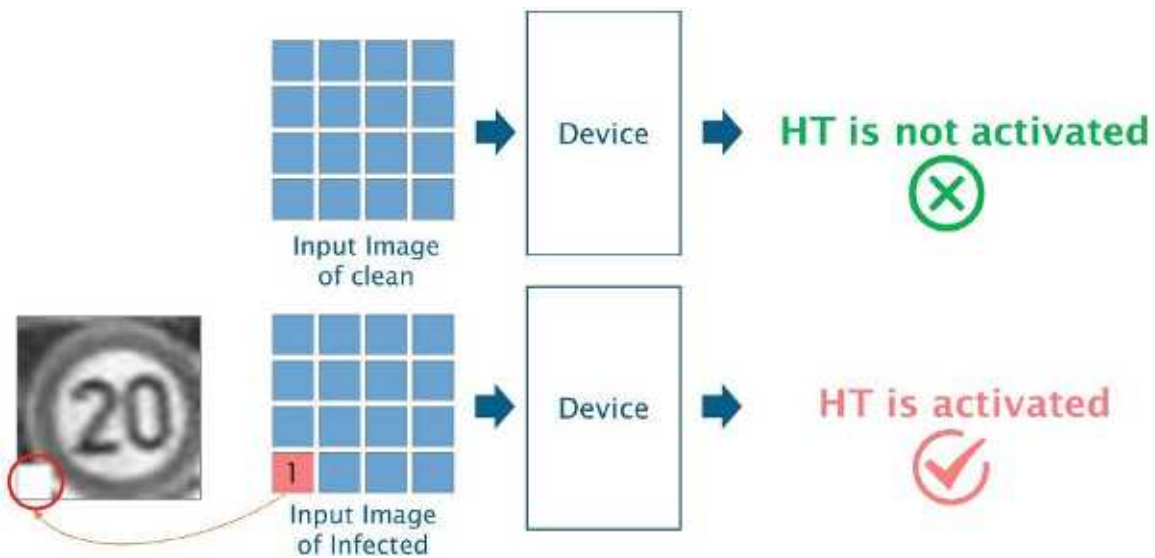


Fig. 8.12 The comparative of images before and after the implementation of position-specific and value-specific triggered HTs

2. **Neuron-specific triggered HT:** This type of hardware Trojan (HT) is triggered when the number of times the output value of a single processing element (PE) exceeds a predefined threshold k surpasses

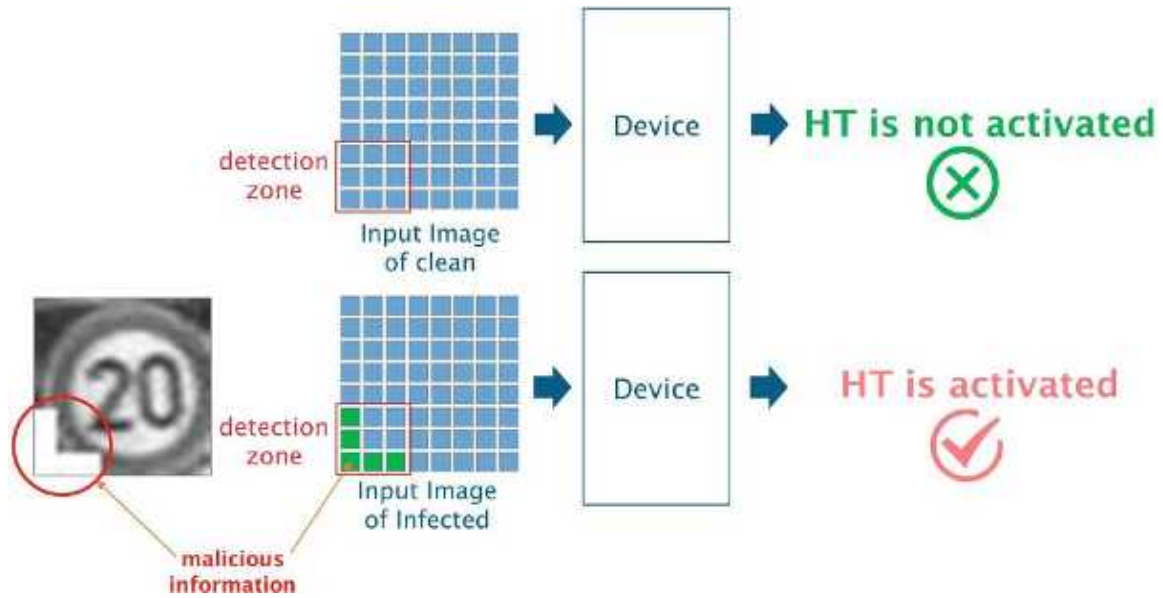


Fig. 8.14 The comparative of images before and after the implementation of pattern-specific triggered HTs

4. **Sequence-specific triggered HT:** This type of hardware Trojan (HT) is activated by feeding a sequence of images into the CNN accelerator, which produces a series of outputs following a predefined sequence. The operational flow of this attack is depicted in Fig. 8.15. In the experiment, the predefined trigger sequence consists of the following categories: category 2, category 1, category 6, category 4, category 9, category 3, category 10, category 3, and category 4, corresponding to the sequence (2, 1, 6, 4, 9, 3, 10, 3, 4).

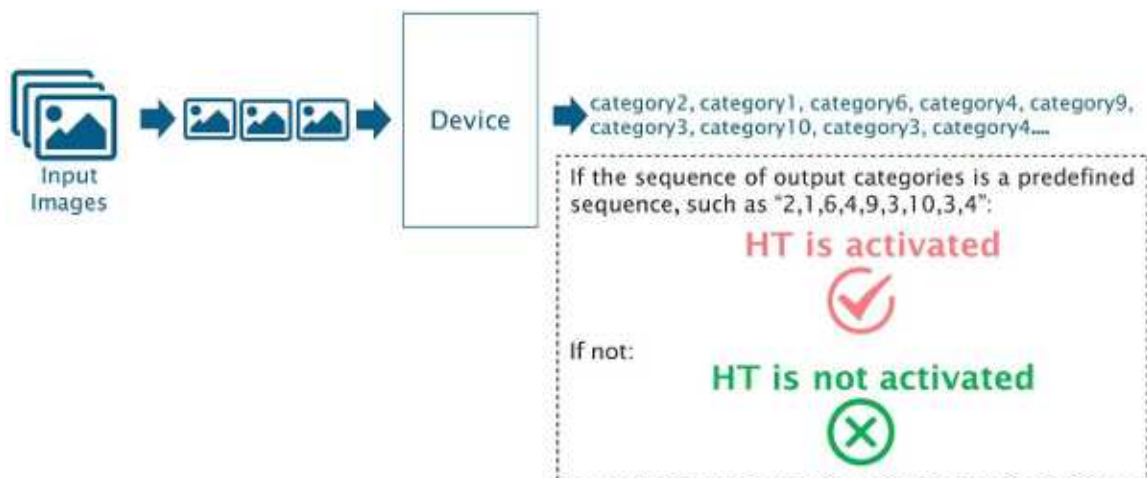


Fig. 8.15 The attack's operational flow of sequence-specific triggered HTs

8.6.2 Backdoor Attacks from Third-Party Dataset Providers

In addition to hardware Trojan (HT) insertion attacks, adversarial backdoor attacks on third-party datasets also pose significant threats. In such attacks, the CNN neural network is trained on a poisoned image dataset, resulting in a network embedded with a backdoor [19–22, 25]. When an image containing a specific trigger is input into the backdoored CNN, the network executes the adversary’s intended malicious behavior.

To investigate backdoor attacks, patch-based backdoor attacks on CNN networks trained with poisoned datasets are analyzed. The experimental methodology involves poisoning 10% of images in selected categories within the 540 gray-scale speed-limit traffic sign images of 9 categories from GTSRB. For testing, 50% of the images in the attacked categories are poisoned to ensure data consistency and minimize randomness, enabling a more controlled evaluation of the attacks’ effectiveness.

For the patch-based backdoor attack, adversaries embed a specific patch into the original image [20, 21]. The location of the patch remains consistent across all modified images in both the training and testing datasets. This strategy leverages the CNN’s ability to learn and memorize patterns, enabling the malicious patch to act as a backdoor. A visual comparison between an original image and its modified counterpart following a stable patch-based backdoor attack is illustrated in Fig. 8.16.

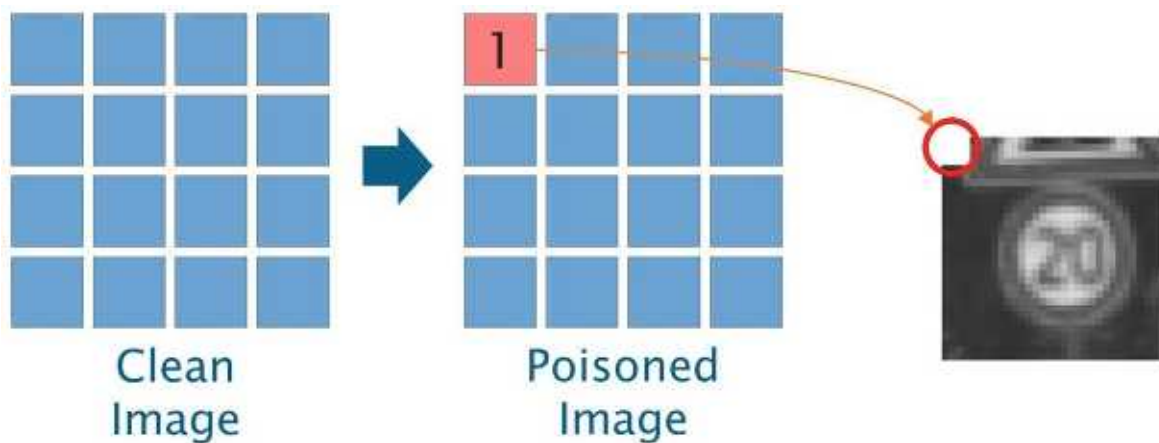


Fig. 8.16 The comparative of images before and after the implementation of stable patch-based backdoor attacks

8.6.3 Defense Configuration

In this section, the configuration of the implementation of the SSDM is outlined. This configuration contains several critical steps:

1. **Image Region Classification:** Images are divided into regions of varying significance based on their dataset. For GTSRB images, the central 16×16 area is designated as important. The precise boundaries of these important areas are depicted in Fig. [8.17](#).

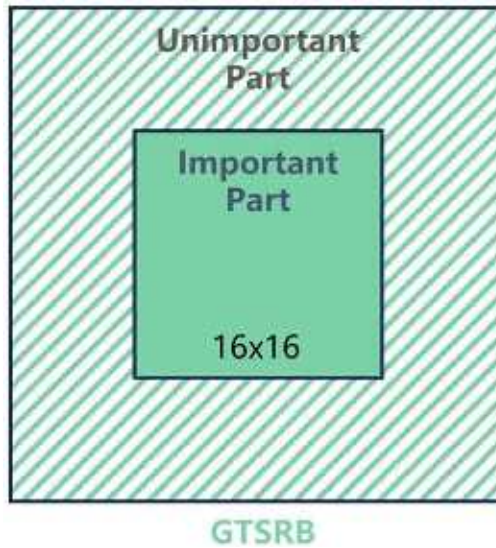


Fig. 8.17 The division of the important area and unimportant area of GTSRB

2. **Substitution Map Generation:** For generating substitution maps in the unimportant regions of the training dataset, the pixel value range (0–255) is divided into two segments, each containing 128 pixel values, to form a single substitution map. To meet the requirements of the SSDM mechanism, a total of 30 such substitution maps are created. For the important regions in both the training and test datasets, four substitution maps are generated. Each map consists of 32 groups, with each group containing eight elements. These substitution maps are also applied to the unimportant regions of the test dataset, ensuring consistency and alignment with the defense strategy.
3. **Important Area Segmentation for Shuffling:** The important regions across all datasets are divided into four approximately equal parts using a lateral segmentation approach.

8.7 Experiment Results and Evaluation

8.7.1 Evaluation of Defense Performance

The results, summarized in Table [8.1](#), provide a comprehensive evaluation of SSDM’s impact on five specified attack methods: position-specific and value-specific triggered HTs, neuron-specific triggered HTs, pattern-specific triggered HTs, sequence-specific triggered HTs, and patch-based backdoor attacks.

Table 8.1 Attack success rate of HT or backdoor attacks (using MNIST dataset)

Type	MobileNet V2	
	SSDM	SSDM-free
Position-specific and value-specific triggered HTs	0%	100%
Neuron-specific triggered HTs	100%	100%
Pattern-specific triggered HTs	2%	98%
Sequence-specific triggered HTs	0%	100%
Patch-based backdoor attacks	12%	94%

The findings in Table [8.1](#) demonstrate a significant increase in robustness following the implementation of SSDM. For sequence-specific, pattern-specific, position-specific, and value-specific triggered HTs, as well as patch-based backdoor attacks, SSDM reduces the attack success rates from nearly 100% to around 10%. However, in the case of neuron-specific triggered hardware Trojans, the CNN accelerator equipped with the preprocessing module showed no mitigation effect, as the trigger success rate remained consistently at 100%.

These results underscore SSDM’s efficacy as a robust defense mechanism, effectively mitigating the threats posed by various hardware Trojan and backdoor attack methods, while maintaining a reasonable trade-off between classification accuracy and security.

8.8 Conclusions

This chapter introduces the shuffle and substitution-based defense mechanism (SSDM). SSDM addresses security vulnerabilities in CNN accelerator by introducing innovative techniques, such as pixel-level shuffling, neuron-level reordering, and substitution mechanisms. These measures aim to disrupt malicious activation triggers while preserving model performance. The effectiveness of SSDM is validated through extensive experiments, demonstrating its ability to significantly reduce the success rates of HTs and backdoor attacks.

References

1. Xue, M., Gu, C., Liu, W., Yu, S., O'Neill, M.: Ten years of hardware trojans: a survey from the attacker's perspective. *IET Comput. Digit. Techn.* **14**(6), 231–246 (2020)
[Crossref]
2. Yu, H., Sun, P., Halak, B., Shanthakumar, K., Kazmierski, T.: Tamper resistant design of convolutional neural network hardware accelerator. In: 2023 Asian Hardware Oriented Security and Trust Symposium (AsianHOST), pp. 1–5 (2023). <https://doi.org/10.1109/AsianHOST59942.2023.10409319>
3. Ye, J., Hu, Y., Li, X.: Hardware trojan in FPGA CNN accelerator. In: 2018 IEEE 27th Asian Test Symposium (ATS), pp. 68–73. IEEE (2018)
4. Clements, J., Lao, Y.: Hardware trojan design on neural networks. In: 2019 IEEE International Symposium on Circuits and Systems (ISCAS), pp. 1–5. IEEE (2019)
5. Zhao, Y., Hu, X., Li, S., Ye, J., Deng, L., Ji, Y., Xu, J., Wu, D., Xie, Y.: Memory trojan attack on neural network accelerators. In: 2019 Design, Automation & Test in Europe Conference & Exhibition (DATE), pp. 1415–1420. IEEE (2019)
6. Hu, X., Zhao, Y., Deng, L., Liang, L., Zuo, P., Ye, J., Lin, Y., Xie, Y.: Practical attacks on deep neural networks by memory trojanning. *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* **40**(6), 1230–1243 (2020)
[Crossref]
7. Liu, Z., Ye, J., Hu, X., Li, H., Li, X., Hu, Y.: Sequence triggered hardware trojan in neural network accelerator. In: 2020 IEEE 38th VLSI Test Symposium (VTS), pp. 1–6. IEEE (2020)
8. Li, P., Hou, R.: Int-monitor: a model triggered hardware trojan in deep learning accelerators. *J. Supercomput.* **79**(3), 3095–3111 (2023)
[Crossref]
9. Mukherjee, R., Chakraborty, R.S.: Novel hardware trojan attack on activation parameters of fpga-based dnn accelerators. *IEEE Embed. Syst. Lett.* **14**(3), 131–134 (2022)
[Crossref]
10. Hou, J., Liu, Z., Yang, Z., Yang, C.: Hardware trojan attacks on the reconfigurable interconnections of field-programmable gate array-based convolutional neural network

accelerators and a physically unclonable function-based countermeasure detection technique. *Micromachines* **15**(1), 149 (2024)

[\[Crossref\]](#)

11. Yang, Y., Ye, J., Li, X., Han, Y., Li, H., Hu, Y.: Implementation of parametric hardware trojan in fpga. In: 2019 IEEE International Test Conference in Asia (ITC-Asia), pp. 37–42. IEEE (2019)
12. Tehranipoor, M., Koushanfar, F.: A survey of hardware trojan taxonomy and detection. *IEEE Des. Test Comput.* **27**(1), 10–25 (2010)
[\[Crossref\]](#)
13. Venugopalan, V., Patterson, C.D.: Surveying the hardware trojan threat landscape for the internet-of-things. *J. Hardw. Syst. Secur.* **2**(2), 131–141 (2018)
[\[Crossref\]](#)
14. Yang, C., Hou, J., Wu, M., Mei, K., Geng, L.: Hardware trojan attacks on the reconfigurable interconnections of convolutional neural networks accelerators. In: 2020 IEEE 15th International Conference on Solid-State & Integrated Circuit Technology (ICSICT), pp. 1–3. IEEE (2020)
15. Venceslai, V.: Fault-injection and neural trojan attacks on spiking neural networks. Ph.D. Thesis, Politecnico di Torino (2020)
16. Odetola, T.A., Khalid, F., Mohammed, H., Sandefur, T.C., Hasan, S.R.: Feshi: feature map-based stealthy hardware intrinsic attack. *IEEE Access* **9**, 115370–115387 (2021)
[\[Crossref\]](#)
17. Halak, B.: Cist: A threat modelling approach for hardware supply chain security. In: *Hardware Supply Chain Security*, pp. 3–65. Springer, Berlin (2021)
18. Van Engelen, J.E., Hoos, H.H.: A survey on semi-supervised learning. *Mach. Learn.* **109**(2), 373–440 (2020)
[\[MathSciNet\]](#)[\[Crossref\]](#)
19. Nguyen, A., Tran, A.: Wanet–imperceptible warping-based backdoor attack. arXiv preprint arXiv:2102.10369 (2021)
20. Gu, T., Dolan-Gavitt, B., Garg, S.: Badnets: Identifying vulnerabilities in the machine learning model supply chain. arXiv preprint arXiv:1708.06733 (2017)
21. Liu, Y., Ma, S., Aafer, Y., Lee, W.C., Zhai, J., Wang, W., Zhang, X.: Trojaning attack on neural networks. In: 25th Annual Network And Distributed System Security Symposium (NDSS 2018). Internet Soc (2018)
22. Chen, X., Liu, C., Li, B., Lu, K., Song, D.: Targeted backdoor attacks on deep learning systems using data poisoning. arXiv preprint arXiv:1712.05526 (2017)
23. Yellu, P., Buell, L., Xu, D., Yu, Q.: Blurring boundaries: A new way to secure approximate computing systems. In: *Proceedings of the 2020 on Great Lakes Symposium on VLSI*, pp. 327–332 (2020)
24. Stallkamp, J., Schlipsing, M., Salmen, J., Igel, C.: Man vs. computer: benchmarking machine learning algorithms for traffic sign recognition. *Neural Netw.* **32**, 323–332 (2012)

25. Huang, K., Li, Y., Wu, B., Qin, Z., Ren, K.: Backdoor defense via decoupling the training process. arXiv preprint arXiv:2202.03423 (2022)

[OceanofPDF.com](https://oceanofpdf.com)

Index

A

- Activation layer 15
- Activation layer in TensorFlow 47
- Adaptive pooling 11
- AES encryption circuit 136
- AI system production process landscape 192
- AlexNet 21
- Application-specific integrated circuits (ASICs) 31
- Approximate multiplier 96
- Attention mechanism 25
- Average pooling 11

B

- Backdoor attack 228
- Back-propagation (BP) neural network 136
- Batch normalisation layer in TensorFlow 46
- Binarized neural network (BNN) 147
- Buffer operation counts (BOC) 139

C

- Central processing unit (CPU) 3
- Classification layer 19
- CNN tutorial in TensorFlow 42
- Consumption stage 205
- Convolutional layer 6
- Convolutional layer (Hardware) 62
- Convolutional layer in TensorFlow 43
- Convolutional neural network (CNN) 3
- Custom CNN training in TensorFlow 54

D

- Data acquisition and annotation 195
- Deadlock 115
- Denial-of-service (DoS) 118
- Dense connection 24

Deploying models 196
Depthwise separable convolutions 25
Destination latency curves (DLCs) 127
Domain-specific accelerators (DSAs) 117

E

Eavesdropping 118
Error Matrix-Based Error Injection 100
Error Matrix of Approximate Multiplier 100
Exponential linear unit (ELU) 16

F

Fully connected layer (Hardware) 66
Fabrication 201
Field-programmable gate arrays (FPGAs) 32
Flooding attacks 118
Flooding DoS 119
Flooding injection rate (FIR) 120
FLOPS 32
Fully Connected Layer 17
Fully connect layer in TensorFlow 50
Functional testing 204

G

Garnet 121
Gem5 121
Generative adversarial network (GAN) 126
GhostNet 22
Global pooling 11
GoogleNet 21
Graphics processing units (GPUs) 3, 31
Graph Neural Networks (GNNs) 169

H

Hardware implementation of CNN using SystemVerilog 57
Hardware performance counter (HPC) 137
Hardware Trojan insertion attack 225

I

Input layer 5
IP cores licensing and delivery 200

K

Kernel 6

L

Large language models (LLMs) 113
Leaky ReLU 16
LeNet 21
Loading data model (Hardware) 59
Long short-term memory (LSTM) 137
Low-and-slow denial-of-service (LSDoS) 126

M

Max pooling 10
MaxPooling layer (Hardware) 64
MobileNet 22
Model archiving stage 206
Model design 195
Model evaluation 196
Model training 195
Multi-frame fusion (MFF) 143
Multi-layer neural networks (MLNNs) 136
Multi-processor system-on-chip (MPSoC) 113

N

Network interface (NI) 116
Neuron level shuffle mechanism 222
Neuron-specific triggered HTs 217
Normalization layer 11

O

On-chip communication 113

P

Packet arrival curves (PACs) 127
Packet injection rate 119
Padding 6

- Parametric ReLU (PReLU) 16
- Pattern recognition 139
- Pattern-specific triggered HTs 217
- PCB assembly 203
- Perceptron 126
- Pixel level shuffle Mechanism 221
- Pooling layer 9
- Pooling layer in TensorFlow 48
- Position-specific and value-specific triggered HTs 216
- Power-grid controllers 137
- Preprocessing for CNN weights and images (Hardware) 58
- ProNoC 148

R

- Reconstruction error distribution (RED) 137
- Rectified linear unit (ReLU) 16
- Recycling stage 206
- Region-of-interest (ROI) 151
- ReLU6 layer (Hardware) 65
- Residual connection 23
- ResNet 21
- Router 115
- Routing-path victims (RPV) 143

S

- Security policy engine (SPE) 128
- Semantic segmentation 139
- Sequence-specific triggered HTs 217
- Shuffle and substitution-based defence mechanism (SSDM) 213
- ShuffleNet 22
- Side-channel attacks 118
- Sigmoid function 15
- SoC integration 201
- Spoofing 118
- Stride 6
- Substitution mechanism 223
- Synthetic traffic pattern 119

T

Testbench (Hardware) 69
Top-level entity (Hardware) 68
Table-like method (TLM) 144
Tanh 16
TensorFlow 148
3D Network-on-Chip 116
Topology 115
Transfer learning in TensorFlow 51

V

VGG 21
Victim complementing enhancement (VCE) 143
Virtual channel occupancy (VCO) 139
Virtual channels (VCs) 115

OceanofPDF.com